深度學習的數學基礎

葉筱倞

2026 年 3 月 25 日[1]

[1]此書稿為 Mathematical Foundations of Deep Learning（https://arxiv.org/abs/2603.18387）的繁體中文翻譯版。此書正式版收於 “Chapman & Hall/CRC Mathematics and Artificial Intelligence Series” 系列，由 Taylor & Francis 公司出版。此書稿不可作商業用途。

```
@book{ye2026mathematical,
  title = {Mathematical Foundations of Deep Learning},
  author = {Ye, Xiaojing},
  year = {2026},
  publisher = {Chapman and Hall/CRC}
}
```

目 錄

序 **vii**

第一章 深度神經網絡 **1**

1.1 函數逼近 1

1.2 淺度和深度神經網絡 7

1.3 普適逼近定理 10

1.4 普適逼近定理的證明 12

1.5 網絡結構設計 24

1.5.1 激活函數示例 24

1.5.2 網絡模塊示例 28

1.5.3 具有特殊性質的網絡 34

1.5.4 網絡訓練準則 38

1.6 備註與參考文獻 42

1.6.1 關於網絡結構設計的備註 42

1.6.2 普適逼近理論的文獻 42

第二章 網絡訓練 **45**

2.1 優化中的基本概念 46

2.2 最優性條件 49

2.2.1 最優性條件的一般原理 51

2.2.2 必要最優性條件 52

2.2.3 充分最優性條件 59

2.3 自動微分 62

2.3.1 一階導數的計算 64

2.3.2 二階導數的計算 68

2.4 確定性優化算法 71

2.4.1 無約束優化算法 73
2.4.2 約束優化算法 84
2.5 隨機優化算法 98
2.5.1 隨機優化的基礎 98
2.5.2 實用隨機優化算法 110
2.6 備註與參考文獻 116
2.6.1 關於網絡訓練優化的備註 116
2.6.2 數值優化的參考文獻 118

第三章 深度最優控制 119
3.1 最優控制理論 119
3.1.1 Euler–Lagrange 方程 120
3.1.2 Hamiltonian 動力學 123
3.1.3 Pontryagin 極大值原理 127
3.1.4 Hamilton–Jacobi–Bellman 方程 146
3.2 最優概率密度控制理論 149
3.2.1 概率控制的極大值原理 152
3.2.2 概率控制的 HJB 方程 158
3.2.3 初值擾動的變分 161
3.3 Neural ODE 方法 164
3.4 備註與參考文獻 172
3.4.1 在 PDE 解算子學習中的應用 172
3.4.2 最優控制的參考文獻 178

第四章 深度強化學習 181
4.1 強化學習基礎 181
4.1.1 強化學習中的基本概念 181
4.1.2 Markov 決策過程 182
4.1.3 Bellman 算子與 Bellman 方程 184
4.1.4 策略最優性與改進理論 193
4.2 強化學習算法 198
4.2.1 基於模型的強化學習算法 198
4.2.2 無模型強化學習算法 204
4.3 備註與參考文獻 222
4.3.1 強化學習與最優控制 222

4.3.2 強化學習的文獻 223

第五章 生成模型 225

5.1 變分自編碼器 226
5.2 生成對抗網絡 233
5.3 擴散模型 237
5.4 概率密度控制 250
5.5 流匹配 253
5.6 備註與參考文獻 259
5.6.1 生成模型中的噪聲採樣 259
5.6.2 條件生成模型 263
5.6.3 生成模型的參考文獻 266

附录 A 補充材料 269

A.1 Monte Carlo 積分 269
A.2 Banach 空間與不動點理論 271
A.3 信息論基礎 275
A.4 隨機微分方程基礎 284

序

Over the past fifteen years, deep learning has rapidly evolved into one of the central pillars of modern science and engineering, and it is now widely regarded as the most promising approach toward general artificial intelligence. Despite its remarkable empirical success, theoretical understanding has lagged far behind practical advances for much of this period. This book is based on a collection of literature and notes I took in the past years. It is written in response to a growing need, particularly from people interested in learning and understanding deep learning from mathematical perspective: A relatively coherent, principled, and mathematically grounded account of deep learning that focuses on the underlying structures, assumptions, and theoretical guarantees of deep learning models.

At its core, deep learning is a mathematical enterprise. Neural networks are function approximators; training corresponds to solving large-scale nonconvex optimization problems; generalization is fundamentally a question of probability and statistics; and expressivity, stability, and robustness are governed by ideas from approximation theory, functional analysis, and dynamical systems. The goal of this book is to provide some mathematical foundations needed to understand why deep neural networks work, how to train them effectively, and how to leverage deep learning to solve problems arising from machine learning, scientific computing, automatic control, etc.

I hope this book can be beneficial to people in several fields. For readers with a background in mathematics, it provides an entry point into deep learning that respects rigor and abstraction while remaining closely connected to modern practice. For readers trained primarily in machine learning or engineering, it offers a deeper theoretical perspective, unifying familiar concepts under precise mathematical formulations. Throughout the book, I tend to emphasize clarity of assumptions, careful definitions, and explicit connections between theory and algorithms.

The materials in this book are organized around several foundational themes. This book begins by viewing neural networks as parametric function classes and studying their expressive power through the lens of approximation theory. Then the discussion focuses on optimization theory and numerical algorithms, which underpin the training of deep neural networks. Subsequent chapters explore the integration of deep learning with optimal control, rein-

forcement learning, and generative modeling, highlighting the mathematical principles that govern their design and behavior. Wherever possible, connections are drawn between classical results and contemporary developments.

This book is definitely not exhaustive nor intended to be a comprehensive survey. Instead, it should be considered as an introduction to deep learning through a mathematics-oriented approach. It emphasizes ideas, concepts, and results that are likely to remain relevant as the field continues to evolve extremely fast. Proofs are provided for central results, and examples and remarks are included to build intuition and to illustrate how abstract theory informs practical modeling choices.

The intended audience includes graduate students (and advanced undergraduates) and researchers in mathematics, statistics, computer science, electrical engineering, and related fields. A solid background in the standard sequence of calculus courses, linear algebra, and probability is required, while familiarity with advanced calculus and real analysis will be very helpful.

Deep learning continues to reshape both applied sciences and theoretical inquiry. By presenting from a mathematical perspective, I hope this book will help readers develop a deeper understanding of this subject, critically assess emerging ideas, and contribute to the ongoing effort to place deep learning on firm theoretical ground.

I wish to thank Yunmei Chen, Hongyuan Zha, and Haomin Zhou for many years of fruitful collaboration and insightful discussions in the field of mathematics and deep learning, and also acknowledges my excellent students and collaborators, including Nathan Gaby, Shu Liu, Hao Wu, Yaohua Zang, and many others, for the valuable discussions and joint work these years. I am grateful to National Science Foundation for the support of my research.

Xiaojing Ye
Atlanta, Georgia, USA

常用記號

$A := B$	將 A 定義為 B（或 $B =: A$）
s.t.	滿足
$\mathbb{N}$	自然數集合 $\{1, 2, \dots\}$
$\mathbb{R}$	實數集合
$\mathbb{R}^n$	n 維實歐幾里得空間
$[N]$	對任意 $N \in \mathbb{N}$，表示 $\{1, 2, \dots, N\}$
I_d	$d \times d$ 單位矩陣
$A^\top$	（復）矩陣 A 的（共軛）轉置
$\mathrm{tr}(A)$	方陣 A 的跡
$A \succeq B$	$A - B$ 是半正定的（若為 $\succ$，則表示正定）
$\lvert \cdot \rvert$	絕對值，或向量與矩陣的 2-範數
$\lVert \cdot \rVert_*$	由下標 $*$ 指定的函數範數
$x \odot y$	$x, y \in \mathbb{R}^n$ 的逐分量乘積
$\langle x, y \rangle$	$\mathbb{R}^n$ 或 Hilbert 空間中 x 與 y 的內積
$x'(t)$ or $\dot{x}(t)$	函數 $x(t)$ 關於 t 的導數
∇_θ	關於 θ 的梯度（簡記 $\nabla = \nabla_x$）
$\nabla\cdot$	關於 x 的散度
∇^2	關於 x 的 Hessian 矩陣
Δ	關於 x 的 Laplace 算子
C^k	k 次連續可微函數的集合
$\arg\min$ ($\arg\max$)	極小值點（極大值點）的集合
$X \sim p$	X 是服從概率分佈 p 的隨機變量
$x \sim p$	x 是從概率分佈 p 中抽取的樣本
i.i.d.	獨立同分布
$\mathcal{N}(\mu, \Sigma)$	均值為 μ、方差為 Σ 的高斯分佈
$\mathcal{N}(\cdot; \mu, \Sigma)$	$\mathcal{N}(\mu; \Sigma)$ 的概率密度函數
$\mathbb{E}[f(X)]$	$f(X)$ 的期望
$\mathbb{E}_{X \sim p(\cdot \mid Y)}[f(X, Y)]$	給定 Y 時 $f(X, Y)$ 的條件期望
$\mathrm{Var}[f(X)]$	$f(X)$ 的方差

第一章　深度神經網絡

深度神經網絡（deep neural network）是現代深度學習（deep learning）系統的核心構建模塊。從數學角度來看，深度神經網絡是一種有高度表達能力的函數逼近器（function approximator）。人們受到生物神經系統分層組織的啟發而構造了深度神經網絡，並發現它們確實可以有效地通過層次化來處理和表示數據。

本章首先介紹深度神經網絡的動機和基本概念。隨後我們將重點討論神經網絡理論中最基本的結果之一，即普適逼近定理（Universal Approximation Theorem），並在逼近定義於歐幾里得空間緊子集上的一類連續函數時，給出帶有定量的誤差估計的嚴格證明。

之後，我們會回顧常用的激活函數和構成現代深度神經網絡架構的核心模塊。這一部分是為缺乏豐富實踐經驗的讀者提供一些深度網絡構建的直觀理解。更重要的是，我們將通過一些示例，展示如何構造滿足特定函數性質的神經網絡。這類構造技巧通常能夠顯著提升網絡訓練效率和實際性能。最後，我們簡要討論一下訓練網絡用的目標函數，並給出一些啟發性示例，說明我們有時只基於物理定律也能構建它們，不必依賴外部提供的數據。

1.1　函數逼近

函數逼近器在統計學和數據科學中被廣泛使用。我們首先回顧統計學中用於線性迴歸和邏輯分類的兩個經典函數逼近器示例。

例 1.1.1 (線性迴歸，linear regression)**.** 假設有 d 個變量 $x_1, \ldots, x_d \in \mathbb{R}$，稱為**特徵**（feature），它們組合形成一個**特徵向量** $x = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$。此外，有一個觀測值 $y \in \mathbb{R}$，已知它是這些變量的一個仿射函數（affine function）再加上一些噪聲：

$$y = \sum_{i=1}^{d} w_i x_i + b + e, \tag{1.1}$$

其中 $w = (w_1, \ldots, w_d) \in \mathbb{R}^d$ 和 b 分別稱為**權重**（weight）和**偏置**（bias），它們是未知的。在 (1.1) 中，我們假設噪聲 e 服從均值為 0、標準差為未知量 s 的高斯分佈 $\mathcal{N}(0, s^2)$。也就是說，e 的概率分佈為

$$\mathcal{N}(e; 0, s^2) := \frac{1}{\sqrt{2\pi s^2}} e^{-\frac{|e|^2}{2s^2}}.$$

問題是：如何利用樣本 (x, y) 以及 (1.1) 中給出的關係來估計 (w, b)?

顯然，我們無法從單個樣本 (x, y) 中估計 (w, b)。實際中，我們需要一個包含多個（例如 M 個）樣本的數據集 $\mathcal{D}$：

$$\mathcal{D} = \Big\{ (x^{(j)}, y^{(j)}) \in \mathbb{R}^d \times \mathbb{R} : \ j \in [M] \Big\},$$

其中每個樣本 $(x^{(j)}, y^{(j)})$ 都滿足 (1.1) 中的相同仿射關係，並且噪聲 $e^{(j)} \sim \mathcal{N}(0, s^2)$ 相互獨立同分布（i.i.d.），$j \in [M] := \{1, 2, \ldots, M\}$。

為簡化記號，我們用 θ 表示 w 和 b 拼接而成的 $(d+1)$ 維列向量：

$$\theta := \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{d+1}.$$

那麼，對於數據集 $\mathcal{D}$，如果以下關係

$$y^{(j)} - \theta^\top \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix} = y^{(j)} - (w^\top x^{(j)} + b) = e^{(j)} \sim \mathcal{N}(0, s^2)$$

對所有 $j \in [M]$ 成立，我們會得到 θ 的**似然函數**（likelihood function）L：

$$\begin{aligned} L(\theta) &:= \prod_{j=1}^{M} \mathcal{N}(e^j; 0, s^2) \\ &= \prod_{j=1}^{M} \mathcal{N}(y^{(j)} - w^\top x^{(j)} - b; 0, s^2) \\ &= (2\pi s^2)^{-\frac{M}{2}} e^{-\frac{1}{2s^2} \sum_{j=1}^{M} |y^{(j)} - w^\top x^{(j)} - b|^2}. \end{aligned}$$

統計學中常用的極大似然估計（maximum likelihood estimate）方法，即認為 $L(\theta)$ 取得最大值的點就是最優的 θ^* 。由於負對數函數是嚴格遞減的，θ^* 同時

也是**負對數似然函數**（negative log-likelihood function）ℓ 的最小化解：

$$\begin{aligned}\ell(\theta) &:= -\log L(\theta)\\ &= \frac{1}{2s^2}\sum_{j=1}^{M}|y^{(j)} - w^\top x^{(j)} - b|^2 + \frac{M}{2}\log(2\pi s^2).\end{aligned}$$

因此我們得到**極大似然估計** θ^*：

$$\theta^* = \begin{pmatrix} w^* \\ b^* \end{pmatrix} = \arg\min_{(w,b)} \frac{1}{2s^2}\sum_{j=1}^{M}|y^{(j)} - w^\top x^{(j)} - b|^2. \tag{1.2}$$

注意，當初未知的那個標準差 s 不影響 (1.2) 的解，因此我們可以忽略它。現在記

$$X = \begin{pmatrix} x^{(1)\top} & 1 \\ \vdots & \vdots \\ x^{(M)\top} & 1 \end{pmatrix} \in \mathbb{R}^{M\times(d+1)} \quad \text{和} \quad Y = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(M)} \end{pmatrix} \in \mathbb{R}^M,$$

則可以將最小化問題 (1.2) 寫為簡潔形式：

$$\min_{\theta\in\mathbb{R}^{d+1}} \frac{1}{2}|Y - X\theta|^2. \tag{1.3}$$

把 (1.3) 中的目標函數對 θ 求梯度，我們得到

$$X^\top X\theta = X^\top Y. \tag{1.4}$$

我們知道 (1.3) 的極小值點必定滿足 (1.4) 。

注意，當 M 足夠大且 $x^{(j)}$ 足夠多樣時（這在問題設置合理、且數據採集得當時，通常是成立的），矩陣 X 具有滿列秩 $d+1$。（更精確地說，$x^{(1)},\ldots,x^{(M)}$ 張成 $\mathbb{R}^{d+1}$ 中一個不包含 $1 = (1,\ldots,1)^\top$ 的 d 維線性子空間，這樣的 X 具有滿列秩。）因此，$X^\top X$ 可逆，且 (1.4) 只有唯一解：

$$\theta^* = \begin{pmatrix} w^* \\ b^* \end{pmatrix} = (X^\top X)^{-1}X^\top Y,$$

這個解也是 (1.2) （或等價地 (1.3)）的唯一極小值點（因為 (1.3) 顯然是關於 θ 的凸函數）。這就是線性迴歸裡，通過利用數據集 $\mathcal{D}$ 在 (1.1) ，來“學習”最優的 w 和 b 的方式。

例 1.1.2 (邏輯分類，Logistic Classification)**.** 假設對於每一個 d 維特徵向量 $x = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$，都有一個取值為二元的分類標識 $y \in \{0, 1\}$，稱為**標籤**（label）。當特徵向量 x 屬於 “真” 類時，標籤 $y = 1$；當 x 屬於 “假” 類時，標籤 $y = 0$。

在標準的邏輯分類（也稱邏輯迴歸，logistic regression）中，若預測特徵向量 x 屬於 “真” 類，即其對應的標籤 y 是 1 的概率 $\Pr(y = 1)$ 被設置為：

$$\Pr(y = 1) = \sigma(w^\top x + b), \tag{1.5}$$

其中

$$\sigma(z) := \frac{1}{1 + e^{-z}}, \qquad \forall\, z \in \mathbb{R} \tag{1.6}$$

稱為**邏輯函數**（logistic function），更常稱為 **Sigmoid 函數**。注意 σ 的值域為 $(0, 1)$，因此可用於表示概率。我們知道

$$\Pr(y = 0) = 1 - \Pr(y = 1) = 1 - \sigma(w^\top x + b).$$

換言之，y 可以看作服從參數為 $\sigma(w^\top x + b) \in (0, 1)$ 的伯努利（Bernoulli）分佈的樣本。

我們知道，若隨機變量 Z 取值為 1 或 0，且服從 Bernoulli(p) 分佈（$p \in (0, 1)$），則 $\Pr(Z = z) = p^z (1 - p)^{1-z}$，這是該分佈的概率質量函數的一種統一表達形式。

為簡化記號，我們記

$$\theta := \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{d+1}.$$

現在我們同樣需要一個包含 M 個獨立同分布樣本的數據集 $\mathcal{D}$

$$\mathcal{D} = \Big\{ (x^{(j)}, y^{(j)}) \in \mathbb{R}^d \times \{0, 1\} : \ j \in [M] \Big\}$$

其中每個樣本 $(x^{(j)}, y^{(j)})$ 都滿足 (1.5)。於是 θ 的似然函數 L 為：

$$L(\theta) = \prod_{j=1}^{M} \big(\sigma(w^\top x^{(j)} + b)\big)^{y^{(j)}} \big(1 - \sigma(w^\top x^{(j)} + b)\big)^{1 - y^{(j)}},$$

其負對數似然函數 ℓ 為:

$$\begin{aligned}\ell(\theta) &= -\log L(\theta) \\ &= -\sum_{j=1}^{M}\Big(y^{(j)}\log\sigma(w^\top x^{(j)}+b)+(1-y^{(j)})\log(1-\sigma(w^\top x^{(j)}+b))\Big).\end{aligned} \tag{1.7}$$

為了求解 (1.7) 中 ℓ 的極小值，我們首先推導一些簡單結果。注意 σ 的導數為

$$\sigma'(z)=\frac{e^{-z}}{(1+e^{-z})^2}=\sigma(z)(1-\sigma(z)).$$

我們定義函數 $h:\mathbb{R}^d\times\mathbb{R}^{d+1}\to\mathbb{R}$ 為

$$h(x,\theta):=\sigma(w^\top x+b).$$

則有

$$\begin{aligned}\nabla_\theta h(x,\theta) &= \sigma'(w^\top x+b)\begin{pmatrix}x\\1\end{pmatrix}^\top \\ &= \sigma(w^\top x+b)\big(1-\sigma(w^\top x+b)\big)\begin{pmatrix}x\\1\end{pmatrix}^\top \in\mathbb{R}^{1\times(d+1)},\end{aligned}$$

從而

$$\begin{aligned}\nabla_\theta\log h(x,\theta) &= (1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}^\top, \\ \nabla_\theta\log(1-h(x,\theta)) &= -\sigma(w^\top x+b)\begin{pmatrix}x\\1\end{pmatrix}^\top.\end{aligned}$$

它們的 Hessian 矩陣分別為

$$\begin{aligned}\nabla_\theta^2\log h(x,\theta) &= -\sigma(w^\top x+b)(1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}\begin{pmatrix}x\\1\end{pmatrix}^\top, \\ \nabla_\theta^2\log(1-h(x,\theta)) &= -\sigma(w^\top x+b)(1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}\begin{pmatrix}x\\1\end{pmatrix}^\top,\end{aligned}$$

對於任意 x 和 θ，它們是相同的負半定矩陣。

基於上述結果，由 (1.7) 得到

$$\nabla_\theta \ell(\theta) = -\sum_{j=1}^{M} \left(y^{(j)}(1-\sigma(w^\top x^{(j)}+b)) - (1-y^{(j)})\sigma(w^\top x^{(j)}+b) \right) \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix}^\top$$

在實際實現中，由於 $y^{(j)}$ 僅取 1 或 0，上式可以進一步簡化。此外，有

$$\nabla_\theta^2 \ell(\theta) = \sum_{j=1}^{M} \sigma(w^\top x^{(j)}+b)(1-\sigma(w^\top x^{(j)}+b)) \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix} \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix}^\top \succeq 0$$

這表明 ℓ 關於 θ 是凸的。

因此，滿足

$$\nabla_\theta \ell(\theta) = (\partial_w \ell(\theta), \partial_b \ell(\theta)) = 0 \tag{1.8}$$

的任意臨界點（critical point）θ 都是 ℓ 的全局最小點，從而也是基於數據集 $\mathcal{D}$ 的極大似然估計。此外，若

$$X = \begin{pmatrix} x^{(1)\top} & 1 \\ \vdots & \vdots \\ x^{(M)\top} & 1 \end{pmatrix} \in \mathbb{R}^{M\times(d+1)}$$

具有滿列秩 $d+1$，則對任意非零 $z \in \mathbb{R}^{d+1}$，有

$$z^\top \nabla_\theta^2 \ell(\theta) z = \sum_{j=1}^{M} \sigma(w^\top x^{(j)}+b)(1-\sigma(w^\top x^{(j)}+b)) \, |(x^{(j)\top},\ 1)z|^2 > 0,$$

從而 $\nabla_\theta^2 \ell(\theta) \succ 0$，因此 (1.8) 僅有一個臨界點，即 ℓ 的唯一最小點。

例 1.1.1 and 1.1.2 引出了關於給定數據集 $\mathcal{D}$ 的函數逼近的若干問題：

1. 如果我們擁有數據集 $\mathcal{D}$，但沒有像 (1.1) 和 (1.5) 中那樣用於描述 x 與 y 關係的具體函數 f，該怎麼辦?

2. 如何找到一個模型（參數化函數），使其在適當參數（例如 w 和 b）下可以（近似地）表示未知函數 f?

3. 如何在模型表示 f 的精度與效率之間取得良好平衡? 即模型能夠在預設函數距離度量下逼近某一類函數中的任意函數，同時模型規模（參數數量）儘可能小?

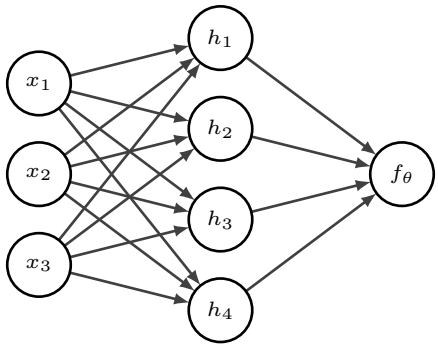


图 1.1: 一個淺層神經網絡 f_θ，其輸入層寬度（維數）為 $d = 3$，隱藏層寬度為 $d_1 = 4$，輸出層寬度為 1。

4. 由於在實際中我們並不知道真實的 f，應如何設定評價標準，以判斷模型是否具有正確的參數來逼近目標函數?

5. 在實際中如何求解模型的參數?

6. 為了獲得模型的正確參數，我們需要多大的數據集 $\mathcal{D}$（即樣本數量）?

在本書中，這些問題將始終圍繞我們，我們也會嘗試逐步給出答案。

1.2 淺度和深度神經網絡

我們考慮如下模型，它結合了線性迴歸和邏輯分類示例（例 1.1.1 和例 1.1.2）中的思想:

$$f_\theta(x) := \sum_{j=1}^{d_1} a_j \sigma(w_j^\top x + b_j), \tag{1.9}$$

其中 $w_j \in \mathbb{R}^d$，$a_j, b_j \in \mathbb{R}$，$d_1 \in \mathbb{N}$，並且 θ 是將所有這些 w_j、a_j 和 b_j（$j = 1, \ldots, d_1$）拼接而成的向量。為方便起見，在本書的其餘部分中，我們通常稱 θ 為 “一個參數”，儘管它實際上是 (1.9) 中模型 f_θ 的所有參數的組合。

例 1.2.1 (淺度神經網絡)**.** 我們用圖 1.1 說明 (1.9) 中的模型 f_θ 在 $d = 3$ 且 $d_1 = 4$ 的情形。

圖 1.1 所示的圖對應一個模型 f_θ，我們稱之為**淺度神經網絡**（或淺層神經網絡），其中 x_1, x_2, x_3 是**輸入層**（input layer）中的神經元，h_1, h_2, h_3, h_4 是**隱藏層**（hidden layer）中的神經元，而 f_θ 是**輸出層**（output layer）中的神經

元。這些神經元的取值關係可以表示為

$$h_j(x) = \sigma(w_j^\top x + b_j) \in \mathbb{R}, \quad \forall\, j \in [d_1],$$

從而有

$$f_\theta(x) = \sum_{j=1}^{d_1} a_j h_j(x).$$

這裡 σ 稱為**激活函數**（activation function），它可以是 (1.6) 中的邏輯函數，也可以是任意一個非仿射函數；不然的話，f_θ 將退化為關於 x 的仿射函數。我們將在第 1.5.1 節介紹現代深度學習中若干常用的激活函數。

奠基性工作 [30] 表明，形如 (1.9) 的淺層神經網絡 在定義於 $\mathbb{R}^d$ 的緊子集 Ω 上的連續函數集合中是稠密的。過去幾十年裡，人們針對更一般的情形發展了大量理論結果，例如使用更深的網絡、其他激活函數以及不同的目標函數集合進行逼近。我們將在第 1.6.2 節給出更多相關參考文獻。

儘管像 (1.9) 這樣的淺層網絡具有可證明的函數逼近能力，但人們發現，它們的複合形式——通常稱為**深度神經網絡**——在函數逼近方面更為強大，而且所需參數數量遠少於理論估計所暗示的規模。深度神經網絡的一個簡單例子是多個淺層網絡的直接複合，稱為**多層感知機**（multilayer perceptron，簡稱 MLP），如下一個例子所示。

例 1.2.2 (多層感知機)**.** 多層感知機（MLP）是多個形如 (1.9) 的淺層網絡的複合。具體地，一個 MLP 可以有 L 個隱藏層，其中第 l 個隱藏層可以有 d_l 個神經元 $h_j^{(l)}$，這裡 $j = 1, \ldots, d_l$ 且 $l = 1, \ldots, L$。輸出層則可以根據需要包含一個或多個神經元。

對於一個具有兩個隱藏層的 MLP，第一隱藏層中的神經元可以表示為

$$h_j^{(1)}(x) = \sigma^{(1)}(w_j^{(1)\top} x + b_j^{(1)}), \quad w_j^{(1)} \in \mathbb{R}^d,\ b_j^{(1)} \in \mathbb{R},\ j = 1, \ldots, d_1.$$

第二隱藏層中的神經元可以表示為

$$h_j^{(2)}(x) = \sigma^{(2)}(w_j^{(2)\top} h_j(x) + b_j^{(2)}), \quad w_j^{(2)} \in \mathbb{R}^{d_1},\ b_j^{(2)} \in \mathbb{R},\ j = 1, \ldots, d_2.$$

這裡 $\sigma^{(1)}$ 和 $\sigma^{(2)}$ 分別是第一隱藏層和第二隱藏層中的激活函數。輸出層神經

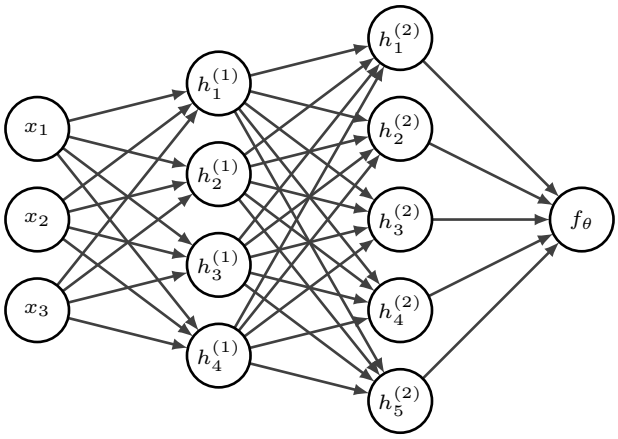


图 1.2: 圖中展示了一個深度神經網絡 f_θ 的例子。這種網絡也稱為多層感知機（MLP），其輸入層寬度為 $d = 3$，兩個隱藏層寬度分別為 $d_1 = 4$ 和 $d_2 = 5$，輸出層寬度為 1。

元由下式給出

$$f_\theta(x) = \sum_{j=1}^{d_2} a_j h_j^{(2)}(x) + b, \quad a_j,\ b \in \mathbb{R},\ j = 1, \ldots, d_2,$$

在這個例子中，它是關於第二層也是最後一層隱藏層的一個仿射函數。

輸入層寬度為 $d = 3$、兩個隱藏層寬度分別為 $d_1 = 4$ 和 $d_2 = 5$、輸出層寬度為 1 的 MLP 如圖 1.2 所示。參數 θ 是將該 MLP 中所有權重和偏置拼接而成的向量，我們可以計算出 θ 的維數為 $(d+1)d_1 + (d_1+1)d_2 + (d_2+1)$。

一個 MLP 可以有很多隱藏層。隱藏層越多，網絡就越深。訓練一個模型 f_θ，無論它是 MLP 還是任意其他結構的深度神經網絡，都是指尋找參數 θ，使得 f_θ 能夠逼近所需的未知函數。這通常統稱為**深度學習**。深度學習是一類新的機器學習技術，並且由於其在廣泛真實世界應用中的驚人經驗成功，它被認為是實現通用人工智能（artificial intelligence，簡稱 AI）最有前景的方法。

在結束本節之前，我們對深度神經網絡作幾點說明，並提出一個問題。首先，我們注意到激活函數在構建深度神經網絡時起著重要作用。特別地，激活函數不應當是仿射的——否則神經網絡不過是仿射函數的複合，最終仍只能得到仿射函數，而其逼近能力非常有限。除了前面見過的邏輯（sigmoid）函數 (1.6) 之外，另一個流行、簡單而有效的激活函數是

$$\sigma(x) = \max(0, x), \tag{1.10}$$

稱為修正線性單元（rectified linear unit，簡稱 ReLU）函數。注意，典型的激活函數，如 (1.6) 和 (1.10)，都是一元函數，即從 $\mathbb{R}$ 到 $\mathbb{R}$ 的映射。當它們作用於向量時，是對輸入向量的各個分量分別作用，從而得到一個與輸入向量維數相同的輸出向量。我們將在第 1.5.1 節展示另外幾種常用的激活函數。

在本書的其餘部分中，我們將深度神經網絡用作逼近複雜目標函數的有前景的參數化模型。特別地，我們將具有未指定網絡參數 θ 的 f_θ 稱為一種**網絡架構**（network architecture）。在一個網絡架構中，神經元（節點）、它們之間的連接（有向邊）以及神經元處的激活函數都是固定的，而參數（如權重和偏置）的取值尚未指定。

與此同時，關於神經網絡我們有一個重要問題：**為什麼神經網絡能夠保證具有逼近未知且複雜函數 f 的能力?**

為了以足夠嚴格的數學方式回答這個問題，我們首先需要指定一個函數集合 F，使得函數 f 屬於其中（這樣我們就可以將搜索範圍縮小到 F 中），並指定一個範數 $\|\cdot\|$ 作為衡量兩個函數之間距離的度量。然後，對於任意 $\epsilon > 0$，我們需要構造——或者至少證明存在——一個網絡架構 f_θ，使得對於任意 $f \in F$，都存在一個參數 θ 滿足 $\|f_\theta - f\| \le \epsilon$。接下來的兩節中，我們將給出更多細節，並回答這個問題。

1.3 普適逼近定理

上一節提出的問題是很有挑戰性的。為了回答這個問題，我們需要對函數 f 及其定義區域作出一些基本假設。在文獻中，通常假設 f 是連續的，並且其定義域 Ω 是 $\mathbb{R}^d$ 的一個有界子集。因此，不失一般性，我們假設定義域為 $\Omega = [0,1]^d$，即 $\mathbb{R}^d$ 中的閉單位立方體，並且 f 屬於一大類函數；除連續性之外，這類函數還可能具有某些光滑性性質（例如可微性）。

在本節和下一節中，我們遵循 [158]，陳述一個關於深度 ReLU 網絡（即所有激活函數都取為 ReLU 的神經網絡）逼近性質的定理，並給出其證明。該定理給出了一個具有代表性的定量誤差估計，而其證明只需要微積分和基礎的數學分析技巧。

現在，我們任意給定 $M > 0$、$k \in \mathbb{N}$，並定義

$$F := \{f \in W^{k,\infty}([0,1]^d) : \|f\|_{W^{k,\infty}([0,1]^d)} \le M\}. \tag{1.11}$$

（我們將在本節稍後對這個索伯列夫（Sobolev）空間 $W^{k,\infty}$ 及其範數作更詳細的說明。）那麼我們希望說明，對於任意 $\epsilon > 0$，都存在一個深度 ReLU 網絡架

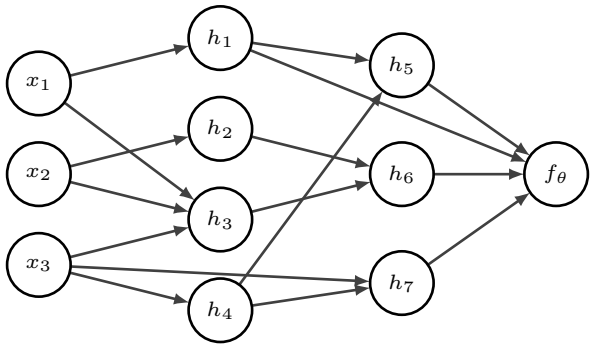


图 1.3: 一個 ReLU 前饋網絡 f_θ，具有 3 個輸入神經元 x_1、x_2 和 x_3。其餘所有神經元都是計算單元。該網絡中權重和偏置的總數為 24，即計算單元的數量（本例中為 8，即 $h_1, \ldots, h_7, f_\theta$）與邊的數量（本例中為 16）之和。

構 f_θ，其規模（我們將在下文中定義 "規模"）依賴於 d、M、k 和 ϵ，使得對任意 $f \in F$，存在某個依賴於 f 的參數 θ 滿足

$$\|f_\theta - f\|_{L^\infty([0,1]^d)} \leq \epsilon.$$

這一結果正式稱為神經網絡的**普適逼近定理**（Universal Approximation Theorem）。（該定理的正式表述及其證明將在下一節給出。）

前饋網絡及其規模 由於將要陳述的逼近定理給出了關於誤差 ϵ 的網絡規模定量估計，我們需要澄清網絡規模的確切含義。為此，我們首先指出，該定理的結論適用於**前饋神經網絡**（Feed-forward network）。前饋網絡可以看作是一類帶有跳躍連接的深度網絡。圖 1.3 展示了一個前饋網絡的例子。

在圖 1.3 中，網絡有 3 個輸入神經元 x_1、x_2 和 x_3。其餘 8 個神經元稱為**計算單元**，因為它們的取值需要根據其父神經元的值計算得到。例如，神經元 h_1 處的取值為

$$h_1(x) = \sigma(w^{(x_1,h_1)}x_1 + w^{(x_3,h_1)}x_3 + b^{h_1}),$$

其中 $w^{(i,j)} \in \mathbb{R}$ 是網絡中有向邊 (i,j) 的權重，$b^j \in \mathbb{R}$ 是神經元 j 處的偏置。在本節和下一節中，為了方便起見，我們也將偏置稱為權重。因此，為了計算 $h_1(x)$，我們需要 3 個權重 $w^{(x_1,h_1)}$、$w^{(x_3,h_1)}$ 和 b^{h_1}。容易看出，總權重數等於邊的數量與計算單元數量之和。對於圖 1.3 中的這個示例網絡，其總權重數為 24。

Lebesgue 空間、Sobolev 空間及其範數 對於可測集合 $\Omega \subset \mathbb{R}^d$ 和可測函數 $f : \Omega \to \mathbb{R}$，$f$ 在 Ω 上的**本性上確界**（essential supremum），記為 $\|f\|_{L^\infty(\Omega)}$，其定義為

$$\|f\|_{L^\infty(\Omega)} = \inf\left\{M \in \mathbb{R} : \begin{array}{l} \exists\, \Omega_0 \subset \Omega \quad \text{with} \quad \mu(\Omega_0) = 0, \\ \text{s.t. } |f(x)| \le M,\ \forall\, x \in \Omega \setminus \Omega_0 \end{array}\right\}. \tag{1.12}$$

也就是說，Ω 上存在一個測度為零的子集 Ω_0 ，在這個子集之外，即 $\Omega \setminus \Omega_0$ 上，$\|f\|_{L^\infty(\Omega)}$ 是 $|f|$ 的最小的上界。具有有限 L^∞ 範數的函數集合稱為 L^∞ **Lebesgue 空間**。如果 Ω 是緊的且 f 在 Ω 上連續，則 $\|f\|_{L^\infty(\Omega)} = \max_{x\in\Omega} |f(x)|$。在第 1.4 節中我們會頻繁使用這一點；為簡化記號，在第 1.4 節中所涉及的任何緊定義域上，我們都用 $\|\cdot\|_\infty$ 代表 $\|\cdot\|_{L^\infty}$。

我們進一步假設 $f : \Omega \to \mathbb{R}$ 存在所有 k 階（弱）偏導數，即對每個滿足 $|\mathbf{k}|_1 := \sum_{i=1}^d k_i \le k$ 的多重指標 $\mathbf{k} = (k_1, \ldots, k_d) \in \{0, 1, \ldots, k\}^d$，$D^{\mathbf{k}} f$ 都存在，並定義範數

$$\|f\|_{W^{k,\infty}(\Omega)} = \max_{\mathbf{k}:|\mathbf{k}|_1 \le k} \|D^{\mathbf{k}} f\|_{L^\infty(\Omega)}. \tag{1.13}$$

（在本節和下一節中，我們將用黑體字母表示向量，以便更清晰地呈現數學公式。對於本書其餘部分，我們仍使用普通字母表示向量，因為那時不會產生混淆。）具有有限 $W^{k,\infty}$ 範數的函數集合稱為 $W^{k,\infty}$ **Sobolev 空間**。我們還知道，$f \in W^{k,\infty}(\Omega)$ 當且僅當對所有滿足 $|\mathbf{k}|_1 \le k-1$ 的 $\mathbf{k}$，$D^{\mathbf{k}} f$ 在 Ω 中存在，並且對所有滿足 $|\mathbf{k}|_1 = k-1$ 的 $\mathbf{k}$，$D^{\mathbf{k}} f$ 在 Ω 中是 Lipschitz 連續的。

由於 $\|\cdot\|_{L^\infty(\Omega)}$ 和 $\|\cdot\|_{W^{k,\infty}(\Omega)}$ 都是絕對齊次的，即對任意 $\alpha \in \mathbb{R}$ 和 $f \in L^\infty(\Omega)$，有 $\|\alpha f\|_{L^\infty(\Omega)} = |\alpha|\, \|\alpha f\|_{L^\infty(\Omega)}$（$\|\cdot\|_{W^{k,\infty}(\Omega)}$ 也同樣如此），因此只需證明本節開頭關於 (1.11) 中 F 的定義在 $M = 1$ 時的結論即可。我們將在下一節中，把剛才這一結論表述為普適逼近定理的一個版本，並給出其證明。

1.4 普適逼近定理的證明

在本節中，我們證明深度 ReLU 前饋網絡的普適逼近定理（在本節中我們簡稱其為 ReLU 網絡）。該定理的正式表述將在本節後面給出。由於該定理還給出了網絡深度、權重數和計算單元數的上界，而這些上界依賴於逼近誤差容許量 $\epsilon > 0$，因此我們將在證明中跟蹤這些數量的（階的）變化。由於權重數總是大於計算單元數，我們通常只跟蹤前者。

現在我們開始證明。我們先從下面這個關於用 ReLU 網絡在區間 $[0, 1]$ 上

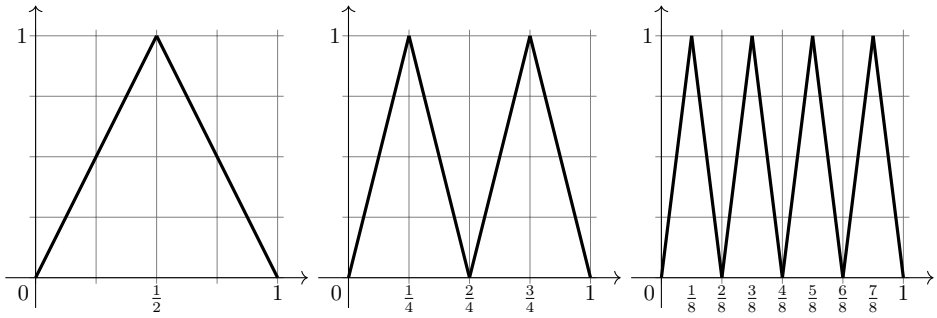


图 1.4: 從左到右：在 (1.14) 中定義的 g_1、g_2 和 g_3。

逼近 x^2 的簡單命題開始。

命題 1.4.1. *對於任意 $\epsilon > 0$，定義在 $[0,1]$ 上的函數 $f(x) = x^2$ 都可以被一個具有 $O(\log(1/\epsilon))$ 個權重的 ReLU 網絡以誤差 $\epsilon > 0$ 逼近。*

证明. 我們考慮一個基本函數 $g : [0,1] \to [0,1]$，定義為

$$g(x) = \begin{cases} 2x, & \text{if } 0 \le x \le \frac{1}{2}, \\ 2(1-x), & \text{if } \frac{1}{2} \le x \le 1, \end{cases}$$

以及 g 的 s 次複合：

$$g_s(x) := \underbrace{g \circ g \circ \cdots \circ g}_{s \text{ in total}}(x)$$

其中 $s \in \mathbb{N}$。容易驗證

$$g_s(x) = \begin{cases} 2^s\left(\frac{2k}{2^s} - x\right), & \text{if } x \in \left[\frac{2k-1}{2^s}, \frac{2k}{2^s}\right],\ k = 1, \ldots, 2^{s-1}, \\ 2^s\left(x - \frac{2k}{2^s}\right), & \text{if } x \in \left[\frac{2k}{2^s}, \frac{2k+1}{2^s}\right],\ k = 0, 1, \ldots, 2^{s-1} - 1. \end{cases} \tag{1.14}$$

$g_1 (= g)$、g_2 和 g_3 的圖像如圖 1.4 所示。

現在，對任意 $m = 0, 1, \cdots$，我們將 $[0,1]$ 劃分為 2^m 個等長區間，並定義一個分段線性函數 f_m 來逼近 $f(x) = x^2$。具體地，我們要求 f_m 在所有網格點 $\frac{k}{2^m}$（其中 $k = 0, \ldots, 2^m$）處與 f 一致：

$$f_m\left(\frac{k}{2^m}\right) = f\left(\frac{k}{2^m}\right) = \left(\frac{k}{2^m}\right)^2.$$

容易驗證 $f_0(x) = x$，並且對所有 $x \in [0,1]$ 和 $m \in \mathbb{N}$，下式都成立：

$$f_{m-1}(x) - f_m(x) = \frac{g_m(x)}{2^{2m}}.$$

從 1 到 m 求和，我們可得

$$f_m(x) = f_0(x) - \sum_{s=1}^{m} \frac{g_s(x)}{2^{2s}} = x - \sum_{s=1}^{m} \frac{g_s(x)}{2^{2s}}. \tag{1.15}$$

這對所有 $m = 1, 2, \ldots$ 都成立。

此外，我們注意到 g 可以寫成一個小型 ReLU 網絡：

$$g(x) = 2\sigma(x) - 4\sigma\Big(x - \frac{1}{2}\Big) + 2\sigma(x-1).$$

也就是說，如果我們定義

$$h_1^{(1)}(x) = \sigma(x), \quad h_2^{(1)}(x) = \sigma\Big(x - \frac{1}{2}\Big), \quad h_3^{(1)}(x) = \sigma(x-1),$$

其中 σ 是 ReLU 函數，那麼有

$$g(x) = 2h_1^{(1)}(x) - 4h_2^{(1)}(x) + 2h_3^{(1)}(x).$$

進一步地，我們定義

$$\begin{aligned}
h_1^{(2)} &= \sigma(g(x)) = \sigma\big(2h_1^{(1)}(x) - 4h_2^{(1)}(x) + 2h_3^{(1)}(x)\big),\\
h_2^{(2)} &= \sigma\Big(g(x) - \frac{1}{2}\Big) = \sigma\Big(2h_1^{(1)}(x) - 4h_2^{(1)}(x) + 2h_3^{(1)}(x) - \frac{1}{2}\Big),\\
h_3^{(2)} &= \sigma(g(x) - 1) = \sigma\big(2h_1^{(1)}(x) - 4h_2^{(1)}(x) + 2h_3^{(1)}(x) - 1\big).
\end{aligned}$$

於是有

$$g_2(x) = 2h_1^{(1)}(x) - 4h_2^{(2)}(x) + 2h_3^{(2)}(x).$$

按照這一模式，我們可以構造一個 ReLU 網絡來表示任意的 g_m 和 f_m（$m \geq 1$）。ReLU 網絡 f_3 的圖像如圖 1.5 所示。

沿著圖 1.5 所示的 ReLU 網絡並增大 m，我們可以看到 ReLU 網絡 f_m 有 $3m+1$ 個計算單元 $\{h_1^{(s)}, h_2^{(s)}, h_3^{(s)}, f_m : \forall\, s \in [m]\}$，以及 $12m-5$ 條邊（對每個 $s \in [m-1]$，x、$h_1^{(s)}$、$h_2^{(s)}$ 和 $h_3^{(s)}$ 各有 4 條出邊，而 $h_1^{(m)}$、$h_2^{(m)}$ 和 $h_3^{(m)}$ 各有 1 條出邊）。因此，f_m 中權重的總數為 $15m - 4 = O(m)$。

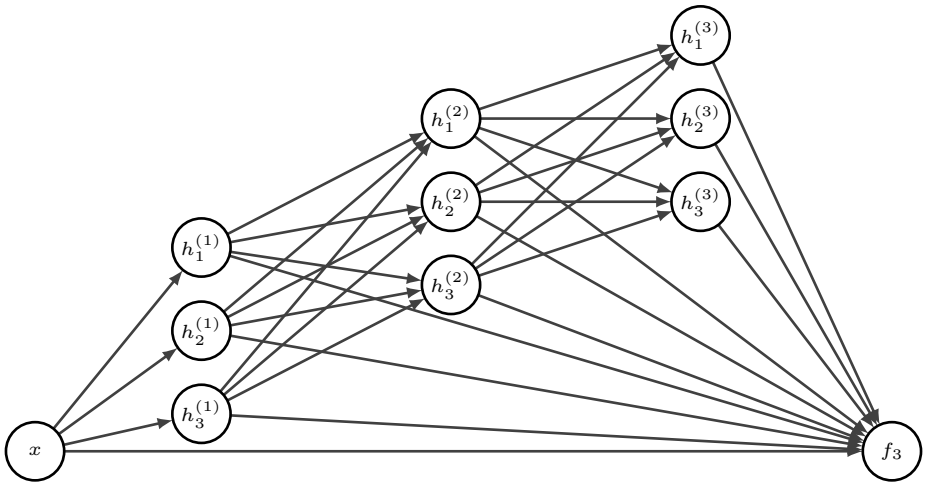


图 1.5: 在 (1.15) 中定義的 ReLU 網絡 f_m 在 $m = 3$ 時的情形。

同樣也很容易求出 f_m 的逼近誤差:

$$\begin{aligned}\|f - f_m\|_\infty &= \max_{x\in[0,1]} |f(x) - f_m(x)| \\ &= f_m(x_m^*) - f(x_m^*) \\ &= \frac{1}{2^{2m+2}},\end{aligned}$$

其中 $x_m^* = 1 - \frac{1}{2^{m+1}}$。因此，為了得到 $\|f - f_m\|_\infty \le \epsilon$，只需令 $\frac{1}{2^{2m+2}} \le \epsilon$。換言之，$f_m$ 的深度和權重數都是 $O(\log(1/\epsilon))$。 □

注意，對任意 $a, b \in \mathbb{R}$，有

$$ab = \frac{1}{2}\Big((a+b)^2 - a^2 - b^2\Big). \tag{1.16}$$

由於我們可以像命題 1.4.1 那樣用 ReLU 網絡逼近平方函數 x^2，因此可以基於恆等式 (1.16) 用 ReLU 網絡逼近兩個實數的乘積。在下一個命題中，我們將說明如何實現這種逼近。

命題 1.4.2. *對於任意 $M \ge 1$ 和 $\epsilon \in (0,1)$，存在一個 ReLU 網絡 $p_{\epsilon,M} : [-M, M] \times [-M, M] \to \mathbb{R}$，使得*

(i) 對任意 $a, b \in [-M, M]$，有 $|p_{\epsilon,M}(a, b) - ab| \le \epsilon$;

(ii) 若 $a = 0$ 或 $b = 0$，則 $p_{\epsilon,M}(a, b) = 0$；並且

(iii) $p_{\epsilon,M}$ 的深度和權重數都是 $O(\log(1/\epsilon)+\log M)$。

证明. 設 $\delta>0$（我們稍後確定其值），設 $q_\delta:[0,1]\to[0,1]$ 是命題 1.4.1 中以誤差 δ 逼近平方函數 $f(x)=x^2$ 的 ReLU 網絡。因此 $q_\delta(0)=0$，並且下式

$$|q_\delta(x)-x^2|\le\delta$$

對所有 $x\in[0,1]$ 成立。定義

$$p_{\epsilon,M}(a,b)=\frac{M^2}{2}\Big[q_\delta\Big(\frac{|a+b|}{2M}\Big)-q_\delta\Big(\frac{|a|}{2M}\Big)-q_\delta\Big(\frac{|b|}{2M}\Big)\Big].$$

顯然，當 $a=0$ 或 $b=0$ 時，$p_{\epsilon,M}(a,b)=0$。

令 $\delta=\frac{8\epsilon}{3M^2}$，則有

$$\begin{aligned}|p_{\epsilon,M}(a,b)-ab|&\le\frac{M^2}{8}\Big[\Big|q_\delta\Big(\frac{|a+b|}{2M}\Big)-\Big(\frac{|a+b|}{2M}\Big)^2\Big|+\Big|q_\delta\Big(\frac{|a|}{2M}\Big)-\Big(\frac{|a|}{2M}\Big)^2\Big|\\&\qquad+\Big|q_\delta\Big(\frac{|b|}{2M}\Big)-\Big(\frac{|b|}{2M}\Big)^2\Big|\Big]\\&\le\frac{M^2}{8}\cdot(3\delta)\\&=\epsilon.\end{aligned}$$

由於 q_δ 的深度和權重數為 $O(\log(1/\delta))=O(\log(1/\epsilon)+\log M)$，我們知道 $p_{\epsilon,M}$ 也滿足同樣的估計。 □

我們將 $W^{k,\infty}([0,1]^d)$ 中的單位球記為

$$B_{k,d}=\{f\in W^{k,\infty}([0,1]^d):\|f\|_{W^{k,\infty}([0,1]^d)}\le1\}. \tag{1.17}$$

考慮 $[0,1]^d$ 上如下均勻網格：我們將每個座標軸上的區間 $[0,1]$ 劃分為 N 個等長區間，則網格點集合為

$$\Big\{\frac{\mathbf{m}}{N}\in[0,1]^d\ :\ \mathbf{m}=(m_1,\dots,m_d)\in N_d\Big\},$$

其中

$$N_d:=\{0,1,\dots,N\}^d.$$

這樣，我們在 $[0,1]^d$ 中總共有 $(N+1)^d$ 個網格點。

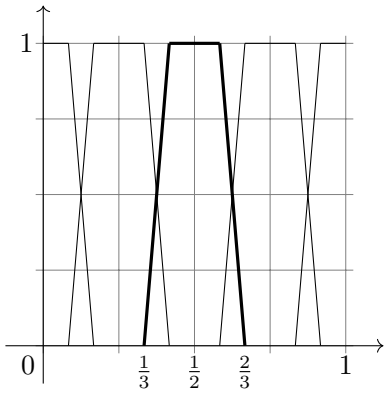


图 1.6: 從左到右：在 $d=1$ 且 $N=4$ 的情形下的 ϕ_0、ϕ_1、ϕ_2（加粗）、ϕ_3 和 ϕ_4。注意對所有 $x\in[0,1]$，都有 $\sum_m \phi_m(x)=1$。

現在定義一個函數 $\psi:\mathbb{R}\to[0,1]$ 為

$$\psi(x)=\begin{cases}1, & \text{if} \quad |x|\le 1,\\ 2-|x|, & \text{if} \quad 1\le|x|\le 2,\\ 0, & \text{if} \quad |x|\ge 2.\end{cases}$$

然後，對任意 $\mathbf{m}=(m_1,\dots,m_d)\in N_d$，定義

$$\phi_{\mathbf{m}}(\mathbf{x})=\prod_{i=1}^{d}\psi\Big(3N\big(x_i-\frac{m_i}{N}\big)\Big) \tag{1.18}$$

其中 $\mathbf{x}=(x_1,\dots,x_d)\in[0,1]^d$。由於 $\|\psi\|_\infty=1$，我們也知道 $\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)}\le 1$。在 $d=1$ 且 $N=4$ 的情形下，$\phi_{\mathbf{m}}$ 的一個例子如圖 1.6 所示。

在下面的命題中，我們將說明 $\{\phi_{\mathbf{m}}:\mathbf{m}\in N_d\}$ 在 $[0,1]^d$ 上構成一個單位分解（partition of unity）。

命題 1.4.3. *對於任意 $N\in\mathbb{N}$，按 (1.18) 定義 $\phi_{\mathbf{m}}$。則*

1. *對任意 $\mathbf{m}\in N_d$ 和 $\mathbf{x}\in[0,1]^d$，有 $0\le\phi_{\mathbf{m}}(\mathbf{x})\le 1$。此外，$\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)}=1$。*

2. *$\phi_{\mathbf{m}}$ 的支撐集（support）滿足*

$$\begin{aligned}\operatorname{supp}(\phi_{\mathbf{m}})&=\Big\{\mathbf{x}\in[0,1]^d:\Big|x_i-\frac{m_i}{N}\Big|\le\frac{2}{3N},\ \forall\, i\in[d]\Big\}\\ &\subset\Big\{\mathbf{x}\in[0,1]^d:\Big|x_i-\frac{m_i}{N}\Big|\le\frac{1}{N},\ \forall\, i\in[d]\Big\}\end{aligned}$$

並且任意 $\mathbf{x} \in [0,1]^d$ 至多屬於 2^d 個 $\phi_{\mathbf{m}}$ 的支撐。

3. 對任意 $\mathbf{x} \in [0,1]^d$，有

$$\sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x}) = 1.$$

证明. 因為 $\|\psi\|_\infty = 1$ 且 $\phi_{\mathbf{m}}(\frac{\mathbf{m}}{N}) = 1$，所以第一個結論成立。

對任意 $\mathbf{x} \in [0,1]^d$，存在 $\mathbf{m} \in N_d$，使得對任意 $i \in [d]$，以下兩種情況中有且只有一種發生:

$$\frac{3m_i - 2}{3N} < x_i < \frac{3m_i - 1}{3N}, \tag{1.19a}$$

$$\frac{3m_i - 1}{3N} \le x_i \le \frac{3m_i}{3N}. \tag{1.19b}$$

這些蘊含了第二個結論成立。

當 (1.19a) 發生時，有

$$\psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 2 - \Big|3N\big(x_i - \frac{m_i}{N}\big)\Big| = 2 + 3N\big(x_i - \frac{m_i}{N}\big).$$

當 (1.19b) 發生時，有

$$\psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 1.$$

不失一般性，設 $d' \in \{0, 1, \ldots, d\}$ ，使得當 $i \le d'$ 時 (1.19a) 發生，而當 $d' + 1 \le i \le d$ 時 (1.19b) 發生。

如果 $d' = 0$，則對所有 $i \in [d]$，都有 $\psi(3N(x_i - \frac{m_i}{N})) = 1$。因此

$$\phi_{\mathbf{m}}(\mathbf{x}) = \prod_{i=1}^{d} \psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 1.$$

如果 $d' = 1$，則

$$\begin{aligned}
\sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}} &= \phi_{\mathbf{m}}(\mathbf{x}) + \phi_{\mathbf{m}-\mathbf{e}_1}(\mathbf{x}) \\
&= \psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big) + \psi\Big(3N\big(x_1 - \frac{m_1 - 1}{N}\big)\Big) \\
&= (2 + 3Nx_1 - 3m_1) + (2 - 3Nx_1 + 3(m_1 - 1)) \\
&= 1,
\end{aligned}$$

其中

$$\mathbf{e}_i := (0,\dots,0,\underbrace{1}_{i\text{th}},0,\dots,0) \in \{0,1\}^d.$$

事實上，我們可以驗證，對任意 $\mathbf{m} \in \{1,\dots,N\}^d$ 和 $i \in [d]$，下式

$$\psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) + \psi\Big(3N\big(x_i - \frac{m_i - 1}{N}\big)\Big) = 1 \tag{1.20}$$

對任意 $\mathbf{x} \in [0,1]^d$ 成立。

如果 $d' = 2$，則

$$\begin{aligned}
\sum_{\mathbf{m}\in N_d} \phi_{\mathbf{m}} &= \phi_{\mathbf{m}}(\mathbf{x}) + \phi_{\mathbf{m}-\mathbf{e}_1}(\mathbf{x}) + \phi_{\mathbf{m}-\mathbf{e}_1-\mathbf{e}_2}(\mathbf{x}) + \phi_{\mathbf{m}-\mathbf{e}_2}(\mathbf{x}) \\
&= \psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big)\psi\Big(3N\big(x_2 - \frac{m_2}{N}\big)\Big) \\
&\quad + \psi\Big(3N\big(x_1 - \frac{m_1 - 1}{N}\big)\Big)\psi\Big(3N\big(x_2 - \frac{m_2}{N}\big)\Big) \\
&\quad + \psi\Big(3N\big(x_1 - \frac{m_1 - 1}{N}\big)\Big)\psi\Big(3N\big(x_2 - \frac{m_2 - 1}{N}\big)\Big) \\
&\quad + \psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big)\psi\Big(3N\big(x_2 - \frac{m_2 - 1}{N}\big)\Big) \\
&= 1
\end{aligned}$$

這是利用 (1.20) 得到的。

按照這一思路並利用 (1.20)，可知對任意 $d' \in \{0,1,\dots,d\}$，都有

$$\sum_{\mathbf{m}\in N_d} \phi_{\mathbf{m}}(\mathbf{x}) = 1.$$

證明完成。 □

如果 $f \in W^{k,\infty}([0,1]^d)$，則我們知道對任意滿足 $|\mathbf{k}|_1 = \sum_{i=1}^d k_i < k-1$ 的 $\mathbf{k} = (k_1,\dots,k_d) \in \{0,\dots,k\}^d$，$D^{\mathbf{k}} f$ 在 Ω 中存在，並且 $\|D^{\mathbf{k}} f\|_{L^\infty([0,1]^d)} \le \|f\|_{W^{k,\infty}([0,1]^d)}$。對任意 $\mathbf{m} \in N_d$，由於 $\frac{\mathbf{m}}{N} \in [0,1]^d$，我們記 f 在 $\frac{\mathbf{m}}{N}$ 處的 $k-1$ 次 Taylor 多項式為

$$P_{\mathbf{m}}(\mathbf{x}) = \sum_{\mathbf{k}:|\mathbf{k}|_1<k} \frac{1}{\mathbf{k}!} D^{\mathbf{k}} f\Big(\frac{\mathbf{m}}{N}\Big)\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}, \tag{1.21}$$

其中 $\mathbf{k}! = k_1! \cdots k_d!$，$(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}} = \prod_{i=1}^{d}(x_i - \frac{m_i}{N})^{k_i}$，並且

$$D^{\mathbf{k}} f = \frac{\partial^k f}{\partial x_1^{k_1} \dots \partial x_d^{k_d}}.$$

接下來，我們說明 $f \in W^{k,\infty}([0,1]^d)$ 可以由對所有 $\mathbf{m} \in N_d$ 的 $\phi_{\mathbf{m}}(\mathbf{x})P_{\mathbf{m}}(\mathbf{x})$ 之和來逼近。

引理 1.4.4. *對於任意 $f \in B_{k,d}$ 和 $\epsilon > 0$，存在 $N = N(\epsilon, k, d) \in \mathbb{N}$ 以及形如*

$$f_N(\mathbf{x}) = \sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x}) P_{\mathbf{m}}(\mathbf{x}), \tag{1.22}$$

的函數 f_N，其中 $P_{\mathbf{m}}(\mathbf{x})$ 按 (1.21) 定義，使得

$$\|f_N - f\|_{L^\infty([0,1]^d)} \le \frac{\epsilon}{2}. \tag{1.23}$$

证明. 注意到，對任意 $\mathbf{x} \in [0,1]^d$，有

$$\begin{aligned}
|f(\mathbf{x}) - f_N(\mathbf{x})| &= \Big| \sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x})(f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})) \Big| && \text{(1.24a)}\\
&\le \sum_{\mathbf{m}: |x_i - \frac{m_i}{N}| < \frac{1}{N}, \forall i \in [d]} |f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})| && \text{(1.24b)}\\
&\le 2^d \max_{\mathbf{m}: |x_i - \frac{m_i}{N}| < \frac{1}{N}, \forall i \in [d]} |f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})| && \text{(1.24c)}\\
&\le 2^d \frac{1}{\mathbf{k}!} \sum_{\mathbf{k}: |\mathbf{k}|_1 = k} \|D^{\mathbf{k}} f\|_{L^\infty([0,1]^d)} \Big| \Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}} \Big| && \text{(1.24d)}\\
&\le 2^d \cdot 1 \cdot d^k \cdot 1 \cdot \Big(\frac{1}{N}\Big)^k, && \text{(1.24e)}
\end{aligned}$$

其中 (1.24a) 利用了 $\{\phi_{\mathbf{m}} : \mathbf{m} \in N_d\}$ 是單位分解;(1.24b) 利用了 $\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)} \le 1$；(1.24c) 利用了 (1.24b) 中求和項至多有 2^d 項；(1.24d) 利用了 (1.21) 中 $P_{\mathbf{m}}(\mathbf{x})$ 的 Taylor 多項式逼近性質；而 (1.24e) 則是因為 $\mathbf{k}! \ge 1$，(1.24d) 中求和項不超過 d^k 項，$\|D^{\mathbf{k}} f\|_{L^\infty([0,1]^d)} \le \|f\|_{W^{k,\infty}([0,1]^d)} \le 1$，並且對所有 $i \in [d]$，都有 $|x_i - \frac{m_i}{N}| \le 1$。

取

$$N = \Big\lceil \Big(\frac{1}{2^d d^k} \cdot \frac{\epsilon}{2}\Big)^{-\frac{1}{k}} \Big\rceil = \lceil 2^{-(d+1)/k} d^{-1} \epsilon^{-1/k} \rceil, \tag{1.25}$$

它依賴於 ϵ、k 和 d, 但不依賴於具體的 f, 於是我們得到所需的估計 (1.23)。 □

現在我們已經準備好證明深度 ReLU 神經網絡的普適逼近定理。

定理 1.4.5 (普適逼近定理)**.** *對於任意 $d, k \in \mathbb{N}$ 和 $\epsilon \in (0,1)$，存在一個 ReLU 網絡架構 h，它只依賴於 d、k 和 ϵ，使得*

1. *架構 h 能夠通過適當選擇參數來表示 $B_{k,d}$ 中的任意函數。*
2. *架構 h 的深度為 $O(\log(1/\epsilon))$，並且權重數至多為 $c\epsilon^{-d/k}\log(1/\epsilon)$，其中 c 依賴於 d 和 k，但不依賴於 ϵ。*

证明. 對任意 $\epsilon \in (0,1)$ 和 $f \in B_{k,d}$，令 N 和 f_N 如引理 1.4.4 中所給。我們將 Taylor 多項式 $P_{\mathbf{m}}(\mathbf{x})$ 改寫為

$$P_{\mathbf{m}}(\mathbf{x}) = \sum_{\mathbf{k}:|\mathbf{k}|_1<k} a^f_{\mathbf{m},\mathbf{k}}\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}$$

其中係數 $a^f_{\mathbf{m},\mathbf{k}} \in \mathbb{R}$ 定義為

$$a^f_{\mathbf{m},\mathbf{k}} = \frac{1}{\mathbf{k}!} D^{\mathbf{k}} f\Big(\frac{\mathbf{m}}{N}\Big).$$

由於 $f \in B_{k,d}$，我們知道對所有 $\mathbf{m}$ 和 $\mathbf{k}$，都有 $|a^f_{\mathbf{m},\mathbf{k}}| \le 1$。

基於 (1.22)，我們可以將 f_N 寫為

$$f_N(\mathbf{x}) = \sum_{\mathbf{m}\in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1<k} a^f_{\mathbf{m},\mathbf{k}} \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}$$

並注意到，只有 $a^f_{\mathbf{m},\mathbf{k}}$ 依賴於給定的 f，而 $\phi_{\mathbf{m}}$ 和 $(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}}$ 都不依賴於 f。

由 (1.4) 可見，f_N 的求和中至多有 $(N+1)^d d^k$ 項：N_d 有 $(N+1)^d$ 個元素，而 $\{\mathbf{k} : |\mathbf{k}|_1 < k\}$ 的大小上界為 d^k。我們還注意到，(1.4) 中每一項的函數部分都具有如下形式：

$$\phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}} \tag{1.26}$$

它是至多 $d+k-1$ 個分段線性函數的乘積：

$$\underbrace{\psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big),\ \ldots,\ \psi\Big(3N\big(x_d - \frac{m_d}{N}\big)\Big)}_{d\ \text{分段线性函数}}, \underbrace{\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}}_{<k\ \text{线性函数}}. \tag{1.27}$$

為簡化記號，我們將 (1.27) 中前 j 項的乘積記為 $Q^j_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，將後 j 項的乘積記為 $Q^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，其中 $j = 1, \ldots, d+k-1$。注意，對所有 j，都有 $|Q^j_{\mathbf{m},\mathbf{k}}(\mathbf{x})| \le 1$ 和 $|Q^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})| \le 1$。

接下來，我們用 ReLU 網絡逼近 (1.26)。令 $M = d + k$，並令 $p_{\delta,M} : [-M, M] \times [-M, M] \to \mathbb{R}$ 為命題 1.4.2 中的 ReLU 網絡，其中 $\delta > 0$ 稍後確定。定義

$$h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = p_{\delta,M}\Big\{\psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big), p_{\delta,M}\Big[\psi\Big(3N\big(x_2 - \frac{m_2}{N}\big)\Big), \dots\Big]\Big\}. \quad (1.28)$$

也就是說，為了計算 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，我們先將 $p_{\delta,M}$ 作用於 (1.27) 中最後兩個因子，然後再將 $p_{\delta,M}$ 作用於所得結果和倒數第三個因子，如此繼續，直到作用到 (1.27) 中的第一個因子結束。因此，為了得到 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，$p_{\delta,M}$ 至多重複應用 $d + k - 2$ 次。為簡化記號，我們將對 (1.27) 中最後 j 個因子應用 $p_{\delta,M}$ 所得的結果記為 $P^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，其中 $j = 2, 3, \dots, d+k-1$。例如，可以看到 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$。

注意，對任意 $i \in [d]$，$\psi(3N(x_i - \frac{m_i}{N}))$ 都可以由一個僅含一層隱藏層且具有 $O(1)$ 個權重的 ReLU 網絡實現。根據命題 1.4.2，ReLU 網絡 $p_{\delta,M}$ 的深度為 $O(\log(1/\delta))$，權重數為 $O(\log(1/\delta) + \log M)$。因此，$h_{\mathbf{m},\mathbf{k}}$ 可以由一個深度為 $O(\log(1/\delta))$、權重數為 $c(d,k)(\log(1/\delta) + \log M)$ 的 ReLU 網絡實現，其中 $c(d,k)$ 是依賴於 d 和 k、但不依賴於 ϵ 和 M 的常數。

現在我們估計 $h_{\mathbf{m},\mathbf{k}}$ 的逼近誤差。注意，對所有 $i \in [d]$ 和 $\mathbf{x} \in [0,1]^d$，都有 $|\psi(3N(x_i - \frac{m_i}{N}))| \le 1$ 和 $|x_i - \frac{m_i}{N}| \le 1$。由命題 1.4.2，對任意滿足 $|a| \le 1$ 和 $|b| \le M$ 的 $a, b \in \mathbb{R}$，有

$$|p_{\delta,M}(a,b)| - |b| \le |p_{\delta,M}(a,b) - ab| \le \delta.$$

將其應用於 (1.28)，我們知道所有 $p_{\delta,M}$ 的輸入幅值都被 M 所控制，因為 $p_{\delta,M}$ 至多隻應用了 $d + k - 2\,(< M)$ 次。

因此，我們有

$$\begin{aligned}
&\Big|h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x})\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}\Big| \\
&\qquad = \Big|P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{d+|\mathbf{k}|_1-1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad \le \Big|P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-2)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad\quad + \Big|Q^{1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-2)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{2}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-3)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad\quad + \dots \\
&\qquad\quad + \Big|Q^{d+|\mathbf{k}|_1-3}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-2}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{d+|\mathbf{k}|_1-3}_{\mathbf{m},\mathbf{k}}(\mathbf{x})Q^{-2}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad \le (d+k)\delta,
\end{aligned}$$

其中最後一個不等式是因為其上方求和中的項數不超過 $d+k-2$，且每一項都被 δ 所控制。此外，若 $\mathbf{x} \notin \operatorname{supp}(\phi_{\mathbf{m}}) := \operatorname{Closure}(\{\mathbf{x} : \phi_{\mathbf{m}}(\mathbf{x}) \geq 0\})$，即 $\mathbf{x}$ 不屬於 $\phi_{\mathbf{m}}$ 的支撐集，則有 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = \phi_{\mathbf{m}}(\mathbf{x})(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}} = 0$。

現在定義

$$h(\mathbf{x}) = \sum_{\mathbf{m} \in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} h_{\mathbf{m},\mathbf{k}}(\mathbf{x}). \tag{1.29}$$

注意，係數 $a^f_{\mathbf{m},\mathbf{k}}$ 依賴於 f，而函數 $h_{\mathbf{m},\mathbf{k}}$ 不依賴於 f。於是對任意 $\mathbf{x} \in [0,1]^d$，有

$$\begin{aligned}
|h(\mathbf{x}) - f_N(\mathbf{x})| &= \Big| \sum_{\mathbf{m} \in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} \Big(h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big) \Big| \\
&= \Big| \sum_{\mathbf{x} \in \operatorname{supp}(\phi_{\mathbf{m}})} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} \Big(h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big) \Big| \\
&\leq 2^d \max_{\mathbf{x} \in \operatorname{supp}(\phi_{\mathbf{m}})} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} \Big| h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big| \\
&\leq 2^d d^k (d+k) \delta,
\end{aligned}$$

其中第一個不等式是因為 $\mathbf{x}$ 至多屬於 2^d 個 $\phi_{\mathbf{m}}$ 的支撐集合且 $|a^f_{\mathbf{m},\mathbf{k}}| \leq 1$，最後一個不等式是因為 $\{\mathbf{k} : |\mathbf{k}|_1 < k\}$ 的大小上界為 d^k。取

$$\delta = \frac{\epsilon}{2^{d+1} d^k (d+k)}, \tag{1.30}$$

則對所有 $\mathbf{x} \in [0,1]^d$，都有 $|h(\mathbf{x}) - f_N(\mathbf{x})| \leq \frac{\epsilon}{2}$。因此

$$\|h - f\|_{L^\infty([0,1]^d)} \leq \|h - f_N\|_{L^\infty([0,1]^d)} + \|f_N - f\|_{L^\infty([0,1]^d)} \leq \frac{\epsilon}{2} + \frac{\epsilon}{2} = \epsilon.$$

最後，注意到 (1.29) 中的 h 可以通過至多 $(N+1)^d d^k$ 個 ReLU 網絡的線性組合來實現，即

$$\big\{ h_{\mathbf{m},\mathbf{k}} : \mathbf{m} \in N_d,\ |\mathbf{k}|_1 \leq k \big\}.$$

這些 ReLU 網絡中的每一個都具有 $O(\log(1/\delta))$ 的深度和 $O(\log(1/\delta) + \log(d+k))$ 個權重。因此，h 的深度仍然是 $O(\log(1/\delta))$，而 h 的總權重數為

$$c(d,k) d^k (N+1)^d (\log(1/\delta) + \log(d+k)).$$

結合 (1.25) 中的 $N = \lceil 2^{-(d+1)/k} d^{-1} \epsilon^{-1/k} \rceil$ 以及 (1.30) 中的 δ，即可得到定理

中關於 h 的深度和權重數的上界。 □

定理 1.4.5 給出瞭如下網絡規模估計：為了在 L^∞ 範數意義下對任意 $f \in F$ 實現 ϵ-逼近，需要網絡規模為 $O(\epsilon^{-d/k}\log(1/\epsilon))$。該上界中關於 ϵ 的主導項是 $\epsilon^{-d/k}$，而 $\log(1/\epsilon)$ 是次要項。常數項還依賴於維數 d，但不依賴於 ϵ。這一估計在現有關於 Sobolev 空間上深度神經網絡逼近理論中看起來是典型的（某些方法可能會有小幅改進，但主導項 $\epsilon^{-d/k}$ 仍然保留在估計中）。在本章末尾，我們將給出關於這一主題的更多相關參考文獻。

1.5 網絡結構設計

在本節中，我們將瀏覽若干重要的網絡設計概念，以獲得對深度網絡在實際中如何使用的基本認識。我們首先介紹一些當前常用的激活函數示例。隨後，我們討論若干重要的網絡結構，這些結構常被用作構建大型網絡架構的基本模塊。

此外，我們還將介紹一些有用的思想與技巧，用於設計具有特定性質的神經網絡。這些方法能夠使訓練過程更加簡單且穩定，更重要的是，在實際應用中往往能夠顯著提升性能與效率。我們還將展示如何利用物理信息（而非外部給定數據）來設計目標函數的思路，這一方向已成為現代科學計算中的一個重要發展趨勢。

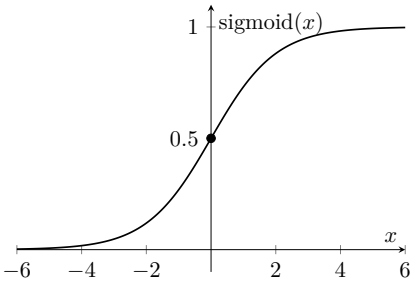
1.5.1 激活函數示例

下面，我們列出若干非線性激活函數示例，它們在現代深度網絡結構中很常見。需要注意的是，激活函數通常指非仿射（因此也是非線性）函數，否則神經網絡僅僅是仿射函數的複合，結果仍為仿射函數，無法逼近複雜函數。

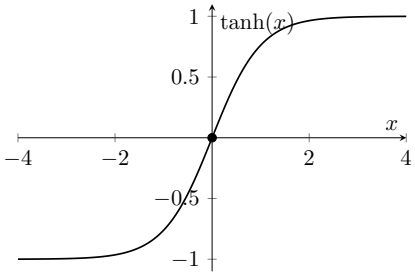
在本小節中，我們僅討論單變量激活函數，並將在本書後續部分介紹多變量激活函數。我們前面提過，當我們說將單變量函數應用於向量時，指的是將該激活函數分別作用於向量的每一個分量，其結果仍為同維度的向量。本小節中介紹的所有激活函數均顯示在圖 1.7 中。接下來我們將給出這些激活函數的數學表達形式，並簡要描述它們的性質。

Sigmoid 標準 sigmoid 函數定義為

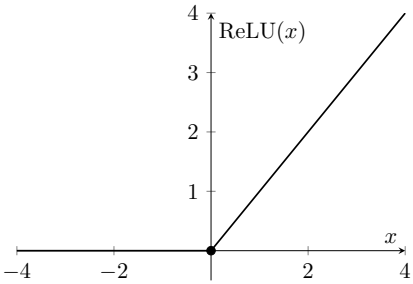
$$\mathrm{sigmoid}(x) = \frac{1}{1+e^{-x}}, \qquad \forall\, x \in \mathbb{R} \tag{1.31}$$



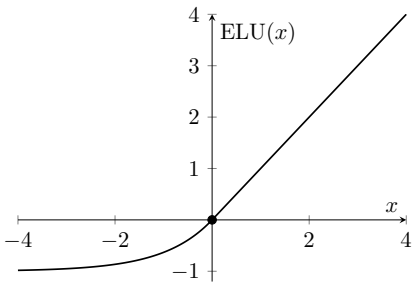
(a) Sigmoid



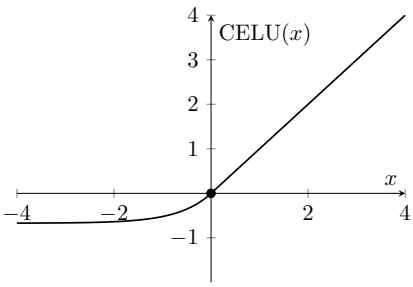
(b) tanh



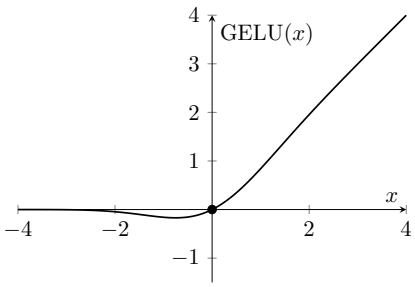
(c) ReLU



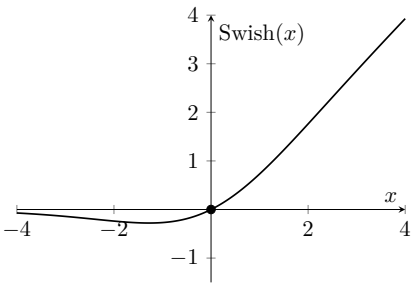
(d) ELU



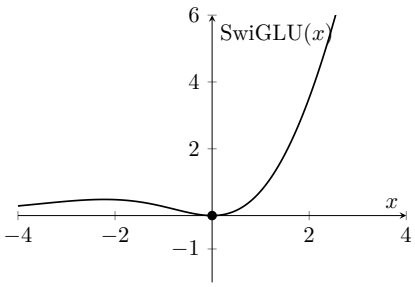
(e) CELU



(f) GELU



(g) Swish



(h) SwiGLU

图 1.7: 激活函數示例。

並如圖 1.7a 所示。在文獻中，sigmoid 函數通常指當 $x \to \infty$ 時趨於 1、當 $x \to -\infty$ 時趨於 0 的函數，這類函數也可以是非單調的。然而，在深度學習領域中，通常默認 (1.31) 為 sigmoid 函數。當輸出已知位於開區間 $(0,1)$ 時，sigmoid 函數可作為神經網絡的最後一層。然而，當 x 遠離 0 時，其導數會迅速減小，這種現象稱為“梯度消失”，往往會導致訓練效率降低。

雙曲正切函數（tanh） tanh 函數 定義為

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \qquad \forall\, x \in \mathbb{R} \tag{1.32}$$

並如圖 1.7b 所示。tanh 函數 (1.32) 可視為 sigmoid 函數的一種變體，其輸出範圍為 $(-1,1)$。tanh 同樣存在梯度消失問題，通常用於循環神經網絡中的門控機制（本節後續將看到相關示例）。

修正線性單元（ReLU） ReLU 函數 定義為

$$\mathrm{ReLU}(x) = \max(0, x), \qquad \forall\, x \in \mathbb{R} \tag{1.33}$$

並如圖 1.7c 所示。ReLU 函數 (1.33) 常用於分段迴歸中，用於描述在某一拐點前後具有不同斜率的關係。在統計建模中，當預測變量的影響發生突變時也會出現該函數。在深度學習中，ReLU 是一種非常常用的激活函數。當 $x \geq 0$ 時，它不會產生梯度消失問題。然而在 $x = 0$ 處不可導。深度 ReLU 神經網絡本質上是連續的分片的仿射函數，其 Lipschitz 常數可以根據網絡權重進行估計。

指數線性單元（ELU） ELU 激活函數 定義為

$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x \geq 0, \\ e^x - 1 & \text{otherwise}, \end{cases} \tag{1.34}$$

並如圖 1.7d 所示。ELU 函數 (1.34) 是 ReLU 函數 (1.33) 的平滑版本。在 $x < 0$ 時，它在一定程度上緩解了梯度消失問題，並有時可以加速訓練過程。該函數在深度網絡中較為常見，訓練穩定性通常優於 ReLU，但計算代價更高。

連續可導指數線性單元（CELU） CELU 激活函數 定義為

$$\mathrm{CELU}(x) = \begin{cases} x & \text{if } x \geq 0, \\ \alpha(e^{x/\alpha} - 1) & \text{otherwise}, \end{cases} \tag{1.35}$$

並在圖 1.7e 中給出（取 $\alpha = \frac{2}{3}$）。CELU 函數 (1.35) 是 ELU 的直接推廣，引入了可調超參數 $\alpha > 0$，可作為 ReLU 與 ELU 的替代方案。

高斯誤差線性單元（GELU） GELU 激活函數 定義為

$$\mathrm{GELU}(x) = \frac{x}{2}\Big(1 + \mathrm{erf}\Big(\frac{x}{2}\Big)\Big), \quad \forall\, x \in \mathbb{R}, \tag{1.36}$$

其中 erf 為高斯誤差函數，定義為

$$\mathrm{erf}\Big(\frac{x}{2}\Big) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

GELU 激活函數 (1.36) 如圖 1.7f 所示。它是光滑且單調的，在現代 transformer 架構中廣泛使用，尤其是在自然語言處理與大語言模型。相比 ReLU，它通常表現更優，在許多實際應用中逐漸流行起來。

Swish 帶有超參數 $\beta > 0$ 的 swish 函數 定義為

$$\mathrm{swish}_\beta(x) = x \cdot \mathrm{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}}, \quad \forall\, x \in \mathbb{R}, \tag{1.37}$$

當 $\beta = 1$ 時退化為 sigmoid linear unit（SiLU）。當 $\beta = 1$ 時的 SiLU 函數如圖 1.7g 所示。swish 是一種光滑的非單調激活函數，可以取負值，通常優於 ReLU，並有助於改善梯度傳播。

Swish 門控線性單元（SwiGLU） SwiGLU 函數 定義為

$$\mathrm{SwiGLU}(x) = (vx + c) \cdot \mathrm{swish}(wx + b), \quad \forall\, x \in \mathbb{R} \tag{1.38}$$

其中參數為 v, w, b, c，並如圖 1.7h 所示。該函數可推廣至向量輸入情形，此時 V, W 為矩陣，b, c 為向量，乘法變為分量乘積 $\odot$。即對於任意 d 維向量 x, y，$x \odot y := (x_1 y_1, \ldots, x_d y_d) \in \mathbb{R}^d$。SwiGLU 是現代大語言模型中最常用的門控激活函數之一。其門控機制可動態控制信息流，提高非線性表達能力。Swish 函

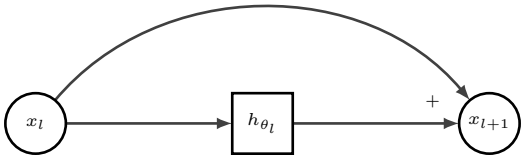


图 1.8: 殘差網絡（ResNet）第 l 層中的一個基本模塊：$x_{l+1} = x_l + h_{\theta_l}(x_l)$，其中 $h_{\theta_l} : \mathbb{R}^d \to \mathbb{R}^d$ 是一個具有參數 θ_l 的任意（淺層或小型多層）網絡，且 d 是 x_l 的維度。

數光滑，上無界而下有界。在相同參數規模下，SwiGLU 通常比 ReLU、GELU 等具有更強的建模能力。

1.5.2 網絡模塊示例

我們討論若干基礎神經網絡模塊的結構，這些模塊常用於構建大型深度網絡架構。

殘差網絡（ResNet）

儘管多層感知機（MLP）展示了通過堆疊淺層網絡構建深層網絡的思想，但其在實驗中的表現往往不穩定。特別是，當層數增加時，訓練誤差與測試誤差反而可能上升，這與“更深網絡具有更強表達能力”的直覺相矛盾。這一現象通常被認為是深層 MLP 難以訓練導致的。

殘差網絡（residual network，ResNet）在廣泛問題中能夠有效克服上述問題。在標準的 MLP 中，第 l 層實現為 $x_{l+1} = h_{\theta_l}(x_l)$。而在 ResNet 中，第 l 層定義為

$$x_{l+1} = x_l + h_{\theta_l}(x_l). \tag{1.39}$$

上式含有從 x_l 到 x_{l+1} 的跳躍連接（skip connection），所以我們僅須學習殘差項 h_{θ_l} 。圖 1.8 展示了這個 ResNet 模塊的結構。

注意，(1.39) 要求 h_{θ_l} 保持其輸入的維度，這可以通過合理設計這些隱藏層來實現。實驗中，如 (1.39) 所示的 ResNet 模塊通常可以堆疊到數千層，同時此類網絡中的梯度消失問題在很大程度上得到了緩解。

如我們所見，ResNet 模塊 (1.39) 學習的是對恆等映射的修正，而不是全新的變換。如果在某一深度處不需要額外層，則該殘差模塊 h_{θ_l} 可以簡單地去逼近 0 ，使 (1.39) 輸出近似等於輸入。這一性質使得 ResNet 對冗餘層更加魯棒（robust），並提供了一種隱式正則化形式。此外，只要殘差模塊 h_{θ_l} 的

Lipschitz 常數保持足夠小，使得整個 ResNet 構成一個可逆映射，恆等跳躍連接可以確保信息在前向與反向傳播過程中不發生失真。這樣，我們通常可以穩定地訓練非常深的 ResNet 。

值得注意的是，ResNet 模塊可以從多個不同的角度進行解釋。其中一個重要觀點將其與動力系統聯繫起來，因為 (1.39) 類似於求解常微分方程（ordinary differential equation，ODE）時前向 Euler 離散化的一步，時間步長為 1。這一視角啟發了更先進的網絡結構，例如具有連續時間的 Neural ODE 方法。我們將在第 3.3 節中詳細討論 Neural ODE 方法。另一個觀點（如前所述）強調，(1.39) 中的殘差路徑在右端梯度中引入了恆等矩陣，這在適當控制 h_{θ_l} 的 Lipschitz 性質時，有助於降低梯度消失問題的風險。

ResNet 模塊的簡單設計使得訓練非常深的神經網絡成為可能，並已成為現代深度學習架構中的基礎元素。這些模塊如今已廣泛應用於各類深度神經網絡的構建，覆蓋極其廣泛的應用領域。

卷積神經網絡（CNN）

標準的多層感知機（MLP）由於具有大量可訓練權重，還存在過參數化的問題。例如，如果 MLP 的第 l 層和第 $(l+1)$ 層分別有 d_l 和 d_{l+1} 個計算單元，則權重數量至少為 $d_l d_{l+1}$。這往往在訓練過程中導致許多不良後果，並且對數據量的需求過高。此外，MLP 的層結構忽略了輸入相鄰分量之間的相關性，從而降低了學習信息的效率。

卷積神經網絡（convolutional neural network，CNN）模塊是一類專門設計用於處理輸入分量之間具有空間關聯的數據（如文本、圖像和視頻）的有效網絡結構。尤其是在視覺識別任務中（包括圖像分類、目標檢測、語義分割和視頻分析），CNN 模塊佔據主導地位。其成功源於一種結構特性，能夠利用局部空間相關性、參數共享以及分層特徵學習。

在圖 1.9 中，我們展示了一個簡單 CNN 卷積核（中間圖）作用於圖像（左圖）後得到的輸出結果（右圖）。這一思想可以很容易推廣到構建更復雜的 CNN 模塊。例如，CNN 模塊的輸入可以是具有紅、綠、藍三個通道的彩色圖像，或是一個多通道特徵張量（多維矩陣）。CNN 模塊本身也可以具有 $C > 1$ 個通道，例如其卷積核可以表示為 $\{w_{ij}^c \in \mathbb{R} : i, j \in [3], c \in [C]\}$。卷積核的尺寸也可以更大，如 5、7 或更大。輸出特徵張量的尺寸由輸入、卷積核大小以及卷積的應用方式共同決定。

在使用 CNN 模塊時還有許多其他技巧。例如，如果希望將卷積核應用於輸入的邊界，可以在輸入外圍填充人工值（如 0），並從圖 1.9 中所示的以 x_{11}

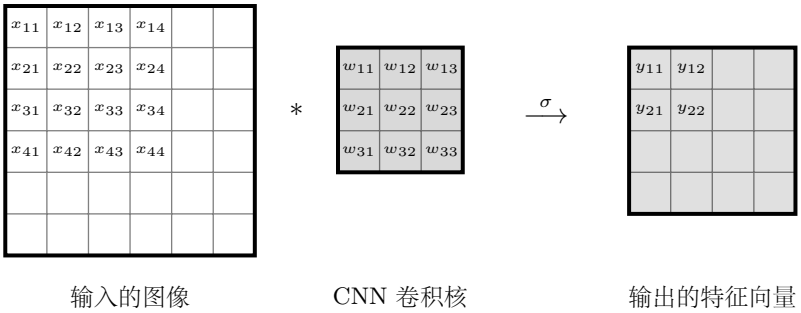


图 1.9: 一個基本卷積神經網絡（CNN）模塊作用於圖像並生成輸出的示意圖。最左側圖像為一個分辨率為 6×6 的二維灰度輸入圖像，其像素強度為 $\{x_{ij} \in \mathbb{R} : i, j \in [6]\}$。中間為一個大小為 3×3 的卷積核，其核值為可訓練參數 $\{w_{ij} \in \mathbb{R} : i, j \in [3]\}$。最右側為大小為 4×4 的特徵張量，其特徵值為 $\{y_{ij} \in \mathbb{R} : i, j \in [4]\}$，表示圖像與卷積核卷積後的輸出。在該示例中，$3 \times 3$ 卷積核首先作用於以 x_{22} 為中心的 3×3 區域，並得到輸出中的第一個特徵值 $y_{11} = \sum_{i=1}^{3} \sum_{j=1}^{3} \sigma(w_{ij} x_{ij})$，其中 σ 為激活函數。在應用激活函數 σ 之前還可以加入偏置項。隨後相同的卷積核作用於以 x_{23} 為中心的區域，並得到 y_{12}，依此類推。該卷積操作通常簡寫為 $y = \sigma(w * x)$。當多個此類模塊堆疊時，前一 CNN 模塊的輸出特徵張量作為下一模塊的輸入。

為中心的區域開始應用卷積核。如果希望壓縮特徵張量的尺寸，可以在卷積時使用更大的步長（圖 1.9 中步長為 1，因此除邊界外尺寸不變）。除了圖 1.9 中提到的加偏置和激活函數外，還有其他常用操作，如批歸一化（對多個輸出求平均）和最大池化。這些示例在網絡上可以輕鬆找到，此處不再贅述。大多數此類操作已集成於現代深度學習軟件包中，可直接在編程中使用。

CNN 模塊的關鍵在於，小型局部卷積核可以作用於整個大規模輸入。通過卷積操作，卷積核在輸入上滑動，並與局部區域計算內積，從而生成對特定模式（如邊緣、角點或紋理）具有強響應的特徵張量。這種局部連接大幅減少了參數數量，相比 MLP 中的全連接層，提高了統計效率與泛化能力，尤其在圖像等高維輸入中表現突出。

由於 CNN 模塊中的卷積核在每個空間位置都會重複使用，因此具有平移不變性：輸入的平移會導致特徵張量相應平移。這一性質與自然圖像高度契合，因為圖像中的物體通常與其位置無關。因此，CNN 能夠學習在空間位置上具有良好泛化能力的穩健特徵，同時保持模型規模可控。

通過堆疊多個卷積模塊，CNN 自然形成分層的數據表示。淺層通常捕捉邊緣和簡單紋理等低級特徵，而深層則編碼物體部件和語義結構等更抽象的概念。這種分層結構在一定程度上類似於哺乳動物大腦皮層中的視覺處理路徑，也是 CNN 在感知任務中表現優異的重要原因。

從數學角度看，CNN 可以被視為具有強歸納能力的結構化函數逼近器。卷積操作引入局部性與平穩性假設，而網絡深度則實現組合式表示。從數值角度看，由於卷積結構適合現代電腦硬件（尤其是 GPU），CNN 可以高效訓練。

CNN 由於其高效性、強歸納偏置以及在數據有限或計算資源受限場景下的優異表現，具有重要意義。因此，CNN 仍在計算機視覺、信號處理和科學計算等領域中發揮核心作用。

圖神經網絡（GNN）

在大量現代實際應用中，數據來源於由圖表示的不規則域，例如社交網絡、引文網絡、分子圖以及物理交互系統。與圖像或序列不同，圖缺乏固定網格結構，其節點的鄰居數量也可能不同。圖神經網絡（graph neural network，GNN）通過從節點、邊以及整體拓撲結構中學習特徵來應對這一挑戰。

給定一個（有向）圖，記為 $G = (V, E)$，其中 $V = \{1, \ldots, n\}$ 是節點集合，$E = \{e_{ij} = (i, j) \in V \times V\}$ 是（有向）邊集合。圖結構由一個加權鄰接矩陣 $A = [A_{ij}]_{i,j=1}^{n} \in \mathbb{R}^{n\times n}$ 表示，其中 $A_{ij} > 0$ 表示存在一條從節點 i 指向節點 j 的邊，即 $e_{ij} \in E$。在無權情形下，若 $e_{ij} \in E$，則只需取 $A_{ij} = 1$，否則取 0。每個節點 i 都對應一個行特徵向量 $x_i \in \mathbb{R}^{1\times d}$，將所有節點特徵堆疊起來可得到特徵矩陣 $X \in \mathbb{R}^{n\times d}$。GNN 的目標是學習同時受 X 和 G 的拓撲結構所影響的節點級、邊級或圖級表示。

大多數現代 GNN 都可以統一到如下的消息傳遞神經網絡框架之下。在第 l 層，每個節點 i 都維護一個 d_l 維的隱藏表示 $h_i^l \in \mathbb{R}^{d_l}$。更新過程由兩步組成：消息聚合和隱藏狀態更新。第 l 層中的消息聚合步驟定義為

$$m_i^l = \sum_{j:e_{ji}\in E} \phi_l(h_i^l, h_j^l),$$

其中求和遍歷節點 i 的所有鄰居節點，而 $\phi_l(\cdot,\cdot)$ 是一個網絡模塊，用於學習如何從相連的兩個節點 i 和 j 的隱藏表示 h_i^l 和 h_j^l 中傳遞消息（信息）。第 l 層中的隱藏表示更新步驟定義為

$$h_i^{l+1} = \psi_l(h_i^l, m_i^l),$$

其中 ψ_l 也可以由神經網絡來實現，例如 MLP 或門控循環單元（GRU）。這一迭代過程使信息能夠在圖上進行傳播，因此在經過 L 層之後，h_i^L 編碼了距離節點 i 最多 L 跳範圍內的節點信息。

圖卷積網絡（graph convolutoinal network，GCN）是 GNN 的一種廣泛使用的實例。給定圖的一個適當加權的鄰接矩陣 $A \in \mathbb{R}^{n\times n}$，定義 $\tilde{A} := A + I_n$，以及相應的對角加權度矩陣 $\tilde{D} := \mathrm{diag}([\tilde{D}_{11}, \ldots, \tilde{D}_{nn}])$，其中 $\tilde{D}_{ii} = \sum_{j=1}^{n} \tilde{A}_{ij}$，而 I_n 是 $n \times n$ 單位矩陣。則第 l 個 GCN 層可設為：

$$H_{l+1} := \sigma\Big(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H_l W_l\Big),$$

其中 $H_0 := X \in \mathbb{R}^{n\times d}$ 是前面給出的輸入節點特徵矩陣，而 $H_l := [h_1^l, \ldots, h_n^l]^\top \in \mathbb{R}^{n\times d_l}$ 是第 l 個隱藏層中的節點特徵矩陣。對稱歸一化 $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ 保證了數值穩定性，並平衡了不同度數節點的貢獻。這裡 $W_l \in \mathbb{R}^{d_l\times d_{l+1}}$ 是一個可訓練的權重矩陣，σ 是諸如 ReLU 的非線性激活函數。這一形式可以解釋為圖上的一種 Laplacian 平滑。

GNN 已經成功應用於社交網絡分析、分子性質預測、推薦系統和知識圖譜。儘管其效果顯著，但仍然存在一些挑戰，包括過平滑、大規模圖上的可擴展性問題，以及表達能力的理論極限。當前的研究正通過改進網絡結構、採樣策略以及與圖論和譜方法建立聯繫來應對這些問題。

儘管表現有效，GNN 仍面臨一些挑戰，例如隨著深度增加，節點表示會變得難以區分的過平滑問題，以及在超大規模圖上的可擴展性問題。這些侷限推動了大量擴展方法的發展，包括基於注意力的聚合、殘差連接、基於採樣的方法以及歸一化技術。儘管如此，圖卷積網絡仍為從關係數據中學習提供了一個具有原則性且強有力的框架，並構成了許多現代圖表示學習方法的基礎。

循環神經網絡（RNN）

在現實世界應用中，生成和觀測到的數據通常表現出很強的序列性和時間相關性，例如文本、語音、音樂、視頻等。循環神經網絡（recurrent neural network，RNN）是一類專門用於建模此類數據的神經網絡結構，其方式是維護一個隨時間演化的內部狀態。不同於標準的前饋神經網絡，RNN 顯式地引入了時間步之間的依賴關係，因此適用於時間序列預測、自然語言處理以及動力系統建模等任務。

設 $\{x_t\}_{t=1}^{T}$ 為一個輸入序列，例如一個由 T 個單詞、標點符號和符號按順序構成的句子，其中 $x_t \in \mathbb{R}^d$ 是序列中第 t 個元素的數值表示。（在語言模型中，單詞、標點和符號通常都表示為向量並嵌入到歐幾里得空間中。）一個基本的 RNN 通常維護一個隱藏狀態 $h_t \in \mathbb{R}^m$，稱為**歷史**（history），用於總結過去的信息。狀態更新通過一個循環模塊 R 來完成，它是當前狀態 x_t 與歷史 h_{t-1} 的函數。該 R 輸出更新後的歷史 h_t，並與 x_{t+1} 一起作為下一時刻 t

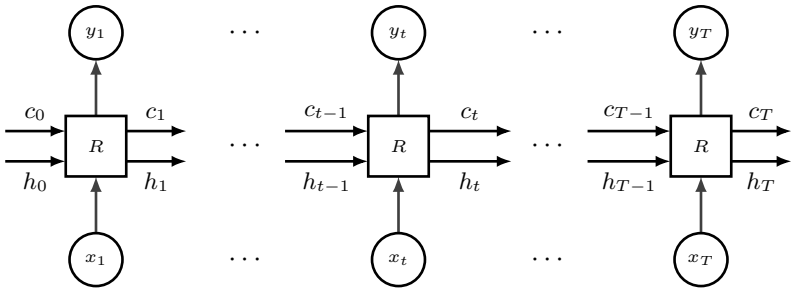


图 1.10: 標準循環神經網絡（RNN）的結構。RNN 的核心組成部分是循環模塊 R，它是一個被參數化為網絡模塊的函數。在時刻 t，R 接收當前輸入數據 x_t（來自長度為 T 的時序數據 $(x_1, \ldots, x_T)$，例如由 T 個單詞、標點和符號組成的句子），以及在前一時刻 $t-1$ 由 R 生成的單元狀態 c_{t-1} 和歷史 h_{t-1}，並計算下一個單元狀態 c_t 和歷史 h_t。它還可能生成一個輸出 y_t，該輸出可與外部給定的觀測值進行比較，以訓練其自身參數。所有這些輸入和輸出都表示為向量。這裡所示的 RNN 模塊 R 被重複使用，因此其結構和參數值在每個時刻 t 都相同。初始的 c_0 和 h_0 在實現中可以是具有平凡取值的佔位符，但在訓練中不是必須的。一個以單元狀態為輸入但沒有輸出的循環模塊 R 的示例，即長短期記憶網絡（LSTM），見 (1.40)。

時 R 的輸入。循環模塊 R 也可以設計成具有其他輸入和輸出，例如單元狀態 c_t 和觀測量 y_t。圖 1.10 展示了一個循環模塊 R 具有單元狀態作為輸入但沒有觀測輸出的 RNN 示例。

從動力系統的角度看，RNN 模塊 R 定義了一個非線性狀態空間模型，它從過去的歷史和狀態中提取信息，以生成下一狀態所需的新歷史。循環結構使信息能夠隨時間持續存在，從而使網絡能夠捕捉輸入序列中的時間相關性和長程依賴。然而，對 R 的重複迭代可能會在 RNN 訓練中導致梯度消失或梯度爆炸。為緩解這些問題，諸如長短期記憶網絡（LSTM）和 GRU 等門控結構引入了乘法門來調節信息流。這些機制改善了梯度傳播，並使得對長程依賴的建模成為可能。儘管基於注意力的模型已經興起，RNN 仍然是理解序列建模與動力系統學習的一個基礎框架。

我們以 LSTM 為例來說明 RNN 的結構。在 LSTM 中，模塊 $R := R_{F,I,O}$ 是一個由三個門組成的模塊，分別稱為遺忘門（forget gate）F、輸入門（input gate）I 和輸出門（output gate）O。這三個門中的每一個都是一個小型神經網絡：F 是一個具有可訓練權重矩陣 W_F 和偏置向量 b_F 的網絡，I 具有權重矩陣 (W_I, W_C) 和偏置 (b_F, b_C)，而 O 具有權重矩陣 W_O 和偏置 b_O。這些權重和偏置的維度都被適當設定，以使得下述計算能夠成立。

總體上，在以上各門定義之後，一個 LSTM 模塊 R 通常被設定為如下輸

入輸出函數:

$$(c_t, h_t) = R_{F,I,O}(x_t, c_{t-1}, h_{t-1}) \tag{1.40}$$

對每個時刻 $t = 1, \ldots, T$，其具體計算實現為

$$\begin{aligned}
f_t &= \mathrm{sigmoid}(W_F[h_{t-1}; x_t] + b_F), \\
c_t &= f_t \odot c_{t-1} + \mathrm{sigmoid}(W_I[h_{t-1}; x_t] + b_I) \odot \tanh(W_C[h_{t-1}; x_t] + b_C), \\
h_t &= \mathrm{sigmoid}(W_O[h_{t-1}; x_t] + b_O) \odot \tanh(c_t),
\end{aligned}$$

其中激活函數 sigmoid 和 tanh 都是按分量作用的，$\odot$ 表示我們前面解釋過的兩個向量的分量乘積，而 $[h_{t-1}; x_t]$ 表示將 h_{t-1} 和 x_t 拼接得到的列向量。如果 R 還具有輸出 y_t，那麼它將被用於與某些觀測數據或給定的真實值 $\hat{y}_t$ 進行比較，並出現在訓練目標函數中。

1.5.3 具有特殊性質的網絡

正如我們所見，神經網絡結構設計具有很大的靈活性。這在實際中尤其有用，因為我們可以巧妙地將所需的函數性質直接構造進網絡結構中，從而使這些性質自動得到滿足。

將所需性質構造進網絡的一個主要優點在於，這通常能夠有效消除（或至少減少）學習最優網絡參數這一優化問題中的約束。這往往會帶來更快、更穩定的訓練計算過程，並在實際中提升網絡性能。下面我們將針對不同的指定性質介紹若干這樣的設計技巧示例。

值域受約束的函數

如果我們知道待逼近的函數 $f : \mathbb{R}^d \to \mathbb{R}^n$ 的輸出值位於某個給定範圍內，那麼我們可以將這一約束顯式地加到用於參數化該函數的神經網絡的最後一步操作中。

例如，如果對所有 x 都有 $f(x) \in (0,1)^n$，那麼我們可以將 f 參數化為複合函數 $\mathrm{sigmoid} \circ \phi$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意神經網絡，而 sigmoid 函數 (1.31) 是按分量作用的。類似地，如果 $f(x) \in (-1,1)^n$，則可以使用 tanh 函數 (1.32)。如果 f 的值域是 $\mathbb{R}^n$ 中的一個長方體 $(a_1, b_1) \times \cdots \times (a_n, b_n)$，其中 $a = (a_1, \ldots, a_n)$ 和 $b = (b_1, \ldots, b_n)$ 是已知的 n 維向量，並滿足對所有 $i \in [n]$

都有 $a_i < b_i$，則我們可以將 f 參數化為

$$f(x) = a + (b - a) \odot \mathrm{sigmoid}(\phi(x)),$$

或

$$f(x) = \frac{1}{2}\Big((a + b) + (b - a) \odot (\tanh(\phi(x))\Big).$$

如果 $f(x) \geq 0$，我們可以將 f 參數化為 $\max(0, \phi)$、$\max(0, \phi)^2$，或者直接取 $\frac{1}{2}|\phi|^2$。注意，只要網絡 ϕ 是連續可微函數（例如 ϕ 僅使用光滑激活函數），那麼後兩者也是連續可微的。這些思想還可以推廣到更復雜的情形。

如果 $f(x) \in \mathbb{R}^n$ 需要表示關於 n 個取值的一個概率分佈，也就是說，$f(x)$ 是一個 n 桶直方圖（n-bin histogram ），那麼 f 的值域就是定義為

$$\Delta^n := \Big\{(y_1, \ldots, y_n) \in \mathbb{R}^n : 0 \leq y_i \leq 1,\ \sum_{i=1}^n y_i = 1\Big\} \subset \mathbb{R}^n$$

的概率單純形（probability simplex）。（這個概率單純形也記作 Δ^{n-1}，以強調它是 $\mathbb{R}^n$ 中的一個 $(n-1)$ 維子流形。）在這種情況下，我們可以將 f 參數化為複合函數 $\mathrm{softmax} \circ \phi$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意網絡，而 softmax 函數定義為

$$\mathrm{softmax}(w) := \frac{1}{\sum_{i=1}^n e^{w_i}}\Big(e^{w_1}, \ldots, e^{w_n}\Big) \in \mathbb{R}^n$$

其中 $w \in \mathbb{R}^n$。於是對所有 $x \in \mathbb{R}^d$，都有 $f(x) \in \Delta^n$。

在一些實際問題中，函數 f 還要求逐點被某個給定函數 g 所界定。例如，已知 $g : \mathbb{R}^d \to \mathbb{R}$，並且待求的 f 滿足對每個 $x \in \mathbb{R}^d$ 都有 $f(x) \geq g(x)$。在這種情況下，我們可以將 f 參數化為

$$f(x) = g(x) + \frac{1}{2}|\phi(x)|^2,$$

其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意神經網絡。

在一些應用中，還會對 f 在某些指定點上的取值提出額外要求。例如，在某些最優控制問題中，我們希望尋找控制系統的 Lyapunov 函數 f。這類函數在某些指定點（稱為平衡點）處已知應為正定。作為一個典型例子，考慮標準情形：平衡點設為原點 $0 \in \mathbb{R}^d$，並且 Lyapunov 函數 f 必須滿足對每個 $x \in \mathbb{R}^d$ 都有 $f(x) \geq 0$，且當且僅當 $x = 0$ 時有 $f(x) = 0$。在這種情況下，我們可以將 f 參數化為

$$f(x) = g(x) + \frac{1}{2}|\phi(x) - \phi(0)|^2,$$

其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意神經網絡，而函數 g 是用戶定義的 f 的非負下界。g 的例子包括 $g(x) = \frac{\alpha}{2}|x|^2$ 和 $g(x) = \alpha \log(1 + |x|^2)$，其中 $\alpha > 0$ 是用戶定義的權重，用於決定下界函數 g 的尺度。

偶函數、奇函數與週期函數

如果已知 $f : \mathbb{R}^d \to \mathbb{R}^n$ 關於任意輸入 $x = (x_1, \ldots, x_d) \in \mathbb{R}^d$ 的第 i 個分量 x_i 是偶函數，那麼我們可以將 f 參數化為

$$f(x) = \phi\Big(x_1, \ldots, x_{i-1}, \frac{\psi(x_i) + \psi(-x_i)}{2}, x_{i+1}, \ldots, x_d\Big), \tag{1.41}$$

其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 和 $\psi : \mathbb{R} \to \mathbb{R}$ 都是任意神經網絡。容易驗證，對每個 $x \in \mathbb{R}^d$，都有 $f(x_1, \ldots, x_i, \ldots, x_d) = f(x_1, \ldots, -x_i, \ldots, x_d)$。如果 f 關於多個分量都是偶函數，也可以採取同樣的方法。如果 f 關於 x_i 是奇函數，那麼只需要將 (1.41) 中的 $\frac{\psi(x_i)+\psi(-x_i)}{2}$ 替換為 $\frac{\psi(x_i)-\psi(-x_i)}{2}$ 即可。這一思想可以逐個作用於 x 的不同分量，使得 f 關於某些分量為偶函數，而關於另一些分量為奇函數。

如果已知 $f : \mathbb{R}^d \to \mathbb{R}^n$ 關於任意 $x = (x_1, \ldots, x_d) \in \mathbb{R}^d$ 的第 i 個分量 x_i 是 T-週期的，那麼我們可以將 f 參數化為

$$f(x) = \phi\Big(x_1, \ldots, x_{i-1}, \cos\Big(\frac{2\pi x_i}{T}\Big), \sin\Big(\frac{2\pi x_i}{T}\Big), x_{i+1}, \ldots, x_d\Big), \tag{1.42}$$

其中 $\phi : \mathbb{R}^{d+1} \to \mathbb{R}^n$ 和 $\psi : \mathbb{R} \to \mathbb{R}$ 是任意神經網絡。這一方法也可以很容易地推廣到 f 關於多個分量都是週期函數的情形，並且不同分量的週期還可以不同。

具有指定邊界值、初值和/或終值的函數

在某些實際應用中，例如求解偏微分方程（partial differential equation，PDE）[37]，解通常需要滿足給定的邊界條件、初始條件和/或終值條件。在這些情況下，我們可以將這些條件直接嵌入到用於參數化解的深度神經網絡中。

例如，如果某個 PDE 的解 $u : \mathbb{R}^d \to \mathbb{R}$ 的定義域是 $\bar{\Omega}$，其中 $\bar{\Omega}$ 是開單位盒 $\Omega := (0, 1)^d \subset \mathbb{R}^d$ 的閉包，並且已知該解在邊界 $\partial\Omega$ 上等於某個給定函數 g，那麼我們可以將 u 參數化為

$$u(x) = g(x) + \phi(x) \prod_{i=1}^{d} x_i(1 - x_i) \tag{1.43}$$

對每個 $x = (x_1, \ldots, x_d) \in \Omega$ 都成立，其中 $\phi : \mathbb{R}^d \to \mathbb{R}$ 是任意神經網絡。我們可以事先在 Ω 的內部任意指定 $g(x)$ 的取值。容易驗證，(1.43) 在 Ω 內部沒有任何約束，並且在 $\partial\Omega$ 上始終等於 g。

雖然 (1.43) 展示瞭如何參數化具有邊界條件的函數這一思想，但我們需要注意 $\prod_{i=1}^d x_i(1-x_i)$ 的選取必須謹慎。我們以 $d = 1$ 且 $g = 0$ 為例。若 $u : \mathbb{R} \to \mathbb{R}$，並且當 $x \to 0$ 時有 $u(x) = O(\sqrt{x})$，則可注意到，當 $x \to 0$ 時，有

$$\frac{u(x)}{x(1-x)} = O\Big(\frac{1}{\sqrt{x}(1-x)}\Big) \to \infty$$

因此，像 (1.43) 那樣將 $u(x)$ 參數化為 $u(x) = \phi(x) \cdot x(1-x)$ 並不合適，因為這會要求 $x \to 0$ 時，有 $\phi(x) \to \infty$，然而這在典型神經網絡結構下是不可能的，因為一般情況下，神經網絡 ϕ 在 $\bar{\Omega}$ 上會處處連續。回到多元情形，我們應將 (1.43) 改寫為

$$u(x) = g(x) + \phi(x) \cdot \sqrt{x_1}(1-x_1) \prod_{i=2}^{d} x_i(1-x_i) \tag{1.44}$$

或類似形式。正如我們所見，(1.44) 的選擇要求我們預先估計當 x 趨於邊界 $\partial\Omega$ 時 $u(x)$ 的階。總之，我們應當對 (1.43) 作適當修改，使得 ϕ 只需逼近一個在 $\bar{\Omega}$ 上的一致有界函數——這正是神經網絡能夠做到的，因為其結構通常只允許其逼近連續函數。

如果時間演化 PDE 的解 $u : \bar{\Omega} \times [0, \infty) \to \mathbb{R}$ 中，Ω 是 $\mathbb{R}^d$ 的一個開子集，並且已知 u 在時刻 $t = 0$ 於 $\bar{\Omega}$ 上具有初值 f_0，那麼我們可以將 u 參數化為

$$u(x,t) = f_0(x) + t\phi(x,t)$$

其中 $\phi : \bar{\Omega} \times [0, \infty) \to \mathbb{R}$ 是任意神經網絡，用以表示關於 x 和 t 的任意函數。如果已知的是 u 的終值，也可以類似地處理。

如果解 $u : \bar{\Omega} \times [0, T] \to \mathbb{R}$ 對某個有限的 $T > 0$ 定義，並且其在 $\bar{\Omega}$ 上的初值 f_0 和終值 f_T 都已知，那麼我們可以將 u 參數化為

$$u(x,t) = (T-t)f_0(x) + tf_T(x) + t(T-t)\phi(x,t),$$

這樣便能自動滿足初值和終值條件。也就是說，對所有 $x \in \bar{\Omega}$，都有 $u(x,0) = f_0(x)$ 且 $u(x,T) = f_T(x)$。與此同時，當 $t \in (0,T)$ 時，$u(x,t)$ 沒有約束，因為 ϕ 是任意神經網絡且 $t(T-t) > 0$。根據當 t 趨於 0 或 T 時 $u(x,t)$ 的變化

速率，我們可能還需要像 (1.44) 中那樣對上式中的 t 的指數作出修改。

1.5.4 網絡訓練準則

在問題已經表述清楚且網絡結構已經確定之後，我們需要指定一個準則，以便求取最優的網絡參數。這一準則通常被設定為一個標量值函數，稱為**目標函數**（objective function），它將任意網絡參數 θ（即網絡中所有參數的集合）映射為一個實數。它也常被稱為**損失函數**（loss function），因為在大多數情況下這一函數是需要被最小化的。此後我們稱其為目標函數，因為在某些應用中我們也可能尋找這一函數的極大值點或鞍點。

求解給定目標函數的最優解（例如給定損失函數的極小值點）的方法稱為優化，這也是第 二 章的主要內容。在深度學習中，這一優化過程也稱為網絡訓練（network training），即尋找網絡的良好參數，使其能夠很好地解決目標問題。在本小節中，我們將討論一些為給定問題設定合適目標函數的規則與思路。

在監督學習問題中，訓練數據同時包含變量 x 和觀測值或標籤 y，如例 1.1.1 和例 1.1.2 所示。在這些問題中，目標函數通常基於 y 與預測結果之間的統計關係來構造，預測結果通常記為 $f_\theta(x)$，其中 f_θ 是帶參數 θ 的神經網絡，用於估計描述該關係的函數。例如，在關於迴歸的例 1.1.1 中，f_θ 被人為設定為一個仿射函數 $f_\theta(x) = w^\top x + b$，其中 $\theta = (w, b)$，對應的觀測值 y 服從分佈 $\mathcal{N}(f_\theta(x), s^2)$，即 y 等於 $f_\theta(x)$ 加上一些具有未知方差 s^2 的獨立高斯噪聲。於是我們可以基於高斯分佈的極大似然估計來構造關於 θ 的目標函數。在關於分類的例 1.1.2 中，f_θ 被人為設定為 $f_\theta(x) = \mathrm{sigmoid}(w^\top x + b)$，其中 $\theta = (w, b)$，對應的標籤 y 被假設服從概率分佈 $\mathrm{Bernoulli}(f_\theta(x))$。於是目標函數也相應地按照極大似然估計來構造。

在無監督學習和強化學習問題中，我們通常沒有標籤數據 y。在這些情況下，我們需要根據問題本身的表述來設定訓練目標函數，而這往往高度依賴具體問題。在第 四 章和第 五 章中，我們將看到許多為相關方法設定目標函數的不同途徑。

在本小節結尾，我們給出一個近年來科學計算研究中目標函數設計的例子。其目標是通過將偏微分方程（PDE）的解參數化為深度神經網絡來求解 PDE。其中的挑戰在於，所考慮的 PDE 往往定義在高維空間上，例如 $\mathbb{R}^d$ 的某個緊子集，其中 $d \geq 5$。儘管數值分析學科已經發展出大量理論與方法，用於數值求解 PDE [4,143,144]，例如有限差分法、有限元法和譜方法，但在高維情形下，使用這些傳統數值 PDE 方法，在計算上經常是非常困難的。困難

原因在於，這些方法通常需要對問題定義域（例如 $[0,1]^d$）進行精細的空間離散化，並在所有離散點上近似解的值。然而，離散點點數量會隨著維數 d 以指數速度增長，這就是所謂的“維數災難”（curse of dimensionality）問題，因此傳統數值 PDE 方法在高維情況下，計算量太大，現代的電腦無法計算。

使用深度神經網絡來逼近 PDE 的解已被證明是在緩解維數問題方面非常有效的方法。其關鍵思想是根據給定 PDE 來設計用於訓練這些深度神經網絡的目標函數，而這些 PDE 往往由問題中的物理規律決定（PDE 通常來源於物理定律）。因此，這類方法中的神經網絡通常被稱為**物理信息神經網絡**（physics-informed neural network）。注意，在這些方法中避免了直接的空間離散化，而目標函數通常通過 Monte Carlo 積分（見附錄 A.1）來近似，比較有效地解決了高維問題中的積分計算問題。下面的例子展示了這些方法是如何工作的。

例 1.5.1 (求解高維 PDE)**.** 我們考慮一個經典的 PDE，即 Poisson 方程。設 $\Omega = (0,1)^d$ 是 $\mathbb{R}^d$ 中的開單位盒，且給定 $f : \Omega \to \mathbb{R}$。則 Poisson 方程寫為

$$-\Delta u(x) = f(x), \quad \forall\, x \in \Omega, \tag{1.45}$$

其中 Δ 是 Laplace 算子，即對每個 $x \in \Omega$，$\Delta u(x) = \sum_{i=1}^d \frac{\partial^2 u}{\partial x_i^2}(x)$。Poisson 方程 (1.45) 還要求 u 滿足某種邊界條件，否則可能存在無窮多個可能解。例如，在 Poisson 方程的 Dirichlet 邊值問題中，邊界函數 $g : \partial\Omega \to \mathbb{R}$ 是給定的，並要求解 u 滿足對所有 $x \in \partial\Omega$ 都有 $u(x) = g(x)$。現在，我們的方法是將 $u : \bar{\Omega} \to \mathbb{R}$ 參數化為一個深度神經網絡，記為 u_θ，其網絡參數為 $\theta \in \mathbb{R}^m$（其中 m 是該網絡中的參數個數），並設計一個適當的目標函數來訓練 θ，使得 u_θ 滿足 Poisson 方程 (1.45)，並且在 $\partial\Omega$ 上等於 g。

我們已經見過一種簡單技巧，可以像 (1.43) 那樣將 u 參數化為自動滿足邊界條件的深度神經網絡。設 u_θ 就是以這種方式構造的，並且在 $\partial\Omega$ 上等於 g。那麼剩下的問題就是構造一個關於 θ 的合適目標函數，使得 θ 達到最優時，意味著對所有 $x \in \Omega$ 都有 $-\Delta u_\theta(x) = f(x)$。

構造目標函數的一種簡單方式，是基於 (1.45) 兩邊的誤差。具體而言，我們通過求解如下關於 θ 的極小化問題來尋找 u 的最優參數 θ:

$$\min_{\theta \in \mathbb{R}^m} \int_\Omega |\Delta u_\theta(x) + f(x)|^2 \, dx. \tag{1.46}$$

由此可見，如果 θ 在 (1.46) 中使目標函數值達到 0，那麼 u_θ 就是滿足 Poisson 方程 (1.45) 的一個解。

由於 (1.46) 中積分的精確計算不可得，並且 $\Omega \subset \mathbb{R}^d$ 的維數較高（例如 $d \geq 5$），我們使用 Monte Carlo 積分（見附錄 A.1）來近似它。更精確地說，我們注意到任意（可積）函數 $h : \Omega \to \mathbb{R}$ 的積分可以按如下方式近似：

$$\int_\Omega h(x)\,dx \approx \frac{1}{N}\sum_{i=1}^{N} h(x^{(i)}), \tag{1.47}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 是從 Ω 上均勻分佈中抽取的 N 個獨立同分布樣本。容易檢驗，上式右端是左端的無偏估計。關於 Monte Carlo 積分的更多細節可見附錄 A.1。

根據 (1.47)，我們可以用定義在 $\mathbb{R}^m \to \mathbb{R}$ 上的函數 ℓ 來近似 (1.46) 中的目標函數：

$$\ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} |\Delta u_\theta(x^{(i)}) + f(x^{(i)})|^2, \tag{1.48}$$

其中 $x^{(i)}$ 與前面一樣，是從 Ω 上均勻分佈中抽取的獨立同分布樣本。在實際中，(1.48) 中的所有項都可以計算。特別地，Laplace 算子是散度算子與梯度算子的複合，可以通過自動微分來計算，其中自動微分方法的細節將在第 2.3 節中給出。隨後，我們可以應用某種數值優化算法（詳見第 二 章）來求解 (1.48) 中 ℓ 的極小值點 θ，於是 u_θ 就成為 Poisson 方程 (1.45) 的近似解。這種利用深度神經網絡求解高維 PDE 的方法在 [114] 中被提出，並命名為物理信息神經網絡。自提出以來，它引起了廣泛關注，並催生了大量後續工作、變體與應用。

在給定 Poisson 方程 (1.45) 的情況下，我們也可以設計其他目標函數來訓練網絡參數 θ。例如，已知 Poisson 方程 (1.45) 的解 u 也滿足如下能量極小化問題：

$$\min_u \int_\Omega \Big(\frac{1}{2}|\nabla u(x)|^2 - f(x)u(x)\Big)\,dx \tag{1.49}$$

前提是 $u = g$ 在 $\partial\Omega$ 上成立。（這一點可由 PDE 基礎理論中的變分法驗證。其思想與第 3.1.1 節中類似。這裡省略細節。）因此，如果我們像 (1.43) 那樣將 $u : \Omega \to \mathbb{R}$ 參數化為滿足邊界值條件的神經網絡，那麼我們可以用其 Monte Carlo 積分來近似 (1.49) 中的積分：

$$\ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} \Big(\frac{1}{2}|\nabla u_\theta(x^{(i)})|^2 - f(x^{(i)})u(x^{(i)})\Big). \tag{1.50}$$

最小化 (1.50) 中的 ℓ 同樣會給出某個 θ，使得 u_θ 成為 Poisson 方程 (1.45) 的

近似解。這種將能量泛函用作訓練損失函數的方法在 [34] 中被提出，並命名為 deep Ritz 方法。

與 (1.46) 需要 Laplace 算子中的二階偏導數相比，(1.50) 的優點在於它只涉及採樣點處 u 的一階梯度。注意，當我們應用基於梯度的優化算法（例如第二 章給出的那些算法）來求解 (1.48) 或 (1.50) 中 ℓ 的極小值點時，在優化過程的每一次迭代中，還需要關於 θ 的另一重梯度。

然而需要注意的是，只有少數 PDE 已知具有像 (1.49) 這樣的能量形式。另一種途徑是，PDE 的理論與應用往往依賴於其弱形式（weak formulation）。事實上，現實應用中的許多 PDE 只具有弱形式意義下的解（這些解稱為弱解，weak solution）。在這種情況下，我們並不期望找到像 (1.45) 中那樣二次可微的強解（也稱經典解），因為它們並不存在。（不過，如果強解確實存在，那麼它也是弱解。因此，當我們考慮 PDE 的弱形式時，並不會遺漏任何強解。）

Poisson 方程 (1.45) 的弱形式可以如下導出。同樣地，我們先假設 u 滿足邊界條件 $u = g$ 於 $\partial\Omega$ 上。將任意測試函數 $\psi \in C_0^\infty(\Omega)$ 乘到 (1.45) 兩邊，其中 $C_0^\infty(\Omega)$ 表示在 Ω 中光滑並且在邊界 $\partial\Omega$ 上取零值的函數集合，然後對兩邊積分，得到

$$-\int_\Omega \Delta u(x)\psi(x)\,dx = \int_\Omega f(x)u(x)\,dx. \tag{1.51}$$

對左側做分部積分並整理 (1.51)，便得到 Poisson 方程 (1.45) 的弱形式：

$$\int_\Omega \Big(\nabla u(x)\cdot\nabla\psi(x) - f(x)\psi(x)\Big)\,dx = 0, \qquad \forall\,\psi \in C_0^\infty(\Omega). \tag{1.52}$$

基於 (1.52)，我們可以將 ψ 參數化為一個任意的邊界值為零的神經網絡 ψ_η，其參數為 η，並嘗試通過求解如下鞍點問題來尋找最優參數 θ 和 η：

$$\min_\theta \max_\eta \frac{1}{2N}\Big|\sum_{i=1}^N \Big(\nabla u_\theta(x^{(i)})\cdot\nabla\psi(x^{(i)}) - f(x^{(i)})\psi(x^{(i)})\Big)\Big|^2, \tag{1.53}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 仍然是從 Ω 上均勻分佈中抽取的 N 個獨立同分布樣本。一旦我們找到最優的 θ，相應的 u_θ 就是 (1.52) 的弱解。在實際中，還存在許多數值技巧，可以更高效、更穩定地處理優化問題 (1.52)。

(1.53) 的優點在於目標函數中只出現梯度。更重要的是，它使我們能夠在弱解存在時求得 Poisson 方程 (1.45) 的弱解。這種利用給定 PDE 的弱形式並將測試函數參數化為對抗網絡的方法在 [159] 中被提出，並命名為弱對抗網絡（weak adversarial network）。

1.6 備註與參考文獻

1.6.1 關於網絡結構設計的備註

我們觀察到，關於深度神經網絡逼近能力的理論研究與其實際設計和部署之間存在顯著差距。從理論角度看，研究通常關注神經網絡能夠逼近哪些函數類、如何在嚴格的數學準則下評估這種逼近，以及為了達到給定精度需要多大的網絡。相比之下，實踐研究主要強調高效網絡結構的設計，通常在啟發式方法的指導下，用於解決特定任務。這包括圖像處理中被廣泛採用的 U-Net [122] 以及自然語言處理中使用的 transformer [150]，它們都是高效且強大的網絡結構，但本書並未討論。然而，關於這些結構逼近誤差的基礎理論問題卻很少被深入研究。

這種差距是有原因的。深度學習在實踐上的顯著成功鼓勵研究者去處理廣泛的實際問題，並發展新的方法，以將神經網絡的適用性擴展到先前被認為難以處理的任務上。這些進展在很大程度上得益於海量數據集的可獲得性以及與幾十年前相比計算能力的巨大提升。值得注意的是，許多現代深度網絡在表示能力方面缺乏正式的理論保證，而且它們的規模通常遠小於逼近理論所給出的理論界——通常是上界。一個體現這種張力的代表性例子是：理論界仍在持續努力研究深度網絡是否能夠在一般情形下克服維數災難，而實踐者則基本繞開這一問題，轉而實現那些具有高度表達能力並在眾多應用領域中表現出強大性能的網絡結構。目前尚不清楚理論與實踐之間的這一鴻溝最終是否會被彌合；然而，這種差距的存在並未顯著阻礙任一方向上的進展。

歸根結底，更現實的看法或許是承認：為了獲得實際上的高效性，設計多樣且精心構造的網絡架構是必要的。在自然界中，動物物種的多樣性使其能夠在特定能力上實現專門化——例如卓越的力量、高速運動、飛行能力或高級視覺感知，而這些能力都依賴於適應相應功能的神經系統。類比而言，我們有理由期待，人工神經網絡同樣應通過面向任務的專門設計原則來構建，以實現最佳性能。

1.6.2 普適逼近理論的文獻

普適逼近定理是神經網絡理論中的一個基礎性結果，它表明在適當條件下，神經網絡能夠以任意精度逼近一大類函數。奠基性工作 [30, 69] 證明了前饋神經網絡可以一致逼近定義在緊域上的任意連續函數。這一結果隨後在 [35, 67, 68, 70] 中被推廣到更廣泛的激活函數類別與逼近情形。在 [83] 中，這一觀點被進一步澄清：其證明了一個前饋網絡是普適逼近器當且僅當其激活函

數不是多項式。特別地，關於使用 sigmoid 激活函數 [8]、週期函數 [97, 98] 以及一般激活函數 [70] 的神經網絡逼近能力都已建立。

關於網絡規模和誤差的顯式定量逼近理論已被廣泛研究。在 [96] 中，這類結果對光滑函數和解析函數也成立。例如，對於任意定義在 $W^{k,p}(\Omega)$ 中的函數（其中 $p \in [1, \infty]$ 且 $\Omega \subset \mathbb{R}^d$ 為緊集），若希望在 L^p 範數下以誤差 $\epsilon > 0$ 進行逼近，則文中表明一個規模為 $O(\epsilon^{-d/k})$、具有非多項式光滑激活函數的淺層網絡就足夠了 [96]。在 [109] 中，對淺層 ReLU 網絡也建立了類似的估計。第 1.4 節中給出的證明遵循 [158]，該文針對 $W^{k,\infty}(\Omega)$ 中的函數，利用深度 ReLU 網絡給出了這樣的估計。上述估計仍然遭受維數災難，因為所估計的網絡規模會隨著維數 d 指數級增長。已有結果表明，在稀疏網格設定下，這一問題可以得到緩解 [100]。如果待逼近的函數已知具有組合結構，那麼這一問題還可以被顯著減輕 [110]。這一領域仍在持續發展，並不斷出現更優的估計結果。

總而言之，普適逼近定理保證了神經網絡的表達能力，而近些年裡新的結果則闡明瞭深度、激活函數以及逼近效率的關係。這些結果共同構成了深度神經網絡在多種應用領域中能夠成功的重要理論基礎。

第二章　網絡訓練

在一個深度學習項目中，一旦網絡結構和用於訓練的目標函數已經確定，下一項主要任務就是確定最優的網絡參數，例如通過最小化目標函數（在這種情況下也稱為損失函數）來實現。這一過程通常稱為**網絡訓練**（network training）。用於完成這一任務的數學技術屬於**優化**（optimization）這一廣義範疇，而優化是應用數學與計算數學中的一個核心主題。

數值優化是應用數學的一個重要領域，其重點在於發展具有可證明收斂性質的算法，以求解參數化的目標函數的極大值點或極小值點。本章介紹優化的基本理論基礎，例如最優性條件，以及自動微分；後者在現代深度學習實現中高效計算導數方面起著關鍵作用。此外，我們還將介紹若干經典的無約束優化算法，例如梯度下降法（包括線搜索策略）、Newton 方法和擬 Newton 方法，以及約束優化方法，例如 Lagrange 算法。本部分的討論僅限於確定性優化（deterministic optimization）的情形，即目標函數及其關於優化變量的梯度都可以被直接計算。

我們還將介紹隨機優化（stochastic optimization）的基本概念，它是了訓練深度神經網絡時占主導地位的一類方法。特別地，我們將討論幾種廣泛使用的隨機優化算法，包括隨機梯度方法、基於動量的變體以及自適應優化方案。

記號說明　在本章中，我們遵循優化中的記號，將目標函數記為 $f : \mathbb{R}^n \to \mathbb{R}$，將待優化變量記為 $x \in \mathbb{R}^n$，並假設約束集 Ω 是 $\mathbb{R}^n$ 中的閉集。

我們通常把一個優化問題寫成最小化問題。在這種情況下，目標函數 f 也稱為損失函數，而最優解稱為極小值點。在設計優化問題時，我們通常需要保證 $f^* := f(x^*) = \min_{x\in\Omega} f(x)$ 是一個有限數（否則可能無解）。我們將在第 2.1 節中給出更多細節。

我們經常使用優化方法來尋找神經網絡的最優參數，這一過程稱為**網絡訓練**。因此，本章中的目標函數 f 和變量 x 在網絡訓練中通常分別對應於損失函數 ℓ 和網絡參數 θ。

2.1 優化中的基本概念

正如我們前面提到的，一個典型的優化問題寫成如下形式：

$$\min_{x\in\Omega} \ f(x), \tag{2.1}$$

其中 $\Omega \subset \mathbb{R}^n$ 稱為**約束集**（或**可行集**），$x \in \Omega$ 稱為**變量**，而 $f : \Omega \to \mathbb{R}$ 稱為**目標函數**（當它需要被最小化時，也稱為**損失函數**或**代價函數**）。(2.1) 的目標是在 Ω 中找到 f 的一個極小值點 x^*，即對所有 $x \in \Omega$ 都有 $f(x^*) \le f(x)$。由於最大化目標函數 f 等價於最小化 $-f$，因此在本章其餘部分中，我們一般都將優化問題表述為最小化問題。

優化問題有若干種典型的分類方式，下面列出幾種。

連續優化與離散優化 離散優化通常指 (2.1) 中的約束集 Ω 是有限集或可數無窮集的情形（例如 $\Omega = \mathbb{N}$，即自然數集）。相比之下，在連續優化問題中，約束集 Ω 是 $\mathbb{R}^n$ 中一個（或若干個）連通區域的並集。在這類區域中，通常除了邊界點外，Ω 中的所有點都是內點。這裡，x 是 Ω 的一個內點是指存在一個包含於 Ω 中的、以 x 為中心的小開鄰域。等價地，x 是一個內點，是指存在 $r > 0$ 使得：

$$B_r(x) := \{y \in \mathbb{R}^n : |y - x| < r\} \subset \Omega. \tag{2.2}$$

集合 $B_r(x)$ 稱為以 x 為中心、半徑為 r 的**開球**。Ω 的內點集合記為 $\mathrm{int}(\Omega)$。

注意，適當定義的優化問題中的約束集幾乎總是閉的，否則 Ω 可能不包含 f 的極小值點。例如，$f(x) = x$ 在 $\Omega = (0, 1) \subset \mathbb{R}$ 中沒有極小值點。

通常，與離散優化相比，連續優化問題更容易求解，因為我們可以利用 f 在 x 鄰域（例如 $B_r(x)$）中的正則性性質（如連續性、可微性和凸性）來獲取有關鄰域中其它點的信息。在本書中，我們只考慮連續優化，因為深度學習中的幾乎所有網絡訓練問題都可以表述為或近似為連續優化。

無約束優化與有約束優化 無約束優化指的是 $\Omega = \mathbb{R}^n$ 的情形。在有約束優化中，Ω 可以是 $\mathbb{R}^n$ 的一個顯式給定子集，例如 $[-1, 1]^n$，或者更一般地由一組**不等式**和/或**等式**約束描述為

$$\Omega = \{x \in \mathbb{R}^n : g(x) \le 0,\ h(x) = 0\}. \tag{2.3}$$

在 (2.3) 中，我們看到約束集 Ω 由

$$\text{不等式约束:} \qquad g(x) \le 0, \tag{2.4a}$$

$$\text{等式约束:} \qquad h(x) = 0, \tag{2.4b}$$

所決定，其中函數 $g : \mathbb{R}^n \to \mathbb{R}^k$ 和 $h : \mathbb{R}^n \to \mathbb{R}^m$ 定義為

$$g(x) = \begin{pmatrix} g_1(x) \\ \vdots \\ g_k(x) \end{pmatrix} \in \mathbb{R}^k \qquad \text{和} \qquad h(x) = \begin{pmatrix} h_1(x) \\ \vdots \\ h_m(x) \end{pmatrix} \in \mathbb{R}^m.$$

注意，$z \le 0$（$z = 0$）表示向量 z 的每一個分量都非正（都等於 0）。還要注意，等式約束 $h(x) = 0$ 可以化為兩個不等式約束 $h(x) \le 0$ 和 $-h(x) \le 0$ 的組合。不過，我們很少這樣做，因為一般來說等式約束比不等式約束更容易處理。

顯然，一般而言無約束優化問題比有約束優化問題更容易處理。雖然本章我們會研究無約束和有約束優化問題的性質，但在網絡訓練中，我們通常更傾向於將問題表述為或近似為無約束優化。這可以通過多種方式實現，並且依賴具體情形；我們將在本章後面詳細討論。

凸優化與非凸優化 如果對任意 $x, y \in \Omega$，都有

$$(1-t)x + ty \in \Omega, \qquad \forall\, t \in [0,1], \tag{2.5}$$

則稱 Ω 是 $\mathbb{R}^n$ 中的一個**凸集**（convex set）。換言之，如果對 Ω 中的任意兩點 x, y，連接 x 和 y 的整條線段都包含在 Ω 中，則 Ω 是一個凸集。

設 Ω 是 $\mathbb{R}^n$ 的一個凸子集，且 $f : \Omega \to \mathbb{R}$ 是定義在 Ω 上的函數。若

$$f((1-t)x + ty) \le (1-t)f(x) + tf(y), \qquad \forall\, t \in [0,1].$$

則稱 f 是一個**凸函數**。換言之，當且僅當對任意兩點 $x, y \in \Omega$，以 $(x, f(x))$ 和 $(y, f(y))$ 為端點的割線段在限制到線段 $\{(1-t)x + ty : t \in [0,1]\}$ 上時，總位於函數 f 的圖像之上或與之重合，我們就稱函數 f 是凸的。圖 2.1 給出了一個凸函數（左）和一個非凸函數（右）的示例。若 $-f$ 是凸函數，則稱 f 為**凹函數**。

當 Ω 是凸集且 f 是凸函數時，問題 (2.1) 稱為一個凸優化（convex optimization）問題。若 Ω 由 (2.4) 那樣的不等式約束 $g(x) \le 0$ 和等式約束 $h(x) = 0$

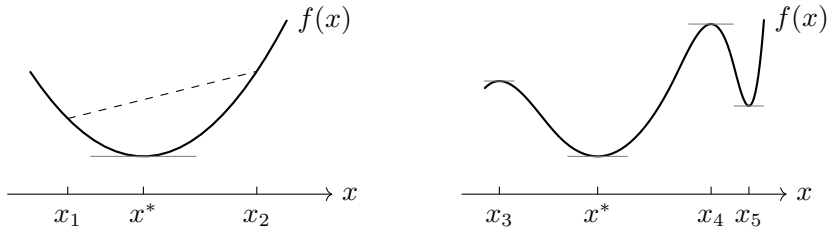


图 2.1: （左）一個凸函數 f。任意臨界點，例如 x^*，如果在該點處 f 的切線是水平的（即若 f 在 x^* 處可微，則 $\nabla f(x^*) = 0$），那麼它必定是 f 的全局極小值點。對任意 x_1 和 x_2，連接 $(x_1, f(x_1))$ 和 $(x_2, f(x_2))$ 的線段（虛線）在 x_1 與 x_2 之間始終位於 $f(x)$ 之上或與之重合；（右）一個非凸函數 f。臨界點可以是局部極小值點，如 x^* 和 x_5，也可以是局部極大值點，如 x_3 和 x_4，或者是鞍點（此圖未畫出）。在圖中所示區間內，x^* 是全局極小值點。

共同確定，則通常要求所有的 g_i 都是凸函數，而所有的 h_j 都是仿射函數，其中 $i \in [k]$ 且 $j \in [m]$。這樣的要求是 Ω 為凸集的一個充分但非必要條件。之所以有這一慣例，是因為當 g_i 為凸函數時，其下水平集 $\{x \in \mathbb{R}^n : g_i(x) \le 0\}$ 必為凸集，而凸集的交 $\{x : g(x) \le 0\} = \cap_{i=1}^k \{x : g_i(x) \le 0\}$ 仍然是凸集。出於同樣的原因，h_j 和 $-h_j$ 都需要是凸的（因為 $h(x) = 0$ 等價於 $h(x) \le 0$ 和 $-h(x) \le 0$），這意味著對每個 j，h_j 必須是仿射函數。

凸函數具有許多重要且有用的性質。因此，圍繞凸優化已經發展出了大量理論與算法結果。然而，在深度學習應用中，優化問題（尤其是網絡訓練）幾乎總是非凸的。因此，在本書中我們將主要考慮非凸優化。

凸性與非凸性會顯著影響我們對於獲得優化問題“最好”或“最優”解的期待。為說明這一點，我們先定義局部最優與全局最優。如果存在 $r > 0$ 使得

$$f(x) \ge f(x^*), \qquad \forall\, x \in \Omega \cap B_r(x^*). \tag{2.6}$$

則稱 x^* 是問題 (2.1) 的一個**局部極小值點**（local minimizer）。此外，如果 (2.6) 中的等號僅當 $x = x^*$ 時成立，則稱 x^* 是一個**嚴格**（strict）局部極小值點。如果

$$f(x) \ge f(x^*), \qquad \forall\, x \in \Omega. \tag{2.7}$$

則稱 x^* 是問題 (2.1) 的一個**全局極小值點**（global minimizer）。如果 (2.7) 中的等號僅當 $x = x^*$ 時成立，則稱 x^* 是 (2.1) 的嚴格全局極小值點。類似地，我們也可以定義 f 在 Ω 中的（嚴格）局部和全局極大值點。圖 2.1 也給出了一些局部/全局極小值點和極大值點的示例。顯然，一個全局極小值點必然是

局部極小值點，但反之不一定成立。

我們指出，凸優化的一個主要性質是：任意局部極小值點都是全局極小值點，正如下述命題所示。

命題 2.1.1. 設 $\Omega \subset \mathbb{R}^n$ 是一個凸集，且 $f : \Omega \to \mathbb{R}$ 是一個凸函數。如果 $x^* \in \Omega$ 是 f 的一個局部極小值點，那麼 x^* 必然是 f 在 Ω 中的一個全局極小值點。

证明. 由於 x^* 是 f 在 Ω 中的一個局部極小值點，根據定義可知存在 $r > 0$ 使得 (2.6) 成立。

現在我們使用反證法。假設存在另一點 $x \in \Omega$ 使得 $f(x) < f(x^*)$。取任意 $\alpha \in (0, \min\{r, |x - x^*|\})$，並設 $t = \alpha/|x - x^*|$ 以及 $y = (1-t)x^* + tx$。那麼 $t \in (0,1)$，$y \in \Omega$（因為 Ω 是凸集且 $x, x^* \in \Omega$），並且

$$|y - x^*| = t|x - x^*| = \alpha < r,$$

這意味著 $y \in B_r(x^*)$。因此，我們得到

$$f(y) = f((1-t)x^* + tx) \le (1-t)f(x^*) + tf(x) < f(x^*),$$

其中第一個不等式來自 f 的凸性，第二個不等式來自 $f(x) < f(x^*)$ 且 $t \in (0,1)$。這與 (2.6) 矛盾。 □

2.2 最優性條件

為了為討論優化問題解的最優性條件（optimality conditions）作準備，我們先固定一些關於梯度、Hessian 矩陣以及鏈式法則（chain rule）的書寫格式，這將在本節其餘部分中使表述更加方便。

梯度與鏈式法則的格式 設 $f : \mathbb{R}^n \to \mathbb{R}^m$ 可微。若 $m = 1$，則我們將 f 在 x 處的**梯度**（gradient）記為一個行向量

$$\nabla f(x) = \big(\partial_1 f(x), \ldots, \partial_n f(x)\big) \in \mathbb{R}^{1\times n}, \tag{2.8}$$

其中 $\partial_i f(x) := \frac{\partial f}{\partial x_i}(x)$。若 $m > 1$ 且我們將 $f(x) = (f_1(x), \ldots, f_m(x))^\top \in \mathbb{R}^m$ 記為列向量，則 f 在 x 處的 **Jacobian 矩陣**是一個 $m \times n$ 矩陣

$$\nabla f(x) = \begin{pmatrix} \nabla f_1(x) \\ \vdots \\ \nabla f_m(x) \end{pmatrix} = \begin{pmatrix} \partial_1 f_1(x) & \partial_2 f_1(x) & \cdots & \partial_n f_1(x) \\ \partial_1 f_2(x) & \partial_2 f_2(x) & \cdots & \partial_n f_2(x) \\ \vdots & \vdots & \ddots & \vdots \\ \partial_1 f_m(x) & \partial_2 f_m(x) & \cdots & \partial_n f_m(x) \end{pmatrix} \in \mathbb{R}^{m \times n}. \tag{2.9}$$

注意，當 $m = 1$ 時，(2.9) 中 Jacobian 的大小與 (2.8) 中梯度的大小是一致的。保持這一格式將有助於我們在下面對函數複合應用鏈式法則時跟蹤相關矩陣的維度。

進一步地，設 $g : \mathbb{R}^m \to \mathbb{R}^k$ 是另一個可微函數。則 $g \circ f : \mathbb{R}^m \to \mathbb{R}^k$ 也是可微的。若我們對 $g \circ f$ 應用鏈式法則，應得到一個 $k \times n$ 的 Jacobian 矩陣，它按照**從左到右**的順序書寫，即先作用在外層函數 g 的梯度，再作用在內層函數 f 的梯度：

$$\underbrace{\nabla(g \circ f)(x)}_{k \times n} = \underbrace{\nabla_f g(f(x))}_{k \times m} \underbrace{\nabla f(x)}_{m \times n}, \tag{2.10}$$

其中每個矩陣的大小都標在相應的大括號下方。類似地，若還有 $h : \mathbb{R}^k \to \mathbb{R}^l$，則 $h \circ g \circ f : \mathbb{R}^n \to \mathbb{R}^l$，並按照上面的從左到右規則得到

$$\underbrace{\nabla(h \circ g \circ f)(x)}_{l \times n} = \underbrace{\nabla_g h(g(f(x)))}_{l \times k} \underbrace{\nabla_f g(f(x))}_{k \times m} \underbrace{\nabla f(x)}_{m \times n}.$$

同樣的規則也適用於三個以上函數的複合。

若 $f : \mathbb{R}^n \to \mathbb{R}$ 二次可微，則我們將 f 在 x 處的 **Hessian 矩陣**記為

$$\nabla^2 f(x) = \begin{pmatrix} \partial_{11}^2 f(x) & \partial_{12}^2 f(x) & \cdots & \partial_{1n}^2 f(x) \\ \partial_{21}^2 f_2(x) & \partial_{22}^2 f(x) & \cdots & \partial_{2n}^2 f(x) \\ \vdots & \vdots & \ddots & \vdots \\ \partial_{n1}^2 f_m(x) & \partial_{n2}^2 f(x) & \cdots & \partial_{nn}^2 f(x) \end{pmatrix} \in \mathbb{R}^{n \times n}, \tag{2.11}$$

其中 $\partial_{ij}^2 f(x) := \frac{\partial^2 f}{\partial x_i \partial x_j}(x)$。我們知道，若 f 在 x 處二次連續可微，則 $\nabla^2 f(x)$ 是對稱矩陣。

我們用 “tr” 表示一個方陣的**跡**（trace）。若 $f : \mathbb{R}^n \to \mathbb{R}^n$ 是一個可微向量

場，則 f 在 x 處的**散度**（divergence）定義為

$$\nabla \cdot f(x) := \mathrm{tr}(\nabla f(x)) = \sum_{i=1}^{n} \partial_i f_i(x), \tag{2.12}$$

它是一個標量。若 $f : \mathbb{R}^n \to \mathbb{R}$ 是一個二次可微的標量值函數，則 f 在 x 處的 **Laplacian** 定義為

$$\Delta f(x) := \mathrm{tr}(\nabla^2 f(x)) = \sum_{i=1}^{n} \partial_{ii}^2 f(x). \tag{2.13}$$

2.2.1 最優性條件的一般原理

給定 $\mathbb{R}^n$ 的一個子集 Ω，若非零向量 $v \in \mathbb{R}^n$ 滿足存在 $r > 0$ 使得對所有 $\alpha \in [0, r)$ 都有 $x + \alpha v \in \Omega$，則稱 v 是點 $x \in \Omega$ 處的一個**可行方向**（feasible direction）。設 $f : \Omega \to \mathbb{R}$ 可微。我們將 f 在 x 處沿方向 v 的**方向導數**（directional derivative）記為

$$D_v f(x) := \langle \nabla f(x), v \rangle = \frac{d}{d\alpha} f(x + \alpha v)\Big|_{\alpha=0} = \lim_{\alpha \to 0^+} \frac{f(x + \alpha v) - f(x)}{\alpha}, \tag{2.14}$$

它是把 f 限制到一條射線 $\{x + \alpha v : \alpha \geq 0\}$ 上時，在 x 點的導數。

一階必要條件 若 $x \in \Omega$ 是 f 的一個局部極小值點，則 f 在 x 處沿任意可行方向 v 的方向導數必然是非負的。這稱為可微函數 f 下優化問題 (2.1) 的局部極小值點的**一階必要條件**（first-order necessary condition）。

注意，若 x 是 Ω 的一個內點，則對某個 $r > 0$ 有 $B_r(x) \subset \Omega$。因此，我們知道任意非零向量 $v \in \mathbb{R}^n$ 都是 $x \in \Omega$ 處的可行方向，因為對任意 $\alpha \in [0, r/|v|)$ 都有 $x + \alpha v \in B_r(x) \subset \Omega$。這就意味著對任意 $v \in \mathbb{R}^n$ 都有 $D_v f(x) = \langle \nabla f(x), v \rangle = 0$，從而 $\nabla f(x) = 0$。因此，一個內點局部極小值點必定是 f 的一個臨界點。因此，在優化問題中我們常常尋找 f 的臨界點，這是因為 f 的所有即是內點又是局部極小值的點，都一定是臨界點。這一事實在無約束情形 $\Omega = \mathbb{R}^n$ 下尤其有用，因為此時每個點都是內點。

二階必要條件 設 $f : \mathbb{R}^n \to \mathbb{R}$ 二次連續可微。若 $x \in \Omega$ 是 f 的一個局部極小值點，且 v 是滿足 $D_v f(x) = \langle \nabla f(x), v \rangle = 0$ 的一個可行方向，則由 Taylor

定理，作為 α 的函數，$f(x+\alpha v)$ 滿足

$$0 \le f(x+\alpha v) - f(x) = \frac{\alpha^2}{2} v^\top \nabla^2 f(x+\xi\alpha v) v \tag{2.15}$$

其中對任意足夠小的 $\alpha > 0$，某個 $\xi = \xi(\alpha, x, v) \in [0,1]$ 成立。由於 $\nabla^2 f$ 連續，我們知道當 $\alpha \to 0^+$ 時，(2.15) 蘊含

$$v^\top \nabla^2 f(x) v \ge 0. \tag{2.16}$$

這個 (2.16) 稱為優化問題 (2.1) 的**二階必要條件**（second-order necessary condition）。

注意，若 x 是 Ω 的一個內點且是 f 的局部極小值點，則我們知道 $\nabla f(x) = 0$，並且任意方向 v 都是可行方向。因此，由 (2.16) 可推出 $\nabla^2 f(x) \succeq 0$，即 $\nabla^2 f(x)$ 是半正定矩陣。因此，當 x 是 (2.1) 中 Ω 的一個內點時，$\nabla^2 f(x) \succeq 0$ 是局部極小值點 x 的二階必要條件。若進一步有 $\nabla^2 f(x) \succ 0$，則 x 是 f 的一個嚴格局部極小值點，而我們稱 $\nabla^2 f(x) \succ 0$ 為 f 的局部極小值點 x 的二階充分條件。

備註. 我們指出，一階與二階必要條件對於一般的非凸函數而言**並非**充分條件。例如，對定義為 $f(x) = x^3$ 的函數 $f : \mathbb{R} \to \mathbb{R}$，有 $f'(0) = f''(0) = 0$，這意味著 f 在 $x = 0$ 處滿足這兩個條件。然而，$x = 0$ 是 f 的一個鞍點，而不是局部極小值點或極大值點。

上述一階與二階必要條件給出了局部極小值點的簡明刻畫，特別是在這些點是 Ω 的內點時尤為有信息量。然而，在實際中，當局部極小值點 x 位於約束優化問題中 Ω 的邊界上時，我們無法直接驗證這些條件。這是因為很難確定 x 處所有可行方向的集合，並逐一檢查這些必要條件。因此，我們需要對約束優化而言更為實用的必要條件，下面我們就來說明。

2.2.2 必要最優性條件

我們首先考慮只有等式約束時優化問題的一階和二階必要條件：

$$\min_{x \in \mathbb{R}^n} \quad f(x), \tag{2.17a}$$

$$\text{s.t.} \quad h(x) = 0. \tag{2.17b}$$

我們要求約束函數 $h:\mathbb{R}^n \to \mathbb{R}^m$ 滿足 $m \le n$；否則約束要麼是過度冗餘的（有些約束是冗餘的，可以刪去），要麼是過定的（約束過多導致約束集 Ω 為空），此時我們可以修改 h，並將問題 (2.17) 轉化為一個滿足 $m \le n$ 的等價問題。在 $m \le n$ 的前提下，我們知道 Jacobian 矩陣 $\nabla h(x)$ 的秩在任意 x 處都不會超過 m。現在我們引入正規點的定義。

定義 2.2.1 (正規點)**.** 若 h 在 x 處的 Jacobian 矩陣具有滿行秩 m，則稱 x 是 (2.17b) 的一個**正規點**。

定義 2.2.2 (切空間)**.** 記 (2.17b) 中的約束集為 $\Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$。則 Ω 在 $x \in \Omega$ 處的**切空間**定義為

$$T(x) = \{v \in \mathbb{R}^n : \nabla h(x)v = 0\} \subset \mathbb{R}^n. \tag{2.18}$$

也就是說，$T(x)$ 是矩陣 $\nabla h(x)$ 的行空間的正交補。

引理 2.2.3. *設 $x \in \Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$ 是一個正規點。則 $v \in T(x)$ 當且僅當存在一條曲線 $c:(-\delta,\delta) \to \Omega$，使得 $c(0) = x$ 且 $c'(0) = v$。*

证明. 我們先證明必要性。設 $c(t)$ 是一條滿足上述條件的曲線。則對所有 $t \in (-\delta,\delta)$，都有 $c(t) \in \Omega$，即 $h(c(t)) = 0$。對 t 求導，得到

$$0 = \frac{d}{dt}h(c(t))\Big|_{t=0} = \nabla h(c(0))c'(0) = \nabla h(x)v.$$

因此，由 (2.18) 可知 $v \in T(x)$。

接下來證明充分性。設 $x \in \Omega$ 是一個正規點，且 $v \in T(x)$。定義 $\bar{h}:\mathbb{R} \times \mathbb{R}^m \to \mathbb{R}^m$ 為

$$\bar{h}(t,w) = h(x + tv + \nabla h(x)^\top w).$$

則有 $\bar{h}(0,0) = h(x) = 0$，並且

$$\partial_w \bar{h}(t,w)|_{(t,w)=(0,0)} = \nabla h(x)\nabla h(x)^\top \succ 0,$$

這是因為 $\nabla h(x) \in \mathbb{R}^{m\times n}$ 具有滿行秩 m 且 $m \le n$。由隱函數定理可知，存在 $t = 0$ 的一個鄰域 $(-\delta,\delta)$，其中某個 $\delta > 0$，以及一個函數 $u:(-\delta,\delta) \to \mathbb{R}^m$，滿足 $(0,u(0)) = (0,0)$ 且

$$\bar{h}(t,u(t)) = h(x + tv + \nabla h(x)^\top u(t)) = 0 \tag{2.19}$$

對所有 $t \in (-\delta, \delta)$ 成立。注意到

$$0 = \frac{d}{dt} h(x + tv + \nabla h(x)^\top u(t))\Big|_{t=0} = \nabla h(x)(v + \nabla h(x)^\top u'(0)).$$

由於 $\nabla h(x)v = 0$ 且 $\nabla h(x)\nabla h(x)^\top \succ 0$，可知 $u'(0) = 0$。

定義曲線 $c(t) = x + tv + \nabla h(x)^\top u(t)$，則由 (2.19) 可知，對所有 $t \in (-\delta, \delta)$ 都有 $h(c(t)) = 0$，即 $c(t) \in \Omega$。此外，$c(0) = x + \nabla h(x)^\top u(0) = x$，且 $c'(0) = v + \nabla h(x)^\top u'(0) = v$。證明完成。 □

定理 2.2.4 (Lagrange)**.** *設 $f, h \in C^1$。若 x 是 (2.17) 的一個局部極小值點，且 $x \in \Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$ 是一個正規點，則存在 $\lambda \in \mathbb{R}^m$ 使得*

$$\nabla f(x) + \lambda \nabla h(x) = 0.$$

证明. 由引理 2.2.3，我們知道對任意 $v \in T(x)$，存在某個 $\delta > 0$ 和一條曲線 $c : (-\delta, \delta) \to \Omega$，使得 $c(0) = x$ 且 $c'(0) = v$。

由於 x 是 f 的一個局部極小值點，我們有

$$0 = \frac{d}{dt} f(c(t))\Big|_{t=0} = \nabla f(c(0))c'(0) = \nabla f(x)v.$$

由於 $v \in T(x)$ 是任意的，我們知道 $\nabla f(x)$ 屬於 $\nabla h(x)$ 的行空間。這就意味著存在（一個行向量）$\lambda \in \mathbb{R}^m$，使得 $\nabla f(x) + \lambda \nabla h(x) = 0$。 □

備註. 注意，在定理 2.2.4 中要求 $x \in \Omega$ 是一個正規點是必要的：考慮問題 (2.17)，其中 $f, h : \mathbb{R} \to \mathbb{R}$ 定義為

$$f(x) = x \quad \text{和} \quad h(x) = \begin{cases} \frac{1}{2}x^2, & \text{if} \quad x < 0, \\ 0, & \text{if} \quad 0 \le x \le 1, \\ \frac{1}{2}(x-1)^2, & \text{if} \quad x > 1. \end{cases}$$

那麼很容易說明，$x = 0$ 是約束 $h(x) = 0$ 下 f 的極小值點。此外，$f, h \in C^1$，$f'(0) = 1$ 且 $h'(0) = 0$（因此 $x = 0$ 不是正規點），而定理 2.2.4 的結論不成立。

為了便於記憶定理 2.2.4 的結果，我們定義 (x, λ) 的 **Lagrange 方程**為

$$L(x, \lambda) = f(x) + \lambda h(x).$$

於是定理 2.2.4 表明，若 $x \in \Omega$ 是一個正規點，並且是等式約束優化問題 (2.17)

的一個局部極小值點，其中 $f \in C^1(\mathbb{R}^n;\mathbb{R})$ 且 $h \in C^1(\mathbb{R}^n;\mathbb{R}^n)$，則存在一個稱為 **Lagrange 乘子**的 $\lambda \in \mathbb{R}^m$，使得

$$\nabla f(x) + \lambda \nabla h(x) = 0, \tag{2.20a}$$

$$h(x) = 0. \tag{2.20b}$$

我們可以將 (2.20) 記成 $(\partial_x L(x,\lambda), \partial_\lambda L(x,\lambda)) = (0,0)$。

若 $f, h \in C^2$，則我們可以在定理 2.2.4 的基礎上進一步建立 (2.17) 的**二階必要條件**。

定理 2.2.5. *設 $f \in C^1(\mathbb{R}^n;\mathbb{R})$ 且 $h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$，其中 $m \le n$。若 $x \in \Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$ 是一個正規點，且是 (2.17) 的一個局部極小值點，則存在一個 Lagrange 乘子 $\lambda \in \mathbb{R}^m$（行向量），使得*

1. *$\nabla f(x) + \lambda \nabla h(x) = 0$；並且*

2. *對任意 $v \in T(x)$，都有*

$$v^\top \Big(\nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 h(x) \Big) v \ge 0. \tag{2.21}$$

证明. 第一條結論就是定理 2.2.4。對於第二條結論，我們沿用定理 2.2.4 的方法，得到曲線 $c : (-\delta,\delta) \to \Omega$，滿足 $c(0) = x$ 且 $c'(0) = v$。現在定義 $\phi : (-\delta,\delta) \to \mathbb{R}$ 為 $\phi(t) := f(c(t))$。由於 x 是局部極小值點且對所有 $t \in (-\delta,\delta)$ 都有 $c(t) \in \Omega$，我們知道

$$0 \le \phi''(0) = v^\top \nabla^2 f(x) v + \nabla f(x) c''(0). \tag{2.22}$$

進一步地，對每個 $i \in [m]$，定義 $\psi_i : (-\delta,\delta) \to \mathbb{R}$ 為 $\psi_i(x) := h_i(c(t))$。於是，由於對所有 $t \in (-\delta,\delta)$ 都有 $h_i(c(t)) = 0$，便有

$$0 = \psi_i''(0) = v^\top \nabla^2 h_i(x) v + \nabla h_i(x)^\top c''(0) \tag{2.23}$$

對每個 $i \in [m]$ 都成立。將 (2.22) 與按 λ_i 加權後的 (2.23) 對所有 $i \in [m]$ 求和，並注意到 $\nabla f(x) + \lambda \nabla h(x) = 0$，即可得到 (2.21)。 □

現在我們考慮同時含有等式約束與不等式約束的優化問題的一階和二階

必要條件:

$$\min_{x\in\mathbb{R}^n} \quad f(x), \tag{2.24a}$$

$$\text{s.t.} \quad g(x) \le 0, \tag{2.24b}$$

$$h(x) = 0, \tag{2.24c}$$

其中 $g = (g_1, \ldots, g_k) : \mathbb{R}^n \to \mathbb{R}^k$，$h = (h_1, \ldots, h_m) : \mathbb{R}^n \to \mathbb{R}^m$。對任意滿足 $g(x) \le 0$ 且 $h(x) = 0$ 的點 x，我們將 $[k]$ 劃分為**活躍集**（active set）

$$A(x) := \{j \in [k] : g_j(x) = 0\} \tag{2.25}$$

和**非活躍集**（inactive set）

$$I(x) := [k] \setminus A(x) = \{j \in [k] : g_j(x) < 0\}. \tag{2.26}$$

記約束集為

$$\Omega = \{x \in \mathbb{R}^n : g(x) \le 0,\ h(x) = 0\}.$$

若 $x \in \Omega$ 滿足向量

$$\nabla h_1(x),\ \ldots,\ \nabla h_m(x), \quad \text{和} \quad \nabla g_j(x),\ \forall\, j \in A(x) \tag{2.27}$$

線性無關（注意這些是 $\mathbb{R}^n$ 中的 $m + |A(x)|$ 個行向量），則稱 $x \in \Omega$ 是一個**正規點**。下面的結果描述了 (2.24) 的一階必要條件，即著名的 **Karush–Kuhn–Tucker**（KKT）定理。

定理 2.2.6 (Karush–Kuhn–Tucker)**.** 對於約束優化問題 (2.24)，設 $f \in C^1(\mathbb{R}^n;\mathbb{R})$，$g \in C^1(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$。若 $x \in \Omega$ 是一個正規點，且是 f 的一個局部極小值點，則存在 $\lambda \in \mathbb{R}^m$ 和 $\mu \in \mathbb{R}^k$（λ 和 μ 都是行向量），使得

$$\nabla f(x) + \lambda \nabla h(x) + \mu \nabla g(x) = 0, \tag{2.28a}$$

$$h(x) = 0, \tag{2.28b}$$

$$g(x) \le 0, \tag{2.28c}$$

$$\mu \ge 0, \tag{2.28d}$$

$$\mu_j g_j(x) = 0, \quad \forall\, j \in [k]. \tag{2.28e}$$

证明. 注意，由於 $x \in \Omega$，所以 (2.28b) 和 (2.28c) 成立。我們令對所有 $j \in I(x)$ 都有 $\mu_j = 0$。於是，由於對所有 $j \in A(x)$ 都有 $g_j(x) = 0$，可知 (2.28e) 成立。

由於各個 g_j 都是 C^1 函數，且對所有 $j \in I(x)$ 都有 $g_j(x) < 0$，因此存在 $r > 0$，使得對所有 $y \in B_r(x)$ 和 $j \in I(x)$ 都有 $g_j(y) < 0$。記

$$\Omega' := \{y \in \Omega \cap B_r(x) : h(y) = 0,\ g_j(y) = 0,\ \forall\, j \in A(x)\}.$$

由於 $x \in \Omega$ 是一個正規點且是 f 的局部極小值點，我們知道 x 在 Ω' 中仍是一個正規點與局部極小值點（因為 $g, h \in C^1$）。注意，Ω' 只包含等式約束。因此，我們應用定理 2.2.4，得到存在 $\lambda \in \mathbb{R}^m$ 以及對每個 $j \in A(x)$ 存在 $\mu_j \in \mathbb{R}$，使得 (2.28a) 成立（這裡我們注意到，對所有 $l \in I(x)$，有 $\mu_l = 0$）。

現在只需證明對所有 $j \in A(x)$ 都有 $\mu_j \geq 0$。我們採用反證法：若不然，則存在某個 $j \in A(x)$ 使得 $\mu_j < 0$。定義

$$\hat{\Omega} := \{x \in \mathbb{R}^n : h(x) = 0,\ g_{j'}(x) = 0,\ \forall\, j' \in A(x) \setminus \{j\}\} \tag{2.29}$$

以及

$$\hat{T}(x) := \{v \in \mathbb{R}^n : \nabla h(x)v = 0,\ \nabla g_{j'}(x)v = 0,\ \forall\, j' \in A(x) \setminus \{j\}\}. \tag{2.30}$$

注意，$\hat{T}(x)$ 是 $\hat{\Omega}$ 在 x 處的切空間，同時也是下式中張成的線性子空間

$$\operatorname{span}\{\nabla h_i(x),\ \nabla g_{j'}(x)\ :\ i \in [m],\ j' \in A(x) \setminus \{j\}\} \tag{2.31}$$

的正交補。由於 x 是正規點，我們知道 $\nabla g_j(x)$ 與張成空間 (2.31) 中的那些向量線性無關，因此 $\nabla g_j(x) \in \hat{T}(x)$。於是存在某個 $v \in \hat{T}(x)$（例如可取 $v = -\nabla g_j(x)^\top$），使得 $\nabla g_j(x)v < 0$。

我們將 v 乘到 (2.28a) 兩邊，得到

$$0 = \nabla f(x)v + \lambda \nabla h(x)v + \mu \nabla g(x)v = \nabla f(x)v + \mu_j \nabla g_j(x)v,$$

其中第二個等號來自 (2.30) 中 $v \in \hat{T}(x)$ 的定義，以及對所有 $l \in I(x)$ 都有 $\mu_l = 0$。由於 $\mu_j < 0$ 且 $\nabla g_j(x)v < 0$，可得 $\nabla f(x)v < 0$。

由於 x 是 $\hat{\Omega}$ 中的正規點，且 $\hat{T}(x)$ 是 $\hat{\Omega}$ 在 x 處的切空間，由定理 2.2.4 可知，存在 $\delta > 0$ 和一條曲線 $c : (-\delta, \delta) \to \hat{\Omega}$，使得 $c(0) = x$ 且 $c'(0) = v$。此

外，

$$\frac{d}{dt}f(c(t))\Big|_{t=0} = \nabla f(c(0))c'(0) = \nabla f(x)v < 0$$

且

$$\frac{d}{dt}g_j(c(t))\Big|_{t=0} = \nabla g_j(c(0))c'(0) = \nabla g_j(x)v < 0.$$

因此，存在足夠小的 $\delta' \in (0,\delta)$，使得對所有 $t \in (0,\delta')$，都有 $c(t) \in B_r(x)$、$f(c(t)) < f(c(0)) = f(x)$、$g_j(c(t)) < g_j(c(0)) = g_j(x) = 0$。同時，對每個 $j' \in A(x) \setminus \{j\}$，都有 $g_{j'}(c(t)) = 0$（因為 $c(t) \in \hat{\Omega}$）；對每個 $l \in I(x)$，都有 $g_l(c(t)) < 0$（因為 $c(t) \in B_r(x)$），且這些都對所有 $t \in (0,\delta')$ 成立。因此，$\{c(t) : t \in (0,\delta')\} \subset \Omega$，這說明曲線 c 上這些點都是可行點，並且取得了更小的 f 值。這與 x 是 f 在 $\hat{\Omega}$ 中的局部極小值點矛盾。因此，對所有 $j \in A(x)$ 都有 $\mu_j \geq 0$，證明完成。 □

為了便於記憶定理 2.2.6 的結果，我們也可以定義 (x,λ,μ) 的 Lagrangian 方程為

$$L(x,\lambda,\mu) = f(x) + \lambda h(x) + \mu g(x).$$

於是，定理 2.2.6 表明：若 $x \in \Omega$ 是一個正規點，且是約束優化問題 (2.24) 的一個局部極小值點，其中 $f \in C^1(\mathbb{R}^n;\mathbb{R})$，$g \in C^1(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$，則存在一個 Lagrange 乘子 $\lambda \in \mathbb{R}^m$ 和一個必須非負的 **KKT 乘子** $\mu \in \mathbb{R}^k$，使得 $\partial_x L(x,\lambda,\mu) = 0$ 且 $\mu g(x) = 0$（稱為**互補鬆弛**條件，complementary slackness condition）。注意，在 (2.28c) 與 (2.28d) 成立的前提下，$\mu g(x) = 0$ 與 (2.28e) 是等價的。由 (2.28) 中這些等式與不等式組成的系統稱為 **KKT 條件**。滿足 KKT 條件的點 x 稱為 **KKT 點**。

若 $f,g,h \in C^2$，則還可以建立 (2.24) 的二階必要條件，如下所示。

定理 2.2.7. *設 $f \in C^2(\mathbb{R}^n;\mathbb{R})$，$g \in C^2(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^2(\mathbb{R}^n;\mathbb{R}^m)$，且 $k+m \leq n$。若 $x \in \Omega = \{x \in \mathbb{R}^n : g(x) \leq 0,\ h(x) = 0\}$ 是一個正規點，且是 (2.24) 的一個局部極小值點，則存在一個 Lagrange 乘子 $\lambda \in \mathbb{R}^m$ 和一個 KKT 乘子 $\mu \in \mathbb{R}^k$，使得*

1. *KKT 條件 (2.28) 成立；並且*

2. *對任意 $v \in T(x)$，都有*

$$v^\top\Big(\nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 h(x) + \sum_{j=1}^k \mu_j \nabla^2 g_j(x)\Big)v \geq 0, \tag{2.32}$$

其中

$$T(x) = \{v \in \mathbb{R}^n : \nabla h(x)v = 0,\ \nabla g_j(x)v = 0,\ \forall\, j \in A(x)\}.$$

证明. 第一條結論就是定理 2.2.4。對於第二條結論，我們沿用定理 2.2.5 的證明方法，得到曲線 $c : (-\delta, \delta) \to \Omega' = \{x \in \mathbb{R}^n : h(x) = 0,\ g_j(x) = 0,\ \forall\, j \in A(x)\}$，使得 $c(0) = x$ 且 $c'(0) = v$。其餘證明與定理 2.2.5 的證明非常類似，因此此處省略。 □

2.2.3 充分最優性條件

在某些情況下，我們還需要判斷一個給定點是否為局部極小點。這就需要局部極小點的充分條件。在本小節中，我們利用目標函數的二階導數來推導這樣的條件。需要注意的是，一般而言，僅依靠一階導數無法給出充分條件。

引理 2.2.8. 設 P 和 Q 為兩個對稱矩陣，其中 Q 是半正定的，而 P 在 Q 的零空間上是正定的，即對於任意滿足 $x^\top Qx = 0$ 的非零向量 x，有 $x^\top Px > 0$，則存在 $\bar{\gamma} > 0$，使得對於任意 $\gamma \in (0, \bar{\gamma})$，都有 $P + \frac{1}{\gamma}Q \succ 0$。

证明. 我們採用反證法。如果結論不成立，則對於任意 $k \in \mathbb{N}$，取 $\gamma_k = 1/k$，則存在 x_k 滿足 $|x_k| = 1$ 且

$$x_k^\top \Big(P + \frac{1}{\gamma_k}Q\Big)x_k = x_k^\top (P + kQ)x_k \le 0 \tag{2.33}$$

由於 $\{x_k\}$ 有界，存在其子序列 $\{x_{k_j}\}$ 以及向量 $\bar{x}$，使得當 $j \to \infty$ 時 $x_{k_j} \to \bar{x}$。

注意到 $|\bar{x}| = 1$，即 $\bar{x} \ne 0$。對 $k = k_j$ 並令 $j \to \infty$，對 (2.33) 取極限，有

$$\bar{x}^\top P\bar{x} + \lim_{j\to\infty} k_j x_{k_j}^\top Q x_{k_j} \le 0.$$

由於 Q 是半正定的，故對所有 j 有 $x_{k_j}^\top Q x_{k_j} \ge 0$。這意味著 $\bar{x}^\top P\bar{x} \le 0$。

注意到 $\bar{x}^\top P\bar{x}$ 是一個固定數，而當 $j \to \infty$ 時 $k_j \to \infty$。因此必須有

$$\bar{x}^\top Q\bar{x} = \lim_{j\to\infty} x_{k_j}^\top Q x_{k_j} = 0,$$

從而由於 P 在 Q 的零空間上正定，有 $\bar{x}^\top P\bar{x} > 0$。這與上面的 $\bar{x}^\top P\bar{x} \le 0$ 矛盾。 □

現在我們可以給出局部極小值點的充分條件。我們首先考慮帶等式約束的優化問題 (2.17)。

命題 2.2.9 (二階充分條件)**.** 設 $f, h \in C^2$, $x^* \in \mathbb{R}^n$ 且 $\lambda^* \in \mathbb{R}^m$ 滿足 *Lagrange* 條件，並且

$$y^\top \Big(\nabla^2 f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla^2 h_i(x^*)\Big) y > 0 \tag{2.34}$$

對所有滿足 $\nabla h(x^*)y = 0$ 的非零向量 $y \in \mathbb{R}^n$ 成立，則 x^* 是在約束 $h(x) = 0$ 下 f 的一個嚴格局部極小點。事實上，存在 $\alpha > 0$ 和 $r > 0$，使得

$$f(x) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$$

對所有滿足 $h(x) = 0$ 的 $x \in B_r(x^*)$ 成立。

证明. 取 $\gamma > 0$。定義增廣 Lagrange 函數 l_A 為

$$l_A(x, \lambda; \gamma) := f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2, \tag{2.35}$$

注意到 l_A 同時也是如下問題的 Lagrange 函數

$$\min_{x \in \mathbb{R}^n} \quad f(x) + \frac{1}{2\gamma}|h(x)|^2, \tag{2.36a}$$

$$\text{s.t.} \quad h(x) = 0. \tag{2.36b}$$

可以看出 (2.36) 與 (2.17) 具有相同的局部極小點。此外，有

$$\nabla_x l_A(x, \lambda; \gamma) = \nabla f(x) + \lambda \nabla h(x) + \frac{1}{\gamma} h(x)^\top \nabla h(x)$$

以及

$$\nabla^2_{xx} l_A(x, \lambda; \gamma) = \nabla^2 f(x) + \sum_{i=1}^m \Big(\lambda_i + \frac{h_i(x)}{\gamma}\Big) \nabla^2 h_i(x) + \frac{1}{\gamma} \nabla h(x)^\top \nabla h(x).$$

由於 (x^*, λ^*) 滿足 Lagrange 條件，即

$$\begin{aligned} \nabla f(x^*) + \lambda^* \nabla h(x^*) &= 0, \\ h(x^*) &= 0, \end{aligned}$$

因此有

$$\nabla_x l_A(x^*, \lambda^*; \gamma) = \nabla f(x^*) + \lambda^* \nabla h(x^*) + \frac{1}{\gamma} h(x^*)^\top \nabla h(x^*) = 0$$

以及

$$\nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma) = \nabla^2 f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla^2 h_i(x^*) + \frac{1}{\gamma} \nabla h(x^*)^\top \nabla h(x^*).$$

注意，對於任何屬於 $\nabla h(x^*)^\top \nabla h(x^*)$ 的零空間的向量 y（等價地，y 滿足 $\nabla h(x^*)y = 0$），不等式 (2.34) 成立。因此，由引理 2.2.8，存在 $\bar{\gamma} > 0$，使得對於所有 $\gamma \in (0, \bar{\gamma})$，有 $\nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma) \succ 0$。

現在考慮固定 $\gamma \in (0, \bar{\gamma})$ 和 λ^* 時的 $l_A(\cdot, \lambda^*; \gamma)$。此時 $l_A(\cdot, \lambda^*; \gamma) : \mathbb{R}^n \to \mathbb{R}$ 是關於 x 的函數。根據前面的推導，我們有 $\nabla_x l_A(\cdot, \lambda^*; \gamma) = 0$ 且 $\nabla_{xx}^2 l_A(\cdot, \lambda^*; \gamma) \succ 0$。因此，$x^*$ 是 $l_A(\cdot, \lambda^*; \gamma)$ 的嚴格局部極小點。此外，

$$\begin{aligned} l_A(x, \lambda^*; \gamma) &= l_A(x^*, \lambda^*; \gamma) + \nabla_x l_A(x^*, \lambda^*; \gamma) \\ &\quad + \frac{1}{2}(x - x^*)^\top \nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma)(x - x^*) + o(|x - x^*|^2) \\ &= f(x^*) + \frac{1}{2}(x - x^*)^\top \nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma)(x - x^*) + o(|x - x^*|^2). \end{aligned}$$

由於 $l_A(x^*, \lambda^*; \gamma) \in C^2$，可知存在 $\alpha > 0$ 和 $r > 0$，使得

$$l_A(x, \lambda^*; \gamma) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$$

對所有 $x \in B_r(x^*)$ 成立。若進一步滿足 $h(x) = 0$，則有 $l_A(x, \lambda^*; \gamma) = f(x)$，上述不等式即化為 $f(x) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$，對所有滿足 $h(x) = 0$ 的 $x \in B_r(x^*)$ 成立。 □

如上所示，二階充分條件（命題 2.2.9）刻畫了嚴格局部極小值點 x^*。因此，我們可以很容易地將其推廣到同時具有等式與不等式約束的優化問題 (2.24)：只需忽略在 x^* 處不活躍的不等式約束（因為在 x^* 處為嚴格不等式），並注意 x^* 在其餘約束上滿足等式條件。因此，可以直接將這些活躍的不等式約束（此時均轉化為等式約束）納入，從而得到與命題 2.2.9 相同的結論。這裡省略具體細節。

2.3 自動微分

在大多數優化算法中，我們都需要計算目標函數的一階和二階導數。一個簡單的方法是使用有限差分。具體地說，設 $f : \mathbb{R}^n \to \mathbb{R}$，則我們可以將偏導數 $\frac{\partial f(x)}{\partial x_i}$ 近似為

$$\frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + \epsilon e_i) - f(x)}{\epsilon}, \tag{2.37}$$

其中 $e_i \in \mathbb{R}^n$ 的第 i 個分量為 1，其餘分量為 0，而 $\epsilon > 0$ 是用戶選取的步長。然而，當 ϵ 較大時，近似 (2.37) 往往不夠準確；而當 ϵ 較小時，又會嚴重受到浮點誤差的影響。因此，這種近似並不適用於諸如網絡訓練這類大規模優化問題。

相比之下，反向傳播（back-propagation）是一種更為優雅的方法，用於高效準確地計算函數的一階和二階導數（也可延伸計算更高階，但實踐中不常見）。反向傳播則是自動微分（automatic differentiation）的一個特例，我們將在本節中詳細討論自動微分。

自動微分是一種基於函數表示和鏈式法則來計算導數解析值的方法。自動微分的基礎在於：任何函數的計算都可以表示為一系列初等運算的複合。所謂初等運算，是指以下這類只接受一個或兩個標量作為變量的函數：

- 單變量運算：sin、cos、tan、exp、log 等。
- 雙變量運算：$+$、$-$、$\times$、$/$ 等。

對於任意函數 f，我們都可以構造一個**計算圖**，利用這些初等運算來表示 f，其中節點表示這些運算的標量輸入與輸出，邊則表示節點之間由所執行運算所決定的關係。

下面的例子說明了如何將一個函數分解為作用於其參數上的這些初等運算，並構造相應的計算圖。

例 2.3.1. 考慮如下函數 $f : \mathbb{R}^3 \to \mathbb{R}$：

$$f(x) = \frac{e^{2x_2} \cos(x_2 x_3)}{x_1 + x_2} \tag{2.38}$$

其中 $x = (x_1, x_2, x_3) \in \mathbb{R}^3$。除了參數節點 x_1、x_2、x_3 外，f 的計算圖還引入

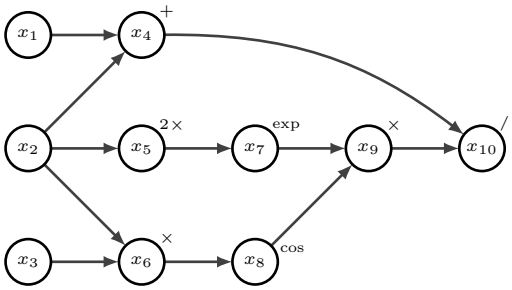


图 2.2: 函數 $f : \mathbb{R}^3 \to \mathbb{R}$（定義見 (2.38)）的計算圖。$f$ 的自變量由節點 x_1、x_2 和 x_3 表示。其餘節點表示中間標量變量。特別地，$x_{10} = f(x_1, x_2, x_3)$。每個節點所執行的初等運算顯示在相應節點的右上角。

了新的節點 $x_4, \ldots, x_{10}$，用於表示中間變量：

$$
\begin{aligned}
x_4 &= x_1 + x_2, \\
x_5 &= 2 \times x_2, \\
x_6 &= x_2 \times x_3, \\
x_7 &= e^{x_5} = e^{2x_2}, \\
x_8 &= \cos(x_6) = \cos(x_2 x_3), \\
x_9 &= x_7 \times x_8 = e^{2x_2} \cos(x_2 x_3), \\
x_{10} &= x_9 / x_4 = e^{2x_2} \cos(x_2 x_3) / (x_1 + x_2).
\end{aligned}
\tag{2.39}
$$

這裡稍微濫用記號，我們之後用 x_i 同時表示節點及該節點處的數值。函數 (2.38) 的計算圖如圖 2.2 所示。之後在本節中，我們將從 x_i 到 x_j 的有向邊記作 (x_i, x_j)。在這種情況下，我們稱 x_i 為 x_j 的**父節點**，而稱 x_j 為 x_i 的**子節點**。

可以看到，(2.39) 中的每個節點都是通過對其一個或兩個父節點施加某種初等運算而得到的。因此，每個節點都有一條或兩條入邊。在圖 2.2 中，每個節點所執行的運算都標註在對應節點的右上角。注意，圖中的運算和連接順序必須與函數 f 的定義正確對應。我們還注意到，計算圖中的所有節點實際上都是 (x_1, x_2, x_3) 的函數。

在計算圖中，一旦 x_i 的所有父節點的值已知，x_i 的值就可以計算出來。如果輸入變量 $x = (x_1, \ldots, x_n)$ 的值已知，我們就可以算出計算圖中所有節點的值。這一按順序進行計算的過程稱為**前向掃描**。

下面我們將說明自動微分如何在第 2.3.1 節中計算一階導數（如梯度和

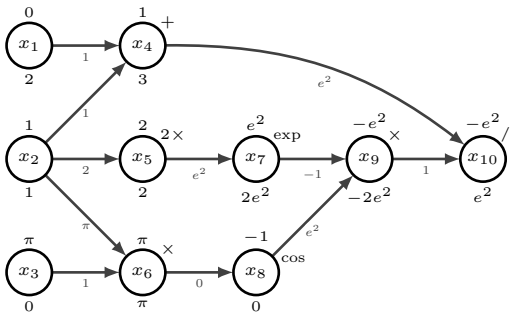


图 2.3: 使用自動微分前向模式計算函數 (2.38) 在 $x = (0, 1, \pi)$ 和 $v = (2, 1, 0)$ 下的 $f(x)$ 與 $D_v f(x)$。x_j 和 $D_v x_j$ 的值分別顯示在對應節點 x_j 的上方和下方。節點 x_j 處所執行的初等運算顯示在其右上角。有向邊 (x_i, x_j) 下方顯示的是 $\frac{\partial x_j}{\partial x_i}$ 的值。在終端節點 x_{10} 處，我們得到 $f(x) = -e^2$ 和 $D_v f(x) = e^2$。

Jacobian 矩陣），以及在第 2.3.2 節中計算二階導數（如 Hessian 矩陣）。無論哪種情況，自動微分都有兩種模式，稱為**前向模式**（forward mode）和**反向模式**（reverse mode），我們將在下文中詳細討論。

2.3.1 一階導數的計算

在本小節中，我們考慮如何使用自動微分計算函數的一階導數。我們首先介紹較為直接且更容易理解的前向模式，然後介紹更常用、也更適合深度學習任務的反向模式。我們還將討論這兩種情況下的計算複雜度和存儲需求。

用前向模式計算梯度與 Jacobian 矩陣

自動微分的前向模式可用於計算方向導數 $D_v f(x)$（當 $f : \mathbb{R}^n \to \mathbb{R}$ 為標量值函數時），或 Jacobian-vector 積 $\nabla f(x)v$（當 $f : \mathbb{R}^n \to \mathbb{R}^m$ 為向量值函數時），其中 x 是定義域中的任意點，$v \in \mathbb{R}^n$ 是任意（列）向量。因此，通過取 $v = e_i$（其中 $i \in [n]$，e_i 是第 i 個分量為 1、其餘分量為 0 的 n 維向量），我們就能得到梯度（或 Jacobian 矩陣）∇f。

我們先考慮 $f : \mathbb{R}^n \to \mathbb{R}$ 為標量值函數的情形。除了輸入變量 $x = (x_1, \ldots, x_n)$ 外，f 的計算圖還會引入中間變量 $x_{n+1}, \ldots, x_N$，它們都是 $x_1, \ldots, x_n$ 的函數。例如，在例 2.3.1 和圖 2.2 中，$N = 10$。

現在我們按照圖 2.2 所示的計算圖，在**前向掃描**中（即圖 2.2 中從左到右）逐步執行這些初等運算。除了數值 x_j 之外，我們還通過鏈式法則計算 $D_v x_j$

的值:

$$D_v x_j = \sum_{x_i \in P_j} \frac{\partial x_j}{\partial x_i} D_v x_i, \tag{2.40}$$

其中 P_j 表示 x_j 的父節點集合。注意，(2.40) 中的求和只包含一項或兩項。我們從給輸入節點 $x_1, \ldots, x_n$ 賦值開始，並注意到對每個 $i \in [n]$，有 $D_v x_i = \langle \nabla x_i, v \rangle = \langle e_i, v \rangle = v_i$。然後我們繼續計算它們的子節點，以及這些子節點處的方向導數。當完成前向掃描時，我們就能在最後一個節點 x_N 處同時得到 $f(x) = x_N$ 和 $D_v f(x) = D_v x_N$。圖 2.3 展示了在輸入 $x = (x_1, x_2, x_3) = (0, 1, \pi)$ 和 $v = (2, 1, 0)$ 時，函數 (2.38) 的中間變量的取值。在圖 2.3 中，x_j 與 $D_v x_j$ 的值分別顯示在節點 x_j 的上方和下方。有向邊 (x_i, x_j) 下方顯示偏導數 $\frac{\partial x_j}{\partial x_i}$ 的值。每個中間變量 x_j 處所執行的初等運算顯示在其右上角，以便參考。例如，給定 $(x_7, D_v x_7) = (e^2, 2e^2)$（這兩個值分別顯示在節點 x_7 的上方與下方）以及 $(x_8, D_v x_8) = (-1, 0)$，我們得到

$$x_9 = x_7 \cdot x_8 = e^2 \cdot (-1) = -e^2$$

以及

$$D_v x_9 = \frac{\partial x_9}{\partial x_7} D_v x_7 + \frac{\partial x_9}{\partial x_8} D_v x_8 = (-1) \cdot (2e^2) + e^2 \cdot 0 = -2e^2.$$

對於上述給定的 x 和 v，最終我們得到 $x_{10} = -e^2$ 且 $D_v x_{10} = e^2$。手工計算也給出同樣的結果：$\nabla f(x) = (e^2, -e^2, 0)$，從而驗證了 $D_v f(x) = \langle \nabla f(x), v \rangle = e^2$。

如果 $f = (f_1, \ldots, f_m) : \mathbb{R}^n \to \mathbb{R}^m$ 是向量值函數，則其計算圖與圖 2.2 類似，只不過終端節點有 m 個而不是一個。我們將在這些終端節點處得到 $(D_v f_1(x), \cdots, D_v f_m(x))$ 的值，從而構成所需的 Jacobian-vector 積

$$\nabla f(x) v = \begin{pmatrix} D_v f_1(x) \\ \vdots \\ D_v f_m(x) \end{pmatrix}.$$

此時，計算成本將是標量值函數情形的 m 倍。

前向模式的優點在於，一旦節點 x_j 的所有子節點的計算都已完成，用於存儲 $(x_j, D_v x_j)$ 的計算機內存就可以釋放。由於計算圖通常非常大，這在實踐中可能是有益的。它的缺點是，如果需要完整的梯度或 Jacobian 矩陣，那麼我們必須依次取 $v = e_i$（$i \in [n]$）。因此，總計算成本將是上述方向導數（或 Jacobian-vector 積）成本的 n 倍，這在 n 非常大時，計算量很高。接下來我

們將說明，自動微分的反向模式可以解決這一問題。

用反向模式計算梯度與 Jacobian 矩陣

我們先直接討論如何計算標量值函數 $f:\mathbb{R}^n\to\mathbb{R}$ 的梯度 $\nabla f(x)$。（對於任意給定的 v，方向導數 $D_v f(x)=\langle\nabla f(x),v\rangle$ 可在之後通過向量內積輕鬆得到。）反向模式與前向模式的主要區別在於：反向模式在前向掃描中除了 x_j 的值之外，不會額外計算其他量。相反，一旦前向掃描完成，自動微分的反向模式就從終端節點 x_N 開始，沿有向邊的反方向依次計算每個節點上的值 $y_j:=\frac{\partial f(x)}{\partial x_j}$，直到輸入變量 $x=(x_1,\ldots,x_n)$ 中所有節點對應的 y_i 都被計算出來。此時，梯度 $\nabla f(x)=(y_1,\ldots,y_n)$ 就得到了。這個過程稱為**反向掃描**，下面我們詳細解釋。

開始反向掃描時，我們注意到 $y_N=\frac{\partial f(x)}{\partial x_N}=1$，因為 $f(x)=x_N$。然後，對於任意節點 x_j，應用鏈式法則得到

$$y_j=\frac{\partial f(x)}{\partial x_j}=\sum_{x_i\in C_j}\frac{\partial f(x)}{\partial x_i}\frac{\partial x_i}{\partial x_j}=\sum_{x_i\in C_j}y_i\frac{\partial x_i}{\partial x_j},$$

其中 C_j 表示 x_j 的所有子節點集合。注意，C_j 可能包含兩個以上的節點，這取決於 f 的定義；這與 (2.40) 中的情況不同。圖 2.4 展示了在輸入 $x=(x_1,x_2,x_3)=(0,1,\pi)$ 和 $v=(2,1,0)$ 時，函數 (2.38) 的節點值 x_j（黑色，位於節點 x_j 上方）和 y_j（灰色，位於節點 x_j 下方）。在圖 2.4 中，x_j 的值是在前向掃描中計算得到的；而每個節點 x_j 上的 y_j 以及每條有向邊 (x_j,x_i) 上的 $\frac{\partial x_i}{\partial x_j}$ 的值，則是在反向掃描中計算得到的。每個中間變量 x_j 所執行的初等運算再次顯示在其右上角，以便參考。例如，已知 $y_4=e^2$、$y_5=-e^2$ 和 $y_6=0$，則我們得到

$$y_2=y_4\frac{\partial x_4}{\partial x_2}+y_5\frac{\partial x_5}{\partial x_2}+y_6\frac{\partial x_6}{\partial x_2}=e^2\cdot 1+(-e^2)\cdot 2+0\cdot\pi=-e^2.$$

當反向掃描完成後，我們得到 $(y_1,y_2,y_3)=(e^2,-e^2,0)$，與前面在 $x=(0,1,\pi)$ 處給出的梯度 $\nabla f(x)$ 一致。

如果 $f=(f_1,\ldots,f_m):\mathbb{R}^n\to\mathbb{R}^m$ 是向量值函數，而我們希望對某個給定的 $u\in\mathbb{R}^m$ 計算 **vector-Jacobian** 積 $u\nabla f(x)\in\mathbb{R}^n$，其中二者都是行向量[1]，

[1]注意這與對於給定 $v\in\mathbb{R}^n$ 的 Jacobian-vector 積 $\nabla f(x)v$ 不同，v 和 $\nabla f(x)v$ 都是列向量。

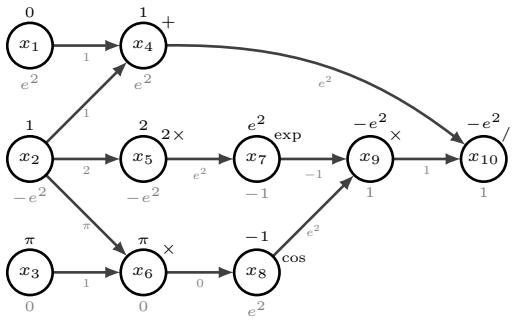


图 2.4: 使用自動微分反向模式計算函數 (2.38) 在 $x = (0, 1, \pi)$ 處的 $\nabla f(x)$。x_j（黑色）和 $y_j = \frac{\partial f(x)}{\partial x_j}$（灰色）的值分別顯示在對應節點 x_j 的上方和下方。節點 x_j 所執行的初等運算顯示在其右上角。有向邊 (x_j, x_i) 下方顯示的是 $\frac{\partial x_i}{\partial x_j}$（灰色）的值。黑色數值在前向掃描中計算，灰色數值在反向掃描中計算。最終結果為 $\nabla f(x) = (e^2, e^{-2}, 0)$。

那麼我們只需定義

$$\bar{f} := \sum_{i=1}^{m} u_i f_i \ : \ \mathbb{R}^n \to \mathbb{R},$$

這就是一個標量值函數，然後像上面那樣使用自動微分的反向模式即可。如果我們希望計算完整的 Jacobian 矩陣 $\nabla f(x)$，那麼就需要在反向掃描過程中，對每個節點 x_j 計算一個（列）向量 $y_j = (y_j^1, \ldots, y_j^m) \in \mathbb{R}^m$，其中對 $k = 1, \ldots, m$，$y_j^k = \frac{\partial f_k(x)}{\partial x_j}$。在反向掃描完成之後，我們得到完整的 Jacobian 矩陣

$$\nabla f(x) = \begin{pmatrix} \nabla f_1(x) \\ \vdots \\ \nabla f_m(x) \end{pmatrix} = \Big(y_1, \ldots, y_n \Big) \in \mathbb{R}^{m \times n}.$$

反向模式的優點在於，計算完整梯度的成本只是求值一個標量值函數 $f : \mathbb{R}^n \to \mathbb{R}$ 所需成本的一個較小常數倍（通常約為 4 到 5 倍）。相比之下，前向模式則需要大約 $O(n)$ 倍於函數求值的計算成本。注意，深度學習領域中所謂的反向傳播，其實正是自動微分反向模式的一個特例。不過，對於向量值函數 $f : \mathbb{R}^n \to \mathbb{R}^m$，隨著 m 的增大，使用前向模式和反向模式計算完整 Jacobian 矩陣 $\nabla f(x)$ 的成本會變得相近。反向模式的一個問題是，前向掃描中計算出的所有值都必須存儲下來，而且用於存儲 y_j 的內存，直到反向掃描中 x_j 的所有父節點都使用完它之後才能釋放。因此，在實際中，自動微分反向模式對內存的需求可能更高。

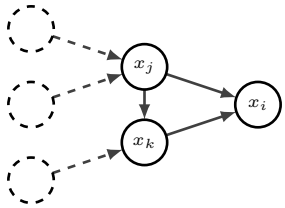


图 2.5: 在前向模式中計算 $D_{uv}f(x)$ 時，與節點 x_i 處量 (2.41) 的計算有關的子圖 (取自 f 的計算圖)。注意，每個節點最多可有兩個父節點。這裡展示的子圖是最複雜的情形: x_i 有兩個父節點 x_j 和 x_k，且 x_j 還是 x_k 的父節點。如果 x_i 只有一個父節點，或者兩個父節點 x_j 和 x_k 彼此不是父節點，那麼 (2.41) 的計算與這裡類似，但更簡單。

2.3.2 二階導數的計算

我們沿用第 2.3.2 節中的相同思路，在本小節中依次考慮如何通過前向模式和反向模式來計算二階導數。

用前向模式計算 Hessian

設 $f:\mathbb{R}^n \to \mathbb{R}$ 是一個標量值、二次連續可微的函數。此時，f 的 Hessian 是一個 $n\times n$ 對稱矩陣 $\nabla^2 f(x) = [\partial_{ij} f(x)]$, 其中對 $i,j \in [n]$, $\partial_{ij} f(x) = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}$。我們首先考慮如何計算

$$D_{uv}x_j := u^\top(\nabla^2 x_j)v \in \mathbb{R}$$

對於計算圖中每個節點 x_j 的值，其中給定任意列向量 $u,v \in \mathbb{R}^n$。與前向模式計算梯度類似，我們仍然只在 f 的計算圖上執行一次前向掃描。不同之處在於，在前向掃描過程中，我們還需要在每個節點 x_j 處計算若干二階偏導數。當整個前向掃描完成時，我們就在終端節點 x_N 處得到了所需的值 $u^\top \nabla^2 f(x) v = u^\top(\nabla^2 x_N)v$。

更具體地說，在 f 的計算圖中的每個節點 x_i 處，我們需要計算以下量：

$$
\begin{aligned}
&x_i,\ \underbrace{\frac{\partial x_i}{\partial x_j},\ \frac{\partial x_i}{\partial x_k},\ D_u x_i,\ D_v x_i,}_{\text{4 个一阶项}} \\
&\underbrace{\frac{\partial^2 x_i}{\partial x_j^2},\ \frac{\partial^2 x_i}{\partial x_k \partial x_j},\ \frac{\partial^2 x_i}{\partial x_k^2},\ D_u\Big(\frac{\partial x_i}{\partial x_j}\Big),\ D_u\Big(\frac{\partial x_i}{\partial x_k}\Big),\ D_v\Big(\frac{\partial x_i}{\partial x_j}\Big),\ D_v\Big(\frac{\partial x_i}{\partial x_k}\Big),\ D_{uv} x_i}_{\text{8 个二阶项}}.
\end{aligned}
\tag{2.41}
$$

回憶一下，對於每個 $i = n+1, \ldots, N$，節點 x_i 只有一個或兩個父節點。不失一般性，我們考慮最複雜的情況：x_i 有兩個父節點 x_j 和 x_k，並且 x_j 還是 x_k 的父節點。如果 x_i 只有一個父節點，或者它有兩個父節點 x_j 和 x_k 但二者互不為父節點，則推導是類似且更簡單的。我們所考慮情形下，f 的計算圖對應的子圖如圖 2.5 所示。為了計算 (2.41) 中的量，我們注意到，這些量在前向掃描過程中已經在父節點 x_j 和 x_k 處被計算並存儲了。由於 x_i 是 x_j 和 x_k 的某個給定初等函數（如 (2.39) 中那些形式），我們可以直接計算

$$
x_i,\ \frac{\partial x_i}{\partial x_j},\ \frac{\partial x_i}{\partial x_k},\ \frac{\partial^2 x_i}{\partial x_j^2},\ \frac{\partial^2 x_i}{\partial x_k \partial x_j},\ \frac{\partial^2 x_i}{\partial x_k^2}.
$$

此外，由鏈式法則可得

$$
\begin{aligned}
D_u x_i &= \frac{\partial x_i}{\partial x_j} D_u x_j + \frac{\partial x_i}{\partial x_k} D_u x_k, \\
D_v x_i &= \frac{\partial x_i}{\partial x_j} D_v x_j + \frac{\partial x_i}{\partial x_k} D_v x_k,
\end{aligned}
$$

因為右端的各項都已知。然後我們有

$$
\begin{aligned}
D_u\Big(\frac{\partial x_i}{\partial x_j}\Big) &= \frac{\partial}{\partial x_j}(D_u x_i) \\
&= \frac{\partial}{\partial x_j}\Big(\frac{\partial x_i}{\partial x_j} D_u x_j + \frac{\partial x_i}{\partial x_k} D_u x_k\Big) \\
&= \frac{\partial^2 x_i}{\partial x_j^2} D_u x_j + \frac{\partial x_i}{\partial x_j}\frac{\partial}{\partial x_j}(D_u x_j) + \frac{\partial^2 x_i}{\partial x_k \partial x_j} D_u x_k + \frac{\partial x_i}{\partial x_j}\frac{\partial}{\partial x_j}(D_u x_k).
\end{aligned}
\tag{2.42}
$$

注意到 $\frac{\partial}{\partial x_j}(D_u x_j) = 0$，而 $\frac{\partial}{\partial x_j}(D_u x_k) = D_u(\frac{\partial x_k}{\partial x_j})$ 在我們計算 x_k 處的量 (2.41) 時已經得到（若 x_j 不是 x_k 的父節點，則該值為 0）。因此，(2.42) 中所有項都是已知的，從而我們可以計算 $D_u(\frac{\partial x_i}{\partial x_j})$。類似地，$D_v(\frac{\partial x_i}{\partial x_j})$、$D_u(\frac{\partial x_i}{\partial x_k})$ 和

$D_v(\frac{\partial x_i}{\partial x_k})$ 也都可以被計算出來。最後，我們有

$$\begin{aligned}
D_{uv}x_i &= D_v(D_u x_i) \\
&= D_v\Big(\frac{\partial x_i}{\partial x_j}D_u x_j + \frac{\partial x_i}{\partial x_k}D_u x_k\Big) \\
&= D_v\Big(\frac{\partial x_i}{\partial x_j}\Big)D_u x_j + \frac{\partial x_i}{\partial x_j}D_{uv}x_j + D_v\Big(\frac{\partial x_i}{\partial x_k}\Big)D_u x_k + \frac{\partial x_i}{\partial x_k}D_{uv}x_k,
\end{aligned}$$

其中右端所有項此時都已知，因此 $D_{uv}x_i$ 也就可以被計算出來。這樣，我們就完成了 x_j 處 (2.41) 中所有量的計算，並可以繼續到下一個節點。需要指出的是，實際計算成本往往比上述公式表面上看起來更低，因為許多項可能為零；這是由於 x_i 僅僅是由 x_j 和 x_k 通過某個初等運算得到的。我們按此方式完成整個前向掃描，並在終端節點得到所需的量 $D_{uv}x_N = u^\top \nabla^2 f(x)v$。

我們說明一下，上述前向模式也可用於計算曲線 $c(t) := f(x+tv)$ 在 $t=0$ 處的曲率：對任意給定的 $x, v \in \mathbb{R}^n$，有 $c(0) = f(x)$、$c'(0) = D_v f(x)$，以及

$$c''(0) = v^\top \nabla^2 f(x) v = D_{vv} f(x).$$

如果我們希望計算整個 Hessian $\nabla^2 f(x)$，則可以對所有 $i \le j \le n$ 且 $1 \le i \le n$，取 $u = e_i$ 與 $v = e_j$ 來得到 Hessian 的上三角部分。由於當 $f \in C^2$ 時 $\nabla^2 f(x)$ 是對稱的，我們便得到完整的 Hessian。這需要對 $\frac{1}{2}n(n+1)$ 組不同的 $(u, v) = (e_i, e_j)$ 進行計算。如果 Hessian 是稀疏的，且非零項的位置已知，那麼所需計算的 (u, v) 對數可以顯著減少，計算成本也可相應降低。

如果我們只想計算某個給定向量 $v \in \mathbb{R}^n$ 的 Hessian-向量積 $\nabla^2 f(x)v$，那麼在 $u^\top \nabla^2 f(x)v$ 中只需讓 $u = e_i$ 對所有 $i \in [n]$ 變化即可。此時總計算成本降為 $O(n)$，但當 n 很大時仍然較高。接下來，我們將說明反向模式如何解決這一問題。

用反向模式計算 Hessian

我們直接考慮如何對 $f \in C^2(\mathbb{R}^n; \mathbb{R})$ 和任意向量 $v \in \mathbb{R}^n$ 計算 Hessian-向量積 $\nabla^2 f(x)v$。首先，我們進行前向掃描，在 f 的計算圖中每個節點 x_j 處計算 x_j 和 $D_v x_j$ 的值。在終端節點 x_N 處，我們同時得到 $f(x) = x_N$ 和 $D_v f(x) = D_v x_N$ 的值。然後我們進行反向掃描，並在每個節點 x_j 處計算

$z_j := \frac{\partial}{\partial x_j}(D_v f(x))$。注意到

$$z_N = \frac{\partial}{\partial x_N}(D_v f(x)) = D_v\Big(\frac{\partial f(x)}{\partial x_N}\Big) = 0$$

因為 $x_N = f(x)$。然後，由鏈式法則可知

$$z_j = \frac{\partial}{\partial x_j}(D_v f(x)) = \sum_{x_j \in C_j} \frac{\partial}{\partial x_i}(D_v f(x))\frac{\partial x_i}{\partial x_j} = \sum_{x_i \in C_j} z_i \frac{\partial x_i}{\partial x_j}, \tag{2.43}$$

其中 x_j 的每個子節點 x_i 對應的 z_i 在此時都已被計算出來。當反向掃描結束後，我們就在輸入變量 $x = (x_1, \ldots, x_n)$ 對應的每個節點 x_j 處得到

$$z_j = \frac{\partial}{\partial x_j}(D_v f(x)) = \frac{\partial}{\partial x_j}\Big(\sum_{i=1}^n v_i \partial_i f(x)\Big) = \sum_{i=1}^n v_i \partial_{ij}^2 f(x)$$

的值。於是有

$$\nabla^2 f(x) v = \begin{pmatrix} \partial_{11}^2 f(x) & \cdots & \partial_{1n}^2 f(x) \\ \vdots & \ddots & \vdots \\ \partial_{n1}^2 f(x) & \cdots & \partial_{nn}^2 f(x) \end{pmatrix} \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix} = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} \in \mathbb{R}^n.$$

按照上述反向模式計算 $\nabla^2 f(x) v$ 的成本，只是計算 f 本身成本的一個常數倍（約為 12 倍）。這顯著低於使用前向模式完成同一任務時所需的 $O(n)$ 成本。唯一的問題是，反向模式要求存儲前向掃描中計算出的所有 x_j，並且在反向掃描中，直到 z_j 被 x_j 的所有父節點使用完之前，存儲 z_j 的內存都不能釋放。

如果需要完整的 Hessian $\nabla^2 f(x)$，則可以對所有 $i \in [n]$ 取 $v = e_i$，並重覆上述過程。總計算成本為 $O(n)$。同樣地，如果已知 Hessian 的稀疏結構，則這一成本還可以進一步降低。

2.4 確定性優化算法

在本節中，我們介紹若干具有代表性的確定性優化（deterministic optimization）算法，即目標函數具有解析表達式，不包含隨機性或期望，其各階導數都可以被精確計算（例如通過自動微分）。這一點可以通過對比下面的確定性優化 (2.44) 與隨機優化 (2.101) 更加清晰地理解。

確定性優化的一般形式寫作

$$\min_{x\in\Omega} \ f(x), \tag{2.44}$$

其中 Ω 是 $\mathbb{R}^n$ 的一個閉子集，目標函數 $f:\mathbb{R}^n\to\mathbb{R}$ 可能是非凸的。我們假設 f 可微，且其梯度 ∇f 滿足 Lipschitz 連續性，具有某個 Lipschitz 常數 $L>0$（在優化領域中通常稱為“光滑性”）。需要注意的是，我們通常假設 f 在整個 $\mathbb{R}^n$ 上是 Lipschitz 的，但這一假設往往可以放寬為在某個有界感興趣區域內 Lipschitz，這一點我們將在後續推導中說明。這一假設較為常見，並且在深度學習中遇到的大多數問題中都成立。

在光滑性假設下，我們有如下重要不等式，它在優化算法的收斂性證明中被頻繁使用。

命題 2.4.1. *設 $f:\mathbb{R}^n\to\mathbb{R}$ 連續可微，且 ∇f 是 Lipschitz 連續的，Lipschitz 常數為 $L>0$，則對所有 $x,y\in\mathbb{R}^n$ 有*

$$f(y)\le f(x)+\langle\nabla f(x),y-x\rangle+\frac{L}{2}|y-x|^2. \tag{2.45}$$

证明. 對任意 $x,y\in\mathbb{R}^n$，定義函數 $\phi:\mathbb{R}\to\mathbb{R}$:

$$\phi(t)=f((1-t)x+ty).$$

由於 f 可微，有

$$\phi(1)=\phi(0)+\int_0^1\phi'(t)\,dt,$$

從而得到

$$f(y)=f(x)+\int_0^1\langle\nabla f((1-t)x+ty),y-x\rangle\,dt.$$

兩邊同時減去 $f(x)+\langle f(x),y-x\rangle$，得到

$$\begin{aligned}
f(y)-f(x)-\langle\nabla f(x),y-x\rangle &= \int_0^1\langle\nabla f((1-t)x+ty)-\nabla f(x),y-x\rangle\,dt\\
&\le \int_0^1|\nabla f((1-t)x+ty)-\nabla f(x)||y-x|\,dt\\
&\le L|y-x|^2\int_0^1 t\,dt\\
&= \frac{L}{2}|y-x|^2,
\end{aligned}$$

其中第二個不等式是由於 ∇f 的 Lipschitz 連續性。 □

2.4.1 無約束優化算法

如果優化問題 (2.44) 是無約束的，即 $\Omega = \mathbb{R}^n$，那麼我們的主要任務是根據一階必要條件找到 f 的臨界點。為此，我們考慮如下迭代格式：

$$x_{k+1} = x_k + \alpha_k v_k, \tag{2.46}$$

其中 $k = 0, 1, 2, \ldots$ 稱為**迭代計數器**（iteration counter），$\alpha_k > 0$ 為**步長**（step size）（在深度學習中通常稱為**學習率**，learning rate），而 v_k 是第 k 次迭代中在 x_k 處的下降方向。這裡所謂下降方向是指 v_k 滿足 $\langle \nabla f(x_k), v_k \rangle < 0$（我們將在本節後面詳細討論下降方向）。在如 (2.46) 所示的優化算法設計中，關鍵在於選擇合適的 v_k 和 α_k，使得由 (2.46) 生成的序列 $\{x_k\}$ 滿足 $|\nabla f(x_k)| \to 0$（當 $k \to \infty$ 時）。

梯度下降法與線搜索

我們首先介紹梯度下降法，並結合一種自動步長選擇策略——回溯線搜索（backtracking line search），以說明理想中的優化算法的特性及實際中潛在的問題。

在梯度下降法中，最簡單的一步是確定下降方向 v_k。我們暫時假設 v_k 是單位向量，因為此時我們的目標是確定方向。特別地，我們希望找到 v_k 使得方向導數 $D_{v_k} f(x_k)$ 為負且儘可能小，從而在該方向上實現最快下降。由此可得：

$$\begin{aligned} v_k &= \arg\min_{|v|=1} D_v f(x_k) \\ &= \arg\min_{|v|=1} \langle \nabla f(x_k), v \rangle \\ &= -\nabla f(x_k)/|\nabla f(x_k)|. \end{aligned}$$

因此，我們可以直接取 $v_k = -\nabla f(x_k)$，並將尺度調整交由步長 α_k 處理（後文將詳細說明）。由於採用負梯度作為下降方向，迭代格式為

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k), \tag{2.47}$$

該迭代方法稱為**梯度下降法**（gradient descent method）。

需要指出的是，在梯度下降法 (2.47) 中選擇 $v_k = -\nabla f(x_k)$ 只是為了保證 v_k 是下降方向，從而簡化討論。實際上，只要存在常數 $c > 0$ 且對所有 k 有 $\nabla f(x_k)v_k \le -c|\nabla f(x_k)|^2$，則任意這樣的 v_k 都可以使用，我們後續推導仍然成立。

在 (2.46) 和 (2.47) 中選擇合適的步長 α_k 非常重要，因為它直接影響算法的收斂性與穩定性。較大的步長可以帶來更快的進展，但可能導致序列 $\{x_k\}$ 發散或劇烈震盪；較小的步長則會使迭代進展緩慢，導致算法效率低下。

為了在第 k 次迭代中確定合適的步長，一個自然的想法是尋找使 $f(x_k + \alpha v_k)$ 在 $\alpha \ge 0$ 上最小的 α。為此，我們定義線搜索函數 $\phi_k(\alpha) := f(x_k + \alpha v_k)$，並求解一維優化問題 $\alpha_k = \arg\min_{\alpha \ge 0} \phi_k(\alpha)$。採用這種步長選擇的梯度下降法稱為**最速下降法**（steepest descent method）。然而，即使 ϕ_k 是一維函數，其最小值的求解仍然需要多次計算 f 和 ∇f，因此並不容易實現。

另一種常見方法是使用常數步長 $\alpha > 0$，即 $\alpha_k = \alpha$。事實上可以證明，當 $\alpha \in (0, 2/L)$ 時，由 (2.47) 生成的序列滿足 $|\nabla f(x_k)| \to 0$。但在實踐中，L 往往未知且難以估計，因此難以確定合適的 α。

為了解決這一問題，人們提出了多種自動選擇步長的策略。這裡我們介紹一種廣泛使用的方法——**回溯線搜索**。其目標是找到滿足如下 **Armijo 條件**的步長 α_k（其中 $c \in (0,1)$ 為給定常數）：

$$f(x_k + \alpha_k v_k) \le f(x_k) + c\alpha_k \nabla f(x_k) v_k. \tag{2.48}$$

在梯度下降法中，$v_k = -\nabla f(x_k)$，因此最後一項變為 $-c\alpha_k |v_k|^2$。

一般而言，只要 v_k 是下降方向，就有 $\nabla f(x_k)v_k < 0$，從而 (2.48) 蘊含 $f(x_k + \alpha_k v_k) \le f(x_k)$。因此，該條件也稱為**充分下降條件**。從另一角度看，利用 $\phi_k(\alpha) = f(x_k + \alpha v_k)$，Armijo 條件 (2.48) 等價於

$$\phi_k(\alpha) \le \phi_k(0) + c\alpha\phi_k'(0). \tag{2.49}$$

右側是通過點 $(0, \phi_k(0))$ 的一條直線，其斜率為 $c\phi_k'(0) < 0$。由於 $f(x)$ 有下界 f^*，該直線必然在某個 $\hat{\alpha} > 0$ 處與 $\phi(\alpha)$ 相交，因此 α_k 應取在 0 與 $\hat{\alpha}$ 之間。因此，Armijo 條件既保證目標函數值下降，又避免步長過大。

從 (2.49) 可知，當 α 足夠小時條件必然成立，但過小的步長會導致收斂緩慢。因此，還常加入**曲率條件**：

$$\phi_k'(\alpha) \ge \hat{c}\phi_k'(0) \tag{2.50}$$

其中 $\hat{c} \in (c, 1)$。該條件防止 α_k 過小。Armijo 條件與曲率條件的組合稱為 **Wolfe 條件**。

接下來介紹回溯線搜索的實現方法。

在回溯線搜索中，我們預先設定步長上界 $\bar{\alpha} > 0$ 和收縮因子 $\rho \in (0, 1)$。在第 k 次迭代中，設 $\alpha_k = \bar{\alpha}\rho^{l_k}$，其中

$$l_k = \arg\min \Big\{ l \in \{0, 1, \dots\} : f(x_k + \bar{\alpha}\rho^l v_k) \le f(x_k) - c\bar{\alpha}\rho^l |v_k|^2 \Big\}. \tag{2.51}$$

具體實現時，從 $\alpha_k = \bar{\alpha}$ 開始，檢查 Armijo 條件：

$$f(x_k + \alpha_k v_k) \le f(x_k) - c\alpha_k |v_k|^2. \tag{2.52}$$

若滿足，則接受該 α_k；否則令 $\alpha_k \leftarrow \rho\alpha_k$ 並重覆上述過程。直到條件滿足為止。該算法總結見算法 2.4.1。

Algorithm 2.4.1 帶回溯線搜索的梯度下降法

Input: $\bar{\alpha}, \epsilon_{\text{tol}} > 0$, $\rho, c \in (0, 1)$, $x_0 \in \mathbb{R}^n$, $k = 0$

1: **repeat**
2: $\quad v_k \leftarrow -\nabla f(x_k)$
3: $\quad \alpha_k \leftarrow \bar{\alpha}$
4: $\quad$ **while** $f(x_k + \alpha_k v_k) > f(x_k) - c\alpha_k |v_k|^2$ **do**
5: $\quad\quad \alpha_k \leftarrow \rho\alpha_k$
6: $\quad$ **end while**
7: $\quad x_k \leftarrow x_k + \alpha_k v_k$
8: $\quad k \leftarrow k + 1$
9: **until** $|v_k| < \epsilon_{\text{tol}}$

Output: x_k

現在，對於算法 2.4.1 仍然有幾個問題尚待回答：(i) 線搜索（第 4–6 行）能否在有限步內終止（如果能，其搜索次數上界是多少）？(ii) 終止判據（第 9 行）能否在有限的 k 下滿足（如果能，需要多大的 k）？

我們分別利用下面的引理和定理來回答上述問題 (i) 和 (ii)。

引理 2.4.2. *設 $f : \mathbb{R}^n \to \mathbb{R}$ 屬於 C^1，且 ∇f 在子水平集 $\{x \in \mathbb{R}^n : f(x) \le f(x_0)\}$ 上關於某個常數 $L > 0$ 是 Lipschitz 連續的，其中 $x_0 \in \mathbb{R}^n$ 是任意初始點。對任意 $\bar{\alpha} > 0$ 以及 $\rho, c \in (0, 1)$，記*

$$\bar{l} := \left\lceil \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log \rho} \right\rceil. \tag{2.53}$$

則在算法 2.4.1 的任意一次迭代 k 中，線搜索（第 4–6 行）的次數至多為 $\bar{l}$。即，對所有 k，都有 $l_k \le \bar{l}$，其中 l_k 由 (2.51) 定義。

证明. 我們採用反證法。若 $l_k > \bar{l}$，則有

$$f(x_k) - c\alpha|v_k|^2 < f(x_k + \alpha v_k) \tag{2.54}$$

其中 $\alpha := \bar{\alpha}\rho^{\bar{l}}$。由 (2.53) 中 $\bar{l}$ 的定義可知

$$\bar{l} \ge \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log\rho},$$

而由於 $\rho \in (0,1)$，故有

$$\alpha = \bar{\alpha}\rho^{\bar{l}} \le \frac{2(1-c)}{L}.$$

另一方面，由 (2.45) 可得

$$\begin{aligned} f(x_k + \alpha v_k) &\le f(x_k) + \langle \nabla f(x_k), \alpha v_k\rangle + \frac{L\alpha^2}{2}|v_k|^2 \\ &= f(x_k) - \alpha\Big(1 - \frac{L\alpha}{2}\Big)|v_k|^2 \\ &\le f(x_k) - c\alpha|v_k|^2, \end{aligned}$$

這與 (2.54) 矛盾。 □

引理 2.4.2 表明:每次迭代所需的線搜索次數有一個一致上界 $\bar{l} = O(\log L)$。注意，線搜索中每次判定準則檢查（算法 2.4.1 的第 4 行）都需要在候選點 $x_k + \alpha_k v_k$ 處計算一次 f。

接下來，我們證明：由算法 2.4.1 生成的序列 $\{x_k\}$ 對任意初始點 x_0 都滿足當 $k \to \infty$ 時 $|\nabla f(x_k)| \to 0$。

定理 2.4.3 (算法 2.4.1 的收斂性)**.** *對任意 $x_0 \in \mathbb{R}^n$、$\bar{\alpha} > 0$ 以及 $\rho, c \in (0,1)$，由算法 2.4.1 生成的序列 $\{x_k\}$ 滿足*

1. *級數 $|\nabla f(x_k)|^2$ 是有界的，即*

$$\sum_{k=0}^{\infty} |\nabla f(x_k)|^2 < \infty\ . \tag{2.55}$$

因此，當 $k \to \infty$ 有 $|\nabla f(x_k)| \to 0$。

2. 對任意 $K > 1$，有

$$\min_{0\le k\le K-1} |\nabla f(x_k)|^2 \le \frac{L(f(x_0)-f^*)}{2c(1-c)\rho K} = O\Big(\frac{1}{K}\Big). \tag{2.56}$$

证明. 由於回溯線搜索得到的步長 α_k 滿足 Armijo 條件 (2.48)，因此有

$$c\alpha_k|\nabla f(x_k)|^2 \le f(x_k) - f(x_{k+1})$$

對 $k = 0, 1, \ldots, K-1$ 求和，並注意到 $\alpha_k = \bar{\alpha}\rho^{l_k} \ge \bar{\alpha}\rho^{\bar{l}}$，可得

$$c\bar{\alpha}\rho^{\bar{l}} \sum_{k=0}^{K-1} |\nabla f(x_k)|^2 \le f(x_0) - f(x_K) \le f(x_0) - f^* < \infty \tag{2.57}$$

因為 $f(x_K) \ge f^* > -\infty$。這就證明了 (2.55) 以及第一條結論。

由 (2.53) 中 $\bar{l}$ 的定義可知

$$\bar{l} - 1 \le \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log\rho}$$

從而

$$\rho^{\bar{l}-1} \ge \frac{2(1-c)}{\bar{\alpha}L}.$$

因此，由 (2.57) 可得

$$\min_{0\le k\le K-1} |\nabla f(x_k)|^2 \le \frac{1}{K}\sum_{k=0}^{K-1} |\nabla f(x_k)|^2 \le \frac{L(f(x_0)-f^*)}{2c(1-c)\rho K}. \tag{2.58}$$

這就證明了第二條結論。 □

定理 2.4.3 使我們確信：由算法 2.4.1 生成的迭代序列 $\{x_k\}$ 以次線性速率 $O(1/k)$ 收斂到某個臨界點。對於一般的光滑非凸優化問題，這一結果通常被認為是最好的收斂性結果之一。

與此同時，該算法本身並不要求對問題有太多先驗瞭解：我們無需為步長的選取而苦惱，因為線搜索會自動找到合適的步長。此外，只要超參數（即算法中需要手動設定的參數）取值落在規定範圍內，收斂性就能得到保證，而這些範圍與具體問題無關。相比之下，回憶一下，如果使用常數步長 $\alpha_k \equiv \alpha \in (0, 2/L)$，則必須知道 ∇f 的 Lipschitz 常數 L，而這通常難以估計，並且會隨著不同的 f 而變化。

儘管算法 2.4.1 具有上述令人滿意的性質，但我們往往更關心優化算法在實踐中的效率。原因在於，從實踐角度看，即使犧牲一些超參數選擇的自由度或收斂速率的理論保證，只要能夠獲得顯著更快的經驗收斂速度（從而減少求解所需的時間、資源和能耗），通常也是值得的。我們知道，當目標函數 f 是病態的（ill-conditioned）時，即使 f 是強凸的（此時條件數 L/μ 很大，其中 μ 是 f 的強凸參數），梯度下降法的收斂也會非常緩慢。因此，人們投入了大量工作來構造更高效的優化算法，尤其是針對大規模優化問題。接下來，我們考慮若干利用目標函數二階信息來緩解優化問題病態性的優化方法。

Newton 法與擬 Newton 法

梯度下降法易於實現，因為其下降方向 v_k 只需在每個迭代點 x_k 處取為負梯度 $-\nabla f(x_k)$。然而，即使步長選取得像最速下降法那樣，梯度下降法的收斂仍然可能非常慢。這主要是由於目標函數 f 的 Hessian 矩陣的病態性所致。現在我們考慮其另一個極端—— **Newton 法**。Newton 法在結構上比大多數下降法更復雜，但從收斂速度的角度來看，它是最快的優化方法之一。

Newton 法的迭代仍具有典型下降法的形式：

$$x_{k+1} = x_k + \alpha_k v_k^N, \tag{2.59}$$

其中關鍵在於 **Newton 方向**的選擇，記為：

$$v_k^N := -(\nabla^2 f(x_k))^{-1} \nabla f(x_k). \tag{2.60}$$

Newton 方向的思想是：更新點 x_{k+1} 應當是原目標函數在 x_k 處二階近似的極小點。我們考慮如下近似：

$$f(x) \approx \tilde{f}_k(x) := f(x_k) + \nabla f(x_k)(x - x_k) + \frac{1}{2}(x - x_k)^\top \nabla^2 f(x_k)(x - x_k).$$

對上述 $\tilde{f}_k$ 關於 x 求梯度可見，只要 $\nabla^2 f(x_k)$ 可逆，由 (2.59) 給出的 x_{k+1} 的確是 $\tilde{f}_k$ 的臨界點。由於 Hessian 提供了目標函數在 x_k 附近的二階信息，因此預期 (2.60) 中的 Newton 方向 v_k^N 比僅使用一階信息的梯度下降法更有效。

由此可見，(2.60) 中的 Newton 方向涉及 Hessian $\nabla^2 f(x_k)$ 的逆。因此，若 $\nabla^2 f(x_k)$ 是奇異的，則 Newton 法無法運行。進一步地，若 $\nabla^2 f(x_k)$ 不是正定的，則 (2.60) 中定義的 v_k^N 甚至可能不是下降方向。這些問題通常可以通過步長線搜索、信賴域（trust region）技術、Gauss–Newton 近似等方式加以

修正。由於這裡的目的是說明思想，並在後文展示 Newton 法及其重要變體的收斂性，我們暫時直接假設 v_k^N 存在且是下降方向。

我們還注意到，(2.60) 中的 Newton 步 v_k^N 計算代價非常高，因為它需要求 Hessian 矩陣的逆，其計算成本通常是關於 n 的三次方級別。我們稍後會討論如何解決這一問題，這裡暫時假設每一步迭代都能得到 v_k^N。

現在考慮使用 Newton 方向 v_k^N 且步長 α_k 為常數時，Newton 法的局部收斂性質。所謂局部收斂，是指對於滿足充分最優性條件的某個局部極小值點 x^*（即 $\nabla f(x^*) = 0$ 且 $\nabla^2 f(x^*) \succ 0$），若初始點足夠接近 x^*，則 Newton 法收斂到 x^*。我們承認這些條件看起來較強，在實踐中也不一定容易滿足；但對於非凸問題而言，它們往往是目前已知最好的收斂性保證。Newton 法的這一具體收斂結果如下所示。

定理 2.4.4 (Newton 法的局部二次收斂)**.** *設 $f : \mathbb{R}^n \to \mathbb{R}$ 屬於 C^2，且 $\nabla^2 f$ 是 Lipschitz 連續的，即存在 $M > 0$ 使得*

$$|\nabla^2 f(\hat{x}) - \nabla^2 f(x)| \le M|\hat{x} - x|$$

對任意 $x, \hat{x} \in \mathbb{R}^n$ 成立，其中 $|A| := \max_{x:|x|\le 1} |Ax|$ 表示矩陣的 2-範數。設 x^* 是一個局部極小點，滿足 $\nabla f(x^*) = 0$ 且 $\nabla^2 f(x^*) \succ 0$，並設 $\{x_k\}$ 是採用 Newton 方向 v_k^N 且每一步步長均取 $\alpha_k = 1$ 的 Newton 法所生成的序列，則有:

1. *若 x_0 足夠接近 x^*，則當 $k \to \infty$ 時，$x_k \to x^*$，且收斂速率為二次（具體將在證明中給出）。*

2. *此外，當 $k \to \infty$ 時，$|\nabla f(x_k)| \to 0$，且收斂速率同樣為二次。*

证明. 注意到

$$\begin{aligned} x_{k+1} - x^* &= x_k - v_k^N - x^* \\ &= x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k) - x^* \\ &= (\nabla^2 f(x_k))^{-1} (\nabla^2 f(x_k)(x_k - x^*) - \nabla f(x_k)). \end{aligned} \tag{2.61}$$

由 Taylor 定理可知

$$\nabla f(x_k) - \nabla f(x^*) = \int_0^1 \nabla^2 f(x_k + t(x^* - x_k))(x_k - x^*)\, dt. \tag{2.62}$$

因此有

$$
\begin{aligned}
&|\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))| \\
&\qquad = \Big| \int_0^1 \Big(\nabla^2 f(x_k) - \nabla^2 f(x_k + t(x_k - x^*))\Big)(x_k - x^*)\, dt \Big| \\
&\qquad \le M|x_k - x^*|^2 \int_0^1 t\, dt \\
&\qquad = \frac{M}{2}|x_k - x^*|^2
\end{aligned}
\tag{2.63}
$$

其中第一個等號來自 (2.62)，而不等號則利用了 $\nabla^2 f$ 的 M-Lipschitz 連續性。

由於 $\nabla^2 f(x^*) \succ 0$ 且 $f \in C^2$，可知在 x^* 的某個鄰域內，$\nabla^2 f(x)$ 在每一點都可逆。進一步地，存在 $r > 0$，使得

$$
|(\nabla^2 f(x))^{-1}| \le 2|(\nabla^2 f(x^*))^{-1}| \tag{2.64}
$$

對所有 $x \in B_r(x^*)$ 成立。

結合 (2.61)、(2.63) 和 (2.64)，可得

$$
\begin{aligned}
|x_{k+1} - x^*| &\le \frac{M}{2} \cdot 2\,|(\nabla^2 f(x^*))^{-1}|\,|x_k - x^*|^2 \\
&= M\,|(\nabla^2 f(x^*))^{-1}|\,|x_k - x^*|^2.
\end{aligned}
$$

若選取 x_0 滿足

$$
|x_0 - x^*| \le \min\Big(r, \frac{1}{2M|(\nabla^2 f(x^*))^{-1}|}\Big),
$$

則可通過歸納法證明存在常數 $C > 0$，使得

$$
|x_{k+1} - x^*| \le C|x_k - x^*|^2 \quad \text{和} \quad |x_k - x^*| < 1/C \tag{2.65}
$$

對所有 k 成立。這表明 $x_k \to x^*$，且 (2.65) 的第一個條件就是所謂的**二次收斂速率**。

此外，由 (2.60) 中 v_k^N 的定義可知

$$
\nabla f(x_k) + \nabla^2 f(x_k) v_k^N = 0
$$

因此

$$
\begin{aligned}
|\nabla f(x_{k+1})| &= |\nabla f(x_{k+1}) - \nabla f(x_k) - \nabla^2 f(x_k) v_k^N| \\
&= \Big| \int_0^1 \nabla^2 f(x_k + t(x_{k+1} - x_k))(x_{k+1} - x_k)\, dt - \nabla^2 f(x_k) v_k^N \Big| \\
&= \Big| \int_0^1 \nabla^2 f(x_k + t v_k^N) v_k^N\, dt - \nabla^2 f(x_k) v_k^N \Big| \\
&\le \int_0^1 |\nabla^2 f(x_k + t v_k^N) - \nabla^2 f(x_k)| |v_k^N|\, dt \\
&\le \frac{M}{2} |v_k^N|^2 \\
&\le \frac{M}{2} |(\nabla^2 f(x_k))^{-1}|^2 |\nabla f(x_k)|^2 \\
&\le 2M |(\nabla^2 f(x^*))^{-1}|^2 |\nabla f(x_k)|^2,
\end{aligned}
$$

這說明 $|\nabla f(x_k)|$ 也以二次速率收斂到 0。 □

現在我們回到之前尚未解決的問題：如何計算 (2.60) 中的 v_k^N。對於大規模問題，即 n 達到百萬量級甚至更高時，直接求 $\nabla^2 f(x_k)$ 的逆是不可行的，因為這需要 $O(n^3)$ 的計算成本。與其直接求逆，我們可以通過求解如下線性方程組來得到 v_k^N：

$$\nabla^2 f(x_k) v = -\nabla f(x_k). \tag{2.66}$$

這就是標準線性系統

$$Qx = b \tag{2.67}$$

的形式，其中 Q 是對稱正定矩陣（這裡假設 (2.66) 中的 $\nabla^2 f(x_k)$ 是正定的；當 x_k 足夠接近滿足 $\nabla^2 f(x^*) \succ 0$ 的 x^* 時，這確實成立）。

針對如 (2.67) 這樣的巨大線性系統，人們已經設計出了大量數值方法。這裡我們只介紹**共軛梯度法**（conjugate gradient，CG），這是求解 (2.67) 最經典且高效的方法之一。求解線性系統的標準 CG 方法見算法 2.4.2。需要注意的是，若所有計算都精確進行，則 CG 方法至多在 n 步內收斂；其每一步的主要計算開銷是矩陣向量乘法 Qd_j（見 (2.68a)），而這一結果在第 j 次迭代中還會被重複利用來計算其他量。在實踐中，通常只需約 $\sqrt{n}$ 步迭代就能非常接近 $Qx = b$ 的真實解，這取決於 Q 的條件數（即 Q 的最大與最小特徵值之比）。若條件數過大，也有許多對 CG 的預條件修正方法可以緩解病態性問題 [105]。若 Hessian 是稀疏的，則計算成本還能進一步降低。因此，通過求解 (2.66) 來估計 v_k^N，CG 是一種在實踐中可行的方法。

Algorithm 2.4.2 用於求解 $Qx = b$ 的共軛梯度（CG）方法

Input: Initial $x_0 \in \mathbb{R}^n$, $g_0 = Qx_0 - b$, $d_0 = -g_0$.

1: **for** $j = 0, 1, , \ldots, n-1$ **do**

2:

$$\alpha_j = \frac{g_j \cdot d_j}{g_j \cdot (Qd_j)}, \tag{2.68a}$$

$$x_{j+1} = x_j + \alpha_j d_j, \tag{2.68b}$$

$$g_{j+1} = Qx_{j+1} - b = g_j + \alpha_j (Qd_j), \tag{2.68c}$$

$$\beta_j = \frac{g_j \cdot (Qd_j)}{d_j \cdot (Qd_j)}, \tag{2.68d}$$

$$d_{j+1} = -g_{j+1} + \beta_j d_j. \tag{2.68e}$$

3: **end for**

Output: x_n.

總之，雖然 Newton 法在其原始形式下對大規模問題而言計算上不可行，但存在許多變通方法可以得到高效的**非精確 Newton 法**。上面使用 CG 來近似 Newton 步的方法，即所謂的 Newton–CG 方法，就是其中之一。需要指出的是，這類非精確 Newton 法通常並不顯式計算或存儲 Hessian 矩陣，而只需要若干 Hessian-vector 乘積。因此，儘管它們使用了 f 的二階導數，仍常被稱為 Hessian-free 方法。

另一類 Hessian-free 方法稱為**擬 Newton 法**（quasi-Newton method），它們通常被認為是在小規模到中等規模的光滑非凸優化問題中最優先考慮的方法。擬 Newton 法的主要思想如下。假設迭代點 x_{k+1} 足夠接近 x_k，則有

$$\begin{aligned}\nabla f(x_k) &= \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}) + o(|x_k - x_{k+1}|) \\ &\approx \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}).\end{aligned}$$

將其整理，並假設該近似足夠準確，則得到

$$\nabla^2 f(x_{k+1})(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k). \tag{2.69}$$

如果能找到一個矩陣 $B_k \in \mathbb{R}^{n \times n}$，在每一步迭代中起到 Hessian $\nabla^2 f(x_k)$ 的替代作用，則 (2.69) 可寫為

$$B_{k+1}(\underbrace{x_{k+1} - x_k}_{=:\, s_k}) = \underbrace{\nabla f(x_{k+1}) - \nabla f(x_k)}_{=:\, y_k}. \tag{2.70}$$

方程 (2.70) 稱為**割線方程**，滿足該條件的優化算法稱為**擬 Newton 法**。若每一步都能得到這樣的 B_k，則擬 Newton 法的更新格式為

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k), \tag{2.71}$$

其中步長 α_k 通常通過線搜索選取，使其滿足 Wolfe 條件 (2.48) 和 (2.50)。

在實踐中，一個真正可用的擬 Newton 法 不僅需要滿足 (2.70)，還要求 B_k 易於更新，並且其逆（或對任意 $b \in \mathbb{R}^n$ 的 $B_k^{-1}b$）易於計算。Broyden–Fletcher–Goldfarb–Shannon（BFGS）方法是滿足這些條件的最著名擬 Newton 法之一。BFGS 中 B_k 的更新公式為（注意 y_k 是行向量，s_k 是列向量）：

$$B_{k+1} = B_k + \frac{y_k^\top y_k}{y_k s_k} - \frac{(B_k s_k)(B_k s_k)^\top}{s_k^\top (B_k s_k)}, \tag{2.72}$$

其中所有分母均為標量，而分子均為秩為 1 的矩陣。

在 BFGS 的實現中，我們還需要在每一步更新 $H_k := B_k^{-1}$，以便利用 H_k 輕鬆計算 (2.71)。為此，我們利用 Sherman–Morrison–Woodbury 公式：若矩陣 $A \in \mathbb{R}^{n\times n}$ 可逆，且兩個列向量 $u, v \in \mathbb{R}^n$ 滿足 $1 + v^\top A u \neq 0$，則

$$(A + uv^\top)^{-1} = A^{-1} - \frac{(A^{-1}u)(v^\top A^{-1})}{1 + v^\top A^{-1} u}. \tag{2.73}$$

該公式可以通過直接計算驗證。

我們重複使用 Sherman–Morrison–Woodbury 公式 (2.73)，可得到 H_k 的更新公式：

$$\begin{aligned}
H_{k+1} &= B_{k+1}^{-1} \\
&= \Big(B_k + \frac{y_k^\top y_k}{y_k s_k} - \frac{(B_k s_k)(B_k s_k)^\top}{s_k^\top (B_k s_k)} \Big)^{-1} \\
&= H_k + \Big(1 + \frac{y_k H_k y_k^\top}{y_k s_k} \Big) \frac{s_k s_k^\top}{y_k s_k} - \frac{H_k (s_k y_k^\top) + (s_k y_k^\top) H_k}{y_k s_k}.
\end{aligned}$$

這就是 BFGS 方法中 H_k 的更新公式。

BFGS 方法（以及所有擬 Newton 法）的一個缺點在於：它們需要存儲一個可能是稠密的 $n \times n$ 矩陣 H_k（即使真實 Hessian 是稀疏的），這限制了其在大規模問題中的應用。有一種變體稱為 limited-memory BFGS（L-BFGS），它只利用最近若干步（通常 3 到 20 步）迭代中的曲率信息來構造 H_k 的近似。更多細節可參見 [105]。當某個迭代點 x_k 足夠接近滿足 $\nabla^2 f(x^*) \succ 0$ 的局部

極小點 x^* 時，可以證明 BFGS 方法以超線性速率收斂，即

$$\lim_{k\to\infty} \frac{|x_{k+1} - x^*|}{|x_k - x^*|} = 0.$$

相比之下，L-BFGS 方法具有線性收斂速率，即

$$\lim_{k\to\infty} \frac{|x_{k+1} - x^*|}{|x_k - x^*|} = \gamma$$

其中常數 $\gamma \in (0, 1)$ [105]。

2.4.2 約束優化算法

在本小節中，我們介紹若干約束優化算法。我們指出，這些算法在網絡訓練中很少直接使用，因為我們通常會避免將這類大規模優化問題構造成約束形式，因為那樣會更具挑戰性。不過，這些算法為如何利用無約束優化算法來求解約束問題提供了很有啟發性的思路。當我們希望對網絡參數施加額外約束與正則化，或者設計自適應網絡結構時，它們可能會很有用。

求解約束優化問題的一大類方法，是將原問題替換為一列無約束優化問題。這些無約束問題的解可以收斂到原約束問題的解。我們首先介紹這一類方法中一個常用的算法。

二次罰函數法

我們考慮帶有等式約束的優化問題 (2.17)，這裡將其重述如下：

$$\min_{x\in\mathbb{R}^n} \quad f(x), \tag{2.74a}$$

$$\text{s.t.} \quad h(x) = 0. \tag{2.74b}$$

我們介紹的第一個算法稱為**二次罰函數法**（quadratic penalty method）。該方法將約束問題 (2.74) 修改為如下無約束問題

$$\min_{x\in\mathbb{R}^n} Q_\gamma(x), \tag{2.75}$$

其中**二次罰函數** Q_γ 定義為

$$Q_\gamma(x) := f(x) + \frac{1}{2\gamma}|h(x)|^2 \tag{2.76}$$

而 $\gamma > 0$ 稱為**罰參數**。該罰參數 γ 用於懲罰約束 $h(x) = 0$ 的違反程度：γ 越小，懲罰越重。因此，設定一列罰參數 $\{\gamma_k\}$ 使得當 $k \to \infty$ 時 $\gamma_k \to 0^+$ 是合理的。同時，對每個 k，我們以某個初值求解對應 $\gamma = \gamma_k$ 的無約束極小化問題 (2.75)，得到一個解 x_k，並期望解序列 $\{x_k\}$ 收斂到原約束問題 (2.74) 的一個解。如果我們明智地選擇 γ_k 和初值，那麼無約束問題 $\min_x Q_{\gamma_k}$ 可能會很快被求解。

現在我們來描述上述二次罰函數法的收斂性質。

定理 2.4.5. 設 x_k 是 Q_{γ_k} 的全局極小值點，並且 $\gamma_k \to 0^+$。那麼，$\{x_k\}$ 的每一個聚點[2] x^* 都是約束問題 (2.74) 的一個全局極小值點。

证明. 設 $\bar{x}$ 是 (2.74) 的一個全局極小值點，即 $h(\bar{x}) = 0$ 且

$$f(\bar{x}) \le f(x)$$

對所有滿足 $h(x) = 0$ 的 x 成立。另一方面，由於 x_k 是 Q_{γ_k} 在 $\mathbb{R}^n$ 上的全局極小值點，因此有 $Q_{\gamma_k}(x_k) \le Q_{\gamma_k}(\bar{x})$，即

$$f(x_k) + \frac{1}{2\gamma_k}|h(x_k)|^2 \le f(\bar{x}) + \frac{1}{2\gamma_k}|h(\bar{x})|^2 = f(\bar{x}), \tag{2.77}$$

或者等價地，

$$|h(x_k)|^2 \le 2\gamma_k(f(\bar{x}) - f(x_k)).$$

由於假設 f 有一個有限下界 f^*，因此有

$$|h(x_k)|^2 \le 2\gamma_k(f(\bar{x}) - f(x_k)) \le 2\gamma_k(f(\bar{x}) - f^*),$$

其中 $f(\bar{x}) - f^*$ 是一個有限常數。

設 x^* 是 $\{x_k\}$ 的一個聚點，則存在 $\{x_k\}$ 的一個子序列 $\{x_{k_j}\}$，使得當 $j \to \infty$ 時 $x_{k_j} \to x^*$。

由 h 的連續性可得

$$|h(x^*)|^2 = \lim_{j\to\infty} |h(x_{k_j})|^2 \le \lim_j 2\gamma_{k_j}(f(\bar{x}) - f^*) = 0,$$

這是因為當 $j \to \infty$ 時，有 $\gamma_{k_j} \to 0$。因此 $h(x^*) = 0$，這說明 x^* 是一個可行點。

[2]如果序列 $\{x_k\}$ 存在一個子序列，例如 $\{x_{k_j}\}$，使得當 $j \to \infty$ 時 $\{x_{k_j}\}$ 收斂到 $\hat{x}$，則稱 $\hat{x}$ 是序列 $\{x_k\}$ 的一個聚點（accumulation point）。聚點也稱為極限點（limit point）。

Algorithm 2.4.3 用於求解 (2.74) 的二次罰函數法

Input: Initial penalty parameter $\gamma_0 > 0$, tolerance ϵ_0 and guess $x_{0,0} \in \mathbb{R}^n$.

1: **for** $k = 0, 1, 2, \ldots$ **do**
2: Start from $x_{k,0}$, find x_k such that $|\nabla Q_{\gamma_k}(x_k)| \leq \epsilon_k$.
3: **if** x_k is "satisfactory" **then**
4: Terminate.
5: **end if**
6: Choose $\gamma_{k+1} \in (0, \gamma_k)$.
7: Choose $\epsilon_{k+1} \in (0, \epsilon_k)$.
8: Choose an initial guess $x_{k+1,0}$ for the next iteration.
9: **end for**

Output: x_k.

進一步地，由 (2.77) 可知，對所有 k 都有 $f(x_k) \leq f(\bar{x})$。由於 f 是連續的，因此有

$$f(x^*) = \lim_{j\to\infty} f(x_{k_j}) \leq f(\bar{x}).$$

由於 $\bar{x}$ 是全局極小值點，因此必有 $f(x^*) = f(\bar{x})$，並且 x^* 也是一個全局極小值點。 □

雖然定理 2.4.5 給出了一個非常理想的收斂性質，但在實踐中很難得到非凸函數 Q_γ 的全局極小值點。

接下來，我們介紹一個更實際的算法。對於任意 k，它不需要 Q_{γ_k} 的全局極小值點，但在一定條件下仍能保證收斂到 (2.74) 的一個 Lagrange 點（即滿足 Lagrange 條件，但不一定是全局極小值點）。該算法見算法 2.4.3。

備註. 關於算法 2.4.3，我們有幾點說明。首先，在適當的光滑性條件下，算法 2.4.3 第 2 行中的 x_k 在實踐中是可以找到的。更具體地說，注意到 $\gamma_k > 0$ 且

$$\nabla Q_{\gamma_k}(x) = \nabla f(x) + \frac{1}{\gamma_k} h(x)^\top \nabla h(x).$$

如果 f 和 $h^\top \nabla h$ 都是 Lipschitz 連續的，那麼 Q_{γ_k} 也是 Lipschitz 連續的。因此，存在某種算法，例如帶回溯線搜索的梯度下降法（算法 2.4.1），能夠從任意初值 $x_{k,0}$ 出發，在有限步內找到 x_k。但是需要注意，$h^\top \nabla h(x)$ 在很多情況下未必是 Lipschitz 連續的，例如 $h(x) = x^2$。不過，如果 h 是仿射函數，則它是 Lipschitz 的。

其次，我們常常令 γ_k 遞減、ϵ_k 非增，但最終都需要滿足當 $k \to \infty$ 時 $\gamma_k \to 0^+$ 且 $\epsilon_k \to 0^+$。一個常用的方法是用某個預先指定的 $(0, 1)$ 中常數來縮

放 γ_k 和 ϵ_k。

第三，初值的一個典型選擇是 $x_{k+1,0} = x_k$。但也可能存在更好的選擇。

最後但同樣重要的是，我們注意到算法 2.4.3 在 x_k “足夠滿意” 時終止，這聽起來似乎比較主觀。在實踐中，我們會在 γ_k、$|h(x_k)|$ 和 $|\nabla Q_{\gamma_k}(x_k)|$（這些量都可以計算）都足夠小時終止算法。

我們看到，算法 2.4.3 是一種很有意思的方法：由於 $\epsilon_k \to 0$，我們有

$$|\nabla Q_{\gamma_k}(x_k)| = \left|\nabla f(x_k) + \frac{1}{\gamma_k} h(x_k)^\top \nabla h(x_k)\right| \le \epsilon_k.$$

若記 $\lambda_k := h(x_k)^\top/\gamma_k$，那麼 $\nabla f(x_k) + \lambda_k \nabla h(x_k)$ 可能會接近於 0。因此，如果 (x_k, λ_k) 有一個聚點 (x^*, λ^*)，那麼對於某個子序列 $\{(x_{k_j}, \lambda_{k_j})\}$，我們將有

$$\nabla f(x^*) + \lambda^* \nabla h(x^*) = \lim_{j\to\infty} \nabla f(x_{k_j}) + \lambda_{k_j} \nabla h(x_{k_j}) = 0$$

從而 (x^*, λ^*) 滿足第一個 Lagrange 條件。因此，剩下要證明的是 $h(x^*) = 0$，即第二個 Lagrange 條件。下面的定理給出了一個完整結果。

定理 2.4.6. *在算法 2.4.3 中，設當 $k \to \infty$ 時 $\epsilon_k \to 0^+$ 且 $\gamma_k \to 0^+$，那麼對於 $\{x_k\}$ 的任意一個聚點 x^*，若 x^* 是正則點，則 x^* 是 (2.74) 的一個 Lagrange 點。此外，對於任意收斂到 x^* 的子序列 $\{x_{k_j}\}$，記 $\lambda_{k_j} := \nabla h(x_{k_j})/\gamma_{k_j}$，則當 $j \to \infty$ 時 λ_{k_j} 也收斂到某個 λ^*，且 (x^*, λ^*) 滿足 Lagrange 條件。*

证明. 設 $\{x_{k_j}\}$ 是 $\{x_k\}$ 的任意一個子序列，滿足當 $j \to \infty$ 時 $x_{k_j} \to x^*$。回憶一下，對任意固定的 $\gamma > 0$，有 $\nabla Q_\gamma(x) = \nabla f(x) + \frac{1}{\gamma} h(x)^\top \nabla h(x)$。根據算法 2.4.3 第 1 行中 x_k 的性質，有

$$\left|\nabla f(x_{k_j}) + \frac{1}{\gamma_{k_j}} h(x_{k_j})^\top \nabla h(x_{k_j})\right| \le \epsilon_{k_j},$$

這意味著

$$|h(x_{k_j})^\top \nabla h(x_{k_j})| \le \gamma_{k_j}(\epsilon_{k_j} + |\nabla f(x_{k_j})|).$$

由於 f、h 和 ∇h 的連續性，以及當 $j \to \infty$ 時 $x_{k_j} \to x^*$、$\epsilon_{k_j} \to 0$、$\gamma_{k_j} \to 0$，可知

$$|h(x^*)^\top \nabla h(x^*)| = \lim_{j\to\infty} |h(x_{k_j})^\top \nabla h(x_{k_j})| = 0.$$

由於 x^* 是正則點，即 $\nabla h(x^*)$ 具有滿行秩，因此有 $h(x^*) = 0$，從而 x^* 是一個可行點。

接下來我們證明 λ_{k_j} 當 $j \to \infty$ 時收斂到某個 λ^*。為此，注意到 ∇h 和行列式 $\det(\cdot)$ 都是連續函數，而 $\nabla h(x^*)$ 具有滿行秩。因此，x^* 是集合

$$\{x \in \mathbb{R}^n : \nabla h(x)\text{是滿行秩的}\} = \{x \in \mathbb{R}^n : \det(\nabla h(x)\nabla h(x)^\top) > 0\}$$

的一個內點。因此，對所有充分大的 j，我們知道 $\nabla h(x_{k_j})\nabla h(x_{k_j})^\top$ 具有滿秩，因此可逆。

令 $\lambda_{k_j} := h(x_{k_j})^\top/\gamma_{k_j}$。則有

$$\lambda_{k_j}\nabla h(x_{k_j}) = \nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j}). \tag{2.78}$$

由於 $\nabla h(x_{k_j})\nabla h(x_{k_j})^\top$ 可逆，因此有

$$\lambda_{k_j} = \big(\nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j})\big)\nabla h(x_{k_j})^\top\big(\nabla h(x_{k_j})\nabla h(x_{k_j})\big)^{-1}.$$

令 $j \to \infty$，我們有 $|\nabla Q_{\gamma_{k_j}}(x_{k_j})| \le \epsilon_{k_j} \to 0$，並且

$$\begin{aligned}\lim_{j\to\infty}\lambda_{k_j} &= \lim_{j\to\infty}\Big(\nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j})\Big)\nabla h(x_{k_j})^\top\Big(\nabla h(x_{k_j})\nabla h(x_{k_j})^\top\Big)^{-1}\\ &= -\nabla f(x^*)\nabla h(x^*)^\top(\nabla h(x^*)\nabla h(x^*)^\top)^{-1}\\ &=: \lambda^*.\end{aligned}$$

這說明 $\{\lambda_{k_j}\}$ 收斂到上述定義的 λ^*。

此外，在 (2.78) 中令 $j \to \infty$，可得

$$\nabla f(x^*) + \lambda^*\nabla h(x^*) = 0.$$

因此 (x^*, λ^*) 滿足 Lagrange 條件 (2.20)。 □

備註. 定理 2.4.6 中要求 x^* 為正則點這一條件是必要的，否則我們無法得到 $h(x^*) = 0$，這意味著 x^* 可能不是可行點。不過，我們仍然可以證明 $h(x^*)^\top\nabla h(x^*) = 0$，這意味著 x^* 是 $|h(x)|^2$ 的一個駐點。

我們還需要注意，當 γ_k 趨於 0 時，在極小化 Q_{γ_k} 的過程中會出現病態性問題。注意到

$$\nabla Q_{\gamma_k}(x) = \nabla f(x) + \frac{1}{\gamma_k}h(x)^\top\nabla h(x) = \nabla f(x) + \frac{1}{\gamma_k}\sum_{i=1}^m h_i(x)\nabla h_i(x)$$

以及

$$\nabla^2 Q_{\gamma_k}(x) = \nabla^2 f(x) + \frac{1}{\gamma_k}\sum_{i=1}^{m} h_i(x)\nabla^2 h_i(x) + \frac{1}{\gamma_k}\nabla h(x)^\top h(x).$$

若令 $x = x_{k_j}$ 且 $\gamma_k = \gamma_{k_j}$，並注意到當 $j \to \infty$ 時 $x_{k_j} \to x^*$ 且 $\lambda_{k_j} = h(x_{k_j})^\top/\gamma_{k_j} \to \lambda^*$，則我們預期 $\nabla^2 Q_{\gamma_{k_j}}(x_{k_j})$ 接近於

$$\nabla^2 f(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla^2 h_i(x^*) + \lim_{j\to\infty} \frac{1}{\gamma_{k_j}} \nabla h(x_{k_j})^\top \nabla h(x_{k_j}). \tag{2.79}$$

前兩項之和趨近於 Lagrange 函數 $l(x,\lambda) = f(x) + \lambda h(x)$ 關於 x 在 (x^*, λ^*) 處的 Hessian，即 $\partial_x^2 l(x,\lambda)$。若 x^* 是嚴格局部極小點，則該 Hessian 的特徵值是固定的且為正。然而，(2.79) 中最後一項是棘手的：我們知道 $\nabla h(x_{k_j})^\top \nabla h(x_{k_j})$ 趨近於 $\nabla h(x^*)^\top \nabla h(x^*) \in \mathbb{R}^{n\times n}$，而後者是不可逆的的，因為 $m < n$。在這種情況下，$1/\gamma_{k_j}$ 會把所有正特徵值放大，從而當 $\gamma_{k_j} \to 0$ 時，使得 $\nabla^2 Q_{\gamma_{k_j}}$ 在 x^* 處的條件數變得非常大。這意味著，在實踐中極小化 $Q_{\gamma_{k_j}}$ 可能非常困難。對此有一種緩解方法；參見 [105, Chapter 17.1]。

Lagrange 方法

我們考慮一種針對帶等式約束優化問題的替代方法，它基於 Lagrange 函數：

$$l(x,\lambda) = f(x) + \lambda h(x), \tag{2.80}$$

其中 $f, h \in C^2$。該 **Lagrange 方法**定義為

$$x_{k+1} = x_k - \alpha_k(\nabla f(x_k) + \lambda_k \nabla h(x_k))^\top, \tag{2.81a}$$

$$\lambda_{k+1} = \lambda_k + \beta_k h(x_k)^\top, \tag{2.81b}$$

其中 k 是迭代計數器，$\alpha_k, \beta_k > 0$ 是步長。轉置 $^\top$ 是為了使向量形狀與我們的約定一致，即 $x \in \mathbb{R}^n$ 是列向量，而 $\lambda \in \mathbb{R}^m$ 是行向量。

在討論 Lagrange 方法 (2.81) 的性質之前，我們先給出一種直觀的方式來記憶該方法。對任意 $x \in \mathbb{R}^n$，注意到

$$\max_{\lambda\in\mathbb{R}^m} l(x,\lambda) = \begin{cases} f(x), & \text{if } h(x) = 0, \\ +\infty, & \text{if } h(x) \neq 0. \end{cases}$$

因此，我們可以將等式約束優化問題 (2.74) 重寫為

$$\min_{x\in\mathbb{R}^n}\{f(x):h(x)=0\}=\min_{x\in\mathbb{R}^n}\max_{\lambda\in\mathbb{R}^m} l(x,\lambda).$$

因此，Lagrange 方法 (2.81) 可以理解為：對 x 做梯度下降以極小化 $l(x,\lambda)$，同時對 λ 做梯度上升以極大化它。由於 (2.81) 中只需要梯度和 Jacobian，我們稱其為一階 Lagrange 方法。

現在我們考察 Lagrange 方法 (2.81) 的收斂性質。為簡化分析，我們假設步長為常數，即對某些 $\alpha,\beta>0$，有 $\alpha_k=\alpha$ 且 $\beta_k=\beta$。通過對 λ 做適當縮放，不失一般性地可設 $\beta=\alpha$。在這種情況下，Lagrange 方法 (2.81) 可以看作一個形如 $z_{k+1}=F(z_k)$ 的**不動點迭代**，其中 $z_k:=(x_k,\lambda_k)\in\mathbb{R}^{n+m}$ 是由 x 與 λ 拼接而成的列向量，而

$$F(z):=\begin{pmatrix} x-\alpha(\nabla f(x)+\lambda\nabla h(x))^\top \\ \lambda^\top+\alpha h(x)\end{pmatrix} \tag{2.82}$$

對任意 $z=(x,\lambda)\in\mathbb{R}^{n+m}$ 成立。這裡所謂 F 的不動點，是指滿足 $z=F(z)$ 的點 z。在此定義下，有如下引理，其證明非常簡單，因此略去。

引理 2.4.7. *一個點 $z^*=(x^*,\lambda^*)$ 滿足 Lagrange 條件*

$$\nabla f(x^*)+\lambda^*\nabla h(x^*)=0, \tag{2.83a}$$

$$h(x^*)=0 \tag{2.83b}$$

當且僅當 z^* 是 F 的一個不動點。

接下來，我們證明：若 (x^*,λ^*) 滿足二階充分條件，即 (x^*,λ^*) 滿足 (2.83) 以及

$$\nabla^2 f(x^*)+\sum_{i=1}^m \lambda_i^*\nabla^2 h_i(x^*)\succ 0, \tag{2.84}$$

則只要初值 (x_0,λ_0) 足夠接近 (x^*,λ^*) 且 α 足夠小，由 Lagrange 方法 (2.81) 生成的序列 $\{(x_k,\lambda_k)\}$ 就會收斂到 (x^*,λ^*)。

定理 2.4.8. *設 (x^*,λ^*) 滿足等式約束優化問題 (2.74) 的二階充分條件，即滿足 (2.83) 和 (2.84)，並且 $\alpha,\beta>0$ 足夠小，則存在 (x^*,λ^*) 的一個鄰域，使得對該鄰域中的任意初值 (x_0,λ_0)，由 Lagrange 方法 (2.81) 生成的序列 $\{(x_k,\lambda_k)\}$ 收斂到 (x^*,λ^*)。*

证明. 如前所述，通過適當縮放 λ，我們可假設 $\alpha = \beta$。注意 Lagrange 方法即為 $z_{k+1} = F(z_k)$。由引理 2.4.7，我們只需證明 z_k 收斂到 F 的一個不動點。由於 $z^* = (x^*, \lambda^*)$ 是 F 的不動點，因此有

$$|z_{k+1} - z^*| = |F(z_k) - F(z^*)|.$$

注意到

$$\nabla F(z) = I + \alpha \begin{pmatrix} -H(x,\lambda) & -\nabla h(x)^\top \\ \nabla h(x) & 0 \end{pmatrix} \in \mathbb{R}^{(n+m)\times(n+m)},$$

其中

$$H(x,\lambda) := \nabla^2 f(x) + \sum_{i=1}^{m} \lambda_i \nabla^2 h_i(x).$$

另一方面，由均值定理，存在一個關於 z_k 和 z^* 的矩陣值函數 M（M 依賴於 F），使得

$$F(z_k) - F(z^*) = M(z_k, z^*)(z_k - z^*),$$

其中 $(M(z_k, z^*))_i$，即 $M(z_k, z^*)$ 的第 i 行，是 ∇F_i 在連接 z_k 與 z^* 的線段上的某一點處的取值。兩邊取範數，得到

$$|F(z_k) - F(z^*)| \le |M(z_k, z^*)||z_k - z^*|. \tag{2.85}$$

現在我們斷言：當 $\alpha > 0$ 足夠小時，有 $|\nabla F(z^*)| < 1$。為此，注意到 $\nabla F(z^*) = I + \alpha M^*$，其中

$$M^* := \begin{pmatrix} -H(x^*,\lambda^*) & -\nabla h(x^*)^\top \\ \nabla h(x^*) & 0 \end{pmatrix}.$$

因此，只需證明 M^* 的所有特徵值都具有負實部，因為這樣一來當 $\alpha > 0$ 足夠小時，$\nabla F(z^*) = I + \alpha M^*$ 的所有特徵值實部都落在 $(0,1)$ 中。

設 ξ 是 M^* 的任意一個特徵值，$z = (x, \lambda)$ 是對應特徵向量。（注意根據特徵向量的定義，$z \neq 0$。）則 $M^* z = \xi z$。左乘 $z^\top$[3]，可得 $z^\top M^* z = \xi |z|^2$。因此

$$\mathrm{Re}(z^\top M^* z) = \mathrm{Re}(\xi)|z|^2 \tag{2.86}$$

[3]注意，由於 z 可能是復向量，這裡的記號 $^\top$ 表示複共軛轉置。

其中 $\mathrm{Re}(\cdot)$ 表示實部。此外，注意到

$$\begin{aligned}\mathrm{Re}(z^\top M^* z) &= -\mathrm{Re}(x^\top H(x^*,\lambda^*)x) - \mathrm{Re}(x^\top \nabla h(x^*)^\top \lambda^*) \\ &\qquad + \mathrm{Re}(\lambda^{*\top} \nabla h(x^*)x) \\ &= -\mathrm{Re}(x^\top H(x^*,\lambda^*)x).\end{aligned} \tag{2.87}$$

由於 (2.84)，有 $H(x^*,\lambda^*) \succ 0$，因此只要 $x \neq 0$，便有 $\mathrm{Re}(x^\top H(x^*,\lambda^*)x) > 0$。

現在可見，若 $x \neq 0$，則比較 (2.86) 與 (2.87)，再注意到 $z \neq 0$，便得 $\mathrm{Re}(\xi) < 0$。

若 $x = 0$，則 $z = (0,\lambda)$。比較 $M^* z = \xi z$ 兩邊的前 n 個分量，可得 $\nabla h(x^*)^\top \lambda = 0$。由於 x^* 是正則點，$\nabla h(x^*)$ 具有滿行秩，因此可知 $\lambda = 0$。但這意味著 $z = 0$，與 $z \neq 0$ 矛盾。因此，M^* 的所有特徵值都具有負實部。

至此，我們知道：當 $\alpha > 0$ 足夠小時，有

$$|\nabla F(z^*)| = |I + \alpha M^*| < 1.$$

由 ∇F 與範數的連續性，存在 $r > 0$ 和 $\kappa \in (0,1)$，使得對所有 $z \in \bar{B}_r(z^*) := \{z : |z - z^*| \leq r\}$，都有 $M(z,z^*) \leq \kappa$。

若 $z_0 = (x_0,\lambda_0) \in \bar{B}_r(z^*)$，則由 (2.85) 有

$$|z_1 - z^*| \leq |M(z_0,z^*)|\,|z_0 - z^*| \leq \kappa |z_0 - z^*| \leq r,$$

因此也有 $|M(z_1,z^*)| \leq \kappa \in (0,1)$。若 $z_k \in \bar{B}_r(z^*)$，則同理可得

$$|z_{k+1} - z^*| \leq \kappa |z_k - z^*| \leq r$$

且 $|M(z_{k+1},z^*)| \leq \kappa \in (0,1)$。由歸納法可知，$z_k$ 以線性速率收斂到 z^*。 □

增廣 Lagrange 方法

在第 2.2.3 節中，我們引入了 Lagrange 函數的一個變體，稱為**增廣 Lagrange 函數**，用以推導約束優化問題的二階充分條件。與之對應，也存在一種求解約束優化問題的方法，稱為**增廣 Lagrange 方法**，下面我們對其進行討論。

需要指出的是，增廣 Lagrange 方法也與二次罰函數法有關。兩者的區別在於：由於引入了 Lagrange 乘子，增廣 Lagrange 方法能夠有效緩解二次罰

函數法中的病態性問題。

為了理解增廣 Lagrange 方法的構造動機，我們先回顧等式約束優化問題：

$$\min_{x\in\mathbb{R}^n} \quad f(x), \tag{2.88a}$$

$$\text{s.t.} \quad h(x)=0. \tag{2.88b}$$

我們還回憶一下，在定理 2.4.6 中證明二次罰函數法收斂時，序列 $\{h(x_k)^\top/\gamma_k\}$ 的聚點實際上正是對應於序列 $\{x_k\}$ 的聚點 x^* 的 Lagrange 乘子 λ^*。因此，我們期望當 k 足夠大時，有 $h(x_k)\approx\gamma_k\lambda^*$。在二次罰函數法中，我們需要 $\gamma_k\to 0^+$（以便得到 $h(x_k)\to 0$），而這會導致 $\nabla^2 Q_{\gamma_k}$ 的病態性。而在增廣 Lagrange 方法中，這一問題可以得到緩解，從而 γ_k 不需要像二次罰函數法那樣非常快地趨於 0，同時仍能保證 $h(x_k)$ 很快趨於 0。

為此，我們回憶增廣 Lagrange 函數 定義如下，它是原問題 (2.88) 的 Lagrange 函數與二次罰函數的 “混合”：

$$l_A(x,\lambda;\gamma):=f(x)+\lambda h(x)+\frac{1}{2\gamma}|h(x)|^2. \tag{2.89}$$

對 l_A 關於 x 求梯度，有

$$\begin{aligned}\nabla_x l_A(x,\lambda;\gamma)&=\nabla f(x)+\lambda\nabla h(x)+\frac{1}{\gamma}h(x)^\top\nabla h(x)\\&=\nabla f(x)+\Big(\lambda+\frac{1}{\gamma}h(x)^\top\Big)\nabla h(x).\end{aligned}$$

假設 k 足夠大，並記 x_k 為 $l_A(x,\lambda_k;\gamma_k)$ 的極小點，則我們期望得到

$$\lambda_k+\frac{h(x_k)^\top}{\gamma_k}\approx\lambda^*,$$

或者等價地，

$$h(x_k)^\top\approx\gamma_k(\lambda^*-\lambda_k),$$

當 (x_k,λ_k) 接近 (x^*,λ^*) 時成立。若 $\lambda_k\to\lambda^*$，則 $\lambda^*-\lambda_k\to 0$，因此我們可能不需要像二次罰函數法那樣讓 $\gamma_k\to 0$ 得那麼快。這也啟發我們在每次更新 x_k 後，作如下更新

$$\lambda_{k+1}=\lambda_k+\frac{h(x_k)^\top}{\gamma_k} \tag{2.90}$$

而 x_k 則通過

$$x_k = \arg\min_{x\in\mathbb{R}^n} \ l_A(x, \lambda_k; \gamma_k). \tag{2.91}$$

若保持二次罰函數法中的設定，即 $\gamma_k \to 0^+$，並進一步假設 $\{\lambda_k\}$ 有界，則可以證明增廣 Lagrange 方法 也能保證收斂，如下所示。

命題 2.4.9. 設 $(\bar{x}, \bar{\lambda})$ 是由 (2.90) 與 (2.91) 生成的序列 $\{(x_k, \lambda_k)\}$ 的任意一個聚點，且 $\gamma_k \to 0$。則 $\bar{x}$ 是 (2.88) 的一個全局極小值點。

证明. 記 $f^* := \min\{f(x) : h(x) = 0,\ x \in \mathbb{R}^n\}$。則由每個 k 上的 (2.91)，對任意 x 有

$$l_A(x_k, \lambda_k; \gamma_k) \le l_A(x, \lambda_k; \gamma_k).$$

對右側所有滿足 $h(x) = 0$ 的 x 取下確界，得到

$$l_A(x_k, \lambda_k; \gamma_k) \le f^*. \tag{2.92}$$

設子序列 $\{(x_{k_j}, \lambda_{k_j})\}$ 當 $j \to \infty$ 時收斂到 $(\bar{x}, \bar{\lambda})$。則令 $j \to \infty$ 可得

$$f(\bar{x}) + \bar{\lambda} h(\bar{x}) + \lim_{j\to\infty} \frac{1}{2\gamma_{k_j}} |h(x_{k_j})|^2 \le f^*. \tag{2.93}$$

注意到 $f(\bar{x}) + \bar{\lambda} h(\bar{x})$ 是某個常數。由於 $\gamma_k \to 0$ 且 $|h(x_k)| \ge 0$，因此必有

$$|h(\bar{x})|^2 = \lim_{j\to\infty} |h(x_{k_j})|^2 = 0,$$

這說明 $h(\bar{x}) = 0$，從而 $\bar{x}$ 是可行的。此外，由 (2.93) 可知 $f(\bar{x}) \le f^*$。因此 $\bar{x}$ 是 (2.88) 的一個全局極小值點。 □

雖然命題 2.4.9 給出了收斂結果，但我們注意到 (2.91) 在實踐中較難實現，因為 $l_A(\cdot, \lambda_k; \gamma_k)$ 可能是非凸的，而我們一般無法得到其全局極小值點 x_k。作為替代，我們可以對所有 k 近似求出 $l_A(\cdot, \lambda_k; \gamma_k)$ 的駐點。我們在算法 2.4.4 中總結了增廣 Lagrange 方法。在下一個命題中，我們將證明：若算法 2.4.4 生成的 $\{x_k\}$ 收斂到某個正則點 x^*，則該 x^* 滿足問題 (2.88) 的 Lagrange 條件。

命題 2.4.10. 設 $f, h \in C^1$，$\{\lambda_k\}$ 有界，且 $\gamma_k, \epsilon_k \to 0^+$。假設對所有 k，$\{x_k\}$ 滿足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$，並且 $x_k \to x^*$，其中 x^* 是正則點（注意我們並未假設 x^* 是局部極小點）。則有 $h(x^*) = 0$，並且 $\tilde{\lambda}_k := \lambda_k + \frac{h(x_k)^\top}{\gamma_k} \to \lambda^*$，其中 (x^*, λ^*) 滿足 *Lagrange* 條件 (2.83)。

Algorithm 2.4.4 用於 (2.88) 的增廣 Lagrange 方法

Input: Initial $\gamma_0 > 0$, $\epsilon_0 > 0$, $x_{0,0} \in \mathbb{R}^n$, $\lambda_0 \in \mathbb{R}^m$.

```
for k = 0, 1, 2, ... do
    Start from x_{k,0}, find x_k such that |∇_x l_A(x_k, λ_k; γ_k)| ≤ ε_k.
    if x_k is "satisfactory" then
        Terminate.
    end if
    Set λ_{k+1} = λ_k + ∇h(x_k)^⊤ / γ_k.
    Choose γ_{k+1} ∈ (0, γ_k).
    Choose ε_{k+1} ∈ (0, ε_k).
    Choose an initial guess x_{k+1,0} for the next iteration.
end for
```

Output: x_k.

证明. 由於 $x_k \to x^*$ 且 x^* 是正則點，即 $\nabla h(x^*)$ 具有滿行秩，我們可設對所有 k 都有 $\nabla h(x_k)$ 具有滿行秩，從而 $\nabla h(x_k)\nabla h(x_k)^\top$ 可逆。注意

$$\begin{aligned}\nabla_x l_A(x_k, \lambda_k; \gamma_k) &= \nabla f(x_k) + \lambda_k \nabla h(x_k) + \frac{1}{\gamma_k} h(x_k)^\top \nabla h(x_k) \\ &= \nabla f(x_k) + \tilde{\lambda}_k \nabla h(x_k), \end{aligned} \tag{2.94}$$

其中

$$\tilde{\lambda}_k := \lambda_k + \frac{h(x_k)^\top}{\gamma_k}.$$

將 (2.94) 兩邊右乘 $\nabla h(x_k)^\top (\nabla h(x_k)\nabla h(x_k)^\top)^{-1}$ 並整理，得到

$$\tilde{\lambda}_k = (\nabla_x l_A(x_k, \lambda_k; \gamma_k) - \nabla f(x_k))\nabla h(x_k)^\top (\nabla h(x_k)\nabla h(x_k)^\top)^{-1}.$$

由於 $x_k \to x^*$ 且當 $k \to \infty$ 時 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \to 0$，因此

$$\tilde{\lambda}_k \to \lambda^* := -\nabla f(x^*)\nabla h(x^*)^\top (\nabla h(x^*)\nabla h(x^*)^\top)^{-1}.$$

在 (2.94) 中令 $k \to \infty$，可得

$$\nabla f(x^*) + \lambda^* \nabla h(x^*) = 0.$$

由於 λ_k 有界，並且 $\lambda_k + \frac{h(x_k)^\top}{\gamma_k} = \tilde{\lambda}_k \to \lambda^*$，故序列 $\{\frac{h(x_k)^\top}{\gamma_k}\}$ 有界。由於 $\gamma_k \to 0^+$，因此有 $h(x_k) \to 0$。當 $x_k \to x^*$ 且 h 連續時，可知 $h(x^*) = 0$。 □

我們注意到，命題 2.4.10 仍然要求 $\gamma_k \to 0$。在實踐中，通常有以下三種可能情況。

第一種情況是：對於某個 k，我們無法找到滿足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$ 的 x_k。這通常發生在 $l_A(\cdot, \lambda_k; \gamma_k)$ 無下界的時候。

第二種情況是：對每個 k，$\{x_k\}$ 都滿足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$，但 $\{x_k\}$ 不收斂，或者它收斂到一個不可行點 x^*。這通常發生在原始問題（等式約束）沒有可行解時。在這種情況下，$\{x_k\}$ 收斂到一個不可行點 x^*，並且該點是 $|h(x)|^2$ 的一個駐點，即 $h(x^*)^\top \nabla h(x^*) = 0$。

第三種情況是：$\{x_k\}$ 存在並收斂到一個點 x^*，該點是局部極小點但不是正則點，即 $\nabla h(x^*)$ 不具有滿行秩。在這種情況下，$\nabla h(x^*) \nabla h(x^*)^\top$ 不可逆，因此 $\lambda_k + \frac{h(x_k)}{\gamma_k}$ 可能不收斂。

我們指出，增廣 Lagrange 方法有時未必高效。這通常是因為當 $\gamma_k \to 0$ 時，$\nabla^2_{xx} l_A(\cdot, \lambda_k; \gamma_k)$ 的條件數會變得非常大。

在不等式約束優化中的應用 我們可以將增廣 Lagrange 方法用於同時帶有等式與不等式約束的優化問題。為此，我們可以先把不等式約束轉化為等式約束。這個過程可以通過引入**平方鬆弛變量**來完成，如下所示。

考慮帶有等式和不等式約束的一般優化問題

$$\min_{x \in \mathbb{R}^n} \quad f(x), \tag{2.95a}$$

$$\text{s.t.} \quad h(x) = 0, \tag{2.95b}$$

$$g_j(x) \le 0, \quad j = 1, \ldots, m, \tag{2.95c}$$

通過引入平方鬆弛變量 $y = (y_1, \ldots, y_m)^\top \in \mathbb{R}^m$，我們可以將 (2.95) 改寫成等價形式

$$\min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad f(x), \tag{2.96a}$$

$$\text{s.t.} \quad h(x) = 0, \tag{2.96b}$$

$$g_j(x) + y_j^2 = 0, \quad j = 1, \ldots, m, \tag{2.96c}$$

問題 (2.96) 的增廣 Lagrange 函數為

$$l_A(x, y, \lambda, \mu; \gamma) = f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2 + \mu \tilde{g}(x, y) + \frac{1}{2\gamma}|\tilde{g}(x, y)|^2, \tag{2.97}$$

其中

$$\tilde{g}(x,y) := \Big(\tilde{g}_1(x,y_1),\ldots,\tilde{g}_m(x,y_m)\Big)^\top \in \mathbb{R}^m$$

且

$$\tilde{g}_j(x,y_j) := g_j(x) + y_j^2, \quad \text{for } j = 1,\ldots,m.$$

當 λ、μ 和 γ 固定時，我們需要求解關於 (x,y) 的無約束子問題：

$$\min_{x,y}\ l_A(x,y,\lambda,\mu;\gamma). \tag{2.98}$$

若在 (2.98) 中固定 x，則 y 存在閉式解：首先，$y = (y_1,\ldots,y_m)$ 的各個分量彼此分離，每個 y_j 都求解

$$\min_{y_j}\ \mu_j(g_j(x)+y_j^2) + \frac{1}{2\gamma}(g_j(x)+y_j^2)^2,$$

這等價於

$$\min_{z_j\ge 0}\ \mu_j(g_j(x)+z_j) + \frac{1}{2\gamma}(g_j(x)+z_j)^2.$$

我們可以對上式中關於 z_j 的項配方，去掉與 z_j 無關的常數項，得到等價優化問題：

$$\min_{z_j\ge 0}\ \frac{1}{2\gamma}z_j^2 + \Big(\frac{1}{\gamma}\nabla g_j(x)^\top + \mu_j\Big)z_j,$$

其極小點可寫成閉式形式：

$$z_j^* = \max(0, -(g_j(x)+\gamma\mu_j)).$$

注意，z_j^* 依賴於 x、μ_j 和 γ。記

$$g_j^+(x,\mu_j;\gamma) := g_j(x) + z_j^* = \max(g_j(x), -\gamma\mu_j).$$

將 $y_j^2 = z_j^*$ 代回增廣 Lagrange 函數 (2.97)，得到一個不含 y 的簡化增廣 Lagrange 函數：

$$\begin{aligned} l_A(x,\lambda,\mu;\gamma) := f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2 \\ + \sum_{j=1}^m \Big(\mu_j g_j^+(x,\mu;\gamma) + \frac{1}{2\gamma}(g_j^+(x,\mu;\gamma))^2\Big). \end{aligned}$$

我們還可以通過在求和項中配方，進一步簡化 $l_A(x,\lambda,\mu;\gamma)$：

$$\begin{aligned}\mu_j g_j^+(x,\mu;\gamma)+\frac{1}{2\gamma}(g_j^+(x,\mu;\gamma))^2 &= \frac{1}{2\gamma}\Big((g_j^+(x,\mu_j;\gamma)+\gamma\mu_j)^2-\gamma^2\mu_j^2\Big)\\ &= \frac{1}{2\gamma}\Big(\max(g_j(x)+\gamma\mu_j,0)^2-\gamma^2\mu_j^2\Big)\\ &= \frac{\gamma}{2}\Big(\max\Big(\frac{1}{\gamma}g_j(x)+\mu_j,0\Big)^2-\mu_j^2\Big).\end{aligned}$$

其餘部分與增廣 Lagrange 方法（算法 2.4.4）相同，只不過乘子變成了 (λ,μ)。不過我們注意到，$\max(\cdot,0)^2$ 並不是二次可微的。這在極小化 $l_A(\cdot,\lambda_k,\mu_k;\gamma_k)$ 來求 x_k 時可能會帶來問題。

儘管如此，根據上面的分析，我們仍然期望當 $k\to\infty$ 時有

$$\lambda_k+\frac{1}{\gamma_k}h(x_k)^\top\to\lambda^*$$

以及

$$\mu_k+\frac{1}{\gamma_k}g^+(x_k,\mu_k;\gamma_k)=\max\Big(\mu_k+\frac{1}{\gamma_k}g(x_k),0\Big)\to\mu^*$$

其中最大值按分量取，且 (λ^*,μ^*) 是對應於 $h(x^*)=0$ 與 $g(x^*)\le 0$ 的 Lagrange 乘子。注意到 $\mu^*\ge 0$，這與 KKT 條件是一致的。

2.5 隨機優化算法

深度學習通常需要大規模的數據樣本，因此目標函數通常在一種特定的設置下構建，在這種設置中，隨機優化算法相比傳統的確定性方法更具適應性。在本節中，我們將介紹隨機優化算法，包括其形式、動機、優缺點，以及當前網絡訓練中廣泛使用的若干算法。特別地，我們關注其在求解非凸無約束優化問題中的應用，這類問題是神經網絡訓練中最常見的。

2.5.1 隨機優化的基礎

回顧標準的監督學習問題，我們通常給定一個數據集 $\mathcal{D}=\{(x_i,y_i)\in\mathbb{R}^d\times\mathbb{R}^m : i\in[N]\}$，其中包含 N 對樣本，且 N 可以非常大，例如 10^6 或更大。我們的目標是找到映射 $u:\mathbb{R}^d\to\mathbb{R}^m$，並將其參數化為具有參數 $\theta\in\mathbb{R}^n$ 的深度神經網絡 u_θ，使得對於所有 $i\in[N]$，$u_\theta(x_i)$ 近似等於 y_i。為簡便起見，假設我們將關於 θ 的損失函數 $\ell:\mathbb{R}^n\to\mathbb{R}$ 設為 $u_\theta(x_i)$ 與 y_i 之間的平方差的

平均值，並通過求解

$$\min_{\theta\in\mathbb{R}^n}\ \ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} f_i(\theta), \quad \text{其中} \quad f_i(\theta) = \frac{1}{2}|u_\theta(x_i) - y_i|^2. \tag{2.99}$$

由於 $\nabla_\theta f_i(\theta)$ 和 $\nabla_\theta^2 f_i(\theta)$ 可以通過自動微分（如第 2.3 節所述）計算，我們可以得到 $\ell(\theta)$ 的梯度和 Hessian。因此，從理論上講，我們可以應用第 2.4.1 節中介紹的任意無約束優化算法，例如梯度下降法、Newton 法以及擬 Newton 方法，來尋找 ℓ 的（局部）極小值。然而，事實證明，對於形式如 (2.99) 的損失函數，採用確定性優化算法在實際中往往不可行，或者至少效率較低。下面我們對此進行分析。

為了與本章中使用的符號保持一致，我們仍用 x 表示待優化變量（而不是 θ），並用 F_N 表示損失函數以強調其為 N 項的平均值，將優化問題 (2.99) 寫為

$$\min_{x\in\mathbb{R}^n}\ F_N(x) := \frac{1}{N}\sum_{i=1}^{N} f_i(x). \tag{2.100}$$

我們指出，F_N 也稱為**有限和**目標函數，這是一類在深度學習問題中常見的目標函數。它是（無約束）隨機優化問題一般形式的一個特例，後者通常寫為

$$\min_{x\in\mathbb{R}^n}\ F(x) := \mathbb{E}_\xi[f(x;\xi)], \tag{2.101}$$

其中 ξ 是一個隨機變量，其樣本（實現）可以是從 $\mathcal{D}$ 中獨立抽取的單個或一部分樣本，而 $\mathcal{D}$ 包含 N 個樣本，這些樣本來自某個（可能未知的）數據分佈。

例如，如果我們有一個數據集 $\mathcal{D} = \{z_i : i \in [N]\}$，稱為一個**批次**（batch），其中每個樣本 z_i 對應一個關於 x 的函數 f_i，並且 ξ 被設為從 $\mathcal{D}$ 中按均勻分佈抽取的一個樣本，則 (2.101) 可簡化為 (2.100)。也可以將 (2.100) 視為通過 Monte Carlo 積分（見附錄 A.1）對 (2.101) 的近似，其中使用的數據集為 $\mathcal{D}$。損失函數 F 和 F_N 分別稱為**期望損失**（expected loss）和**經驗損失**（empirical loss）。在本節的其餘部分，我們主要關注最小化 (2.100) 中的 F_N，並在需要時討論其與最小化 (2.101) 中 F 的聯繫。

如果將 (2.100) 視為一個確定性優化問題來求解，則標準梯度下降法在第 k 次迭代中的更新為

$$x_{k+1} = x_k - \frac{\alpha_k}{N}\sum_{i=1}^{N} \nabla f_i(x_k), \tag{2.102}$$

其中 $\alpha_k > 0$ 為步長。我們稱 (2.102) 為**批量梯度**（batch gradient method）（或**全梯度**）方法。這實際上是將梯度下降法直接應用於 (2.100)。

為了理解隨機優化，我們考慮求解 (2.100) 的基本隨機更新形式，在第 k 次迭代中為

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x), \tag{2.103}$$

其中 i_k 是從 $\{1, \ldots, N\}$ 中隨機抽取的，並且對所有 $j < k$，i_k 與 i_j 在統計上相互獨立。我們稱 (2.103) 為**基本隨機梯度**（basic stochastic gradient）方法。需要注意的是，(2.103) 中的 x_k 不僅依賴於 F_N、初始值 x_0 和步長 α_k，還依賴於隨機序列 $\{i_j : j < k\}$。實際上，$\{x_k : k \geq 1\}$ 可以看作一個隨機過程。

通過比較批量梯度方法 (2.102) 與基本隨機梯度方法 (2.103)，我們可能會覺得前者這種確定性優化方法更具吸引力。這是可以理解的，因為存在許多成熟的確定性優化算法，並且我們通常期望通過這些算法獲得良好的收斂速度和解的精度。此外，(2.100) 中 F_N 的結構還允許我們利用並行計算技術，從而顯著降低每次迭代的計算時間。

然而，從多個方面來看，隨機梯度方法 (2.103) 在深度學習應用中已被證明更加高效 [20]。例如，考慮一個網絡訓練問題，其中給定的數據集 $\mathcal{D}$ 構造不佳：$\mathcal{D}$ 由某個較小數據集 $\mathcal{D}_{\text{sub}}$ 的十個完全相同的副本組成。（儘管我們通常會避免這種情況，但在實際中採集的數據往往具有相似性和重複性。這裡僅用於提供一個直觀動機，說明批量梯度方法可能不如基本隨機梯度方法高效。）在這種情況下，批量梯度方法僅使用 $\mathcal{D}_{\text{sub}}$ 也會產生相同的迭代序列，但由於實際使用的是 $\mathcal{D}$，其計算成本卻增加了 10 倍。相比之下，基本隨機梯度方法的計算成本和收斂行為保持不變。這表明，與 (2.102) 中使用全部樣本相比，基本隨機梯度方法 (2.103) 可能更加高效。

實際上，已有大量文獻對基本隨機梯度方法與先進的確定性優化方法進行了比較。例如，一些研究結果表明，適當調節常數步長的基本隨機梯度方法在性能上顯著優於 limited-memory BFGS（L-BFGS）方法——這是 BFGS 方法（第 2.4.1 節）的一種高效且廣泛使用的變體。更具體地，在一個使用邏輯損失函數和 RCV1 數據集的二分類問題中，基本隨機梯度方法在前幾個 epoch 內顯著快於 L-BFGS 並達到較好的結果，而 L-BFGS 則在較後期才追趕並超過基本隨機梯度方法 [20]。

這一結果看起來令人驚訝且反直覺。然而，[13] 中的如下示例對此現象提供了一個有力的解釋。

例 2.5.1. 設 N 為奇整數，並令 $M = \frac{N-1}{2}$。我們將區間 $[-1, 1]$ 劃分為 $2M$

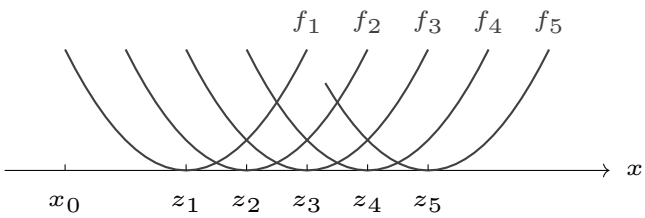


图 2.6: 當 $N=5$ 且 $M=\frac{N-1}{2}=2$ 時，(2.104) 中的 z_i 與 (2.105) 中的 f_i 的示意圖。這裡 $z_1,\ldots,z_N$ 是將區間 $[-1,1]$ 等分為 $2M=4$ 段後的網格點，其中 $z_1=-1$ 且 $z_N=1$。函數 $f_i(x)=(x-z_i)^2$ 是圖中所示的二次函數，而其平均函數 $F_N(x):=\frac{1}{N}\sum_{i=1}^N f_i(x)=x^2+\frac{1}{N}\sum_{i=1}^N z_i^2$（圖中未畫出）在 $x=0$ 處有唯一極小值（在此圖中也對應點 z_3）。我們可以考察在初值為 x_0 時，應用批量梯度下降法 (2.102) 和基本隨機梯度法 (2.103) 的收斂行為。基本隨機梯度下降法在初期以比批量梯度下降法更低的計算代價收斂到極小點 $x=0$，但當 x_k 接近 $x=0$ 時，它可能會 “迷失方向”，因為每次迭代它只使用一個隨機選取的 f_i 的梯度。

個等長小段，每段長度為 $\frac{1}{M}=\frac{2}{N-1}$。記該劃分的 $N=2M+1$ 個網格點為

$$z_i=-1+\frac{2(i-1)}{N-1},\quad i=1,\ldots,N. \tag{2.104}$$

注意到 $z_1=-1$ 且 $z_N=1$。現在定義

$$f_i(x):=(x-z_i)^2 \tag{2.105}$$

對所有 $i=1,\ldots,N$，並定義

$$F_N(x):=\frac{1}{N}\sum_{i=1}^N f_i(x)=\frac{1}{N}\sum_{i=1}^N (x-z_i)^2. \tag{2.106}$$

圖 2.6 給出了當 $N=5$ 且 $M=2$ 時 z_i 和 f_i 的示意圖。

注意到 $f_i(x)=(x-z_i)^2$ 是一個在 z_i 處取得最小值的二次函數。與此同時，平均函數 F_N 也是一個簡單的二次函數：

$$F_N(x)=x^2+\underbrace{\frac{1}{N}\sum_{i=1}^N z_i^2}_{\text{常数}}, \tag{2.107}$$

其在 $x=0$ 處取得唯一的最小值。

如果我們從一個距離極小點 0 很遠的初值 x_0 出發（見圖 2.6），對 (2.106) 應用批量梯度下降法 (2.102) 或者更先進的確定性優化算法，如 L-BFGS，那麼它們實際上是在最小化 (2.107)，並能夠生成一個穩定收斂到極小點 $x = 0$ 的迭代序列 $\{x_k\}$。相比之下，如果我們對同樣的初值 x_0 使用基本隨機梯度法 (2.103)，則可以預期生成的迭代序列 $\{x_k\}$ 在前幾個 epoch 中仍然會快速收斂到 0，但每次迭代的計算代價要低得多：批量梯度下降法需要計算所有 f_i 的梯度，而基本隨機梯度下降法在每次迭代中只需計算一個隨機選取的 f_i 的梯度，因此其代價僅為前者的 $1/N$。

該例子還表明，如果使用常數步長，那麼基本隨機梯度法可能只能保證收斂到極小點附近的某個鄰域。換言之，為了確保收斂到極小點，可能必須採用遞減步長策略。

從理論角度看，我們還可以對最小化 F_N 時，批量梯度下降法和基本隨機梯度法所需總計算成本的上界進行估計。注意到 F_N 是一個強凸且光滑的函數，批量梯度下降法實際上就是標準梯度下降法，並且已知具有線性收斂速度。更具體地，可以證明批量梯度法以線性速度收斂：

$$F_N(x_k) - F_N^* \leq O(\gamma^k),$$

其中 $F_N^* > \infty$ 是 F_N 在 $\mathbb{R}$ 上的（可能未知的）最小值，而 $\gamma \in (0,1)$ 依賴於比值 $L/\mu \geq 1$。這裡 L 是 ∇F_N 的 Lipschitz 常數，而 μ 是 F_N 的強凸參數，即

$$F_N(\hat{x}) \geq F_N(x) + \nabla F_N(x)(\hat{x} - x) + \frac{\mu}{2}|\hat{x} - x|^2$$

對所有 $x, \hat{x} \in \mathbb{R}^n$ 都成立。我們知道，當 L/μ 更大時，F_N 的病態性更嚴重，而 γ 也更接近於 1。（在本例中，由 (2.107) 定義的 F_N 滿足 $L = \mu = 1$。不過我們指出，即使在實際應用中 F_N 是強凸的，比值 L/μ 仍然可能非常大。）

由於上述線性收斂速度，可以很容易推出：為了得到 F_N 的一個 ϵ-最優解 x_k，即

$$F_N(x_k) - F_N^* \leq \epsilon$$

對任意用戶選定的誤差容忍度 $\epsilon > 0$，所需迭代次數 k 的量級為 $O(\log(1/\epsilon))$。由於批量梯度下降法每次迭代的計算代價為 N（因為它需要計算 $f_1, \ldots, f_N$ 的梯度以得到 F_N 的梯度），因此獲得一個 ϵ-最優解所需的總計算代價與 $N \log(1/\epsilon)$ 成正比。

另一方面，我們也可以考察基本隨機梯度下降法 (2.103) 在最小化 (2.106) 所定義的 F_N 時的計算代價。由於 F_N 是強凸的，可以證明基本隨機梯度法以

次線性（sublinear）速度收斂：

$$\mathbb{E}[F_N(x_k)] - F_N^* = O(1/k),$$

其中 x_k 是由基本隨機梯度下降法生成的第 k 個迭代點。上述收斂速度僅為次線性的，因此從**漸近**（asymptotically）意義上看，它比線性收斂慢得多。然而，上述次線性收斂速度並不依賴於樣本數 N。因此，為達到一個 ϵ-最優解所需的迭代次數為 $O(1/\epsilon)$。就每次迭代的計算代價而言，它為 1，因為基本隨機梯度下降法每次只需計算 $f_1, \ldots, f_N$ 中一個隨機選取函數的梯度。總而言之，其總計算代價與 $1/\epsilon$ 成正比，並且同樣不依賴於 N。

現在我們看到，批量梯度下降法 (2.102) 和基本隨機梯度下降法 (2.103) 所需的計算代價分別與 $N\log(1/\epsilon)$ 和 $1/\epsilon$ 成正比。當 N 固定時，隨著 $\epsilon \to 0$，我們知道從漸近意義上看，$N\log(1/\epsilon)$ 遠小於 $1/\epsilon$。然而，實際情況是，$\epsilon > 0$ 往往會事先被設定為一個足夠小的數，而樣本數 N 在收斂速度中起著重要作用，特別是在優化過程的早期階段。這就解釋了為什麼基本隨機梯度下降法在前幾個 epoch 中的收斂速度甚至可能快於 L-BFGS。

我們還要指出，隨機優化算法（例如基本隨機梯度下降法 (2.103)）的另一個優點在於：即使是在最小化如 (2.101) 所示的期望損失函數 F 時，它們仍然適用。相比之下，確定性優化算法（例如批量梯度算法 2.102）則變得不可行，因為在實踐中我們無法精確計算 F 的梯度。這些優點正是隨機優化算法如今在深度學習研究與應用中變得如此普遍的原因。

接下來，我們考慮隨機梯度方法的收斂性質和計算複雜度。比較批量梯度方法 (2.102) 與基本隨機梯度方法 (2.103)，可以看出它們每次迭代的計算代價分別是 N 和 1，因為 (2.102) 計算了 N 個梯度，而 (2.103) 只計算了一個。按照深度學習中的慣例，當一個方法在期望意義下或等價意義下用完給定數據集 $\mathcal{D}$ 中的全部 N 個樣本時，我們稱它完成了一個 epoch。因此，批量梯度方法 (2.102) 每次迭代完成一個 epoch，而基本隨機梯度方法 (2.103) 則每經過 N 次迭代完成一個 epoch。我們還將看到所謂的小批量隨機梯度方法，它在第 k 次迭代中使用 $\mathcal{D}$ 中的 N_k 個樣本（稱為一個 mini-batch），其中 N_k 介於 1 和 N 之間（為簡便起見，假設 N_k 不隨 k 改變），因此它每經過 N/N_k 次迭代完成一個 epoch。

隨機梯度方法的收斂性

現在我們來研究隨機梯度方法的收斂行為。正如前面指出的那樣，隨機梯度方法（例如 (2.103)）既可以用於尋找經驗損失 (2.100) 的極小值，也可以用於尋找期望損失 (2.101) 的極小值。對於後者，我們用 p_ξ 表示 ξ 的概率分佈。因此，在本節的其餘部分中，我們用 $F:\mathbb{R}^n\to\mathbb{R}$ 表示損失函數，它可以是上述兩種情形中的任意一種。隨機梯度（stochastic gradient，SG）方法的一般形式見算法 2.5.1。下面我們將更詳細地說明該算法中的各個步驟。

Algorithm 2.5.1 用於求解 (2.100) 或 (2.101) 的隨機梯度方法

Input: 初始猜測 $x_1\in\mathbb{R}^n$。
1: **for** $k=1,2,\ldots$ **do**
2: 生成 ξ_k 的一個實現值。
3: 計算 $g(x_k,\xi_k)$。
4: 選擇步長 $\alpha_k>0$。
5: 令 $x_{k+1}\leftarrow x_k-\alpha_k g(x_k,\xi_k)$。
6: **end for**

我們首先解釋算法 2.5.1 中第 2 行和第 3 行是如何實現的。具體來說，我們考慮 SG 方法在期望損失情形和經驗損失情形下的三種實現方式：基本 SG、小批量 SG，以及帶有二階信息的小批量 SG（為簡潔起見，後者下文稱為二階小批量 SG）。我們指出，在這三種情形中，$\{\xi_k:k\geq 1\}$ 都是一列聯合獨立的隨機變量。一旦確定了 ξ_k，向量 $g(x_k,\xi_k)$ 就根據所指定的 SG 方法定義為：

$$g(x_k,\xi_k)=\begin{cases}\nabla_x f(x_k;\xi_k) & \text{(基本 SG)}\\ \dfrac{1}{N_k}\displaystyle\sum_{i=1}^{N_k}\nabla_x f(x_k;\xi_{k,i}) & \text{(小批量 SG)}\\ H_k\dfrac{1}{N_k}\displaystyle\sum_{i=1}^{N_k}\nabla_x f(x_k;\xi_{k,i}) & \text{(二阶小批量 SG)}\end{cases}\tag{2.108}$$

在基本 SG 情形下，我們已經在有限和情形 (2.100) 中給出了更新公式 (2.103)。在這種情況下，ξ_k 指的是從 $\{1,\ldots,N\}$ 中按均勻分佈抽取的一個樣本索引。如果抽取到的索引為 $i_k\in\{1,\ldots,N\}$，則在算法 2.5.1 的第 2 行中有 $\xi_k=i_k$，而在第 3 行中有 $g(x_k,\xi_k)=\nabla f_{i_k}(x_k)$。如果將基本 SG 應用於 (2.101) 中定義的期望損失 F，那麼 ξ_k 是從 p_ξ 中抽取的一個樣本，並且我們令 $g(x_k,\xi_k)=\nabla_x f(x_k,\xi_k)$。

在小批量 SG 情形下，對於經驗損失，ξ_k 指的是從 $\{1,\ldots,N\}$ 中均勻抽取的 N_k 個索引（其中 N_k 稱為小批量大小，它由用戶預先選定），而 $g(x_k,\xi_k)$

是在 ξ_k 所選索引處對 f_i 的梯度取平均。其顯式形式見 (2.108)。類似地，在期望損失情形中，ξ_k 是從 p_ξ 中獨立抽取的 N_k 個樣本組成的集合，而 $g(x_k, \xi_k)$ 則是在 ξ_k 所指定樣本上梯度的平均值。

二階小批量 SG 情形與前一種情況類似，只是額外引入了一個對稱正定的縮放矩陣 H_k。這個矩陣是基於不同的二階信息引入策略計算出來的。選擇 H_k 的一個一般性原則是：在對 ξ_k 取期望的意義下，$-g(x_k, \xi_k)$ 應當是一個下降方向。關於 H_k 的構造，我們將在後面給出更多細節。

現在我們列出建立算法 2.5.1 中隨機梯度下降方法收斂性所需的一些假設。

假設 2.5.2. 設損失函數 $F : \mathbb{R}^n \to \mathbb{R}$ 連續可微，並且滿足以下假設。

1. $F^* := \inf_{x \in \mathbb{R}^n} F(x)$ 是有限的。

2. 梯度 ∇F 是 L-Lipschitz 連續的，即

$$|\nabla F(\hat{x}) - \nabla F(x)| \le L|\hat{x} - x|, \quad \forall\, x, \hat{x} \in \mathbb{R}^n.$$

3. 存在標量 μ 和 μ_G，滿足 $0 < \mu \le \mu_G$，使得對所有 k 都有

$$\nabla F(x_k) \cdot \mathbb{E}_{\xi_k}[g(x_k, \xi_k)] \ge \mu |\nabla F(x_k)|^2, \tag{2.109}$$

$$|\mathbb{E}_{\xi_k}[g(x_k, \xi_k)]| \le \mu_G |\nabla F(x_k)|. \tag{2.110}$$

4. 存在標量 $M, M_G \ge 0$，使得對所有 k 都有

$$\mathbb{E}_{\xi_k}[|g(x_k, \xi_k)|^2] \le M + M_G |\nabla F(x_k)|^2. \tag{2.111}$$

假設 2.5.2 中前兩個條件較為常見，並且在實際應用中通常都成立。條件 (2.109) 要求在對 ξ_k 取期望的意義下，$g(x_k, \xi_k)$ 與 $\nabla F(x_k)$ 之間的夾角應小於 $\pi/2$。這是要求 $-g(x_k, \xi_k)$ 在期望意義下成為下降方向的一個典型條件。條件 (2.110) 實質上要求 $g(x_k, \xi_k)$ 的期望（即一階矩，first moment）對每個 k 都是 $O(|\nabla F(x_k)|)$ 量級。結合 (2.109) 可知，在對 ξ_k 取期望時，$|g(x_k, \xi_k)|$ 應與 $|\nabla F(x_k)|$ 成比例。條件 (2.111) 要求 $g(x_k, \xi_k)$ 的二階矩（second moment）對每個 k 都能被某個常數 $M > 0$ 與 $O(|\nabla F(x_k)|^2)$ 之和所控制。

注意，如果 $g(x_k, \xi_k)$ 是 $F(x_k)$ 的無偏估計，即

$$\mathbb{E}_{\xi_k}[g(x_k, \xi_k)] = \nabla F(x_k),$$

那麼條件 (2.109) 和 (2.110) 都以 $\mu = \mu_G = 1$ 成立。在許多 SG 方法的標準設定中，例如基本 SG 方法，可以證明 $g(x_k, \xi_k)$ 確實是 $\nabla F(x_k)$ 的無偏估計。

在假設 2.5.2 下，我們可以得到第 k 次迭代中 $\mathbb{E}_{\xi_k}[F(x_{k+1})]$ 與 $F(x_k)$ 之差的一個簡單上界，如下引理所示。

引理 2.5.3. *設假設 2.5.2 中的條件成立。令 x_k 為從任意初始點 x_0 出發，由算法 2.5.1 中的隨機梯度方法生成的迭代序列。則有*

$$\mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) \le -\Big(\mu - \frac{1}{2}\alpha_k L M_G\Big)\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 L M. \quad (2.112)$$

证明. 由於 ∇F 是 L-Lipschitz 連續的，我們有

$$\begin{aligned} F(x_{k+1}) - F(x_k) &\le \nabla F(x_k) \cdot (x_{k+1} - x_k) + \frac{L}{2}|x_{k+1} - x_k|^2 \\ &= -\alpha_k \nabla F(x_k) \cdot g(x_k, \xi_k) + \frac{\alpha_k^2 L}{2}|g(x_k, \xi_k)|^2. \end{aligned}$$

對兩邊關於 $\xi_k \sim p_\xi$ 取期望，並注意到 x_k 與 ξ_k 相互獨立（因為 ξ_k 是在 x_k 計算出來之後獨立生成的），可得

$$\mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) \le -\alpha_k \nabla F(x_k) \cdot \mathbb{E}_{\xi_k}[g(x_k, \xi_k)] + \frac{\alpha_k^2 L}{2}\mathbb{E}_{\xi_k}[|g(x_k, \xi_k)|^2].$$

將條件 (2.109) 和 (2.111) 分別應用於上式右端的第一項和第二項，得到

$$\begin{aligned} \mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) &\le -\alpha_k \mu |\nabla F(x_k)|^2 + \frac{\alpha_k^2 L}{2}\Big(M + M_G[|\nabla F(x_k)|^2]\Big) \\ &= -\Big(\mu - \frac{1}{2}\alpha_k L M_G\Big)\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 L M, \end{aligned}$$

從而完成證明。 □

現在我們已準備好證明算法 2.5.1 中隨機梯度方法的收斂性。我們在兩種步長 α_k 的設定下考察該方法的收斂性。第一種設定是在所有迭代中使用常數步長，第二種設定是使用預先給定的自動遞減步長。

定理 2.5.4 (具有常數步長的 SG)**.** *設假設 2.5.2 中的條件成立。令 x_k 為由算法 2.5.1 中的隨機梯度方法從任意初值 x_0 出發生成的迭代序列，並且對所有 k 取常數步長 $\alpha_k = \alpha$，其中*

$$0 < \alpha \le \frac{\mu}{L M_G}. \quad (2.113)$$

則有

$$\mathbb{E}\Big[\sum_{k=1}^{K}|\nabla F(x_k)]|^2\Big] \le \frac{K\alpha LM}{\mu} + \frac{2(F(x_1)-F^*)}{\mu\alpha}. \tag{2.114}$$

因此，

$$\mathbb{E}\Big[\frac{1}{K}\sum_{k=1}^{K}|\nabla F(x_k)]|^2\Big] \le \frac{\alpha LM}{\mu} + \frac{2(F(x_1)-F^*)}{\mu\alpha K}. \tag{2.115}$$

當 $K\to\infty$ 時，上式收斂到正標量 $\alpha LM/\mu$。

证明. 對 (2.112) 關於 $\xi_1,\ldots,\xi_k$ 取期望（注意對每個 k，x_k 由 $\xi_1,\ldots,\xi_{k-1}$ 決定），並利用 (2.113) 中對 α 的約束，可得

$$\begin{aligned}\mathbb{E}[F(x_{k+1})]-\mathbb{E}[F(x_k)] &\le -\Big(\mu-\frac{1}{2}\alpha LM_G\Big)\alpha|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM\\ &\le -\frac{1}{2}\mu\alpha|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM.\end{aligned}$$

對兩邊從 $k=1,2,\ldots,K$ 求和，得到

$$F^*-F(x_1)\le\mathbb{E}[F(x_{k+1})]-F(x_1)\le-\frac{1}{2}\mu\alpha\sum_{k=1}^{K}|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM,$$

其中第一個不等式來自於對所有 k 都有 $F(x_{k+1})\ge F^*$。對該不等式整理，即得 (2.114)。再將 (2.114) 兩邊除以 K，即可得到 (2.115)。 □

我們可以看到，(2.115) 中得到的上界並不好，因為除非 $M=0$，否則當 $K\to\infty$ 時它並不會趨於 0。然而，由 (2.111) 可知，$M=0$ 意味著二階矩 $\mathbb{E}_{\xi_k}[|g(x_k,\xi_k)|^2]$ 必須以與 $|\nabla F(x_k)|^2$ 一樣快的速度衰減。即使 $g(x_k,\xi_k)$ 是 $\nabla F(x_k)$ 的無偏估計，這仍然很難做到，因為這實際上要求 $g(x_k,\xi_k)$ 中的噪聲（方差）趨於 0，而這在現實應用中很少能夠實現。

儘管如此，我們仍然可以從估計式 (2.115) 中得到一些啟發。其收斂到一個依賴於 M 和步長 α 的正數，這解釋了我們在上面的例 2.5.1 中看到的現象：基本 SG 方法在初期可以很高效，但最終可能只是在極小點 $x=0$ 的某個鄰域中徘徊，而不是收斂到 0 本身。通過使用更小的步長 α，這個鄰域的大小可以減小，但這顯然會拖慢整體收斂速度，因此未必值得。

接下來，我們考慮算法 2.5.1 中 SG 方法在遞減步長下的收斂性。我們採

用經典文獻 [119] 中刻畫的步長規則:

$$\sum_{k=1}^{\infty} \alpha_k = \infty \qquad \text{和} \qquad \sum_{k=1}^{\infty} \alpha_k^2 < \infty. \tag{2.116}$$

於是我們得到如下結果。

定理 2.5.5 (具有遞減步長的 SG)**.** *設假設 2.5.2 中的條件成立。令 x_k 為由算法 2.5.1 中的隨機梯度方法從任意初值 x_0 出發生成的迭代序列，其步長 α_k 滿足規則* (2.116)*。則有*

$$\lim_{K\to\infty} \mathbb{E}\Big[\sum_{k=1}^{K} \alpha_k |\nabla F(x_k)]|^2\Big] < \infty, \tag{2.117}$$

從而

$$\lim_{K\to\infty} \mathbb{E}\Big[\frac{1}{A_K}\sum_{k=1}^{K} \alpha_k |\nabla F(x_k)]|^2\Big] = 0, \tag{2.118}$$

其中 $A_K := \sum_{k=1}^{K} \alpha_k$。

证明. 注意到 (2.116) 中的條件 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$ 蘊含 $\alpha_k \to 0$ 當 $k \to \infty$。因此，不妨設對所有 k 都有 $\alpha_k \le \frac{\mu}{LM_G}$。於是，對 (2.112) 關於 $\xi_1, \ldots, \xi_k$ 取期望，得到

$$\begin{aligned}\mathbb{E}[F(x_{k+1})] - \mathbb{E}[F(x_k)] &\le -\Big(\mu - \frac{1}{2}\alpha_k LM_G\Big)\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 LM \\ &\le -\frac{1}{2}\mu\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 LM.\end{aligned}$$

對兩邊從 $k = 1, 2, \ldots, K$ 求和，得到

$$F^* - F(x_1) \le \mathbb{E}[F(x_{k+1})] - F(x_1) \le -\frac{1}{2}\mu \sum_{k=1}^{K} \alpha_k |\nabla F(x_k)|^2 + \frac{1}{2} LM \sum_{k=1}^{K} \alpha_k^2.$$

整理該不等式，可得

$$\sum_{k=1}^{K} \alpha_k |\nabla F(x_k)|^2 \le \frac{2(F(x_1) - F^*)}{\mu} + \frac{LM}{\mu}\sum_{k=1}^{K} \alpha_k^2. \tag{2.119}$$

再次利用 (2.116) 中 $\sum_{k=1}^{\infty} \alpha_k^2 < \infty$，我們知道右端隨著 $K \to \infty$ 單調增加且有有限上界，這就推出了 (2.117)。

Algorithm 2.5.2 子採樣無 Hessian 不精確 Newton 法

Input: 初始猜測 $x_1 \in \mathbb{R}^n$，常數 $\rho, \gamma, \eta \in (0,1)$，$K_{\mathrm{CG}} \in \mathbb{N}$。

1: **for** $k = 1, 2, \ldots$ **do**

2: 生成 ξ_k 和 ξ_k^H 的實現值。

3: 計算 $g(x_k, \xi_k)$。

4: 通過無 Hessian 的 CG 求解

$$H(x_k; \xi_k^H)s = -g(x_k; \xi_k) \tag{2.120}$$

來計算 s_k；如果達到最大迭代次數 K_{CG}，或者滿足

$$|H(x_k; \xi_k^H)s + g(x_k, \xi_k)| \le \rho |g(x_k; \xi_k)| \tag{2.121}$$

則終止 CG。

5: 通過回溯線搜索並使用衰減率 γ 選擇步長 $\alpha_k > 0$，使得

$$f(x_{k+1}; \xi_k) \le f(x_k; \xi_k) + \eta\, \alpha_k\, g(x_k; \xi_k) \cdot s_k \tag{2.122}$$

其中 $x_{k+1} = x_k + \alpha_k s_k$。

6: **end for**

進一步地，將 (2.119) 兩邊除以 A_K，並注意到 (2.116) 中條件 $A_K = \sum_{k=1}^K \alpha_k \to \infty$ 當 $K \to \infty$，即可得到 (2.118)。 □

基於定理 2.5.5，我們很容易作出如下判斷：

$$\liminf_{k\to\infty} \mathbb{E}[|\nabla F(x_k)|^2] = 0.$$

換言之，序列 $\{\mathbb{E}[|\nabla F(x_k)|^2] : k \ge 1\}$ 必然存在一個收斂到 0 的子序列。

二階隨機優化方法

正如 [20] 所指出的，二階信息（也稱曲率信息）對於消除 SG 方法中的某些關鍵問題（例如病態性）並顯著提升其性能是非常有幫助的。這受到二階方法在求解廣泛的確定性優化問題中取得巨大成功的啟發，例如擬 Newton 方法這一類方法。我們再次強調，使用二階信息並不意味著必須計算或存儲 F 的 Hessian 矩陣；對於深度學習問題而言，當維數 n 達到數百萬甚至數十億時，這通常是不可行的，或者至少在計算上代價極其高昂。

我們用不精確 Newton 法的一個變體來說明二階信息在隨機優化算法中的使用。具體地，我們考慮子採樣的 Hessian-free inexact Newton 法作為一個例子。該方法總結於算法 2.5.2。

為了實現該方法，我們注意到，除了如 (2.108) 中用於計算 $g(x_k, \xi_k)$ 的大小為 N_k 的 ξ_k 之外，還需要再採樣一個大小為 N_k^H 的實現 ξ_k^H，從而得到一個隨機 Hessian 估計

$$H(x_k; \xi_k^H) := \frac{1}{N_k^H} \sum_{i=1}^{N_k^H} \nabla^2 f(x_k; \xi_{k,i}^H). \tag{2.123}$$

研究發現，該方法對估計 $H(x_k; \xi_k^H)$ 中的噪聲似乎不像對 $g(x_k; \xi_k)$ 中的噪聲那樣敏感。因此，通常將 N_k^H 取得比 N_k 更小，以降低計算代價。注意，這裡並不會顯式地計算和存儲 $H(x_k; \xi_k^H)$。相反，該方法使用共軛梯度（CG）算法（算法 2.4.2）來通過線性系統 (2.120) 求解 $s_k \in \mathbb{R}^n$，而這只需要若干次矩陣-向量乘法（關於如何利用自動微分計算這類乘法的細節，見第 2.3 節）。這正是該方法被稱為 Hessian-free 的原因。

雖然在 $H(x_k; \xi_k^H)$ 正定的前提下，CG 算法被證明能夠在有限步內精確求解 (2.120)，但實際中通常會提前終止 CG：要麼當達到預設的最大迭代次數 K_{CG}，要麼當 (2.120) 兩邊之差足夠小，即滿足 (2.121) 時。由 CG 算法為 (2.120) 得到的解 s_k 起到下降方向的作用，而隨後在固定 ξ_k 的情況下，採用標準的回溯線搜索策略來確定步長 α_k，並進而得到下一個迭代點 x_{k+1}。

2.5.2 實用隨機優化算法

儘管上述介紹的標準隨機梯度下降方法仍然是最基本的方法，但它們的侷限性促使人們發展出自適應優化方法（adaptive optimization methods）。在本小節中，我們介紹若干當前深度學習中廣泛使用的自適應優化器，並對其動機與性質作一些說明。然而，在寫作本文時，這些方法在僅假設溫和正則性條件下求解具有光滑非凸目標函數的隨機優化問題時，尚未擁有完整的收斂性保證。對其收斂行為的研究目前仍是一個活躍的研究方向。

儘管如此，這些方法在經驗上看起來比標準的基本/小批量 SG 方法更快、更高效（也更穩定）。因此，它們往往是深度學習研究和工業應用中訓練網絡的首選方法。與此同時，未來極有可能還會不斷髮明出更高效、更穩定的隨機優化算法。

動量方法

為簡化記號，我們用 g_k 表示諸如 (2.108) 中的隨機下降方向：

$$g_k := g(x_k; \xi_k).$$

加速收斂的兩個關鍵技術之一是使用動量（momentum）。動量的思想是引入過去梯度的加權滑動平均來改進 g_k。這有助於平滑優化軌跡，從而通過減小振盪使算法更快收斂。帶有**動量**的基本隨機梯度下降方法寫為

$$m_{k+1} = \beta_1 m_k + (1-\beta_1) g_k, \tag{2.124a}$$

$$x_{k+1} = x_k - \alpha_k m_{k+1}, \tag{2.124b}$$

其中 $m_k \in \mathbb{R}^n$ 是第 k 次迭代中的動量，α_k 是步長（學習率）。動量 m_1 初始化為 0，權重參數 β_1 在實踐中通常取為 0.9。

儘管 (2.124) 能帶來速度提升，步長 α_k 仍然依賴於待最小化目標函數梯度的 Lipschitz 常數，並且為了收斂還需要滿足遞減策略。這給實踐者帶來了額外的調參負擔，以便為這些步長找到最優表現。下面將介紹的**自適應方法**具有抵消 Lipschitz 常數影響的特徵，並且常常可以在固定步長下正常工作。這些自適應方法實際上是在使用對角縮放矩陣 H_k，參見 (2.108)，以融入目標函數的二階信息。

自適應梯度（AdaGrad）

Adaptive gradient（AdaGrad）是最早為大規模和稀疏優化問題提出的自適應方法之一 [33]。它在每次迭代中引入一個向量形式的原始二階矩估計 $v_k \in \mathbb{R}^n$。然後利用 v_k 的各個分量分別對 g_k 中對應的分量進行縮放。AdaGrad 的形式為

$$v_{k+1} = v_k + g_k \odot g_k, \tag{2.125a}$$

$$x_{k+1} = x_k - \alpha_k \frac{g_k}{\sqrt{v_{k+1}} + \epsilon}, \tag{2.125b}$$

其中 $\odot$ 表示兩個向量的分量乘積，結果仍為同維向量，參數 $\epsilon > 0$ 通常取為 10^{-8}，以保證分母遠離 0。(2.125b) 中的平方根和除法也都是逐分量進行的。因此，在 (2.125b) 中，g_k 的各個分量是分別被縮放的，而該比值對於目標函數乘以任意正的常數縮放是不變的（這種縮放會改變目標函數的 Lipschitz 常

數)。因此，步長 α_k 不受這種縮放影響，從而在迭代過程中可以保持固定。

AdaGrad 在具有稀疏特徵的情形中尤其有效，例如自然語言處理，因為不常出現的參數會獲得相對更大的更新。然而，它的主要缺點是分母單調增加，這使得 (2.125b) 中的比值迅速趨於 0。這往往會導致過早收斂以及訓練過程停滯。

均方根傳播（RMSProp）

Root mean square propagation（RMSProp）[145] 被提出作為對 AdaGrad 的一種改進，以緩解其學習率衰減過快的問題。RMSProp 的形式為

$$v_{k+1} = \beta_2 v_k + (1-\beta_2) g_k \odot g_k, \tag{2.126a}$$

$$x_{k+1} = x_k - \alpha_k \frac{g_k}{\sqrt{v_{k+1}} + \epsilon}, \tag{2.126b}$$

其中 β_2 通常取為 0.99，且 v_1 初始化為 0。由此可見，(2.126a) 中的加權求和緩解了平方梯度不斷累積的問題，因此 (2.126b) 中的分母向量不會像 AdaGrad 中那樣快速增大。

由於最近的梯度有較高權重，RMSProp 在許多應用中表現良好，並曾廣泛用於訓練循環神經網絡。然而，RMSProp 似乎對超參數選擇較為敏感。

自適應矩估計（Adam）

Adaptive Momentum Estimate（Adam）優化器 [76] 結合了動量與 RMSprop 技術，從而提供更平衡、更高效的優化過程。Adam 的關鍵方程如下：Adam 通過同時維護梯度的一階矩和二階矩的指數滑動平均，融合了動量方法與 RMSProp 的思想。具體地，在第 k 次迭代中，Adam 按如下方式更新一階矩估計 m_k 和原始二階矩估計 v_k：

$$m_{k+1} = \beta_1 m_k + (1-\beta_1) g_k, \tag{2.127a}$$

$$v_{k+1} = \beta_2 v_k + (1-\beta_2) g_k \odot g_k, \tag{2.127b}$$

其初值為 $m_1 = v_1 = 0$。由於這樣的初值會使 m_k 和 v_k 向 0 偏置，Adam 試圖通過它們的修正版本來校正這種偏差：

$$\hat{m}_{k+1} = \frac{m_{k+1}}{1-\beta_1^k}, \tag{2.128a}$$

$$\hat{v}_{k+1} = \frac{v_{k+1}}{1-\beta_2^k}, \tag{2.128b}$$

其中，只要 $\beta_1, \beta_2 \in (0,1)$，(2.128) 中兩個修正分母都會在 $k \to \infty$ 時趨於 1。然後 Adam 按如下方式更新 x_k：

$$x_{k+1} = x_k - \alpha_k \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}. \tag{2.129}$$

超參數通常仍與前面類似：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$。步長 α_k 可以固定為常數，並且通常取 10^{-3}，這一般效果較好，但有時也可能小到 10^{-5}。

Adam 因其在廣泛的網絡結構和數據集上都具有快速收斂性和魯棒性而被廣泛採用。然而，後續研究表明，在某些情形下 Adam 的泛化性能可能不如帶動量的 SG，並且在某些理論設定下甚至可能不收斂 [116]。

帶解耦權重衰減的 Adam（AdamW）

控制 x 的取值並穩定優化過程的一種典型方法，是直接在目標函數中加入 Tikhonov 正則化。也就是說，可以將某個優化算法，例如 Adam，應用於最小化 $f(x) + \frac{\lambda}{2}|x|^2$，而不是 $f(x)$。其中 λ 是用於權衡項 $|x|^2$ 的超參數。正如我們所見，Tikhonov 正則化傾向於減小 x 各個分量的幅值。然而，這種修改會改變目標函數、它的一階矩和二階矩，最重要的是，它還會改變最優解本身。

Adam with Decoupled Weight Decay（AdamW） [90] 則提供了一種替代方案：在更新 x 時直接添加一個衰減項。更具體地說，AdamW 進行與 Adam 中相同的計算 (2.127)–(2.128)，但僅將 (2.129) 改為

$$x_{k+1} = x_k - \alpha_k \lambda x_k - \alpha_k \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}, \tag{2.130}$$

這與 (2.129) 相比多了一個額外項 $\alpha_k \lambda x_k$。這裡 λ 被稱為衰減權重，在實際中通常取為 0.01。

在多種經驗測試中，相較於 Adam，AdamW 似乎能夠改善泛化能力和穩定性，特別是在諸如 transformer 這樣的大規模模型中。因此，AdamW 已成

為許多現代深度學習框架中的默認優化器。

由 Newton-Schulz 正交化的動量（Muon）

Momentum Orthogonalized by Newton-Schulz（Muon） [73] 是一種利用深度神經網絡中權重參數奇異值的優化方法。回憶一下，深度神經網絡中的大多數構件都具有形如 $Wx+b$ 的仿射操作，其中 W 是權重矩陣。為便於表述，我們稱 W 為一個二維（2D）權重矩陣，並在下文中假設其大小為 $d_{\text{out}} \times d_{\text{in}}$。

注意，在我們前面討論的所有優化算法中，W 都被重新展平為一個向量，並與 b 一起拼接成總參數向量 $\theta \in \mathbb{R}^n$ 的一部分（本章中我們使用 $x \in \mathbb{R}^n$，但在網絡訓練中它指的就是參數 $\theta \in \mathbb{R}^n$）。因此，在網絡訓練的優化過程中，W 的矩陣結構被忽略了。

然而，權重矩陣在仿射變換中扮演著重要角色，並且對網絡的 Lipschitz 常數有很強的影響；該網絡本質上是一個從輸入映射到對應輸出的映射。因此，Muon 提出通過正交化來正則化關於權重矩陣的梯度。為此，Muon 對矩陣動量 $\mu M_k + g_k$ 施加若干次 Newton–Schulz (NS) 迭代，以將所有奇異值歸一化為 1。Muon 的形式為

$$M_{k+1} = \mu M_k + g_k, \tag{2.131a}$$

$$O_{k+1} = \mathrm{NS}(M_{k+1}; (a,b,c), \epsilon, K_{\mathrm{NS}}), \tag{2.131b}$$

$$x_{k+1} = x_k - \alpha\lambda x_k - \alpha O_k, \tag{2.131c}$$

其中權重參數 μ 通常取為 0.95，λ 取為 0.1，步長 α 取為 0.001。

現在我們來解釋 (2.131) 中的各項。首先注意，Muon 中的 g_k 指的是目標函數關於神經網絡中每一個單獨權重矩陣的（隨機）梯度。Muon 也可以用於更新四維卷積參數，只需將四個維度中的三個恰當地展平，使其變成類似權重矩陣那樣的二維矩陣。至於其他形式為標量或向量的參數，其梯度仍使用先前介紹的算法來更新，例如 Adam 和 AdamW。因此，下文中我們假設 g_k 是一個二維矩陣，且其大小與對應權重矩陣 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 相同。動量 M_k 也是一個同樣大小的矩陣。

Muon 方案中的關鍵步驟是 (2.131b)。對於任意一個大小為 $m \times n$ 的矩陣 M 作為輸入（不失一般性，下文中假設 $m \geq n$），該步驟返回一個與其同樣大小的矩陣 O，作為 $UV^\top$ 的近似，其中 $M = U\Sigma V^\top$ 是 M 的**奇異值分解**（singular value decomposition，SVD）。我們知道，SVD 可以將任意實矩陣或復矩陣 M 分解為 $U\Sigma V^\top$，其中 $U \in \mathbb{R}^{m\times n}$ 的 n 個列向量在 $\mathbb{R}^m$ 中正交歸一

（即，U 的列向量是 $\mathbb{R}^m$ 的某組標準正交基的一個子集），V 是一個 $n \times n$ 的正交矩陣，而 Σ 是一個 $n \times n$ 的對角矩陣，其非負對角元稱為 M 的**奇異值**。因此，U 和 V 的列向量分別稱為 M 的左、右**奇異向量**。關於 SVD 的細節可參見 [49,148]。因此，步驟 (2.131b) 的作用是從 $M = U\Sigma V^\top$ 中去掉 Σ，並近似得到 M 的正交版本 $O = UV^\top$。也就是說，O 是如下問題的解

$$\min_{O \in \mathbb{R}^{m \times n}} \frac{1}{2} \|O - M\|_F^2, \tag{2.132}$$

其中 $\|A\|_F := (\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2)^{1/2}$ 是 $m \times n$ 矩陣 $A = [A_{ij}]_{i,j}$ 的 Frobenius 範數。這實際上等價於將 A 重新看作一個 mn 維向量並施加標準的向量 2-範數。此外，Frobenius 範數是由內積 $\langle A, B \rangle_F := \mathrm{tr}(A^\top B)$（對任意 $A, B \in \mathbb{R}^{m \times n}$）誘導出的範數。由於 Frobenius 範數具有這些性質，很容易看出 (2.132) 的極小化解是 $UV^\top$。這就是 Muon 中所謂的正交化。

然而，SVD 的計算代價較高，因此在實踐中往往希望避免用它來求解 (2.132)。NS 迭代是一種能夠快速逼近 (2.132) 解的方法，因此 Muon 在 (2.131b) 中採用了它。

NS 迭代的思想如下。我們定義關於 σ 的一個五次多項式 $\phi : \mathbb{R} \to \mathbb{R}$（這裡的 σ 指矩陣的一個正奇異值）：

$$\phi(\sigma) := a\sigma + b\sigma^3 + c\sigma^5. \tag{2.133}$$

我們也將這一定義擴展到矩陣 M 上，並把 ϕ 作用於矩陣：

$$\begin{aligned}
\phi(M) &:= aM + b(MM^\top)M + c(MM^\top)^2 M \\
&= \Big(aI_m + b(MM^\top) + c(MM^\top)^2\Big)M \\
&= \Big(aI_m + b(U\Sigma^2 U^\top) + c(U\Sigma^4 U^\top)\Big)U\Sigma V^\top \\
&= U\Big(a\Sigma + b\Sigma^3 + c\Sigma^5\Big)V^\top,
\end{aligned} \tag{2.134}$$

其中我們重複利用了 SVD 分解 $M = U\Sigma V^\top$ 及 $U^\top U = I_n$ 來得到上述等式。我們看到，(2.134) 中的 $\phi(M)$ 實際上是將 (2.133) 分別作用於 M 的每一個奇異值 σ。

因此，為了讓 NS 迭代快速將所有奇異值歸一化為 1，我們只需找到 (2.133) 中的係數 (a, b, c)，使得對任意正數 σ，經過少數幾次 ϕ 的迭代後都能映射到

1。換句話說，我們希望找到 (2.133) 中的 (a,b,c)，使得

$$\phi^{(j)}(\sigma) \approx 1, \quad \forall \sigma > 0$$

並儘可能讓 j 小，其中 $\phi^{(j)}$ 表示 ϕ 複合 j 次。如果能夠找到這樣的 (a,b,c)，那麼對於任意 $M = U\Sigma V^\top$，我們就可以用 $\phi^{(j)}(M)$ 來近似 $O = UV^\top$。

在實踐中，簡單的取值 $(a,b,c) = (2,-1.5,0.5)$ 看起來已經足夠有效。經過一些細緻優化後，取值 $(a,b,c) = (3.4445,-4.775,2.0315)$ 似乎效果更好，因此目前已成為 PyTorch 等流行深度學習編程包中的默認實現設置 [107]。在 Muon 方法中，(2.131b) 中的 NS 迭代通常在達到最大迭代次數 K_{NS}（通常 5 次就足夠）時終止，或者當 (a,b,c) 的相對變化小於 ϵ（通常取為 10^{-7}）時終止。

我們還可以將 Nesterov 動量的思想融入 Muon 方法 (2.131) 中。這通常通過在 (2.131b) 中用 $g_k + \mu B_{k+1}$ 代替 B_{k+1} 來實現，其中 μ 的默認取值為 0.95。

在實際實現中，對步長也有若干修改方式。特別地，在 [73] 中，步長 α 被乘以 $(\max(1, d_{\mathrm{out}}/d_{\mathrm{in}}))^{1/2}$；而在 [88] 中，則乘以 $0.2(\max(d_{\mathrm{out}}, d_{\mathrm{in}}))^{1/2}$。這些做法在大語言模型訓練中經驗上都表現得較為高效。

2.6 備註與參考文獻

2.6.1 關於網絡訓練優化的備註

本章涵蓋了優化理論中的大多數基本概念，以及若干優化算法和技術。重點放在非凸光滑優化問題上，這類問題在現代深度學習中極為常見。事實上，在過去幾十年中，人們已經發展出大量關於確定性優化方法的結果，包括系統的收斂性分析和成熟的軟件包。然而，在求解現代大規模優化問題時，它們似乎表現乏力 [20]。相比之下，深度學習領域的研究人員正在不斷髮明新的隨機優化算法，以應對此類問題帶來的挑戰。儘管這些算法通常尚未具備完整而嚴格的收斂性保證，但它們驚人的效率使其在當今大型深度網絡訓練中佔據主導地位。

除了 [20] 中給出的評論之外，我們還從實踐角度對這一現象補充幾點說明。眾所周知，訓練大規模神經網絡對資源的需求極其巨大。它需要大量圖形處理單元（graphics processing unit，GPU）以及巨大的電力消耗。一些大型網絡，例如為現代大語言模型設計的網絡，每個都擁有數十億到數千億個參數，

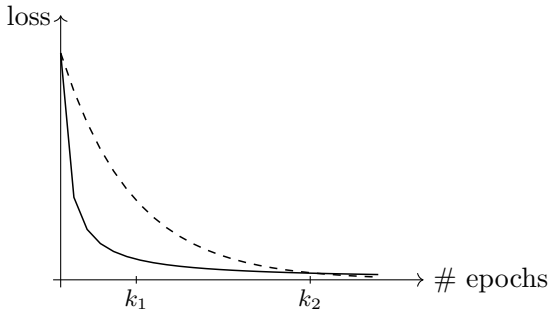


图 2.7: 兩種優化算法下損失函數值衰減的示意性比較。第一種算法（實線）以次線性速率 $O(1/k)$ 收斂，其中 k 為 epoch 數；第二種算法（虛線）以線性速率 $O(\gamma^k)$ 收斂，其中某個 $\gamma \in (0,1)$。第一種算法具有較快的初始收斂，並且可能在 k_1 個 epoch 後就達到令人滿意的解，而第二種算法具有更快的漸近收斂，並將在 k_2 個 epoch 後超過第一種算法。

其訓練通常需要在 GPU 集群系統上運行數月才能完成。考慮到當前人工智能的迅猛發展、新的深度學習技術層出不窮，以及不同實體之間的激烈競爭，人們更傾向於選擇那些能夠在較短時間內以略微降低解質量為代價獲得滿意結果的、更快更高效的算法，這並不令人意外。相比之下，傳統優化領域的方法更側重於精心設計的算法結構、嚴格的收斂性分析、上下複雜度界下的緊緻複雜度估計，以及靈活的超參數設置。

圖 2.7 給出了一個示意圖來解釋這種情況。在圖 2.7 中，橫軸表示 epoch 數，也即實際計算成本；縱軸表示訓練數據集上的損失函數值。實線表示一個具有次線性收斂率 $O(1/k)$ 的優化算法的表現，而虛線表示另一個具有線性收斂率 $O(\gamma^k)$ 的算法，其中某個 $\gamma \in (0,1)$。從理論上看，我們自然更傾向於第二種算法，因為它的線性收斂優於次線性收斂率，並且在漸近意義下會更快地收斂到最小損失函數值。

然而，面對圖 2.7 中的兩條衰減曲線，實踐者可能會更偏好具有次線性收斂率（實線）的第一種算法，因為它在初始階段收斂更快。如果該算法在第 k_1 個 epoch 時給出的結果已經足夠令人滿意，那麼這種選擇尤其合理。儘管具有線性收斂率（虛線）的算法會在 k_2 個 epoch 之後超過第一種算法，但解質量的提升可能不足以證明額外大量計算投入的合理性。例如，在推薦系統或生成式預訓練變換器（generative pre-trained transformer，GPT）中，訓練數據集上的評估指標提升幾個百分點，人類可能根本察覺不到，但在實踐中訓練時間卻可能是 4 個月（k_2）與 1 個月（k_1）。在這種情形下，大多數實踐者都會選擇使用（至少先嚐試使用）具有次線性收斂率的算法。

由於深度神經網絡規模巨大且結構複雜，許多因素都可能強烈影響深度學習模型的性能。在這些因素中，方法學與網絡結構的選擇往往最為關鍵。例如，一個經過中等程度訓練的卷積神經網絡，其圖像分類精度可能遠高於一個經過精心訓練的有相同參數量的全連接網絡。此外，數據樣本的質量、初始化方式的選擇以及預處理過程，都會對網絡訓練的經驗性能產生顯著影響。因此，在設計和訓練深度學習模型時，實踐者往往會優先關注這些方面，而通常直接選擇一個在實踐中以高效和快速收斂著稱的常用優化器（如 Adam 或 AdamW），而不會過多在意其理論收斂性質。

2.6.2 數值優化的參考文獻

關於優化理論及其應用的系統性介紹可參見 [29]。對於凸分析和優化算法中的基礎理論結果，我們參考 [11, 13, 16, 19, 21, 102, 120]。一般連續優化理論與算法可見經典著作 [14, 105]。

第 2.1 節和第 2.2 節中關於基礎優化理論與最優性條件的討論主要遵循 [29]。第 2.3 節關於自動微分的內容遵循 [105]，並在全文中補充了一些額外細節與解釋，特別是關於二階導數計算的部分。第 2.4 節關於確定性優化算法的內容主要遵循 [14, 29, 105]。第 2.5.1 節給出的隨機優化基礎內容遵循 [20]。我們在第 2.5.2 節中對實用隨機優化算法給出概述，其內容主要基於互聯網上的討論和簡短教程，因為這些方法都是近幾年才出現的。

第三章　深度最優控制

最優控制（optimal control）為設計一系列決策提供了嚴格的數學框架，這些決策在物理與操作約束條件下，將動力系統引導至期望目標，例如最大化回報（或等價地，最小化代價）。它在廣泛的現實應用中發揮著核心作用。

在本章中，我們首先系統研究最優控制的基本概念與核心結果。特別地，我們將介紹最優控制的基礎理論，包括 Pontryagin 極大值原理和 Hamilton–Jacobi–Bellman 方程，並給出它們的證明。隨後，我們將這些經典結果推廣到概率密度的最優控制，也稱為平均場控制，其動機在於現代深度學習系統常常涉及對大量智能體的控制。因此，在這些情形下，許多感興趣的問題都可以自然地表述為關於概率分佈的控制問題。

儘管最優控制具有重要性並被廣泛應用，但經典方法在高維情形或複雜非線性動力學存在時會遭遇嚴重的可擴展性限制。為應對這些挑戰，我們引入一種基於深度學習的方法，稱為 neural ordinary differential equation（Neural ODE）方法，將其作為求解廣泛一類最優控制問題的有力工具。該方法既可以處理標準最優控制問題，也可以處理概率控制問題。最後，我們將以學習演化偏微分方程解算子這一任務為代表性例子，展示在解決新興科學計算問題方面時，將深度學習與最優控制相結合產生的巨大潛力。

3.1　最優控制理論

在本節中，我們將對最優控制理論中的基礎結果作簡明而系統的回顧。我們首先介紹變分法及其在推導 Euler–Lagrange 方程中的作用。隨後，我們引入 Hamiltonian 動力學，它為一般控制系統提供了統一且富有洞見的視角。最後，我們將介紹 Pontryagin 極大值原理與 Hamilton–Jacobi–Bellman 方程，並給出嚴格證明。

3.1.1 Euler–Lagrange 方程

最優控制是一個複雜的優化問題，其性質與第 二 章中討論的標準優化非常不同。在最優控制問題中，被優化的對象是一個函數，稱為**控制**（control），它通常表示為從時間區間 $[0,T]$（也可以擴展到 $[0,\infty)$ 或 $\mathbb{R}$，但我們將大部分討論限制在有限時間窗口上，因為這已經具有足夠的一般性）映射到某個多維歐幾里得空間 $\mathbb{R}^m$ 的子集 $\mathcal{U}$ 的映射。這個子集 $\mathcal{U}$ 稱為**控制的容許集**（admissible set of controls）。

儘管最優控制與標準優化有著相似的目標，即尋求使某個目標函數最小化或最大化的解，但其理論與求解方法與第 二 章中給出的標準優化方法非常不同（但仍然相關）。由於被優化的對象是一個隨時間變化的控制函數 $u:[0,T]\to\mathcal{U}\subset\mathbb{R}^m$，我們需要新的理論、分析方法和計算工具來求解最優控制問題。

最優控制的基本方法稱為**變分法**（calculus of variations），這種方法也稱為**變分方法**。我們通過一個經典例子來介紹變分法和 Lagrangian 函數，並進一步導出 Euler–Lagrange 方程。

設給定一個光滑函數 $L:\mathbb{R}^d\times\mathbb{R}^d\to\mathbb{R}$。我們希望找到一條曲線 $x^*:[0,T]\to\mathbb{R}^d$，使得泛函

$$I[x]:=\int_0^T L(x(t),\dot{x}(t))\,dt, \tag{3.1}$$

在約束 $x(0)=x_0$ 和 $x(T)=x_1$ 下取最小值，其中 $x_0,x_1\in\mathbb{R}^d$ 為給定值。在 (3.1) 中，$\dot{x}$ 是 x 關於時間的導數（目前我們可以將 $\dot{x}$ 看作控制）。我們通常將 $x(t)$ 解釋為位置（更一般地，我們稱之為狀態），將 $\dot{x}(t)$ 解釋為時刻 t 的速度。與優化問題類似，我們的第一步是尋找 x^* 的最優性條件。也就是說，假設 x^* 是 (3.1) 中定義的 I 的一個極小化解，我們希望知道 x^* 必須滿足哪些條件。這些關於最優控制的條件，對應於第 二 章中約束優化的 Lagrange 條件與 KKT 條件。

在文獻中，(3.1) 中的被積函數 L 被稱為 **Lagrangian 函數**。注意，這應與第 二 章中等式約束優化問題的 Lagrangian 函數區分開來，因為它們的定義和用途不同。我們通常將 L 寫成關於 $x,v\in\mathbb{R}^d$ 的函數（其中 v 替代 $\dot{x}$，並被解釋為速度）。在本節餘下部分中，我們還使用如下記號表示偏導數：

$$\partial_x L:=(\partial_{x_1}L,\ldots,\partial_{x_d}L)\,,$$
$$\partial_v L:=(\partial_{v_1}L,\ldots,\partial_{v_d}L)\,,$$

其中

$$\partial_{x_i} L = \frac{\partial L}{\partial x_i} \quad \text{和} \quad \partial_{v_i} L = \frac{\partial L}{\partial v_i}$$

對每個 $i = 1, \ldots, d$ 成立。下面的定理表明，如果 x^* 是 (3.1) 的一個最優解，那麼它必須滿足所謂的 **Euler–Lagrange 方程**。該定理的證明使用了變分法的思想，具體見下文。

定理 3.1.1 (Euler–Lagrange equation)**.** *假設 x 是帶邊界約束的 (3.1) 的一個最優解，則 x 滿足 Euler–Lagrange 方程:*

$$\frac{d}{dt}\Big(\partial_v L(x(t), \dot{x}(t))\Big) = \partial_x L(x(t), \dot{x}(t)). \tag{3.2}$$

证明. 回憶一下，無約束光滑優化問題的解位於目標函數梯度為零的臨界點之中。我們希望採用類似的方法，嘗試找到那些使 (3.1) 中定義的泛函 I 的 "梯度" 為零的曲線。然而，由於這裡的 "梯度" 是關於曲線而不是向量的，我們不能直接這樣求梯度。因此，我們先把問題轉化為一個可以應用標準微積分的問題。

為此，我們任取一條光滑曲線 $y : [0, T] \to \mathbb{R}^d$，滿足 $y(0) = y(T) = 0$。我們通常稱 y 為 x 的一個擾動。現在我們看到，對任意 $\epsilon \in \mathbb{R}$，曲線 $x + \epsilon y : [0, T] \to \mathbb{R}^d$ 仍然滿足邊界約束，因為在 $t = 0$ 和 $t = T$ 時分別有 $x(0) + \epsilon y(0) = x_0$ 和 $x(T) + \epsilon y(T) = x_1$。定義 $\phi : \mathbb{R} \to \mathbb{R}$ 為

$$\phi(\epsilon) := I[x + \epsilon y] = \int_0^T L(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t))\, dt. \tag{3.3}$$

由於 x 是 (3.1) 的一個極小化解，我們有

$$\phi(0) = I[x] \le I[x + \epsilon y] = \phi(\epsilon) \tag{3.4}$$

對任意 $\epsilon \in \mathbb{R}$ 成立。

由於 $L : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ 是一個光滑函數，我們知道 ϕ 是光滑的，並由 (3.4) 得到

$$\phi'(0) = 0.$$

現在我們需要求出 ϕ' 的表達式。為此，我們回憶 (3.3) 並得到

$$\phi'(\epsilon) = \int_0^T \Big[\partial_x L\Big(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t)\Big) \cdot y(t) \\ + \partial_v L\Big(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t)\Big) \cdot \dot{y}(t)\Big] dt.$$

於是當 $\epsilon = 0$ 時，我們得到

$$0 = \phi'(0) = \int_0^T \Big[\partial_x L(x(t), \dot{x}(t)) \cdot y(t) + \partial_v L(x(t), \dot{x}(t)) \cdot \dot{y}(t)\Big] dt. \tag{3.5}$$

現在我們來研究，在對所有滿足 $y(0) = y(T) = 0$ 的 $y : [0, T] \to \mathbb{R}^d$ 都成立的前提下，(3.5) 蘊含什麼。設我們從 $1, \ldots, d$ 中任選一個 i，並令

$$y(t) = (0, \ldots, 0, \underbrace{z(t)}_{i\text{th}}, 0, \ldots, 0) \in \mathbb{R}^d \tag{3.6}$$

對所有 $t \in [0, T]$ 成立，其中 $z : [0, T] \to \mathbb{R}$ 是一條滿足 $z(0) = z(T) = 0$ 的光滑函數。那麼，(3.6) 中定義的 y 在時刻 0 和 T 也滿足約束。將 (3.6) 中定義的這個 y 代入 (3.5)，得到

$$0 = \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) z(t) + \partial_{v_i} L(x(t), \dot{x}(t)) \dot{z}(t)\Big) dt.$$

對上式右側積分中的第二項做分部積分，得到

$$0 = \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt. \tag{3.7}$$

這對所有滿足 $z(0) = z(T) = 0$ 的光滑曲線 z 都成立。

由 (3.7)，我們斷言

$$\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) = 0 \tag{3.8}$$

對所有 $t \in [0, T]$ 都成立。為了證明 (3.8)，我們可以採用反證法：如果在某個 $t \in [0, T]$ 處有 $\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) > 0$，那麼由於 L 的光滑性，存在一個子區間 $(t - \delta, t + \delta)$（若 $t = 0$ 或 $t = T$，則取該區間的一半），使得在該區間內有 $\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) > 0$。取 $z > 0$ 於區間

$(t-\delta, t+\delta)$ 上，而在其他地方取 0，則有

$$\begin{aligned} 0 &< \int_{t-\delta}^{t+\delta} \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt \\ &= \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt \end{aligned}$$

這與 (3.7) 矛盾。

由於 (3.8) 對任意 $i = 1, \ldots, d$ 都成立，將 (3.8) 重新整理即可得到 Euler–Lagrange 方程 (3.2)。 □

注意，Euler–Lagrange 方程 (3.2) 是關於 x 的一個由 d 個二階常微分方程（second-order ODEs）構成的非線性系統。這些 ODE 的形式如 (3.8) 所示：

$$\frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t)) = \partial_{x_i} L(x(t), \dot{x}(t))$$

其中 $i = 1, \ldots, d$。

我們還指出，對 L 和 x 的光滑性要求可以進一步放寬（例如，所涉及的梯度/導數除了在 $[0, T]$ 上某個測度為零的集合外處處存在）。在這種情況下，可以藉助測度論給出嚴格證明，而結論保持不變。

3.1.2 Hamiltonian 動力學

Euler–Lagrange 方程 (3.2) 給出了 (3.1) 極小化解的一個必要條件，其中 x 的約束設置在區間 $[0, T]$ 的端點處。速度 $\dot{x}$ 可以被視為 x 的控制。

接下來，我們介紹另一個重要概念，稱為 Hamiltonian 動力學，它與 Euler–Lagrange 方程 (3.2) 密切相關，但在一般最優控制問題中更為常見。與 (3.2) 是一個由 d 個二階 ODE 構成的系統不同，Hamiltonian 動力學構成的是一個由 $2d$ 個一階 ODE 構成的系統。

為此，設給定 $x : [0, T] \to \mathbb{R}^d$，我們定義另一條曲線 $p : [0, T] \to \mathbb{R}^d$，稱為**廣義動量**，如下：

$$p(t) := \partial_v L(x(t), \dot{x}(t)) \tag{3.9}$$

對所有 $t \in [0, T]$ 成立。我們的目標是將 (3.2) 重寫為關於 (x, p) 的一個由 $2d$ 個一階 ODE 構成的系統。

注意，將 Euler–Lagrange 方程轉換為 Hamiltonian 動力學通常要求

$$p = \partial_v L(x, v) \tag{3.10}$$

能夠在給定 (x,p) 時唯一確定 v。這可以通過例如假設在每個 x 和 v 處，$\partial^2_{vv}L(x,v)$ 非奇異來實現，從而可以應用隱函數定理說明這一要求成立。在本節餘下部分中，我們假設 (3.10) 的這一要求成立，並將其解記為 $v(x,p)$。也就是說，$p=\partial_v L(x,v(x,p))$。

定義 3.1.2 (Hamiltonian)**.** 設 $x,p\in\mathbb{R}^d$，且 $v(x,p)$ 是 (3.10) 的解，則定義 $H:\mathbb{R}^d\times\mathbb{R}^d\to\mathbb{R}$ 為

$$H(x,p)=p\cdot v(x,p)-L(x,v(x,p)). \tag{3.11}$$

我們稱 H 為 **Hamiltonian**函數。

與上面的 Lagrangian 函數 L 類似，我們將 H 的偏導數記為

$$\begin{aligned}\partial_x H &:= (\partial_{x_1}H,\dots,\partial_{x_d}H)\,,\\ \partial_p H &:= (\partial_{p_1}H,\dots,\partial_{p_d}H)\,,\end{aligned}$$

其中

$$\partial_{x_i}H=\frac{\partial H}{\partial x_i}\quad 和\quad \partial_{p_i}H=\frac{\partial H}{\partial p_i}$$

對每個 $i=1,\dots,d$ 成立。現在我們準備說明，Euler–Lagrange 方程 (3.2) 可以利用 H 轉化為 Hamiltonian 動力學。

定理 3.1.3 (Hamiltonian dynamics)**.** *設 x 是 Euler–Lagrange 方程 (3.2) 的解，並按 (3.9) 定義 p。則 $(x,p):[0,T]\to\mathbb{R}^d\times\mathbb{R}^d$ 這一對函數滿足 Hamiltonian 動力學:*

$$\begin{cases}\dot x(t)=\partial_p H(x(t),p(t)),\\ \dot p(t)=-\partial_x H(x(t),p(t)).\end{cases} \tag{3.12}$$

此外，$H(x(t),p(t))$ 是 t 的常函數。

证明. 由 (3.11) 中 Hamiltonian H 的定義,其中 $v=v(x,p)$ 滿足 $p=\partial_v L(x,v)$，我們有

$$\begin{aligned}\partial_x H(x,p)&=p\cdot\partial_x v(x,p)-\partial_x L(x,v(x,p))-\partial_v L(x,v(x,p))\cdot\partial_x v(x,p)\\ &=-\partial_x L(x,v(x,p))\end{aligned} \tag{3.13}$$

因為 $p = \partial_v L(x, v(x,p))$。因此，我們有

$$\begin{aligned}\dot{p}(t) &= \frac{d}{dt}\Big(\partial_v L(x(t), \dot{x}(t))\Big) \\ &= \partial_x L(x(t), \dot{x}(t)) \\ &= \partial_x L(x(t), v(x(t), p(t))) \\ &= -\partial_x H(x(t), p(t)),\end{aligned}$$

其中第一步是對 (3.9) 兩邊關於 t 求導所得，第二步由 Euler–Lagrange 方程 (3.2) 得到，第三步是因為 $p(t) = \partial_v L(x(t), \dot{x}(t))$ 蘊含 $\dot{x}(t) = v(x(t), p(t))$，最後一步來自 (3.13)。

此外，由 (3.11) 中 H 的定義可知

$$\begin{aligned}\partial_p H(x,p) &= v(x,p) + p \cdot \partial_p v(x,p) - \partial_v L(x, v(x,p)) \cdot \partial_p v(x,p) \\ &= v(x,p)\end{aligned}$$

因為 $p = \partial_v L(x, v(x,p))$。因此，對每個 $t \in [0,T]$，有

$$\partial_p H(x(t), p(t)) = v(x(t), p(t))$$

成立。同樣，由於 $p(t) = \partial_v L(x(t), \dot{x}(t))$ 蘊含 $\dot{x}(t) = v(x(t), p(t))$，於是得到

$$\dot{x}(t) = \partial_p H(x(t), p(t)).$$

至此，我們已經說明 $(x(t), p(t))$ 滿足 ODE 系統 (3.12)。

為了說明 $H(x(t), p(t))$ 是常數，我們對其關於 t 求導:

$$\begin{aligned}\frac{d}{dt} H(x(t), p(t)) &= \partial_x H(x(t), p(t)) \cdot \dot{x}(t) + \partial_p H(x(t), p(t)) \cdot \dot{p}(t) \\ &= -\dot{p}(t) \cdot \dot{x}(t) + \dot{x}(t) \cdot \dot{p}(t) \\ &= 0,\end{aligned}$$

其中第二步由 (3.12) 得到。證明完成。 □

例 3.1.4. 我們考慮經典力學中的一個例子，其 Lagrangian 函數為

$$L(x,v) = \frac{1}{2} m |v|^2 - V(x), \tag{3.14}$$

其中 x 是物體的位置，m 是物體的質量，v 是物體的速度。函數 $V:\mathbb{R}^d\to\mathbb{R}$ 是物體的勢能。因此，(3.14) 中的 L 是動能與勢能之差。

我們還可以看到

$$\partial_x L(x,v) = -\nabla V(x) \quad \text{和} \quad \partial_v L(x,v) = mv.$$

將它們代入 Euler–Lagrange 方程 (3.2)，得到

$$m\ddot{x}(t) = -\nabla V(x(t)).$$

因此，函數 (3.14) 的 Euler–Lagrange 方程就是描述 Newton 第二運動定律的二階 ODE。進一步地，廣義動量恰好就是經典力學中的動量：

$$p = \partial_v L(x,v) = mv,$$

這意味著

$$v = v(x,p) = \frac{p}{m}.$$

將 x 和 $v(x,p)=\frac{p}{m}$ 代入 Hamiltonian H (3.11)，可得

$$\begin{aligned} H(x,p) &= p\cdot\frac{p}{m} - L\Big(x,\frac{p}{m}\Big) \\ &= \frac{|p|^2}{m} - \frac{1}{2}m\frac{|p|^2}{m^2} + V(x) \\ &= \frac{|p|^2}{2m} + V(x), \end{aligned}$$

它就是動能與勢能之和。如果再將 $p(t)=mv(t)$ 代回上式中的 H，則得到

$$H(x(t),p(t)) = \frac{1}{2}m|v(t)|^2 + V(x(t)).$$

由定理 3.1.3 可知，$H(x(t),p(t))$ 關於 t 保持為常數。這意味著動能與勢能之和隨時間不變，也就是說，總能量守恆。

我們還注意到，Hamiltonian 動力學為

$$\begin{cases} \dot{x}(t) = \partial_p H(x(t),p(t)) = \frac{p(t)}{m}, \\ \dot{p}(t) = -\partial_x H(x(t),p(t)) = -\nabla V(x(t)). \end{cases}$$

如果 $V(x) := mgx$，其中 x 是高度，g 是重力加速度（地球表面上約為

9.81m/sec^2），那麼有 $\dot{p}(t) = -mg$。

3.1.3 Pontryagin 極大值原理

上述討論揭示了 Lagrangian 力學與 Hamiltonian 力學之間的一些基本聯繫。同時，Hamiltonian 動力學在控制理論中展現出重要意義：最優控制的必要條件可以通過 Hamiltonian 動力學來刻畫。這正是本小節的目標。

首先，我們考慮一個關於商品交易的最優控制問題作為示例。假設我們是一名小麥交易商，在時刻 t 擁有現金 $x_1(t)$ 和庫存中的 $x_2(t)$ 單位小麥。在初始時刻 $t = 0$，我們有 $x_1(0) = x_{1,0}$ 和 $x_2(0) = x_{2,0}$，其中 $x_{1,0}, x_{2,0} \geq 0$ 為給定值。設 $c > 0$ 為每單位小麥每單位時間的存儲成本。為簡便起見，假設我們已知任意時刻 t 每單位小麥的價格 $q(t)$。現在我們需要找到最優的 $u(t)$，它表示在時刻 t 買入或賣出小麥的速率（若 $u(t) > 0$ 表示買入，若 $u(t) < 0$ 表示賣出），使得在給定終止時刻 T 的總持有價值最大化。

為了將這個商品交易問題數學化，我們注意到 $x(t) = (x_1(t), x_2(t)) \in \mathbb{R}^2$ 的動力學由下式給出

$$\begin{cases} \dot{x}_1(t) = -cx_2(t) - q(t)u(t), \\ \dot{x}_2(t) = u(t), \end{cases} \tag{3.15}$$

其初值為 $x_1(0) = x_{1,0}$ 和 $x_2(0) = x_{2,0}$。之所以有 (3.15)，是因為金錢支出的變化率 $\dot{x}_1(t)$ 等於庫存小麥 $x_2(t)$ 的存儲成本與以速率 $u(t)$ 買入小麥的支出（或賣出小麥的收入）；而庫存小麥的變化率 $\dot{x}_2(t)$ 就等於 $u(t)$。時刻 T 的總持有價值由下式給出

$$I[u] := x_1(T) + q(T)x_2(T), \tag{3.16}$$

它是時刻 T 的手頭現金 $x_1(T)$ 與庫存小麥價值 $q(T)x_2(T)$ 之和。因此，該最優控制問題就是尋找速率函數 $u : [0, T] \to \mathbb{R}$，使得在初值 $x(0) = (x_1(0), x_2(0)) = (x_{1,0}, x_{2,0})$ 以及約束 (3.15) 下，由 (3.16) 定義的總持有價值 I 最大化。這裡為簡便起見，我們允許 $u(t)$ 取任意實數。在實際中，我們通常要求對每個 t 都有 $u(t) \in \mathcal{U}$，其中 $\mathcal{U}$ 是 $\mathbb{R}$ 的一個子集（若 $u(t)$ 是一個 m 維向量，則 $\mathcal{U}$ 是 $\mathbb{R}^m$ 的一個子集）。在下文中，我們用 $\mathcal{U}_T := \{u : [0, T] \to \mathcal{U} \subset \mathbb{R}^m\}$ 表示整個時間區間 $[0, T]$ 上控制 u 的容許集。

上面的商品交易例子代表了一類典型的一般最優控制問題的構造形式。更

準確地說，這些最優控制問題寫成

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(x(t),u(t))\,dt + g(x(T)), \tag{3.17a}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t),u(t)), \tag{3.17b}$$

$$x(0) = x_0, \tag{3.17c}$$

其中待優化的變量是控制軌跡 $u \in \mathcal{U}_T$。

將上面的商品交易例子與 (3.17) 相比較，我們看到動力學 (3.15) 對應於 ODE 約束 (3.17b)，初值 $x(0) = (x_{1,0}, x_{2,0})$ 對應於 (3.17c)，而 (3.16) 中的總持有價值 $x_1(T) + q(T)x_2(T)$ 就對應於 (3.17a) 中的 $g(x(T))$，它被稱為**終端獎勵**，terminal reward（或稱終端收益）。在 (3.17a) 中，我們還有一個在商品交易例子 (3.16) 中未出現的附加項 $\int_0^T r(x(t),u(t))\,dt$。這是所謂**運行獎勵**（running reward）$r : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}$ 的積分，正如我們稍後將說明的那樣，它可以很容易地併入終端獎勵 g 中。

注意，我們沒有在 (3.17b) 中顯式地把時間變量 t 寫入 ODE 的定義函數 f 中，也沒有寫成 $f(t,x(t),u(t))$ 的形式。這是因為時間變量 t 可以通過引入一個導數恆為 1 的附加分量併入 $x(t)$ 中。更準確地說，我們可以定義

$$\bar{x}(t) := (t, x(t)),$$
$$\bar{f}(\bar{x}(t),u(t)) := (1, f(t,x(t),u(t))),$$

於是有

$$\dot{x}(t) = f(t,x(t),u(t)) \quad \Longleftrightarrow \quad \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t),u(t))$$

而右側仍然具有 (3.17b) 的形式。

最優控制問題 (3.17) 常被稱為**固定終止時刻且終點自由**問題，因為終止時刻 T 是固定的，而終點 $x(T)$ 不是固定的。最優控制還有另一種變體，稱為固定終點且終止時刻自由問題，它與 (3.17) 具有類似的最優性理論。本節中我們也會討論這種變體。

接下來，我們給出關於 (3.17) 解的最優性條件中最重要的結果之一。這一結果稱為 **Pontryagin 極大值原理**。我們先給出 control Hamiltonian 函數的定義，該函數將在 Pontryagin 極大值原理及其證明中使用。

定義 3.1.5 (Control Hamiltonian)**.** 設 f 和 r 分別是 (3.17) 中的 ODE 定義函數和運行獎勵函數。則 **control Hamiltonian**函數 $H : \mathbb{R}^d \times \mathbb{R}^d \times \mathcal{U} \to \mathbb{R}$

定義為

$$H(x,p,u) := p \cdot f(x,u) + r(x,u) \tag{3.18}$$

對任意 $(x,p,u) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathcal{U}$ 成立。

與 (3.11) 中的 Hamiltonian 函數比較可知，(3.18) 中的 control Hamiltonian 函數多了控制變量 $u \in \mathcal{U} \subset \mathbb{R}^m$，(3.11) 中的控制速度 v 被 (3.17b) 中由 u 控制的 ODE 中的函數 f （我們稱 f 為此 ODE 的**定義函數**）所替代，而負的 Lagrangian，$-L$（其中 (3.1) 中對 L 的積分是要最小化的）則被 (3.17a) 中要最大化的運行獎勵函數 r 所替代。

現在我們給出兩個在 Pontryagin 極大值原理及其證明中至關重要的引理。

引理 3.1.6. 設 $x : [0,T] \to \mathbb{R}^d$ 是如下初值問題的解

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)), \quad \forall\, t \in [0,T], \\ x(0) = x_0 \end{cases} \tag{3.19}$$

其中 $f : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^d$ 為給定定義函數, $u : [0,T] \to \mathbb{R}^m$ 為控制函數, $x_0 \in \mathbb{R}^d$ 為給定初值。假設對每個 t，$f(\cdot, u(t))$ 關於 x 是 L_f-*Lipschitz* 連續且為 C^1。令 $y_0 \in \mathbb{R}^d$ 任意固定。對任意 $\epsilon > 0$，設 x^ϵ 是初值問題

$$\begin{cases} \dot{x}^\epsilon(t) = f(x^\epsilon(t), u(t)), \\ x^\epsilon(0) = x_0 + \epsilon y_0 + o(\epsilon). \end{cases} \tag{3.20}$$

的解。則有

$$x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon) \tag{3.21}$$

在 $[0,T]$ 上關於 t 一致成立，即

$$\lim_{\epsilon \to 0} \sup_{t \in [0,T]} \frac{|x^\epsilon(t) - x(t) - \epsilon y(t)|}{\epsilon} = 0,$$

其中 y 是初值問題

$$\begin{cases} \dot{y}(t) = \partial_x f(x(t), u(t)) y(t), \\ y(0) = y_0 \end{cases} \tag{3.22}$$

的解。

证明. 我們定義

$$\delta(t) := \frac{1}{2}|x^\epsilon(t) - x(t)|^2. \tag{3.23}$$

則有

$$\begin{aligned}
\dot{\delta}(t) &= (x^\epsilon(t) - x(t)) \cdot (\dot{x}^\epsilon(t) - \dot{x}(t)) \\
&= (x^\epsilon(t) - x(t)) \cdot (f(x^\epsilon(t), u(t)) - f(x(t), u(t))) \\
&= (x^\epsilon(t) - x(t)) \cdot \partial_x f(\xi^\epsilon(t), u(t))(x^\epsilon(t) - x(t)) \\
&\leq 2L_f \delta(t),
\end{aligned}$$

其中由於 Taylor 定理，$\xi^\epsilon(t)$ 是位於 $x(t)$ 與 $x^\epsilon(t)$ 連線段上的某一點。由 Grönwall 不等式，對所有 $t \in [0, T]$ 有

$$\delta(t) = \delta(0)e^{2L_f t} \leq \delta(0)e^{2L_f T}.$$

因此，當 ϵ 足夠小時，

$$|x^\epsilon(t) - x(t)| = \sqrt{2\delta(t)} \leq \sqrt{2\delta(0)}e^{L_f T} = |\epsilon y_0 + o(\epsilon)|e^{L_f T} \leq C\epsilon|y_0|e^{L_f T}$$

對所有 $t \in [0, T]$ 成立，其中 C 是某個與 ϵ 和 t 無關的常數。因此，當 $t \to 0$ 時，$\|x^\epsilon - x\|_{L^\infty([0,T])} \to 0$，即 $x^\epsilon(t) - x(t) = O(\epsilon)$，並且當 $\epsilon \to 0$ 時在 $[0, T]$ 上一致趨於 0。

對每個 $t \in [0, T]$ 定義

$$y(t) := \lim_{\epsilon \to 0} \frac{x^\epsilon(t) - x(t)}{\epsilon}.$$

則顯然有

$$y(0) = \lim_{\epsilon \to 0} \frac{x^\epsilon(0) - x(0)}{\epsilon} = \lim_{\epsilon \to 0} \left(y_0 + \frac{o(\epsilon)}{\epsilon} \right) = y_0,$$

並且

$$
\begin{aligned}
\dot{y}(t) &= \lim_{\epsilon\to 0}\frac{\dot{x}^\epsilon(t)-\dot{x}(t)}{\epsilon}\\
&= \lim_{\epsilon\to 0}\frac{f(x^\epsilon(t),u(t))-f(x(t),u(t))}{\epsilon}\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\Big(\partial_x f(x(t),u(t))(x^\epsilon(t)-x(t))\\
&\qquad\qquad +\frac{1}{2}(x^\epsilon(t)-x(t))\partial^2_{xx}f(\eta^\epsilon(t),u(t))(x^\epsilon(t)-x(t))\Big)\\
&= \lim_{\epsilon\to 0}\Big(\partial_x f(x(t),u(t))\frac{x^\epsilon(t)-x(t)}{\epsilon}\\
&\qquad\qquad +\frac{\epsilon}{2}\frac{x^\epsilon(t)-x(t)}{\epsilon}\partial^2_{xx}f(\eta^\epsilon(t),u(t))\frac{x^\epsilon(t)-x(t)}{\epsilon}\Big)\\
&= \partial_x f(x(t),u(t))y(t),
\end{aligned}
$$

其中 $\eta^\epsilon(t)$ 是位於 $x(t)$ 與 $x^\epsilon(t)$ 連線段上的某一點。注意，第一步要求 $\partial_\epsilon\partial_t x^\epsilon(t)=\partial_t\partial_\epsilon x^\epsilon(t)$。這成立的一個充分條件是 Clairaut 定理：若二階偏導都存在且連續，則偏導次序可以交換。當 f 足夠光滑時，例如 f 為 C^1，這一點成立。

總結而言，我們得到 $x^\epsilon(t)=x(t)+\epsilon y(t)+o(\epsilon)$ 在 $[0,T]$ 上一致成立，其中 y 是初值問題 (3.22) 的解。 □

為了嚴格證明 Pontryagin 極大值原理，我們需要覆蓋控制軌跡的所有可能變化。理論上說，最優控制問題並不要求控制 u 關於 t 連續。因此，我們考慮下面這種所謂的簡單變分，它允許對控制函數 u 進行不連續的改變。

定義 3.1.7 (簡單變分，simple variation)**.** 對任意 $\tau\in(0,T]$ 和充分小的 ϵ，使得 $0<\tau-\epsilon<\tau\le T$，定義 $u(t)$ 的**簡單變分**（也稱為**針狀變分**或 **needle-like variation**）為

$$
u^\epsilon(t):=\begin{cases} w, & \text{if } \tau-\epsilon<t<\tau,\\ u(t), & \text{otherwise.}\end{cases}
\tag{3.24}
$$

對於簡單變分 u^ϵ，令 x^ϵ（注意這裡的 x^ϵ 不再是上面 (3.20) 中定義的那個）為如下初值問題的解

$$
\begin{cases} \dot{x}^\epsilon(t)=f(x^\epsilon(t),u^\epsilon(t)), & \forall\, t\in[0,T],\\ x^\epsilon(0)=x_0. \end{cases}
\tag{3.25}
$$

定量刻畫原軌道 x 與該 x^ϵ 之間的差異是很重要的。為此，我們從現在起要求 $f\in C^1(\mathbb{R}^d\times\mathbb{R}^m;\mathbb{R}^d)$。然後我們說明，下面定義的函數 y 能夠捕捉它們

差異的一階項。

定義 3.1.8 (擾動函數，perturbation)**.** 在 $t \in [0, \tau)$ 時，令 $y(t) = 0$ 成立。而在 $[\tau, T]$ 上，$y(t)$ 是初值問題

$$\begin{cases} \dot{y}(t) = \partial_x f(x(t), u(t))y(t), \\ y(\tau) = f(x(t), w) - f(x(t), u(t)) \end{cases} \tag{3.26}$$

的解。我們稱此 y 為**擾動函數**。

引理 3.1.9. *假設 x^ϵ 是帶有簡單變分 u^ϵ 的問題 (3.25) 的解，則當 $\epsilon \to 0$ 時，*

$$x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon) \tag{3.27}$$

在 $[0, T]$ 上關於 t 一致成立，其中 $y : [0, T] \to \mathbb{R}^d$ 是定義 3.1.8 中的擾動函數。

证明. 令 $t \in (0, \tau)$ 任意固定，則對任意 $\epsilon \in (0, \tau - t)$，有 $0 < t < \tau - \epsilon$，從而 $x^\epsilon(t) = x(t)$。

對於 $t \in [\tau, T]$，有

$$\begin{aligned} x^\epsilon(t) - x(t) &= \int_{\tau-\epsilon}^{t} \Big(f(x(s), w) - f(x(s), u(s)) \Big) \, ds \\ &= \Big(f(x(\tau), w) - f(x(\tau), u(\tau)) \Big) \epsilon + o(\epsilon) \end{aligned}$$

當 $\epsilon \to 0$ 時成立，這是由於 f 關於 u 的連續性。由於在區間 $[\tau, T]$ 上，$x(t)$ 與 $x^\epsilon(t)$ 滿足以同一個 u 構成的定義函數 f 的 ODE，只是在時刻 τ 的初值不同：

$$x^\epsilon(\tau) = x(\tau) + \Big(f(x(\tau), w) - f(x(\tau), u(\tau)) \Big) \epsilon + o(\epsilon).$$

由引理 3.1.6 可知，當 $\epsilon \to 0$ 時，對所有 t 都有 $x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon)$，其中 y 是定義 3.1.8 中的擾動函數。 □

在證明 Pontryagin 極大值原理之前，我們還要引入控制系統中狀態（state）$x : [0, T] \to \mathbb{R}^d$ 的協態（co-state）$p : [0, T] \to \mathbb{R}^d$ 的概念。在下面的討論中，我們將每個時刻的 $p(t) \in \mathbb{R}^d$ 看作一個行向量（注意它與 $x(t)$ 具有相同維數），並且它的作用類似於約束（由初值問題 (3.19) 構成）對應的 Lagrange 乘子。協態也稱為伴隨函數（adjoint function），它滿足所謂的伴隨方程（adjoint equation），如下所示。

定義 3.1.10 (協態與伴隨方程)**.** 設 x 是在控制 u 下初值問題 (3.19) 的解。若 $p:[0,T]\to\mathbb{R}^d$ 滿足如下終值問題

$$\begin{cases}\dot{p}(t)=-p(t)\partial_x f(x(t),u(t))-\partial_x r(x(t),u(t)), & \forall\, t\in[0,T],\\ p(T)=\nabla g(x(T)).\end{cases}\tag{3.28}$$

則稱 p 為**協態**。(3.28) 中的 ODE 稱為**伴隨方程**。

無運行獎勵時的 Pontryagin 極大值原理

Pontryagin 極大值原理給出了最優控制的一個必要條件。因此，要建立該原理，必須假設給定控制是最優的（正如在優化中推導必要最優性條件時，我們要假設局部極小點在其鄰域內取得最小目標函數值一樣）。現在我們的目標函數 $I[u]$ 由 (3.17a) 給出。我們先從 I 僅含終端獎勵 $g(x(T))$ 的情形開始。

引理 3.1.11. 終端獎勵 $I[u]=g(x(T))$（假設 (3.17a) 中沒有運行獎勵）的變分由下式給出

$$\frac{d}{d\epsilon}I[u^\epsilon]\Big|_{\epsilon=0}=p(\tau)\Big(f(x(\tau),w)-f(x(\tau),u(\tau))\Big).\tag{3.29}$$

证明. 根據引理 3.1.9，我們有

$$I[u^\epsilon]=g(x^\epsilon(T))=g(x(T)+\epsilon y(T)+o(\epsilon)),$$

其中 y 是定義 3.1.8 中的擾動函數。因此，

$$\frac{d}{d\epsilon}I[u^\epsilon]\Big|_{\epsilon=0}=\nabla g(x(T))y(T).\tag{3.30}$$

另一方面，在沒有 r 的情況下，由伴隨方程 (3.28) 可得

$$\begin{aligned}\frac{d}{dt}(p(t)y(t))&=\dot{p}(t)y(t)+p(t)\dot{y}(t)\\&=-p(t)\partial_x f(x(t),u(t))y(t)+p(t)\partial_x f(x(t),u(t))y(t)\\&=0,\end{aligned}$$

這說明 $p(t)y(t)$ 關於 t 為常數。因此有

$$\nabla g(x(T))y(T)=p(T)y(T)=p(\tau)y(\tau),\tag{3.31}$$

其中第一個等號來自 (3.28) 中 p 的終端條件。注意到 $y(\tau)$ 的值由 (3.26) 給出，將 (3.30) 與 (3.31) 結合即可得到 (3.29)。 □

現在我們已準備好給出並證明 Pontryagin 極大值原理（Pontryagin Maximum Principle）。同樣地，我們仍考慮目標函數中沒有運行獎勵的最優控制問題 (3.17)。

定理 3.1.12 (Pontryagin 極大值原理)**.** *設 $u(\cdot)$ 是 (3.17) 的最優解，x 是滿足 (3.17b) 與 (3.17c) 的對應軌跡。則存在一個函數 $p:[0,T]\to\mathbb{R}^n$，使得 (x,p) 滿足 control Hamiltonian 動力學：*

$$\begin{cases} \dot{x}(t) = \partial_p H\left(x(t), p(t), u(t)\right), \\ \dot{p}(t) = -\partial_x H\left(x(t), p(t), u(t)\right), \end{cases} \tag{3.32}$$

其中 x 的初始條件與 p 的終端條件分別為

$$x(0) = x_0 \quad \text{和} \quad p(T) = \nabla g(x(T)).$$

此外，對每個 t，下面這一恆等式成立，它被稱為 ***Pontryagin* 極大值原理**：

$$H(x(t), p(t), u(t)) \;=\; \max_{w\in\mathcal{U}} H(x(t), p(t), w). \tag{3.33}$$

证明. 當沒有運行獎勵 r 時，control Hamiltonian (3.18) 為

$$H(x,p,u) = p\cdot f(x,u).$$

因此，(3.32) 就是 (3.17b) 中給出的 x 的 ODE 與 (3.28) 中給出的 p 的伴隨方程的組合。x 的初始條件和 p 的終端條件分別由 (3.17c) 與 (3.28) 給出。

令 $t\in(0,T)$ 固定，且 $w\in\mathcal{U}$ 任意，則由於 u 是最優的，$\epsilon\mapsto I[u^\epsilon]$ 在 $\epsilon=0$ 處取得極大值。因此

$$0 \geq \frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} = p(\tau)\Big(f(x(t),w) - f(x(t),u(t))\Big). \tag{3.34}$$

因此，我們得到

$$\begin{aligned} H(x(t), p(t), w) &= p(t) f(x(t), w) \\ &\le p(t) f(x(t), u(t)) \\ &= H(x(t), p(t), u(t)). \end{aligned}$$

又由於 $u(t) \in \mathcal{U}$，於是得到極大值原理 (3.33)。 □

注意，帶控制的 Hamiltonian 動力學 (3.32) 與無控制時的 (3.12) 具有相同的形式。為了記住這一動力學，只需記住它是關於狀態與協態二元組 (x, p) 的一階 ODE 非線性系統，其定義函數為 $(\partial_p H, -\partial_x H)$。這種動力學結構稱為**辛形式**（symplectic form）。

帶運行獎勵的 Pontryagin 極大值原理

現在考慮目標函數 $I[u]$ 包含運行獎勵的一般情形：

$$I[u] := \int_0^T r(x(t), u(t))\, dt + g(x(T)). \tag{3.35}$$

此時，control Hamiltonian 保持其原始形式 (3.18)：

$$H(x, p, a) = p \cdot f(x, a) + r(x, u).$$

為了推導這種情形下的 Pontryagin 極大值原理，我們只需引入一個輔助變量 $x_{d+1} : [0, T] \to \mathbb{R}$ 來記錄運行獎勵，這樣目標函數就可以寫成不含運行獎勵的形式，從而化歸到上面已討論過的情形。

為此，定義 x_{d+1} 滿足

$$\begin{cases} \dot{x}_{d+1}(t) = r(x(t), u(t)), \quad \forall\, t \in [0, T], \\ x_{d+1}(0) = 0, \end{cases} \tag{3.36}$$

其中 x 滿足原始初值問題：

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases} \tag{3.37}$$

將 x 與 x_{d+1} 連接成一個單一的擴展狀態變量 $\bar{x}$ ，可得

$$\bar{x} = \begin{pmatrix} x \\ x_{d+1} \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \\ x_{d+1} \end{pmatrix} \in \mathbb{R}^{d+1}. \tag{3.38}$$

接著，我們將擴展狀態變量 $\bar{x}$ 與原控制 u 的定義函數 $\bar{f} : \mathbb{R}^{d+1} \times \mathcal{U} \to \mathbb{R}^{d+1}$ 記為

$$\bar{f}(\bar{x}, u) := \begin{pmatrix} f(x,u) \\ r(x,u) \end{pmatrix} = \begin{pmatrix} f_1(x,u) \\ \vdots \\ f_d(x,u) \\ r(x,u) \end{pmatrix} \in \mathbb{R}^{d+1} \tag{3.39}$$

並將終端獎勵函數 $\bar{g} : \mathbb{R}^{d+1} \to \mathbb{R}$ 記為

$$\bar{g}(\bar{x}) := g(x) + x_{d+1}. \tag{3.40}$$

注意到 (3.39) 中的 $\bar{f}(\bar{x}, u)$ 不依賴於 x_{d+1}，並且有

$$\partial_{\bar{x}} \bar{f}(\bar{x}, u) = \begin{pmatrix} \partial_x f(x,u), & \partial_{x_{d+1}} f(x,u) \\ \partial_x r(x,u), & \partial_{x_{d+1}} r(x,u) \end{pmatrix} = \begin{pmatrix} \partial_x f(x,u), & 0 \\ \partial_x r(x,u), & 0 \end{pmatrix} \in \mathbb{R}^{(d+1)\times(d+1)}.$$

因此，我們的控制問題通過 (3.40) 中的終端獎勵函數 $\bar{g}$ 化為一個等價的無運行獎勵問題：

$$\min_{u \in \mathcal{U}_T} \ \bar{I}[u] := \bar{g}(\bar{x}(T)) = g(x(T)) + x_{d+1}(T) \tag{3.41}$$

並滿足由 (3.36) 與 (3.37) 合併而成的 ODE 與初值約束：

$$\begin{cases} \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t), u(t)), & \forall\, t \in [0, T], \\ \bar{x}(0) = \bar{x}_0. \end{cases} \tag{3.42}$$

將定理 3.1.12 應用於帶有獎勵 $\bar{I}$ 的控制問題 (3.41)，我們得到關於 control Hamiltonian

$$\bar{H}(\bar{x}, \bar{p}, u) = \bar{p} \cdot \bar{f}(\bar{x}, u)$$

的 Pontryagin 極大值原理。於是存在一個協態函數 $\bar{p}:[0,T]\to\mathbb{R}^{d+1}$，滿足伴隨方程

$$\begin{aligned}\Big(\dot{p}(t),\ \dot{p}_{d+1}(t)\Big) &= \dot{\bar{p}}(t)\\ &= -\partial_{\bar{x}}\bar{H}\Big(\bar{x}(t),\bar{p}(t),u(t)\Big)\\ &= -\bar{p}(t)\cdot\partial_{\bar{x}}\bar{f}\Big(\bar{x}(t),\bar{p}(t),u(t)\Big)\\ &= -\Big(p(t),\ p_{d+1}(t)\Big)\cdot\begin{pmatrix}\partial_x f(x(t),u(t)), & 0\\ \partial_x r(x(t),u(t)), & 0\end{pmatrix}\\ &= \Big(-p(t)\cdot\partial_x f(x(t),u(t)) - p_{d+1}\partial_x r(x(t),u(t)),\ 0\Big)\end{aligned} \tag{3.43}$$

以及終值

$$\bar{p}(T) = (p(T),\ p_{d+1}(T)) = (\nabla g(x(T)),\ 1). \tag{3.44}$$

由 (3.43) 與 (3.44) 可見，$p_{d+1}:[0,T]\to\mathbb{R}$ 滿足終值問題

$$\begin{cases}\dot{p}_{d+1}(t)=0, & \forall\, t\in[0,T],\\ p_{d+1}(T)=1,\end{cases}$$

從而可得 $p_{d+1}(t)=1$ 對所有 $t\in[0,T]$ 都成立。

由 $p_{d+1}(t)=1$，我們有

$$\bar{H}(\bar{x},\bar{p},u) = \bar{p}\cdot\bar{f}(\bar{x},u) = p\cdot f(x,u)+r(x,u) = H(x,p,u), \tag{3.45}$$

這就是帶運行獎勵的 control Hamiltonian。同時，由 (3.43) 與 (3.44) 可知，伴隨變量 p 滿足

$$\begin{cases}\dot{p}(t) = -p(t)\cdot\partial_x f(x(t),u(t)) - \partial_x r(x(t),u(t)), & \forall\, t\in[0,T],\\ p(T)=\nabla g(x(T)),\end{cases}$$

其中上式中關於 p 的 ODE 正是採用 (3.45) 中定義的 control Hamiltonian H 後的 $\dot{p}(t) = -\partial_x H(x(t),p(t),u(t))$。總之，定理 3.1.12 中的 Pontryagin 極大值原理對於帶運行獎勵的最優控制問題 (3.17) 同樣成立。

固定終點問題的 Pontryagin 極大值原理

現在我們考慮固定終點（固定到達狀態）且終止時刻自由的最優控制問題。這類問題的一般形式為

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(x(t),u(t))\,dt, \tag{3.46a}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t),u(t)), \tag{3.46b}$$

$$x(0) = x_0, \tag{3.46c}$$

$$x(T) = x_1, \tag{3.46d}$$

其中 $x_0, x_1 \in \mathbb{R}^d$ 是給定的初值和終值，而 T 是首次到達 x_1 的時間，並依賴於控制 u。

我們說明如下幾點：(i) 對 u 的唯一要求是在任意時刻 t 都有 $u(t) \in \mathcal{U}$，但終端時刻 T 可以變化；(ii) (3.46a) 中的目標函數不再含有終端獎勵；(iii) 通過適當設置運行獎勵（例如在 $\mathbb{R}^d \times \mathcal{U}$ 上一致取 $r \le -\delta$，其中 $\delta > 0$ 為給定常數），可以假設首次到達時間 T 是有限的，使得延遲到達會減少最終累積的運行獎勵。

為了推導固定終點最優控制問題 (3.46) 的 Pontryagin 極大值原理，我們需要引入如下更複雜的控制變分。

定義 3.1.13 (多重變分，multiple variations of a control)**.** 對任意 $K \in \mathbb{N}$，選取時間點 $0 < \tau_1 < \tau_2 < \cdots < \tau_K$、正數 $\lambda_1, \cdots, \lambda_K > 0$，以及控制變量 $w_1, w_2, \ldots, w_K \in \mathcal{U}$。對於足夠小的 $\epsilon > 0$，使得區間 $[\tau_k - \lambda_k\epsilon, \tau_k]$（$k \in [K] := \{1, \ldots, K\}$）彼此不重疊，我們定義控制 $u : [0,T] \to \mathcal{U}$ 的**多重變分**為

$$u^\epsilon(t) := \begin{cases} w_k, & \text{if } \tau_k - \lambda_k\epsilon \le t < \tau_k \text{ for any } k \in [K], \\ u(t), & \text{otherwise.} \end{cases} \tag{3.47}$$

對於 (3.47) 中定義的多重變分 u^ϵ，令 x^ϵ 為如下初值問題的解

$$\begin{cases} \dot{x}^\epsilon(t) = f(x^\epsilon(t), u^\epsilon(t)) \quad \text{for any } t \ge 0, \\ x^\epsilon(0) = x_0. \end{cases} \tag{3.48}$$

為了簡化後續表述，我們記矩陣值函數 $\Phi(t,s)$ 為如下問題的解

$$\begin{cases} \frac{d}{dt}\Phi(t,s) = \partial_x f(x(t),u(t)) \quad \text{for any } t \geq s \geq 0, \\ \Phi(s,s) = I_d. \end{cases} \tag{3.49}$$

我們還記

$$y_{\tau_k} = f(x(\tau_k), w_k) - f(x(\tau_k), u(\tau_k))$$

其中 $k = 1, \ldots, K$，這是根據 (3.47) 中定義的多重變分 u^ϵ 給出的。於是，沿著引理 3.1.9 的思路，很容易得到下面的引理。這裡省略其證明。

引理 3.1.14. 設 x^ϵ 是 (3.48) 的解，而 x 是原控制 u 的解。則對任意有限的 $T > 0$，當 $\epsilon \to 0$ 時，有

$$\max_{0 \leq t \leq T} \frac{|x^\epsilon(t) - x(t) - \epsilon y(t)|}{\epsilon} \to 0$$

其中 y 是如下的**擾動函數**:

$$y(t) = \begin{cases} 0, & \textit{if } 0 \leq t < \tau_k, \\ \sum_{k=1}^{k'} \lambda_k \Phi(t,\tau_k) y_{\tau_k}, & \textit{if } \tau_k \leq t < \tau_{k+1},\ k' = 1, \ldots, K-1, \\ \sum_{k=1}^{K} \lambda_k \Phi(t,\tau_k) y_{\tau_k}, & \textit{if } \tau_k \leq t. \end{cases}$$

定義 3.1.15 (Cone of variations)**.** 我們將時刻 t 處的**變分錐**定義為

$$C(t) := \left\{ \sum_{k=1}^{K} \lambda_k \Phi(t,\tau_k) y_{\tau_k} \ : \ \begin{array}{l} K \in \mathbb{N}, \text{ and } \lambda_k > 0, w_k \in \mathcal{U}, \\ 0 < \tau_1 \leq \cdots \leq \tau_K < t,\ \forall\, k \in [K] \end{array} \right\}. \tag{3.50}$$

注意，$C(t)$ 是 $\mathbb{R}^d$ 中的一個凸錐，因為對所有 k 都有 $\lambda_k \geq 0$，並且它由對控制 u 施加某種多重變分後在狀態 $x(t)$ 上產生的所有到 ϵ 一階（即 $O(\epsilon)$ 量級）的變化構成。

$C(t)$ 的性質對於建立固定終點問題 (3.46) 的 Pontryagin 極大值原理非常重要。為了研究 $C(t)$ 的性質，我們需要下面這個拓撲引理。

引理 3.1.16 (Zeros of a vector field)**.** 設 S 是 $\mathbb{R}^n$ 中一個緊（閉且有界）且凸的子集，q 是 S 的一個內點。若 $\phi : S \to \mathbb{R}^n$ 是一個連續向量場，並滿足如下嚴格不等式:

$$|\phi(x) - x| < |q - x|, \quad \forall\, x \in \partial S. \tag{3.51}$$

則存在 $x^* \in S$，使得 $\phi(x^*) = q$。

证明. 我們先考慮一個簡單情形，即 $S = \bar{B}_1(0) := \{x \in \mathbb{R}^n : |x| \le 1\}$，它是以 0 為中心、半徑為 1 的閉單位球，而 $q = 0 \in \mathbb{R}^n$ 是 $\bar{B}_1(0)$ 的一個內點。此時，(3.51) 化為

$$|\phi(x) - x| < |x|, \quad \forall\, x \in \partial\bar{B}_1(0).$$

將上式兩邊平方，得到

$$|\phi(x)|^2 - 2\phi(x) \cdot x < 0,$$

這意味著當 $|x| = 1$ 時，對所有 x 都有 $\phi(x) \cdot x > \frac{1}{2}|\phi(x)|^2 \ge 0$。

由於 $\bar{B}_1(0)$ 與 $\partial\bar{B}_1(0)$ 都是 $\mathbb{R}^n$ 中的緊集，且 ϕ 連續，因此存在 $M, m > 0$ 使得

$$\phi(x) \cdot x \ge m \quad \text{和} \quad |\phi(x)| \le M$$

對所有 $x \in \bar{B}_1(0)$（包括其邊界）成立。取 $\alpha \in (0, 2m/M)$ 並定義 $\psi(x) := x - \alpha\phi(x)$，則有

$$\begin{aligned} |\psi(x)|^2 = |x - \alpha\phi(x)|^2 &= |x|^2 - 2\alpha\phi(x) \cdot x + \alpha^2|\phi(x)|^2 \\ &\le |x|^2 - 2m\alpha + \alpha^2 M = |x|^2 - \alpha(2m - \alpha M) < |x|^2 \le 1 \end{aligned}$$

對所有 $x \in \bar{B}_1(0)$ 成立。由 Brouwer 不動點定理，存在 $x^* \in \bar{B}_1(0)$ 使得 $\psi(x^*) = x^*$。即 $x^* - \alpha\phi(x^*) = x^*$，或等價地 $\phi(x^*) = 0 = q$。

在一般情形下，我們總可以先通過平移使 S 的內點 q 移到 0，再以 $q = 0$ 為中心對 S 作徑向伸縮，將 S 變換為 $\bar{B}_1(0)$。這兩步操作的組合是一個同胚（即從 $\mathbb{R}^n$ 到 $\mathbb{R}^n$ 的連續映射且其逆映射也連續），從而就把一般情形化歸為上面的簡單情形 $S = \bar{B}_1(0)$。 □

現在考慮給定固定終點 $x_1 \in \mathbb{R}^d$ 的最優控制問題。仍令 $T > 0$ 為滿足初值問題

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)), & \forall\, t \ge 0 \\ x(0) = x_0 \end{cases}$$

的軌跡 x 首次到達終點 x_1 的時刻。則總獎勵泛函為

$$I[u] = \int_0^T r(x(t), u(t))\, dt,$$

其中 $r:\mathbb{R}^d\times\mathcal{U}\to\mathbb{R}$ 是給定的運行獎勵函數。

我們再次考慮滿足如下初值問題的輔助函數 $x_{d+1}:[0,T]\to\mathbb{R}$

$$\begin{cases}\dot{x}_{d+1}(t)=r(x(t),u(t)), & \forall\, t\geq 0,\\ x_{d+1}(0)=0.\end{cases}$$

記

$$\bar{x}:=\begin{pmatrix}x\\ x_{d+1}\end{pmatrix}\in\mathbb{R}^{d+1},\quad \bar{x}_0:=\begin{pmatrix}x_0\\ 0\end{pmatrix}\in\mathbb{R}^{d+1},$$

並記

$$\bar{f}(\bar{x},u):=\begin{pmatrix}f(x,u)\\ r(x,u)\end{pmatrix}\in\mathbb{R}^{d+1},$$

以及終端獎勵泛函 $\bar{g}(\bar{x}):=x_{d+1}$。那麼，具有固定終點 $x_1\in\mathbb{R}^d$ 的最優控制問題 (3.46)（回憶一下，$x_0,x_1\in\mathbb{R}^d$ 分別是 $x:[0,T]\to\mathbb{R}^d$ 的給定起點和終點，而這裡的 x_1 不是 $x(t)$ 的第一個分量；在本證明中我們不討論 $x(t)$ 的各個單獨分量）可以寫成

$$\max_u \quad I[u]:=\bar{g}(\bar{x}(T))=x_{d+1}(T), \tag{3.52a}$$

$$\text{s.t.}\quad \begin{cases}\dot{\bar{x}}(t)=\bar{f}(\bar{x}(t),u(t)), & \forall\, t\in[0,T],\\ \bar{x}(0)=\bar{x}_0,\\ x(T)=x_1.\end{cases} \tag{3.52b}$$

其中 u 的容許集為 $\{u:[0,\infty)\to\mathcal{U}\}$。

為了為最優控制問題 (3.52) 建立 Pontryagin 極大值原理，我們令 u 表示最優控制，x 表示對應的最優軌跡。我們的第一個目標是構造 Pontryagin 極大值原理中的協態 p。為此，我們將使用上面定義的記號 $\bar{x}$、$\bar{f}$ 和 $\bar{g}$。具體地，我們按 (3.50) 定義此時的變分錐為

$$C:=C(T)=\left\{\sum_{k=1}^{K}\lambda_k\bar{\Phi}(T,\tau_k)\bar{y}_{\tau_k}\ :\ \begin{array}{l}K\in\mathbb{N},\ \text{and}\ \lambda_k\geq 0, w_k\in\mathcal{U},\\ 0<\tau_1\leq\cdots\leq\tau_K<T,\ \forall\, k\in[K]\end{array}\right\}. \tag{3.53}$$

其中

$$\bar{y}_{\tau_k}=\bar{f}(\bar{x}(\tau_k),w_k)-\bar{f}(\bar{x}(\tau_k),u(\tau_k)),$$

而 $\bar{\Phi}(t,s)$ 滿足

$$\begin{cases} \frac{d}{dt}\bar{\Phi}(t,s) = \partial_{\bar{x}}\bar{f}(\bar{x}(t),u(t)), & \text{for any } t \geq s \geq 0 \\ \bar{\Phi}(s,s) = I_{d+1}. \end{cases}$$

下面的引理指出了 C 的一個關鍵性質。

引理 3.1.17 (變分錐的幾何性質)**.** 單位向量 $e_{d+1} = (0,\ldots,0,1) \in \mathbb{R}^{d+1}$ 不是 (3.53) 中 C 的內點。

证明. 我們採用反證法。假設 e_{d+1} 是 C 的內點,則存在線性無關向量 $z_1,\ldots,z_{d+1} \in \mathbb{R}^{d+1}$ 以及正數 $\lambda_1,\ldots,\lambda_{d+1} > 0$，使得

$$e_{d+1} = \sum_{k=1}^{d+1} \lambda_k z_k,$$

其中

$$z_k = \bar{\Phi}(T,\tau_k)\bar{y}_{\tau_k}$$

對某些時間點 $0 < \tau_1 < \cdots < \tau_{d+1} < T$ 成立（由於 e_{d+1} 是 C 的內點，我們總可以安全地假設這些時間點互不相同，因為我們可以讓 $\tau_1,\ldots,\tau_{d+1}$ 兩兩不同）。

現在令 $\eta > 0$ 並定義

$$S := \Big\{ \sum_{k=1}^{d+1} \lambda_k z_k : 0 \leq \lambda_k \leq \eta,\ \forall\, k \in [d+1] \Big\}.$$

由於 $z_1,\ldots,z_{d+1}$ 線性無關，我們知道 S 有內點。此外，S 是一個緊的凸多面體，其所有邊長都與 η 成比例，並且有一個位於 $0 \in \mathbb{R}^{d+1}$ 的頂點。

由於 e_{d+1} 是 C 的內點，因此存在足夠小的 $\mu > 0$ 使得 $q := \mu e_{d+1}$ 是 S 的內點。此外，存在 $\delta := \delta(\eta,\mu) > 0$，使得對所有 $\bar{y} \in \partial S$ 有 $|\bar{y} - q| > \delta$。

對任意小的 $\epsilon > 0$，定義 $\phi^\epsilon : S \to \mathbb{R}^{d+1}$ 為

$$\phi^\epsilon(z) := \frac{\bar{x}^\epsilon(T) - \bar{x}(T)}{\epsilon} \tag{3.54}$$

其中 z 是 S 中的任意點，且 $\bar{x}$ 是初值問題

$$\begin{cases} \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t), u(t)), & \forall\, t \in [0, T], \\ \bar{x}(0) = \bar{x}_0, \end{cases}$$

的解，而 $\bar{x}^\epsilon$ 是初值問題

$$\begin{cases} \dot{\bar{x}}^\epsilon(t) = \bar{f}(\bar{x}^\epsilon(t), u^\epsilon(t)), & \forall\, t \in [0, T], \\ \bar{x}^\epsilon(0) = \bar{x}_0, \end{cases}$$

的解，且 u^ϵ 是控制 u 的多重變分，定義為

$$u^\epsilon(t) = \begin{cases} w_k, & \text{if } \tau_k - \lambda_k \epsilon \le t < \tau_k,\ k = 1, \ldots, K, \\ u(t), & \text{otherwise.} \end{cases}$$

由引理 3.1.14 可知，對任意有限 T，當 $\epsilon \to 0$ 時，

$$\max_{0 \le t \le T} \frac{|\bar{x}^\epsilon(t) - \bar{x}(t) - \epsilon \bar{y}(t)|}{\epsilon} \to 0$$

其中 $\bar{y}$ 是 $\bar{x}^\epsilon$ 與 $\bar{x}$ 之間的一階擾動，它的定義與引理 3.1.14 中相同，只不過其中的 $\Phi(t, \tau_k)$ 與 y_{τ_k} 要替換成這裡在 (3.54) 中決定 z 的對應對象。因此，

$$|\phi^\epsilon(z) - \bar{y}(T)| = \left| \frac{\bar{x}^\epsilon(T) - \bar{x}(T)}{\epsilon} - \bar{y}(T) \right| = \left| \frac{\bar{x}^\epsilon(T) - \bar{x}(T) - \epsilon \bar{y}(T)}{\epsilon} \right| \le |O(\epsilon)|.$$

我們可以取 ϵ 足夠小，使得 $|O(\epsilon)| < \delta = \delta(\eta, \mu)$，從而有

$$|\phi^\epsilon(z) - \bar{y}(T)| \le |O(\epsilon)| < \delta = \delta(\eta, \mu) \le |\bar{y} - q|, \quad \forall\, \bar{y} \in \partial S.$$

由引理 3.1.16，存在 $z^* \in S$，使得 $\phi^\epsilon(z^*) = q = \mu e_{d+1}$。令 u^ϵ 是與該 z^* 對應的控制 u 的多重變分，x^ϵ 是沿著 u^ϵ 的軌跡。則 $\phi^\epsilon(z^*) = \frac{1}{\epsilon}(\bar{x}^\epsilon(T) - \bar{x}(T)) = \mu e_{d+1}$，即 $x^\epsilon(T) = x(T) = x_1$ 且 $x^\epsilon_{d+1}(T) - x_{d+1}(T) = \mu\epsilon > 0$。這意味著在時刻 T，$x^\epsilon(T) = x_1$，同時獲得了更大的獎勵：

$$\int_0^T r(x^\epsilon(t), u^\epsilon(t))\, dt = x^\epsilon_{d+1}(T) > x_{d+1}(T) = \int_0^T r(x(t), u(t))\, dt,$$

這與 u 和 x 的最優性矛盾。因此，e_{d+1} 不是 C 的內點。 □

引理 3.1.17 表明，e_{d+1} 要麼位於凸錐 C 的邊界上，要麼在 C 之外。若 $e_{d+1} \in \partial C$，則稱這是所謂的**異常問題**（abnormal problem），此時 Pontryagin 極大值原理 沒有作用。若 $e_{d+1} \notin C$，則下面給出的原理成立。

定理 3.1.18 (固定重點問題的 Pontryagin 極大值原理)**.** *假設最優控制問題 (3.46) 不是異常的，且 u 是該問題的一個最優解，x 是與 u 對應的軌跡。則存在函數 $p : [0,T] \to \mathbb{R}^d$ 滿足伴隨方程，並且如定理 3.1.12 中所述的極大化原理對三元組 (x,p,u) 成立。*

证明. 當 (3.46) 不是異常問題時，有 $e_{d+1} \notin C$。注意 C 是閉的且凸的。由支撐超平面定理，存在 $v = (v_1, \ldots, v_{d+1}) \in \mathbb{R}^{d+1}$，使得 $v_{d+1} > 0$ 且對所有 $z \in C$ 都有 $v \cdot z \le 0$。不失一般性，我們通過將 v 替換為 v/v_{d+1}，可以假設 $v_{d+1} = 1$。

設 $\bar{p} = (p, p_{d+1}) : [0,T] \to \mathbb{R}^{d+1}$ 滿足終值問題

$$\begin{cases} \dot{\bar{p}}(t) = -\bar{p}(t)\partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [0,T], \\ \bar{p}(T) = v. \end{cases} \tag{3.55}$$

因此，對所有 $t \in [0,T]$，有 $\dot{p}_{d+1}(t) = 0$，且 $p_{d+1}(T) = v_{d+1} = 1$，從而推出 $p_{d+1}(t) = 1$。於是，根據 (3.55)，p 滿足的伴隨方程

$$\begin{aligned} \dot{p}(t) &= -\bar{p}(t)\partial_x \bar{f}(\bar{x}(t), u(t)) \\ &= -p(t)\partial_x f(x(t), u(t)) - p_{d+1}(t)\partial_x r(x(t), u(t)) \\ &= -p(t)\partial_x f(x(t), u(t)) - \partial_x r(x(t), u(t)) \end{aligned}$$

在整個區間 $[0,T]$ 上成立。

對任意 $\tau \in [0,T)$ 和任意 $w \in \mathcal{U}$，定義

$$\bar{y}_\tau := \bar{f}(\bar{x}(\tau), w) - \bar{f}(\bar{x}(\tau), u(\tau))\bar{y}(t)$$

並定義 $\bar{y}(T) := \bar{\Phi}(T,\tau)\bar{y}_\tau$，它是如下初值問題的解

$$\begin{cases} \dot{\bar{y}}(t) = \partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [\tau, T], \\ \bar{y}(\tau) = \bar{y}_\tau. \end{cases}$$

此外，由 (3.53) 中 C 的定義可知，$\bar{y}(T) \in C$。

現在注意到，由本證明開頭給出的 C 的幾何性質以及終端條件 $\bar{p}(T) = v$，

有

$$0 \geq v \cdot \bar{y}(T) = \bar{p}(T) \cdot \bar{y}(T). \tag{3.56}$$

此外，還有

$$\begin{aligned}\frac{d}{dt}(\bar{p}(t) \cdot \bar{y}(t)) &= \dot{\bar{p}}(t) \cdot \bar{y}(t) + \bar{p}(t) \cdot \dot{\bar{y}}(t) \\ &= -\bar{p}(t)\partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)) \cdot \bar{y}(t) + \bar{p}(t) \cdot \partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t))\bar{y}(t) \\ &= 0,\end{aligned}$$

因此 $\bar{p}(t) \cdot \bar{y}(t)$ 關於 t 為常數，從而有

$$\bar{p}(T) \cdot \bar{y}(T) = \bar{p}(\tau) \cdot \bar{y}(\tau). \tag{3.57}$$

結合 (3.56) 與 (3.57)，可得 $0 \geq \bar{p}(\tau)\bar{y}(\tau) = \bar{p}(\tau)\bar{y}_\tau$，即

$$0 \geq \bar{p}(\tau) \cdot (\bar{f}(\bar{x}(\tau), w) - \bar{f}(\bar{x}(\tau), u(\tau)).$$

因此，

$$\begin{aligned}\bar{H}(\bar{x}(\tau), \bar{p}(\tau), w) &= \bar{p}(\tau) \cdot \bar{f}(\bar{x}(\tau), w) \\ &\leq \bar{p}(\tau) \cdot \bar{f}(\bar{x}(\tau), u(\tau)) \\ &= \bar{H}(\bar{x}(\tau), \bar{p}(\tau), u(\tau)).\end{aligned}$$

由於 $w \in \mathcal{U}$ 與 $\tau \in [0, T)$ 都是任意的，並且 $u(\tau) \in \mathcal{U}$，於是得到

$$\bar{H}(\bar{x}(\tau), \bar{p}(\tau), u(\tau)) = \max_{w \in \mathcal{U}} \bar{H}(\bar{x}(\tau), \bar{p}(\tau), w) \tag{3.58}$$

對每個 τ 都成立。注意，由 $\bar{H}$ 的定義，有

$$\bar{H}(\bar{x}, \bar{p}, u) = \bar{p} \cdot \bar{f}(\bar{x}, u) = p \cdot f(x, u) + r(x, u), \tag{3.59}$$

因為 $p_{d+1} = v_{d+1} = 1$。將 (3.59) 代入 (3.58)，即可得到固定終點最優控制問題 (3.52)，或等價地 (3.46) 的極大值原理。 □

3.1.4 Hamilton–Jacobi–Bellman 方程

在本節中，我們考慮一種不同的求解最優控制問題的方法。該方法基於動態規劃（dynamic programming，DP）的思想，即將給定問題嵌入到一類更大的問題集合中，並一次性求解整個問題族。然後，該整體問題族的解可以直接推出原始問題的解。

回顧具有固定時間 T 且終點自由的最優控制問題，其形式為

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(z(s),u(s))\,ds + g(z(T)), \tag{3.60a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(s) = f(z(s),u(s)), \quad \forall\, s\in[0,T], \\ z(0) = x, \end{cases} \tag{3.60b}$$

其中初始值 $x\in\mathbb{R}^d$ 已知。我們可以將該問題 (3.60) 嵌入到一個更一般的問題族中，其中初始值 $x\in\mathbb{R}^d$ 和起始時間 $t\in[0,T]$ 可以任意給定：

$$\max_{u} \quad I_{x,t}[u] := \int_t^T r(z(s),u(s))\,ds + g(z(T)), \tag{3.61a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(s) = f(z(s),u(s)), \quad \forall\, s\in[t,T], \\ z(t) = x, \end{cases} \tag{3.61b}$$

其中 u 只需滿足對每個 $s\in[t,T]$ 有 $u(s)\in\mathcal{U}$。於是，原始最優控制問題 (3.60) 就是 (3.61) 的一個特例，對應於給定的 x 且 $t=0$。

我們定義**值函數**（value function）$v:\mathbb{R}^d\times[0,T]\to\mathbb{R}$ 為

$$v(x,t) := \sup_u I_{x,t}[u]. \tag{3.62}$$

顯然，對於所有 $x\in\mathbb{R}^d$，都有 $v(x,T)=g(x)$。

現在的問題是，是否可以通過構造關於 v 的問題並求解它來得到 $v(x,t)$，然後再利用得到的值函數 v 來確定最優控制 u。這正是動態規劃的典型策略。

定理 3.1.19 (Hamilton–Jacobi–Bellman equation)**.** *假設 v 關於 (x,t) 是 C^1 的，則 v 滿足如下非線性偏微分方程，稱為* ***Hamilton–Jacobi–Bellman*** *(HJB) 方程：*

$$\partial_t v(x,t) + \max_{w\in\mathcal{U}} \Big\{ \nabla v(x,t)\cdot f(x,w) + r(x,w) \Big\} = 0 \tag{3.63}$$

並滿足終端條件 $v(x,T)=g(x)$，對所有 $x\in\mathbb{R}^d$ 和 $t\in[0,T]$ 成立。

证明. 我們首先證明 (3.63) 在取上確界且不等式（小於等於 0）的形式下成立，然後在最優控制 u 存在的情況下說明其原始形式成立。

為此，我們沿用之前的記號，定義

$$\mathcal{U}_T := \{u:[0,T]\to\mathcal{U} \text{ measurable}\} \tag{3.64}$$

對於任意 $x\in\mathbb{R}^d$、$t\in[0,T)$、滿足 $t+\epsilon\le T$ 的 $\epsilon>0$，以及任意 $w\in\mathcal{U}$，考慮在區間 $[t,t+\epsilon]$ 上的初值問題：

$$\begin{cases} \dot{z}(s)=f(z(s),w), \quad \forall\, s\in[t,t+\epsilon], \\ z(t)=x. \end{cases} \tag{3.65}$$

則在區間 $[t,t+\epsilon]$ 上獲得的收益為 $\int_t^{t+\epsilon} r(z(s),w)ds$。記 $x(t+s)=z(t+s)$，則在區間 $[t+\epsilon,T]$ 上能夠獲得的最大收益為 $v(x(t+\epsilon),t+\epsilon)$，這是由值函數 v 的定義得到的。

另一方面，由於 $v(x,t)$ 是在 (x,t) 處的最大收益，其最優性意味著

$$v(x,t)=v(z(t),t)\ge\int_t^{t+\epsilon} r(z(s),w)\,ds+v(x(t+\epsilon),t+\epsilon) \tag{3.66}$$

對 (3.66) 進行整理併除以 ϵ，得到

$$\frac{v(x(t+\epsilon),t+\epsilon)-v(x,t)}{\epsilon}+\frac{1}{\epsilon}\int_t^{t+\epsilon} r(z(s),w)\,ds\le 0.$$

令 $\epsilon\to 0$，則有 $x(t+s)=z(t+s)\to z(t)=x$，因此

$$\partial_t v(x,t)+\nabla v(x,t)\cdot\dot{z}(t)+r(x,w)\le 0.$$

由於 $\dot{z}(t)=f(z(t),w)$，我們得到

$$\partial_t v(x,t)+\nabla v(x,t)\cdot f(x,w)+r(x,w)\le 0. \tag{3.67}$$

由於 (3.67) 對任意 $w\in\mathcal{U}$ 都成立，我們得到

$$\partial_t v(x,t)+\sup_{w\in\mathcal{U}}\Big\{\nabla v(x,t)\cdot f(x,w)+r(x,w)\Big\}\le 0. \tag{3.68}$$

接下來，我們說明該上確界實際上可以取到最大值，並且 (3.68) 中的不等式可以取等號。為此，假設從時間 t 開始始終使用最優控制 u，則有

$$v(x,t) = \int_t^{t+\epsilon} r(x(s),u(s))\,ds + v(x(t+\epsilon),t+\epsilon)$$

對上述等式整理併除以 ϵ，得到

$$\frac{v(x(t+\epsilon),t+\epsilon) - v(x,t)}{\epsilon} + \frac{1}{\epsilon}\int_t^{t+\epsilon} r(x(s),u(s))\,ds = 0.$$

令 $\epsilon \to 0$，得到

$$\partial_t v(x,t) + \nabla v(x,t)\cdot f(x(t),u(t)) + r(x(t),u(t)) = 0. \tag{3.69}$$

由於對每個 t 都有 $u(t) \in \mathcal{U}$，我們知道 (3.68) 中的上確界實際上可以取到最大值，並且等式成立，從而得到 HJB 方程 (3.63)。 □

現在我們來看如何利用 HJB 方程來求解最優控制。假設我們能夠求解 HJB 方程並得到其解 v，則定義 $\pi : \mathbb{R}^d \times [0,T] \to \mathcal{U}$ 為

$$\pi(x,t) := \arg\max_{w\in\mathcal{U}}\{\nabla v(x,t)\cdot f(x,w) + r(x,w)\}. \tag{3.70}$$

此時有

$$\partial_t v(x,t) + \nabla v(x,t)\cdot f(x,\pi(x,t)) + r(x,\pi(x,t)) = 0. \tag{3.71}$$

對於任意 $x \in \mathbb{R}^d$ 和 $t \in [0,T)$，我們可以求解如下初值問題（假設 $\pi(\cdot,t)$ 足夠光滑）得到 x^*：

$$\begin{cases} \dot{x}^*(s) = f(x^*(s),\pi(x^*(s),s)), \quad \forall\, s \in [t,T], \\ x^*(t) = x. \end{cases} \tag{3.72}$$

則可以斷言，問題 (3.61) 的最優控制 u^* 滿足

$$u^*(s) := \pi(x^*(s),s)$$

對所有 $s \in [t, T]$ 成立。為了說明該 u^* 確實是最優的，我們注意到

$$
\begin{aligned}
I_{x,t}[u^*] &= \int_t^T r(x^*(s), u^*(s))\, ds + g(x^*(T)) \\
&= -\int_t^T \Big(\partial_s v(x^*(s), s) + \partial_x v(x^*(s), s) \cdot f(x^*(s), u^*(s))\Big) ds + g(x^*(T)) \\
&= -\int_t^T \Big(\partial_s v(x^*(s), s) + \partial_x v(x^*(s), s) \cdot \dot{x}^*(s)\Big) ds + g(x^*(T)) \\
&= -\int_t^T \frac{d}{ds}\Big(v(x^*(s), s)\Big) ds + g(x^*(T)) \\
&= v(x^*(t), t) - v(x^*(T), T) + g(x^*(T)) \\
&= v(x, t) \\
&= \sup_u I_{x,t}[u],
\end{aligned}
$$

其中第二個等號來自 (3.71)，第三個來自 (3.72) 中的 ODE，第六個等號來自終端條件 $v(x, T) = g(x)$。這說明 u^* 是 (3.61) 的一個最優控制。

至此，我們可以看到，通過求解 HJB 方程 (3.63) 可以作為求解最優控制問題的一種替代方法，其步驟如上所述。

然而需要指出的是，HJB 方程 (3.63) 是關於 v 的一個非線性一階偏微分方程，並且由於其中包含最大化運算，還需要在每個時間 t 上對 $\mathcal{U}$ 求解一個優化問題，使其更加複雜。在某些特殊情況下，最大值可以以閉式形式給出，從而可以消去 (3.63) 中的最大化運算。然而，即便沒有該最大化運算，由於其高維性（空間維度為 d，對應變量 x），該偏微分方程在實際中仍然可能難以甚至無法求解。儘管如此，HJB 方程（從動態規劃角度）與最優控制之間的緊密聯繫，為研究和求解控制問題提供了重要的理論分析工具。

3.2 最優概率密度控制理論

深度學習問題通常涉及海量的數據點或智能體。例如，在一個大型多智能體控制系統中，每個智能體的狀態都可以表示為一個向量，其分量描述其位置、朝向、速度和加速度。在許多情況下，這些智能體可以被視為服從某些特定概率分佈的樣本。通常，這類學習問題被表述為使這些樣本隨時間演化，這等價於它們所服從的概率分佈的演化，從而使某個目標函數在給定時間區間 $[0, T]$ 上被最小化或最大化。因此，這類問題可以表述為概率密度的最優控制。在本節中，我們將把第 3.1 節中經典最優控制的理論結果推廣到最優概率控制的情

形。

設 Ω 是 $\mathbb{R}^d$ 的一個開子集，其中 d 是狀態向量的維數（例如上文提到的某個智能體的位置、朝向和加速度的拼接後的總維度）。在本節的其餘部分中，我們通常把一個樣本稱為一個智能體。設有 N 個智能體，其狀態為 $\{x_i \in \Omega : i = 1, \ldots, N\}$，並且它們服從某個初始分佈 p 於 Ω 上，即 $x_i \sim p$。在固定時間區間 $[0, T]$ 上，一個隨時間演化的控制向量場 $u : \Omega \times [0, T] \to \mathbb{R}^d$ 的作用下，智能體狀態按照

$$\dot{x}_i(t) = u(x_i(t), t), \quad i = 1, \ldots, N. \tag{3.73}$$

進行變化。每個智能體在 $[0, T]$ 上積累某些運行獎勵，並在時刻 T 獲得一個終端獎勵。不同於經典最優控制，這些獎勵**不僅依賴於該特定智能體本身，還依賴於所有智能體的整體行為**。下面我們給出若干運行獎勵與終端獎勵函數的例子。

我們用 $\rho(\cdot, t)$ 表示在每個時刻 t 時狀態 $x_i(t)$ 的時間邊緣分佈，即 $x_i(t) \sim \rho(\cdot, t)$。則 (3.73) 對應於連續方程

$$\partial_t \rho(x, t) + \nabla \cdot \big(\rho(x, t) u(x, t)\big) = 0 \tag{3.74}$$

定義在 $\Omega \times [0, T]$ 上。為簡化記號，我們將 $\rho(\cdot, t)$ 和 $u(\cdot, t)$ 分別記為 ρ_t 和 u_t，並且在後文中常常省略積分中的 (x)，除非為了說明清楚而必須寫出。

我們定義 Ω 上概率密度函數的空間：

$$P = \Big\{p : \Omega \to [0, \infty) : \int_\Omega p(x)\, dx = 1\Big\}. \tag{3.75}$$

此外，對於給定的終端時間 $T > 0$，我們定義隨時間演化的概率密度函數空間為：

$$P_T = \big\{\rho : \Omega \times [0, T] \to \mathbb{R} : \rho(\cdot, t) \in P,\ \forall\, t \in [0, T]\big\},$$

我們假設在任意固定時刻的可容許控制向量場集合為

$$U = \big\{w \in C(\bar{\Omega}; \mathbb{R}^d) : \mathrm{Lip}(w) \le M_U,\ (w \cdot n)|_{\partial\Omega} = 0\big\}, \tag{3.76}$$

其中某個常數 $M_U > 0$ 已給定；也就是說，w 在 $\bar{\Omega}$ 上是 M_U-Lipschitz 連續的，並滿足邊界條件 $w(x) \cdot n(x) = 0$，這裡 $n(x) \in \mathbb{R}^d$ 表示 $\partial\Omega$ 在點 x 處的外法向量。w 的這個邊界條件保證總概率密度保持為 1。於是我們將隨時間演化

的可容許控制向量場集合定義為

$$U_T = \big\{u : \bar{\Omega} \times [0,T] \to \mathbb{R}^d : u(\cdot,t) \in U,\ \forall\, t \in [0,T]\big\}. \tag{3.77}$$

注意，(3.77) 並不要求 $u(x,\cdot)$ 關於 t 是連續函數，因此允許 u 瞬時改變方向，從而使下面得到的結果適用於更廣泛的情形。

現在，我們可以將最優概率控制問題寫成如下的一般形式：

$$\max_{u\in U_T} \quad I[u] := \int_0^T R(\rho_t, u_t)\,dt + G(\rho_T), \tag{3.78a}$$

$$\text{s.t.} \quad \partial_t \rho_t + \nabla\cdot(\rho_t u_t) = 0, \quad \forall\,(t,x) \in [0,T]\times\Omega, \tag{3.78b}$$

$$\rho_0 = p, \quad \forall\, x \in \Omega, \tag{3.78c}$$

其中 I 是總獎勵泛函，U_T 是可容許控制向量場集合，而 p 是給定的初始概率分佈，即 $x_i(0) \sim p$。在 (3.78) 中，$R : P \times U \to \mathbb{R}$ 和 $G : P \to \mathbb{R}$ 分別是運行獎勵泛函（running reward functional）和終端獎勵泛函（terminal reward functional）。我們假設 $I^* := \max_{u\in U_T} I[u]$ 是有限的。

對於概率分佈和控制，我們作出以下若干假設，以確保後續推導可以嚴格進行。

假設 3.2.1. 我們作如下假設：

(i) 對所有 $x \in \Omega$，$\rho(x,\cdot)$ 在 $[0,T]$ 上關於 t 可微。

(ii) 對所有 $t \in [0,T]$，$\rho(\cdot,t)u(\cdot,t)$ 在 Ω 上可微。

(iii) 對所有 $x \in \Omega$，$\rho(x,\cdot)u(x,\cdot)$ 在 $(0,T)$ 上處處有 Lebesgue 點。

(iv) 對所有 $w \in U$，$R(\cdot,w)$ 在 P 上關於 L^2 意義是 Fréchet 可微的。

(v) $G(\cdot)$ 在 P 上關於 L^2 意義是 Fréchet 可微的。

假設 3.2.1 的 (i) 和 (ii) 可以放寬為弱可微，只要 (3.74) 成立即可。假設 3.2.1 的 (iii) 保證了對所有 $t \in (0,T)$，有 $\frac{d}{dt}\int_0^t \rho(x,s)u(x,s)\,ds = \rho(x,t)u(x,t)$。這一點會在下面引理 3.2.7 的證明中的某一步被使用（我們會在證明中明確指出）。假設 3.2.1 的 (iv) 和 (v) 使得在任意 $p \in P$ 處，R 和 G 都有泛函導數（functional derivative），而這對於後續建立 control Hamiltonian 動力系統是必要的。注意，在實際應用中，運行獎勵泛函和終端獎勵泛函通常都滿足假設 3.2.1 的 (iv) 和 (v)。

3.2.1 概率控制的極大值原理

現在，在本小節中，我們為具有任意初始值 $p \in P$ 的最優概率控制問題 (3.78) 建立一個極大值原理（maximum principle，MP）。作為第 3.1 節中經典最優控制 Pontryagin 極大值原理的對應，這裡建立的 MP 是關於最優解 u 的一個逐時刻必要條件，它定義在無限維空間 P_T 與 U_T 上。為了發展這一 MP，我們首先引入伴隨偏微分方程（PDE）和控制 Hamilton 泛函的定義。

定義 3.2.2 (Adjoint PDE)**.** 設 $u \in U_T$ 是一個控制向量場，$\rho \in P_T$ 是滿足 (3.78b) 和初值條件 (3.78c) 的相應演化概率函數。定義如下演化 PDE:

$$\partial_t \phi_t + u_t \cdot \nabla \phi_t = -\frac{\delta}{\delta \rho_t} R(\rho_t, u_t) \tag{3.79}$$

定義在 $\Omega \times [0,T]$ 上。我們稱 (3.79) 為與 (ρ, u) 相關聯的**伴隨 PDE**。若 $\phi : \Omega \times [0,T] \to \mathbb{R}$ 滿足 (3.79) 且終端條件為 $\phi_T = \frac{\delta}{\delta \rho_T} G(\rho_T)$，則稱 ϕ 為 (ρ, u) 的**伴隨函數**。

在全文中，我們假設在控制問題 (3.78) 中，對任意 $w \in U$，$\frac{\delta}{\delta \rho} R(\cdot, w)$、$\frac{\delta}{\delta \rho} G(\cdot)$ 以及 u 都足夠光滑，從而使得下述假設成立。

假設 3.2.3. 對於任意初始值 $p \in P$, 與 (3.78) 對應的伴隨函數 $\phi : \Omega \times [0,T] \to \mathbb{R}$ 存在。

注意，伴隨函數 ϕ 具有空間變量 x 和時間變量 t，而伴隨方程是一個演化 PDE。相比之下，在經典最優控制中，伴隨函數僅是 t 的函數，而伴隨方程是一個 ODE。

定義 3.2.4 (Hamiltonian functional)**.** 設 $p \in P$，$w \in U$，並令 $f : \Omega \to \mathbb{R}$ 為一個可微函數，我們定義 **Hamiltonian 泛函**為

$$H(p, f, w) = \langle p,\ w \cdot \nabla f \rangle + R(p, w). \tag{3.80}$$

其中 R 是 (3.78) 中的運行獎勵泛函。

現在我們可以驗證，由任意 $u \in U_T$ 所誘導的 (ρ, ϕ) 滿足辛形式的控制 Hamilton 動力系統，如下命題所述。

命題 3.2.5 (Control Hamiltonian dynamics)**.** 設 $u \in U_T$ 是一個控制向量場，$\rho \in P_T$ 是相應的演化概率，ϕ 是 (ρ, u) 的伴隨函數。則 (ρ, ϕ) 滿足控制

Hamilton 動力系統:

$$\begin{cases} \partial_t \rho_t = \dfrac{\delta}{\delta \phi_t} H(\rho_t, \phi_t, u_t) = -\nabla \cdot (\rho_t u_t), \\ \partial_t \phi_t = -\dfrac{\delta}{\delta \rho_t} H(\rho_t, \phi_t, u_t) = -u_t \cdot \nabla \phi_t - \dfrac{\delta}{\delta \rho_t} R(\rho_t, u_t) \end{cases} \tag{3.81}$$

對所有 t 都成立。

利用 (3.74)、(3.79) 和 (3.80)，動力系統 (3.81) 很容易驗證，因此我們省略命題 3.2.5 的證明。

在下文中，我們記 u^* 為 (3.78) 的一個最優控制向量場，ρ^* 為其對應的概率分佈。然後我們如下定義 u^* 的一個針狀變分 u^ϵ：對於任意時刻 $\tau \in (0,T)$、$\epsilon \in (0,\tau)$，以及 $w \in U$，定義

$$u_t^\epsilon := \begin{cases} w, & \tau - \epsilon < t \le \tau, \\ u_t^*, & \text{otherwise}. \end{cases} \tag{3.82}$$

我們記由控制向量場 $u^\epsilon \in U_T$ 和初值 $\rho_0^\epsilon = p$ 所確定的演化概率為 $\rho_t^\epsilon \in P_T$。接下來，我們引入擾動函數，它描述了 ρ_t^ϵ 相對於 ρ_t 的一階近似。

定義 3.2.6 (Perturbation function)**.** 對任意 $\tau \in (0,T)$，我們如下定義函數 $\sigma : [0,T] \times \Omega \to \mathbb{R}$：若 $t \in [0,\tau)$，則

$$\sigma_t = 0\,; \tag{3.83}$$

若 $t \in [\tau, T]$，則 σ 定義為如下初值問題的解:

$$\begin{cases} \partial_t \sigma_t = -\nabla \cdot \big(u_t^* \sigma_t\big), & \forall\, t > \tau, \\ \sigma_\tau = -\nabla \cdot \big(\rho_\tau^* (w - u_\tau^*)\big). \end{cases} \tag{3.84}$$

注意，對於固定的 x，$\sigma_t(x)$ 在 $t = \tau$ 處未必可微，並且 σ_t 依賴於 τ 和 w，但**不**依賴於 ϵ。在下面的引理中，我們證明 ρ_t^ϵ 相對於 ρ_t 的一階近似正是 σ_t。

引理 3.2.7. 設 $\tau \in (0,T)$，$\epsilon \in (0,\tau)$，$w \in U$，並且 u_t^ϵ 由 (3.82) 定義。令 σ_t 為定義 *3.2.6* 中的擾動函數。則對每個 $t \in [0,T]$，都有

$$\lim_{\epsilon \to 0} \frac{1}{\epsilon} \|\rho_t^\epsilon - \rho_t^* - \epsilon \sigma_t\|_2 = 0. \tag{3.85}$$

证明. 我們分三種情形來證明 (3.85)：$t \in [0,\tau)$、$t = \tau$ 和 $t \in (\tau, T]$。

對於任意 $t \in [0,\tau)$，取 $\epsilon \in (0,\tau-t)$，則有 $t<\tau-\epsilon$。由 (3.83)，我們知道 $\sigma_t=0$。由 (3.82)，對所有 $s<\tau-\epsilon$，有 $\rho_s^\epsilon=\rho_s^*$。因此 $\rho_t^\epsilon=\rho_t^*$。

對於 $t=\tau$,我們只需證明:由 (3.84) 中初值條件定義的 σ_τ 等於 $\lim_{\epsilon\to 0}\frac{\rho_\tau^\epsilon-\rho_\tau^*}{\epsilon}$，並且這一結論在 Ω 上幾乎處處成立。為此，任取 $\psi\in H_0^1(\Omega)$，則有

$$
\begin{aligned}
\int_\Omega \Big(\lim_{\epsilon\to 0}\frac{\rho_\tau^\epsilon-\rho_\tau^*}{\epsilon}\Big)\psi\,dx &= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_\Omega(\rho_\tau^\epsilon-\rho_\tau^*)\psi\,dx\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_\Omega\Big(\int_{\tau-\epsilon}^{\tau}(\partial_s\rho_s^\epsilon-\partial_s\rho_s^*)\,ds\Big)\psi\,dx\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_{\tau-\epsilon}^{\tau}\int_\Omega(w\rho_s^\epsilon-u_s^*\rho_s^*)\cdot\nabla\psi\,dxds \qquad (3.86)\\
&= \int_\Omega\Big(\lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_{\tau-\epsilon}^{\tau}(w-u_s^*)\rho_s^*ds\Big)\cdot\nabla\psi\,dx\\
&= \int_\Omega(w-u_\tau^*)\rho_\tau^*\cdot\nabla\psi\,dx\\
&= -\int_\Omega\nabla\cdot\big((w-u_\tau^*)\rho_\tau^*\big)\psi\,dx\\
&= \int_\Omega\sigma_\tau\psi\,dx
\end{aligned}
$$

其中，(3.86) 的第一個等號使用了 Lebesgue 控制收斂定理；第二個等號來自 $\rho_{\tau-\epsilon}^\epsilon=\rho_{\tau-\epsilon}^*$ 以及對時間積分；第三個等號來自 ρ^ϵ 和 ρ^* 的連續方程、對 Ω 的分部積分，以及 $w,u_\tau^*\in U$（從而 $(w\cdot n)|_{\partial\Omega}=0$ 且 $(u_\tau^*\cdot n)|_{\partial\Omega}=0$）；第四個等號由 Lebesgue 控制收斂定理得到；第五個等號由假設 3.2.1 的 (iii) 得到；第六個等號來自分部積分以及 $w,u_\tau^*\in U$；最後一個等號來自 (3.84) 中的初值條件。

對於 $t>\tau$，定義 $\delta(t):=\frac{1}{2}\|\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t\|_2^2$，則有

$$
\begin{aligned}
\dot\delta(t) &= \int_\Omega(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\;\partial_t(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\,dx\\
&= -\int_\Omega(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\;\nabla\cdot\big(u_t^*(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\big)\,dx\\
&= \int_\Omega u_t^*\cdot\nabla\Big(\frac{1}{2}|\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t|^2\Big)\,dx \qquad (3.87)\\
&\le dM_U\delta(t),
\end{aligned}
$$

其中第二個等號來自 ρ_t^ϵ、ρ_t^* 和 σ_t 在 $(\tau,T]$ 上滿足的連續方程；第三個等號來自分部積分以及 $(u_t^*\cdot n)|_{\partial\Omega}=0$；最後一個不等式來自 $\mathrm{Lip}(u_t)\le M_U$，從而

$\|\nabla \cdot u_t^*\|_\infty \le dM_U$。由 Grönwall 不等式，我們有對所有 $t \in (\tau, T]$，

$$\delta(t) \le \delta(\tau) e^{dM_U(T-\tau)} \le \delta(\tau) e^{dM_U T}.$$

由於我們已經證明了 (3.85) 在 $t = \tau$ 時成立，即 $\delta(\tau) = o(\epsilon^2)$，從而得到 $\delta(t) = o(\epsilon^2)$，這就推出了對所有 $t \in (\tau, T]$，(3.85) 成立。 □

在下一個引理中，我們推導 I 的一階變分，這對於利用 u^* 的最優性來證明極大值原理是至關重要的。

引理 3.2.8. 令 u^ϵ 如 (3.82) 所定義，並令 I 為 (3.78) 中對應於控制向量場 u^ϵ 的泛函，則

$$\frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} = R(\rho_s^*, w) - R(\rho_s^*, u_s^*) + \Big\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \Big\rangle + \int_0^T \Big\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \Big\rangle \, dt$$

其中 σ_t 由定義 *3.2.6* 給出。

证明. 首先，我們有

$$I[u^\epsilon] = \int_0^{\tau-\epsilon} R(\rho_t^\epsilon, u_t^*)\, dt + \int_{\tau-\epsilon}^{\tau} R(\rho_t^\epsilon, w)\, dt + \int_\tau^T R(\rho_t^\epsilon, u_t^*)\, dt + G(\rho_T^\epsilon).$$

對 ϵ 求導，利用引理 3.2.7，並令 $\epsilon = 0$，可得

$$\begin{aligned}
\frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} &= \int_0^\tau \Big\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \Big\rangle \, dt - R(\rho_\tau^*, u_\tau^*) + R(\rho_\tau^*, w) \\
&\quad + \int_\tau^T \Big\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \Big\rangle \, dt + \Big\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \Big\rangle \\
&= R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \int_0^T \Big\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \Big\rangle \, dt + \Big\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \Big\rangle,
\end{aligned}$$

從而完成證明。 □

現在我們可以建立極大值原理，它給出了任意最優控制向量場 u^* 的每個時刻滿足的必要條件。

定理 3.2.9 (極大值原理)**.** 設 u^* 是 (3.78) 的一個最優解，其對應密度為 ρ^*，並設 ϕ^* 是 (ρ^*, u^*) 的伴隨函數。則 (ρ^*, ϕ^*, u^*) 滿足控制 *Hamilton* 動力系統 (3.81)，其中 $\rho_0^* = p$ 且 $\phi_T^* = \frac{\delta}{\delta \rho_T} G(\rho_T)$，並且對所有 $\tau \in [0, T]$，下式成立：

$$H(\rho_\tau^*, \phi_\tau^*, u_\tau^*) \;=\; \max_{w \in U} H(\rho_\tau^*, \phi_\tau^*, w). \tag{3.88}$$

证明. 取任意 $\tau \in [0,T]$ 與 $w \in U$，並用該 τ 和 w 按照 (3.82) 定義 u^ϵ。注意到 u^* 是 (3.78) 的最優解，並應用引理 3.2.8，可得

$$0 \geq \frac{d}{d\epsilon} I[u^\epsilon]\big|_{\epsilon=0} = R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \left\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \right\rangle + \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt. \tag{3.89}$$

另一方面，我們有

$$\langle \phi_T^*, \sigma_T \rangle = \langle \phi_\tau^*, \sigma_\tau \rangle + \int_\tau^T \Big(\langle \partial_t \phi_t^*, \sigma_t \rangle + \langle \phi_t^*, \partial_t \sigma_t \rangle \Big) dt. \tag{3.90}$$

將 (3.79) 中關於 (ρ^*, u^*) 的伴隨 PDE 以及 σ 在 (3.84) 中滿足的條件代入 (3.90)，並進行分部積分，得到

$$\begin{aligned} \langle \phi_T^*, \sigma_T \rangle &= \langle \phi_\tau^*, \sigma_\tau \rangle - \int_\tau^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt \\ &= \langle \phi_\tau^*, \sigma_\tau \rangle - \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt, \end{aligned} \tag{3.91}$$

其中第二個等號是因為對所有 $t \in [0, \tau)$，有 $\sigma_t = 0$。

結合 (3.89) 和 (3.91)，以及終端條件 $\phi_T^* = \frac{\delta}{\delta \rho_T^*} G(\rho_T^*)$，我們得到

$$0 \geq R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \langle \phi_\tau^*, \sigma_\tau \rangle. \tag{3.92}$$

另一方面，有

$$\begin{aligned} \langle \phi_\tau^*, \sigma_\tau \rangle &= \big\langle \phi_\tau^*, -\nabla \cdot \big(w - u_\tau^*) \rho_\tau^* \big) \big\rangle \\ &= \langle u_\tau^* \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle - \langle w \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle, \end{aligned} \tag{3.93}$$

其中第一個等號由 (3.84) 中初值條件給出的 σ_τ 得到，最後一個等號來自分部積分。結合 (3.92) 和 (3.93)，我們有

$$\begin{aligned} 0 &\geq \big(R(\rho_\tau^*, w) + \langle u_\tau^* \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle \big) - \big(R(\rho_\tau^*, u_\tau^*) + \langle w \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle \big) \\ &= H(\rho_\tau^*, \phi_\tau^*, w) - H(\rho_\tau^*, \phi_\tau^*, u_\tau^*). \end{aligned}$$

由於這對每個 $\tau \in [0,T]$ 以及任意 $w, u_\tau^* \in U$ 都成立，因此斷言 (3.88) 得證。 □

例 3.2.10. 設 $T>0$ 任意固定，且 $\Omega=\mathbb{R}^d$。考慮控制問題 (3.78)，其中運行獎勵為

$$R(\rho_t,u_t)=-\frac{1}{2}\int_\Omega |u_t(x)|^2\rho_t(x)\,dx \tag{3.94}$$

且終端獎勵為

$$G(\rho_T)=-\frac{1}{2}\int_\Omega |x|^2\rho_T(x)\,dx. \tag{3.95}$$

我們可以取任意正的 $p\in P$ 作為 (3.78) 中的初始 ρ_0，也就是說，對所有 $x\in\Omega$，都有 $p(x)>0$。

我們也可以把終端獎勵 (3.95) 取為 $G(\rho_T)=-\frac{1}{2}\int_\Omega |x-x^*|^2\rho_T(x)\,dx$，其中 x^* 任意；下面的計算是類似的。這裡為簡便起見，我們取 $x^*=0$。

由定義 3.2.4，Hamiltonian 泛函為

$$H(\rho_t,\phi_t,u_t)=\langle\rho_t,u_t\cdot\nabla\phi_t\rangle-\frac{1}{2}\int_\Omega |u_t|^2\rho_t\,dx.$$

由 (3.81)，我們得到 (ρ,ϕ) 的控制 Hamilton 動力系統為

$$\begin{cases}\partial_t\rho_t=-\nabla\cdot(\rho_t u_t),\\ \partial_t\phi_t=-u_t\cdot\nabla\phi_t+\frac{1}{2}|u_t|^2,\end{cases} \tag{3.96}$$

其中 ρ 的初值為

$$\rho_0=p \tag{3.97}$$

而 ϕ 的終值為

$$\phi_T=\frac{\delta}{\delta\rho_T}G(\rho_T)=-\frac{1}{2}|x|^2. \tag{3.98}$$

由極大值原理 (3.88)，有

$$\begin{aligned}H(\rho_t,\phi_t,u_t)&=\max_{w\in U}H(\rho_t,\phi_t,w)\\ &=\max_{w\in U}\Big\{\langle\rho_t,w\cdot\nabla\phi_t\rangle-\frac{1}{2}\int_\Omega |w|^2\rho_t\,dx\Big\}.\end{aligned} \tag{3.99}$$

對上面的極大化表達式做配方可得：極大化子 u_t 滿足 $\rho_t(u_t-\nabla\phi_t)=0$。若進一步假設 $\rho_t>0$，則有

$$u_t=\nabla\phi_t. \tag{3.100}$$

將 (3.100) 代入控制 Hamilton 動力系統 (3.96)，並結合 (3.97) 與 (3.98)，可

知最優 ρ 滿足初值問題

$$\begin{cases} \partial_t \rho_t + \nabla \cdot (\rho_t \nabla \phi_t) = 0, \quad \forall\, t \in [0, T], \\ \rho_0 = p, \end{cases} \tag{3.101}$$

而其對應的伴隨函數 ϕ 滿足終值問題

$$\begin{cases} \partial_t \phi_t + \frac{1}{2}|\nabla \phi_t|^2 = 0, \quad \forall\, t \in [0, T], \\ \phi_T = -\frac{1}{2}|x|^2. \end{cases} \tag{3.102}$$

我們可以驗證，(3.102) 的解為

$$\phi_t(x) = \frac{|x|^2}{2(t - T - 1)}. \tag{3.103}$$

這說明最優控制向量場為

$$u_t(x) = \frac{x}{t - T - 1} \tag{3.104}$$

對所有 x 和 t 都成立。下一小節中，我們將繼續這個例子，考察其值泛函與 HJB 方程。

3.2.2 概率控制的 HJB 方程

在本小節中，我們推導與最優概率控制問題 (3.78) 相關的 Hamilton–Jacobi–Bellman（HJB）方程。為此，我們需要定義價值泛函，它對應的是經典最優控制中價值函數。注意，這是一個定義在乘積空間 $P \times [0, T]$ 上的隨時間演化的泛函。該價值泛函定義如下。

定義 3.2.11 (價值泛函)**.** 對任意 $p \in P$ 和 $t \in [0, T]$，定義隨時間演化的泛函 V 為

$$V(p, t) = \sup_{u \in U_T} \left\{ \int_t^T R(\rho_s, u_s)\, ds + G(\rho_T) \right\} \tag{3.105}$$

其中 ρ_s 滿足初值問題：

$$\begin{cases} \partial_s \rho_s + \nabla \cdot (u_s \rho_s) = 0, \quad \forall\, s \in [t, T], \\ \rho_t = p. \end{cases} \tag{3.106}$$

我們稱 H 為 (3.78) 的**價值泛函**（value functional）。

注意，對於任意 t，$V(\cdot,t):P\to\mathbb{R}$ 是定義在 Ω 上概率密度函數的無限維空間上的泛函。相比之下，經典最優控制中的價值函數是定義在某個有限維歐幾里得空間子集上的函數。

現在我們可以建立價值泛函 V 的 HJB 方程如下。

定理 3.2.12 (概率控制的 HJB 方程)**.** *若對所有 t，$V(\cdot,t)$ 在 P 上是連續 Fréchet 可微的，且對所有 $p\in P$，$V(p,\cdot)$ 在 $[0,T]$ 上屬於 C^1，則 $V:[0,T]\times P\to\mathbb{R}$ 滿足*

$$\partial_t V(p,t)+\max_{w\in U}\Big\{\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t),p\Big\rangle+R(p,w)\Big\}=0, \tag{3.107}$$

並滿足 Ω 上的終值條件 $V(\cdot,T)=G(\cdot)$。

证明. 由 (3.105) 和 (3.106) 可知，對任意 $p\in P$，有 $V(p,T)=G(p)$。現在取 $p\in P$，$t\in[0,T)$。對於任意 $\epsilon>0$，考慮區間 $[t,t+\epsilon]\subset[0,T)$ 上的常值控制 $w\in U$，其對應的演化 ρ_s 滿足

$$\begin{cases}\partial_s\rho_s(x)+\nabla\cdot(w\rho_s)=0, & \forall\, s\in(t,t+\epsilon),\\ \rho_t=p.\end{cases}$$

由價值泛函的定義 (3.105)，有

$$V(p,t)\geq\int_t^{t+\epsilon}R(\rho_s,w)\,ds+V(\rho_{t+\epsilon},t+\epsilon).$$

將上式整理併除以 ϵ，得到

$$0\geq\frac{1}{\epsilon}\int_t^{t+\epsilon}R(\rho_s,w)\,ds+\frac{V(\rho_{t+\epsilon},t+\epsilon)-V(p,t)}{\epsilon}.$$

令 $\epsilon\to0$ 且注意 $\rho_t=p$，得到

$$0\geq R(p,w)+\partial_t V(p,t)+\Big\langle\frac{\delta}{\delta p}V(p,t),\partial_t\rho_t\Big\rangle.$$

注意到 $\partial_t\rho_t=-\nabla\cdot(w\rho_t)=-\nabla\cdot(wp)$，並利用分部積分，得到

$$0\geq R(p,w)+\partial_t V(p,t)+\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t),p\Big\rangle.$$

由於 w 是任意選取的，可得

$$0 \geq \partial_t V(p,t) + \max_{w\in U}\Big\{\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t), p\Big\rangle + R(p,w)\Big\}. \tag{3.108}$$

下面說明 (3.108) 中的不等式實際上為等式。為此，取任意 $(t,p)\in[0,T)\times P$，設 $u^*:[t,T]\times\Omega\to\mathbb{R}$ 與 $\rho^*:[t,T]\times\Omega\to\mathbb{R}$ 為在 (3.105) 中實現 $V(p,t)$ 的最優解。則對任意 $\epsilon>0$，由最優性及 $\rho_t^*=p$，有

$$V(\rho_t^*,t) = \int_t^{t+\epsilon} R(\rho_s^*,u_s^*)\,ds + V(\rho_{t+\epsilon}^*, t+\epsilon).$$

與之前相同，除以 ϵ 並令 $\epsilon\to 0$，再利用分部積分，得到

$$0 = \partial_t V(\rho_t^*,t) + \Big\langle u_t^*\cdot\nabla\frac{\delta}{\delta p}V(\rho_t^*,t), \rho_t^*\Big\rangle + R(\rho_t^*,u_t^*). \tag{3.109}$$

結合 (3.108) 與 (3.109)，並利用 $\rho_t^*=p$ 以及 $u_t^*\in U$，可知最大值可以達到，從而 (3.108) 中的不等式為等式，證明完成。 □

例 3.2.13. 我們繼續考慮例 3.2.10，其運行獎勵為 (3.94)，終端獎勵為 (3.95)，且初值為任意 $\rho_0=p>0$。

由 (3.107) 可知，HJB 方程為

$$\partial_t V(p,t) + \max_{w\in U}\Big\{\Big\langle p, w\cdot\nabla\frac{\delta}{\delta p}V(p,t)\Big\rangle - \frac{1}{2}\int_\Omega |w|^2 p\,dx\Big\} = 0. \tag{3.110}$$

受 (3.99) 中配方技巧和最優解 (3.104) 的啟發，我們嘗試令 $w=\frac{x}{t-T-1}$ 作為 (3.110) 中的極大值點。通過做配方可得 $\nabla\frac{\delta}{\delta p}V(p,t)=w$，從而推出

$$V(p,t) = \frac{1}{2(t-T-1)}\int_\Omega |x|^2 p(x)\,dx. \tag{3.111}$$

可以驗證，(3.111) 中的 $V(p,t)$ 確實滿足 HJB 方程 (3.110)，且在所有時間點 t，都在 $w=\frac{x}{t-T-1}$ 取得最大值。此外，其終值為

$$V(p,T) = -\frac{1}{2}\int_\Omega |x|^2 p(x)\,dx = G(p). \tag{3.112}$$

注意到 (3.111) 還表明最優控制為

$$u_t(x) = \nabla\frac{\delta}{\delta\rho_t}V(\rho_t,t) = \frac{x}{t-T-1}, \tag{3.113}$$

這與 (3.104) 一致。

3.2.3 初值擾動的變分

在本小節中，我們考慮 $G(\rho_T)$ 關於初值 p 的 Fréchet 導數，其中 $\rho : \Omega \times [0,T] \to \mathbb{R}$ 滿足連續方程 (3.74)，其初值為 $\rho_0 = p$。該結果揭示了伴隨函數 ϕ 的一個重要性質。

假設智能體的初始狀態 $\{x_i \in \Omega : i = 1, \ldots, N\}$ 發生擾動，使得初始概率 p 變為

$$p^\epsilon := p + \epsilon h + o(\epsilon) \in P, \tag{3.114}$$

其中某些函數 $h, o(\epsilon) : \Omega \to \mathbb{R}$ 滿足

$$\int_\Omega h(x)\,dx = 0, \quad \int_\Omega o(\epsilon)\,dx = 0. \tag{3.115}$$

注意，(3.115) 是 $p^\epsilon \in P$ 的必要條件。令 ρ^ϵ 滿足如下初值問題：

$$\begin{cases} \partial_t \rho_t^\epsilon + \nabla \cdot (\rho_t^\epsilon u) = 0, \quad \forall\, t \in [0,T], \\ \rho_0^\epsilon = p^\epsilon. \end{cases} \tag{3.116}$$

我們關注 (3.116) 中的 ρ_t^ϵ 與 (3.78b)–(3.78c) 中的 ρ_t 之間的差異，它們使用相同的控制向量場 $u \in U_T$，但初值分別為 p^ϵ 與 p。

引理 3.2.14. *設 ρ_t^ϵ 和 ρ_t 分別以 p^ϵ 和 p 為初值滿足 (3.116)。則*

$$\lim_{\epsilon \to 0} \sup_{0 \le t \le T} \frac{1}{\epsilon} \|\rho_t^\epsilon - \rho_t - \epsilon s_t\|_2 = 0, \tag{3.117}$$

其中 s_t 滿足初值問題：

$$\begin{cases} \partial_t s_t + \nabla \cdot (s_t u_t) = 0, \quad \forall\, t \in [0,T], \\ s_0 = h. \end{cases} \tag{3.118}$$

证明. 定義 $\delta(t) := \frac{1}{2}\|\rho_t^\epsilon - \rho_t - \epsilon s_t\|_2^2$。仿照 (3.87) 並將其中的 σ_t 替換為 s_t，可得對所有 $t \in [0,T]$ 都有 $\delta(t) \le \delta(0) e^{dM_U T}$。由於 $\delta(0) = \frac{1}{2}\|p^\epsilon - p\|_2^2 = o(\epsilon^2)$，因此有

$$\lim_{\epsilon \to 0} \sup_{0 \le t \le T} \frac{(2\delta(t))^{1/2}}{\epsilon} \le \lim_{\epsilon \to 0} \frac{(2\delta(0))^{1/2}}{\epsilon} e^{dM_U T/2} = 0,$$

這就證明了 (3.117)。 □

命題 3.2.15. 對任意 $T>0$、$u\in U_T$、初值 $p\in P$，設 ρ_t 是如下初值問題的解

$$\begin{cases}\partial_t\rho_t+\nabla\cdot(\rho_t u_t)=0, \quad \forall t\in[0,T],\\ \rho_0=p.\end{cases} \tag{3.119}$$

則對於任意在 P 上 *Fréchet* 可微的泛函 $G:P\to\mathbb{R}$，有

$$\frac{\delta}{\delta p}G(\rho_T)=\phi_0, \tag{3.120}$$

其中 $p=\rho_0$ 是 ρ 的初值，而 ϕ 滿足終值問題:

$$\begin{cases}\partial_t\phi_t+u_t\cdot\nabla\phi_t=0, \quad \forall t\in[0,T],\\ \phi_T=\frac{\delta}{\delta\rho_T}G(\rho_T).\end{cases} \tag{3.121}$$

证明. 對任意滿足 (3.115) 的 h，令 p^ϵ 為 (3.114) 中定義的初值，並令 ρ^ϵ 滿足初值問題 (3.116)。由引理 3.2.14 可知

$$G(\rho_T^\epsilon)=G\big(\rho_T+\epsilon s_T+o(\epsilon)\big), \tag{3.122}$$

其中 s_t 是 (3.118) 的解。因此

$$\frac{d}{d\epsilon}G(\rho_T^\epsilon)\Big|_{\epsilon=0}=\Big\langle\frac{\delta}{\delta\rho_T}G(\rho_T),s_T\Big\rangle=\langle\phi_T,s_T\rangle, \tag{3.123}$$

其中第二個等號是由 (3.121) 中 ϕ_t 的終值條件得到的。

另一方面，注意到

$$\begin{aligned}\frac{d}{dt}\langle\phi_t,s_t\rangle&=\langle\partial_t\phi_t,s_t\rangle+\langle\phi_t,\partial_t s_t\rangle\\&=-\langle u_t\cdot\nabla\phi_t,s_t\rangle-\langle\phi_t,\nabla\cdot(s_t u_t)\rangle\\&=0,\end{aligned}$$

其中第二個等號使用了 (3.121) 中 ϕ 的 PDE 以及 (3.118) 中 s 的 PDE；第三個等號則來源於分部積分以及邊界上 $(u_t\cdot n)|_{\partial\Omega}=0$。因而，$\langle\phi_t,s_t\rangle$ 關於 t 為常數，從而

$$\langle\phi_T,s_T\rangle=\langle\phi_0,s_0\rangle=\langle\phi_0,h\rangle. \tag{3.124}$$

結合 (3.123) 與 (3.124)，並注意到 $\frac{\delta}{\delta p}G(\rho_T) = \frac{d}{d\epsilon}G(\rho_T^\epsilon)|_{\epsilon=0}$，可得

$$\left\langle \frac{\delta}{\delta p}G(\rho_T), h \right\rangle = \langle \phi_0, h \rangle. \tag{3.125}$$

由於 h 是任意的，所以 (3.120) 成立。 □

例 3.2.16. 令 $T > 0$ 和 $p \in P$ 任意給定，並取 $\Omega = \mathbb{R}^d$。定義 $u \in U_T$ 為

$$u_t(x) = \frac{x}{2(t - T - 1)} \tag{3.126}$$

對所有 $x \in \Omega$ 與 $t \in [0, T]$ 成立。令 ρ 為帶有向量場 (3.126) 中 u 且初值為 p 的 (3.119) 的解。定義泛函 $G : P \to \mathbb{R}$ 為

$$G(q) = -\frac{1}{2}\int_\Omega |x|^2 q(x)\, dx, \quad \forall\, q \in P. \tag{3.127}$$

注意，對所有 q 都有 $\frac{\delta}{\delta q}G(q) = -\frac{1}{2}|x|^2$。

我們可以驗證

$$\phi_t(x) = \frac{|x|^2}{2(t - T - 1)} \tag{3.128}$$

是終值問題 (3.121) 的解，其中 u 由 (3.126) 給出，終值為 $\phi_T = \frac{\delta}{\delta \rho_T}G(\rho_T) = -\frac{1}{2}|x|^2$。

現在，由命題 3.2.15，我們斷言

$$\frac{\delta}{\delta p}G(\rho_T) = \phi_0 = -\frac{|x|^2}{2(T+1)}. \tag{3.129}$$

為了說明斷言 (3.129)，先注意到

$$\rho_t(x) = \left(\frac{T+1}{T+1-t}\right)^{d/2} p\left(\left(\frac{T+1}{T+1-t}\right)^{1/2} x\right)$$

是帶有 (3.126) 中向量場 u 且初值為給定 p 的初值問題 (3.119) 的解。因此

$$\rho_T(x) = (T+1)^{d/2} p\big(\sqrt{T+1}x\big).$$

於是我們得到

$$
\begin{aligned}
G(\rho_T) &= -\frac{1}{2}\int_\Omega |x|^2 \rho_T(x)\,dx \\
&= -\frac{1}{2}\int_\Omega |x|^2 (T+1)^{d/2} p\big(\sqrt{T+1}x\big)\,dx \\
&= -\frac{1}{2(T+1)}\int_\Omega |y|^2 p(y)\,dy,
\end{aligned}
$$

其中最後一個等號來自變量替換 $y=\sqrt{T+1}x$。因此 $\frac{\delta}{\delta p}G(\rho_T) = -\frac{|x|^2}{2(T+1)}$，從而證明了 (3.129)。

3.3 Neural ODE 方法

儘管我們在上文已經看到了許多關於最優控制的重要理論結果，但在實際中如何用數值方法求解一般控制問題仍然是一個問題。在本節中，我們介紹一種稱為 **Neural Ordinary Differential Equation** 方法的途徑來解決這一問題。下文中我們簡稱其為 Neural ODE 方法。

我們關注第 3.1 節中給出的如下具有固定終止時間 T 和自由終點的一般最優控制問題：

$$
\max_{u\in\mathcal{U}_T} \quad I[u] := g(x(T)), \tag{3.130a}
$$

$$
\text{s.t.} \quad \begin{cases} \dot{x}(t) = f(t, x(t), u(t)), \quad \forall\, t\in[0,T], \\ x(0) = x_0 \end{cases} \tag{3.130b}
$$

其中，終端獎勵函數 $g:\mathbb{R}^d\to\mathbb{R}$、將控制作為自變量之一的定義函數 $f:[0,T]\times\mathbb{R}^d\times\mathcal{U}\to\mathbb{R}^d$，以及某個初值 $x_0\in\mathbb{R}^d$ 都是給定的。

我們之所以專注 (3.130)，是因為它涵蓋了現實應用中大多數最優控制問題。這也部分出於一個實際問題：我們將使用深度神經網絡來逼近控制 $u(t)$，而這種逼近有時難以精確滿足某些約束（例如在固定終點最優控制問題中精確到達某個指定狀態）。

我們還說明一下：為了便於後續表述，在 (3.130) 中時間變量 t 又重新作為顯式輸入出現在 f 中。與此同時，(3.130) 中沒有運行獎勵 r，因為我們可以像 (3.36) 那樣簡單地添加一個輔助變量來將運行獎勵納入考慮。

Neural ODE 方法的思想是將控制函數 u 參數化為一個深度神經網絡 $u_\theta:[0,T]\to\mathcal{U}\subset\mathbb{R}^m$，其中 $\theta\in\mathbb{R}^n$ 表示由其 n 個網絡參數組成的向量。

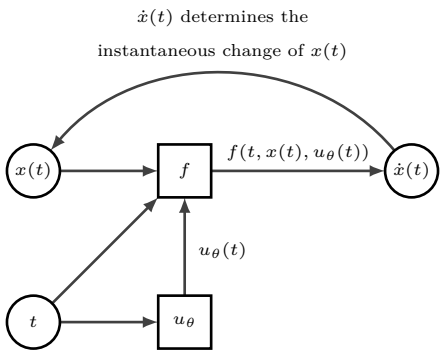


图 3.1: 由 Neural ODE 方法求解的控制系統。控制 ODE 為 $\dot{x}(t) = f(t, x(t), u(t))$，其中 $f : [0, T] \times \mathbb{R}^d \times \mathcal{U} \to \mathbb{R}^d$ 由控制問題決定，而控制 $u : [0, T] \to \mathcal{U}$ 被參數化為參數為 θ 的深度神經網絡 u_θ。在原始的 Neural ODE 方法 [27] 中，函數 $f : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ 被參數化為參數為 θ 的深度神經網絡 f_θ，而 ODE 為 $\dot{x}(t) = f_\theta(t, x(t))$。

現在，最優控制問題 (3.130) 就化為

$$\max_{\theta \in \mathbb{R}^n} \quad J(\theta) := I[u_\theta] = g(x(T)), \tag{3.131a}$$

$$\text{s.t.} \quad \begin{cases} \dot{x}(t) = f(t, x(t), u_\theta(t)), & \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases} \tag{3.131b}$$

注意，此時待優化的變量不再是時間相關的控制函數 $u : [0, T] \to \mathcal{U}$。相反，它變成了待優化的向量 $\theta \in \mathbb{R}^n$，就像標準優化中那樣。一旦找到最優的 θ 值，神經網絡 $u_\theta : [0, T] \to \mathcal{U}$ 就是控制問題 (3.130) 所期望的（近似）最優解 u。以控制函數 u_θ 為基礎的控制系統如圖 3.1 所示。

鑑於待優化的變量是向量 θ，自然可以採用我們在第 2.4 節中介紹的一些基於梯度的優化算法。使用此類算法時的關鍵要素，是目標函數 J 關於 θ 的梯度。因此，我們的主要目標是求出 (3.131) 中定義的函數 J 的 $\nabla_\theta J(\theta)$。如果我們能夠在任意 θ 處計算 $\nabla_\theta J(\theta)$，那麼就可以將其提供給任意基於梯度的優化算法，以尋找 $J(\theta)$ 的（局部）極大值點，它們對應於 u_θ 的（次）最優網絡參數 θ。Neural ODE 方法正提供了一種計算 $\nabla_\theta J(\theta)$ 的途徑。

為了說明 Neural ODE 方法如何工作，我們先引入兩個簡單的輔助函數。第一個 $\tau : [0, T] \to [0, T]$ 是恆等函數：

$$\tau(t) = t$$

對所有 $t \in [0,T]$ 都成立，因此它滿足初值問題

$$\begin{cases} \dot{\tau}(t) = 1, \quad \forall\, t \in [0,T], \\ \tau(0) = 0. \end{cases} \tag{3.132}$$

第二個 $\sigma : [0,T] \to \mathbb{R}^n$ 是關於時間的常函數：

$$\sigma(t) = \theta$$

對所有 $t \in [0,T]$ 都成立，因此它滿足初值問題

$$\begin{cases} \dot{\sigma}(t) = 0, \quad \forall\, t \in [0,T], \\ \sigma(0) = \theta. \end{cases} \tag{3.133}$$

進一步地，我們將增廣函數 $z : [0,T] \to \mathbb{R}^{d+1+n}$ 記為

$$z(t) := \begin{pmatrix} x(t) \\ \tau(t) \\ \sigma(t) \end{pmatrix} \in \mathbb{R}^{d+1+n}, \quad \text{其初值為} \quad z(0) := \begin{pmatrix} x_0 \\ 0 \\ \theta \end{pmatrix} \in \mathbb{R}^{d+1+n}, \tag{3.134}$$

將 z 的增廣 ODE 定義函數 h 記為

$$h(z(t)) := \begin{pmatrix} f(\tau(t), x(t), u_{\sigma(t)}(\tau(t)) \\ 1 \\ 0 \end{pmatrix} \in \mathbb{R}^{d+1+n}, \tag{3.135}$$

並將增廣終端獎勵函數 $\bar{g} : \mathbb{R}^{d+1+n} \to \mathbb{R}$ 記為

$$\bar{g}(z) := g(x)$$

對任意 $z = (x, \tau, \sigma) \in \mathbb{R}^{d+1+n}$ 成立。注意，我們知道 u_θ（由我們選擇的神經網絡結構給出）與 f（控制問題 (3.130) 的給定 ODE 定義函數）的表達式。此外，我們看到 ODE $\dot{z}(t) = h(z(t))$ 是自治的（h 不顯式依賴於 t）。

現在我們可以將最優控制問題 (3.131) 改寫成如下等價形式:

$$\max_{\theta\in\mathbb{R}^n} \quad J(\theta) := \bar{g}(z(T)), \tag{3.136a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(t) = h(z(t)), \quad \forall\, t \in [0,T], \\ z(0) = z_0. \end{cases} \tag{3.136b}$$

對任意 $y_0 \in \mathbb{R}^{d+1+n}$ 和 $\epsilon > 0$，考慮 $z^\epsilon : [0,T] \to \mathbb{R}^{d+1+n}$ 滿足初值問題

$$\begin{cases} \dot{z^\epsilon}(t) = h(z^\epsilon(t)), \quad \forall\, t \in [0,T], \\ z^\epsilon(0) = z_0 + \epsilon y_0. \end{cases}$$

由引理 3.1.6，當 $\epsilon \to 0$ 時，

$$z^\epsilon(t) = z(t) + \epsilon y(t) + o(\epsilon)$$

在 $[0,T]$ 上一致成立。也就是說，當 $\epsilon \to 0$ 時，

$$\sup_{0\le t\le T} \frac{|z^\epsilon(t) - z(t) - \epsilon y(t)|}{\epsilon} \to 0,$$

其中 $y : [0,T] \to \mathbb{R}^{d+1+n}$ 是如下初值問題的解

$$\begin{cases} \dot{y}(t) = \nabla_z h(z(t))y(t), \quad \forall\, t \in [0,T], \\ y(0) = y_0. \end{cases}$$

因此有

$$\frac{d}{d\epsilon}\bar{g}(z^\epsilon(T))\Big|_{\epsilon=0} = \nabla_z \bar{g}(z(T)) \cdot y(T).$$

此外，我們知道與狀態 z 對應的協態 $p : [0,T] \to \mathbb{R}^{1\times(d+1+n)}$（$p(t)$ 是行向量）滿足終值問題:

$$\begin{cases} \dot{p}(t) = -p(t)\nabla_z h(z(t)), \quad \forall\, t \in [0,T], \\ p(T) = \nabla_z \bar{g}(z(T)) \end{cases}$$

並且像之前一樣，對所有 t 有

$$\begin{aligned}\frac{d}{dt}\Big(p(t)\cdot y(t)\Big) &= \dot{p}(t)\cdot y(t) + p(t)\cdot \dot{y}(t)\\ &= -p(t)\nabla_z h(z(t))\cdot y(t) + p(t)\cdot \nabla_z h(z(t))y(t)\\ &= 0.\end{aligned}$$

這意味著 $p(t)\cdot y(t)$ 關於 t 為常數。於是有

$$\frac{d}{d\epsilon}\bar{g}(z^{\epsilon}(T))\Big|_{\epsilon=0} = \nabla_z \bar{g}(z(T))\cdot y(T) = p(T)\cdot y(T) = p(0)\cdot y_0.$$

我們注意到 $p(t)$ 有三個分量塊

$$p(t) = (p_x(t), p_\tau(t), p_\sigma(t)) \in \mathbb{R}^{1\times(d+1+n)}$$

並記

$$z := \begin{pmatrix} x \\ \tau \\ \sigma \end{pmatrix} \in \mathbb{R}^{d+1+n} \quad \text{和} \quad h(z) := \begin{pmatrix} f(\tau, x, u_\sigma(\tau)) \\ 1 \\ 0 \end{pmatrix} \in \mathbb{R}^{d+1+n}.$$

於是有

$$\partial_z h(z) = \begin{pmatrix} \partial_x f & \partial_\tau f + \partial_u f \partial_\tau u & \partial_u f \partial_\sigma u \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{(d+1+n)\times(d+1+n)}.$$

注意，由於對所有 $t \in [0,T]$ 都有 $\tau(t) = t$ 和 $\sigma(t) = \theta$，因此 $\partial_\tau f = \partial_t f$、$\partial_\tau u = \partial_t u$，以及 $\partial_\sigma u = \partial_\theta u$。

因此，協態 $p = (p_x, p_\tau, p_\sigma) : [0,T] \to \mathbb{R}^{1\times(d+1+n)}$ 滿足以如下終值問題：

$$\dot{p}(t) = -p(t)\nabla_z h(z(t)),$$

也即

$$\begin{cases} \dot{p}_x(t) = -p_x(t)\partial_x f(t, x(t), u_\theta(t)) \in \mathbb{R}^{1\times d} \\ \dot{p}_\tau(t) = -p_x(t)\Big(\partial_t f(t, x(t), u_\theta(t)) + \partial_u f(t, x(t), u_\theta(t))\partial_t u_\theta(t)\Big) \in \mathbb{R}^{1\times 1} \\ \dot{p}_\sigma(t) = -p_x(t)\Big(\partial_u f(t, x(t), u_\theta(t))\partial_\theta u_\theta(t)\Big) \in \mathbb{R}^{1\times n} \end{cases} \tag{3.137}$$

其終值為

$$p(T) = \bar{g}(z(T)) = g(x(T)),$$

也即

$$\begin{cases} p_x(T) = \nabla g(x(T)) \in \mathbb{R}^{1\times d}, \\ p_\tau(T) = 0 \in \mathbb{R}^{1\times 1}, \\ p_\sigma(T) = 0 \in \mathbb{R}^{1\times n}. \end{cases} \tag{3.138}$$

假設我們任意選取 $\eta \in \mathbb{R}^n$，並令 $y_0 = (0, 0, \eta^\top)^\top \in \mathbb{R}^{d+1+n}$，則上述推導給出

$$\nabla_\theta J(\theta) \cdot \eta = \frac{d}{d\epsilon} J(\theta + \epsilon\eta)\Big|_{\epsilon=0} = \frac{d}{d\epsilon}\bar{g}(z^\epsilon(T))\Big|_{\epsilon=0} = p(0) \cdot y_0 = p_\sigma(0) \cdot \eta.$$

由於 η 是任意的，因此我們知道

$$\nabla_\theta J(\theta) = p_\sigma(0). \tag{3.139}$$

現在我們看到，為了計算任意 θ 下的 $\nabla_\theta J(\theta)$，只需先求解初值問題 (3.131b) 得到 $x(T)$，再求解終值問題 (3.137)–(3.138) 得到 $p_\sigma(0)$。這樣便得到 (3.139)。我們在算法 3.3.1 中總結了計算 $\nabla_\theta J(\theta)$ 的過程。注意，初值問題和終值問題都可以用成熟的數值 ODE 求解器來求解，例如中點（midpoint）方法和 Runge–Kutta 方法 [24]。由於 $\nabla_\theta J(\theta)$ 可以被計算出來，我們便可以採用任意基於梯度的優化算法來尋找 $J(\theta)$ 的一個（局部）極大值點。

關於 Neural ODE 方法的若干說明

關於使用 Neural ODE 方法求解最優控制問題，我們有如下幾點說明。

首先，Neural ODE 方法將控制 $u : [0, T] \to \mathcal{U}$ 參數化為神經網絡 u_θ。由於在實際中我們幾乎總是將神經網絡設計為時間上的連續函數，因此所得控制解在時間上也是連續的。問題在於，如果最優控制本身在時間上不是連續的，

Algorithm 3.3.1 在 Neural ODE 方法中計算 (3.131) 的 $\nabla_\theta J(\theta)$

Input: 當前的 $\theta \in \mathbb{R}^n$。

1: 通過如下初值問題求解 $x(T)$：

$$\begin{cases} \dot{x}(t) = f(t, x(t), u_\theta(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases}$$

2: 通過如下終值問題求解 $p(0) = (p_x(0), p_\tau(0), p_\sigma(0))$：

$$\begin{cases} \dot{p}(t) = -p(t)\nabla_z h(z(t)), \quad \forall\, t \in [0, T], \\ p(T) = (\nabla g(x(T)), 0, 0), \end{cases}$$

其中伴隨方程的顯式形式由 (3.137) 給出。

Output: $\nabla_\theta J(\theta) = p_\sigma(0)$.

那麼我們無法精確找到它；但如果網絡結構具有足夠強的表達能力，仍有可能得到一個很好的逼近。與此同時，用深度網絡來參數化控制的一個優點是，我們可以適當設計網絡，使其既具有強表示能力，又能保證其輸出始終落在 $\mathcal{U}$ 中（例如 $\mathcal{U}$ 是 $\mathbb{R}^m$ 中的有界集合），從而在學習控制時避免顯式施加約束。關於神經網絡設計的一些例子見第 1.5.3 節。

第二，Neural ODE 方法最初由 [27] 提出，作為殘差網絡（ResNet）結構的連續時間版本，它可以看作是對由深度網絡參數化定義函數 f 的 ODE 使用 Euler 格式所得的結果。更具體地說，在 [27] 中，ODE 具有一種簡單而靈活的形式，其中 f 直接被參數化為一個參數為 $\theta \in \mathbb{R}^n$ 的深度神經網絡 $f_\theta : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$：

$$\begin{cases} \dot{x}(t) = f_\theta(t, x(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases} \tag{3.140}$$

與之對應的協態 p 的伴隨方程也可以像上面那樣推導出來，其形式比 (3.137) 更簡單：

$$\begin{cases} \dot{p}_x(t) = -p_x(t)\partial_x f_\theta(t, x(t)) \in \mathbb{R}^{1\times d}, \\ \dot{p}_\tau(t) = -p_x(t)\partial_t f_\theta(t, x(t)) \in \mathbb{R}^{1\times 1}, \\ \dot{p}_\sigma(t) = -p_x(t)\partial_\theta f_\theta(t, x(t)) \in \mathbb{R}^{1\times n}, \end{cases} \tag{3.141}$$

其終值仍為 $p(t) = (\nabla g(x(T)), 0, 0) \in \mathbb{R}^{1\times(d+1+n)}$。於是我們可以將終端獎勵 $g(x(T))$ 作為目標函數 $J(\theta)$，並且可以像算法 3.3.1 中那樣計算 $\nabla_\theta J(\theta)$，只不

過此時需將其中的初值問題和終值問題分別替換為 (3.140) 和 (3.141)。

第三，Neural ODE 方法也可以被視為一種一般性的深度學習方法。例如，假設我們有一個標準監督學習問題，其給定數據集為 $\mathcal{D} = \{(x_{i,0}, x_i^*) \in \mathbb{R}^d \times \mathbb{R}^d : i \in [M]\}$，我們的目標是尋找映射 $F : \mathbb{R}^d \to \mathbb{R}^d$，使得對所有 $i \in [M]$ 都有 $F(x_{i,0}) \approx x_i^*$。與其直接將 F 參數化為一個深度神經網絡（這可能較難訓練），我們可以利用 Neural ODE 方法，將 ODE 的定義函數參數化為神經網絡 f_θ，並把 F 設為從初始值 $x_i(0) := x_{i,0}$ 到其終值 $x_i(T)$ 的映射，即對所有 $i \in [M]\}$ 定義 $F(x_{i,0}) := x_i(T)$，再訓練 θ 使得對所有 $i \in [M]$ 都有 $x_i(T) \approx x_i^*$。此時，網絡訓練問題可以表述為

$$\max_{\theta \in \mathbb{R}^n} \quad J(\theta) := -\frac{1}{2M} \sum_{i=1}^{M} |x_i(T) - x_i^*|^2, \tag{3.142a}$$

$$\text{s.t.} \quad \begin{cases} \dot{x}_i(t) = f_\theta(t, x_i(t)), & \forall\, t \in [0, T], \\ x_i(0) = x_{i,0}, & i = 1, \ldots, M. \end{cases} \tag{3.142b}$$

只要問題能夠被表述為帶有 ODE 約束的優化問題，我們都可以使用 Neural ODE 方法來求解。

第四，我們也可以利用 Neural ODE 方法來求解許多概率控制問題。其思想是，在任意時刻 t，用足夠多的運動粒子（如智能體）$\{x_i(t) \in \mathbb{R}^d : i \in [M]\}$ 來表示定義在 $\mathbb{R}^d$ 上的概率密度函數 $\rho(\cdot, t)$。粒子數 M 可以人為設定，也可以由問題中可用的數據決定。當我們將向量場參數化為深度神經網絡 $f_\theta : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ 時，這些粒子將按照 ODE (3.142b) 運動。相應地，$\rho(\cdot, t)$ 的演化由如下連續方程給出：

$$\partial_t \rho(x, t) = -\nabla \cdot (\rho(x, t) f_\theta(t, x)). \tag{3.143}$$

因此，我們可以把粒子作為 ρ 的表示，並學習最優的 f_θ，使得 $\rho(\cdot, t)$ 在給定目標函數下達到最大化或最小化。關於這種方法的一個具體例子，見第 五 章中第 5.4 節。

第五，當我們將 Neural ODE 方法用於概率控制時，有時需要估計 $\rho(x, t)$ 的取值。例如，正如我們將在第 5.4 節中展示的那樣，我們需要估計負熵

$$\int \rho(x, t) \log \rho(x, t) dx \approx \frac{1}{M} \log \rho(x_i(t), t).$$

其中右端的近似來自 Monte Carlo 積分（關於這種積分近似方法的細節，見

第 A.1 節），並且 $x_i(t) \sim \rho(\cdot, t)$。因此，我們需要利用這些運動粒子來估計 $\log \rho(x,t)$。為此，我們可以採用如下 ODE：

$$\frac{d}{dt} \log \rho(x(t), t) = -\nabla \cdot f_\theta(t, x(t)), \tag{3.144}$$

其中左側的 d/dt 是關於 t 的全導數，也就是說，對 $\rho(x(t), t)$ 中的 $x(t)$ 與 t 都要對 t 求導，而 ∇ 與 $\nabla\cdot$ 分別表示關於 x 的梯度與散度。為了證明 (3.144)，我們注意到

$$\begin{aligned}
\frac{d}{dt}\rho(x(t), t) &= \partial_t \rho(x(t), t) + \nabla \rho(x(t), t) \cdot \dot{x}(t) \\
&= \partial_t \rho(x(t), t) + \nabla \rho(x(t), t) \cdot f_\theta(t, x(t)) \\
&= -\nabla \cdot (\rho(t, x) f_\theta(t, x)) + \nabla \rho(x(t), t) \cdot f_\theta(t, x(t)) \\
&= -\rho(x(t), t) \nabla \cdot f_\theta(t, x(t)),
\end{aligned}$$

其中第二個等號使用了 ODE $\dot{x}(t) = f_\theta(t, x(t))$，第三個等號使用了連續方程 (3.143)。假設 ρ 處處為正，將兩邊同時除以 $\rho(x(t), t)$，便得到 (3.144)。

注意，(3.143) 只提供了一種計算 $\log \rho(x(t), t)$ 變化量的方法。在實際中，我們未必知道初始分佈 $\rho(\cdot, 0)$（或者某個給定 $T > 0$ 時刻的終端分佈 $\rho(\cdot, T)$）。不過，這通常不會影響 θ 的學習，因為初始或終端分佈往往是預先給定的，並且不依賴於 θ。因此，在建模過程中，我們通常可以將它從目標函數中消去，而不會影響網絡訓練過程中所求得的解 θ。

3.4 備註與參考文獻

3.4.1 在 PDE 解算子學習中的應用

在本小節中，我們介紹深度最優控制在學習演化偏微分方程（evolution PDE）解算子中的一個應用。我們首先概述演化 PDE 以及解算子的定義。然後說明如何利用深度最優控制來學習解算子。在下面的討論中，我們聚焦於關鍵組成部分和核心思想，並簡要提及向一般情形推廣的方法。

演化 PDE 是指描述系統如何隨時間變化的一類 PDE，用於刻畫諸如熱傳導、波動、流體等動態過程。設 Ω 是 $\mathbb{R}^d$ 的一個開子集，F 是定義某個演化 PDE 的微分算子，則該演化 PDE 可寫為

$$\partial_t v(x, t) = F[v](x, t), \quad \forall\, (x, t) \in \Omega \times [0, T], \tag{3.145}$$

其中 $T > 0$ 是某個給定的終止時刻。

下面給出幾個出現在廣泛現實問題中的演化 PDE 示例（其中 $\gamma > 0$ 是某個給定常數）：

- 熱方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t). \tag{3.146}$$

- Hamilton–Jacobi 方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t) + \frac{\gamma}{2}|\nabla v(x,t)|^2. \tag{3.147}$$

- Allen–Cahn 方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t) - \gamma v(x,t)(v(x,t)^2 - 1). \tag{3.148}$$

在上述例子中，∇ 和 Δ 分別照常表示 u 關於 x 的梯度和 Laplace 算子，而 $v : \bar{\Omega} \times [0,T] \to \mathbb{R}$ 若滿足相應 PDE，並滿足某些初始條件和邊界條件（我們稍後會詳細說明），則稱其為該方程的一個解。微分算子 F 是指將 v 映射為由 v 及其到某一階的導數所確定的函數。上述例子中的微分算子 F 由於包含 Laplace 算子，因此都是二階的。

我們說明一下，微分算子 F 可以是非線性的，例如 Hamilton–Jacobi 方程 (3.147) 和 Allen–Cahn 方程 (3.148) 中的情形。非線性往往會給求解這些 PDE 帶來巨大挑戰，因為這類 PDE 的解可能會隨著時間變化而變得剛性很強且不穩定。

數值求解 PDE 的另一個主要挑戰是維數。如果維數 d 很大（例如 $d \geq 5$），那麼經典的數值 PDE 求解器，如有限差分法、有限元法和譜方法，這些在數值 PDE 文獻中已經被深入研究的方法 [9]，往往會由於所謂的維數災難而在計算上變得不可行。這是因為這些方法通常需要對區域 $\Omega \subset \mathbb{R}^d$（或某個變換域）做空間離散化，而離散化後未知變量的數量會隨著 d 指數級增長，從而使得大維情形下的離散問題變得無法計算。

我們還注意到，在上面的例子 (3.146)、(3.147) 和 (3.148) 中，這些微分算子是**自治的**，因為它們不顯式依賴於 t。相反，它們通過 $v(x,t)$ 隱式依賴於 t。不過，我們可以很容易地將稍後要介紹的方法推廣到處理非自治的 F 所對應的 PDE，只需使用增廣變量 $\bar{x} := (t,x)$ 並將 PDE 轉化為自治形式即可。這種做法我們在本章見過，所以我們在此省略細節。

此外，還值得指出的是，我們將要介紹的方法也可以處理更高階的演化

PDE。例如，對於具有二階時間導數的演化 PDE

$$\partial_{tt}^2 v(x,t) = F[v](x,t), \tag{3.149}$$

我們可以將 (3.149) 轉化為一個僅含一階時間導數的等價 PDE:

$$\begin{cases} \partial_t v(x,t) = w(x,t), \\ \partial_t w(x,t) = F[v](x,t), \end{cases}$$

於是需要尋找滿足上述兩個 PDE 組成的系統的解 $(w,v):\bar{\Omega}\times[0,T]\to\mathbb{R}^2$。

為簡化起見，下面我們假設 F 是自治的且 $\Omega=\mathbb{R}^d$。在這種情形下，我們可以不考慮邊界條件。回憶一下，演化 PDE 往往需要某種初始條件來保證解的唯一性。例如，在 (3.146) 和 (3.147) 中，只要 v 是該演化 PDE 的一個解，那麼對於任意常數 $C\in\mathbb{R}$，$v+C$ 也滿足同一個 PDE。

初始條件由某個給定的初值函數 $g:\bar{\Omega}\to\mathbb{R}$ 決定。我們用 $v^g:\bar{\Omega}\times[0,T]\to\mathbb{R}$ 表示如下初值問題的解

$$\begin{cases} \partial_t v(x,t) = F[v](x,t), \quad \forall\,(x,t)\in\Omega\times[0,T], \\ v(x,0) = g(x), \quad x\in\bar{\Omega}. \end{cases} \tag{3.150}$$

然後，我們稱由

$$S_F(g) := v^g. \tag{3.151}$$

定義的映射 $S_F : C^1(\bar{\Omega};\mathbb{R}) \to C^{2,1}(\bar{\Omega}\times[0,T];\mathbb{R})$ 為與 F 相關聯的**解算子**（solution operator），它對應於問題 (3.150)，其中值域 $C^{2,1}(\bar{\Omega}\times[0,T];\mathbb{R})$ 是指定義在 $\bar{\Omega}\times[0,T]$ 上、在內部點關於 x 二次連續可導且關於 t 一次連續可導的實值函數集合。換言之，與 F 相關聯的解算子 S_F，就是將任意初值 g 映射到其在 (3.150) 中對應解 v^g 的映射。我們的目標是發展一種方法，用於求得任意給定 F 的 S_F（或其近似）。

我們強調一下，學習解算子 S_F 的問題與給定 g 下求解 (3.150) 的問題在本質上是不同的。學好解算子 S_F 後，我們可以用很小的計算量得到任意 g 對應的 PDE 解 v^g。相比之下，求解 (3.150) 意味著在給定初值 g 的情況下采用某種方法（例如例 1.5.1 中的 PINN 方法），而當換成另一個初值 g 時就必須從頭再來。如果我們需要許多不同初值所對應的解，這樣的計算代價是很高的。當維數 d 很高時，擁有解算子 S_F 的好處就更加明顯，因為這時即使只對單個 g 求解 (3.150) 也已經代價高昂。

現在我們說明，深度最優控制如何為任意給定 F 的解算子 S_F 提供一種逼近方法。為此，注意到如果我們將函數 $v:\mathbb{R}^d\to\mathbb{R}$ 參數化為一個參數為 $\theta\in\mathbb{R}^n$ 的深度神經網絡 u_θ，那麼我們實際上就將一個 n 維向量 θ 作為函數 v 的代表，而 v 本身是某個無限維函數空間中的點。如果 $v:\mathbb{R}^d\times[0,T]\to\mathbb{R}^d$ 隨時間演化（如演化 PDE 的那些隨時間 t 變化的解），那麼我們就可以用參數軌跡 $\theta:[0,T]\to\mathbb{R}^n$ 來近似表示 v。也就是說，通過適當地引導（控制）$\theta(t)$，我們用 $u_{\theta(t)}(\cdot)$ 作為任意 $t\in[0,T]$ 時刻上 $v(\cdot,t)$ 的一個近似。

在這裡，我們想暫停一下並提出如下問題：一個參數隨時間演化的深度神經網絡 $u_{\theta(t)}:\mathbb{R}^d\to\mathbb{R}$，是否與一個參數固定的深度神經網絡 $\hat{u}_\eta:\mathbb{R}^d\times[0,T]\to\mathbb{R}$ 具有同樣的表達能力？這裡說 $u_{\theta(t)}$ 與 $\hat{u}_\eta$ 具有同樣的表達能力，是指對任意 t 都有 $u_{\theta(t)}(\cdot)=\hat{u}_\eta(\cdot,t)$。如果答案是肯定的，那麼我們就知道 $u_{\theta(t)}$ 是一個與 $\hat{u}_\eta$ 具有相同表示能力的函數逼近器，而後者根據萬能逼近定理（定理 1.4.5）已知能夠表示一大類函數（例如歐氏空間某個緊子集上的 $W^{k,\infty}$ 函數）。下面這個引理對此問題給出了肯定回答。

引理 3.4.1. *對於任意可微的前饋神經網絡結構 $\hat{u}_\eta:\mathbb{R}^d\times[0,T]\to\mathbb{R}$，其定義為 $(x,t)\mapsto\hat{u}_\eta(x,t)$，參數為 η，都存在一個前饋神經網絡 $u_{\theta(t)}:\mathbb{R}^d\to\mathbb{R}$，其定義為 $x\mapsto u_{\theta(t)}(x)$，並以 $\theta(t)$ 作為其在時刻 t 的參數，其中 $\theta(t)$ 關於時間可微，使得對所有 $x\in\mathbb{R}^d$ 和 $t\in\mathbb{R}$，都有 $u_{\theta(t)}(x)=\hat{u}_\eta(x,t)$。*

证明. 我們將前饋網絡 $\hat{u}_\eta$ 記為

$$\hat{u}_\eta(x,t)=\hat{w}_L^\top\hat{h}_{L-1}(x,t)+\hat{b}_L$$

其中

$$\hat{h}_l(x,t)=\sigma(\hat{W}_l\hat{h}_{l-1}(x,t)+\hat{b}_l),\quad l=1,\ \ldots,\ L-1,$$

這裡 σ 是可微激活函數，

$$\hat{h}_0(x,t)=\sigma(\hat{W}_0x+\hat{w}_0t+\hat{b}_0),$$

參數集合為

$$\eta=(\hat{w}_L,\ \hat{b}_L,\ \hat{W}_{L-1},\ \ldots,\ \hat{W}_0,\ \hat{w}_0,\ \hat{b}_0).$$

注意，η 的所有分量都是與 x 和 t 無關的常數。

類似地，我們將網絡 $u_{\theta(t)}$ 記為

$$u_{\theta(t)}(x)=w_l(t)^\top h_{l-1}(x,t)+b_l(t)$$

其中

$$h_l(x,t) = \sigma(W_l(t)h_{l-1}(x,t) + b_l(t)), \quad l = 1,\ \ldots,\ L-1,$$

其中 $h_0(x,t) = \sigma(W_0(t)x + b_0(t))$，且

$$\theta(t) = (w_L(t),\ b_L(t),\ W_{L-1}(t),\ \ldots,\ W_0(t),\ b_0(t)).$$

注意，$\theta(t)$ 是隨時間演化的。因此，我們可以令對所有 t 都有 $b_0(t) = \hat{w}_0 t + \hat{b}_0$，並令 $\theta(t)$ 的所有其他分量都是常數，且分別與 η 中對應分量相同，即

$$w_L(t) \equiv \hat{w}_L, \quad b_L(t) \equiv \hat{b}_L, \quad \ldots, \quad b_1(t) \equiv \hat{b}_1, \quad W_0(t) \equiv \hat{W}_0$$

對所有 t 成立。按此設置，我們不難驗證，對所有 x 和 t，都有 $u_{\theta(t)}(x) = \hat{u}_\eta(x,t)$，並且 $\theta(t)$ 是可微的。 □

備註. 雖然我們在引理 3.4.1 中是針對前饋神經網絡陳述的，但很容易看出該結果對一般神經網絡同樣成立。要看出這一點，只需注意，對於任意神經元 $\hat{y}(x,t)$，我們都可以像引理 3.4.1 中那樣將其改寫為 $y_{\eta(t)}(x)$。對於任意神經元 $z(t,x,\hat{y}(x,t))$，我們可以將其寫成

$$\hat{z}(t,x,\hat{y}(x,t)) = \hat{z}(t,x,y_{\eta(t)}(x)) = z_{(\eta(t),\xi(t))}(x)$$

其中 $z_{\eta,\xi}$ 是某個以 x 為輸入、參數 (η,ξ) 隨 t 演化的神經網絡。重複應用這一結果即可證明該結論。

現在我們看到，使用一個結構固定但參數隨時間演化的深度神經網絡 $u_{\theta(t)}$ 來逼近演化 PDE 的解 $v(\cdot,t)$ 是合理的。下一個問題是如何控制 $\theta(t)$，使得 $u_{\theta(t)}$ 能夠表現得像初值問題 (3.150) 的解。為了滿足 (3.150) 中的初始條件，我們可以直接設定

$$\begin{aligned}\theta(0) &= \arg\min_{\theta_0} \|u_{\theta_0} - g\|^2_{L^2(\Omega)} \\ &= \arg\min_{\theta_0} \int |u_{\theta_0}(x) - g(x)|^2\, dx. \end{aligned} \tag{3.152}$$

注意，在實際中，(3.152) 中的積分通常通過 Monte Carlo 積分來逼近，特別是在高維情形下；而這個最小化問題通常較容易求解，因為我們只是在用一個神經網絡去逼近已知函數 g。

為了滿足 (3.150) 中的 PDE，我們用其近似 $u_{\theta(t)}(x)$ 替代解 $v(x,t)$，得到

$$\partial_t u_{\theta(t)}(x) = F[u_{\theta(t)}](x), \quad \forall\,(x,t) \in \Omega \times [0,T]. \tag{3.153}$$

注意，由於我們假設 F 是自治的，因此在 (3.153) 右側的括號中省略了 t。另一方面，由鏈式法則可得

$$\partial_t u_{\theta(t)}(x) = \nabla_\theta u_{\theta(t)}(x) \cdot \dot{\theta}(t). \tag{3.154}$$

將 (3.153) 和 (3.154) 結合起來，我們看到，為了使 $u_{\theta(t)}$ 成為演化 PDE (3.150) 的解，參數 θ 應滿足

$$\nabla_\theta u_{\theta(t)}(x) \cdot \dot{\theta}(t) = F[u_{\theta(t)}](x) \tag{3.155}$$

對所有 t 都成立。因此，應當以某種方式控制 $\theta(t)$，使得 (3.155) 兩側之差在每個 t 時刻都儘量小。

由於我們的目標是找到可作用於任意初始值 g 的解算子 S_F，我們需要一個控制向量場，它能夠引導 $\dot{\theta}(t)$，使得從由 g 決定的任意初始 $\theta(0)$ 出發時，(3.155) 兩邊之間的差都能夠被最小化。為此，我們收集一個由 M 個初始函數組成的集合 $\{g_i : \bar{\Omega} \to \mathbb{R} : i \in [M]\}$（理想情況下，它們應儘可能與實際應用中可能輸入給 S_F 的那些初值相似）。然後我們像 (3.152) 那樣為它們求得相應的初始參數 $\theta_{i,0}$：

$$\theta_i(0) = \operatorname*{arg\,min}_{\theta_{i,0} \in \mathbb{R}^n} \int |u_{\theta_{i,0}}(x) - g_i(x)|^2 \, dx. \tag{3.156}$$

我們注意到上面的問題可以對 i 並行求解。我們用 $\mathcal{D}_{\text{init}} = \{\theta_i(0) \in \mathbb{R}^n : i \in [M]\}$ 表示這個初值集合。接著，我們嘗試尋找一個控制向量場 $h : \mathbb{R}^n \to \mathbb{R}^n$，它也被參數化為一個參數為 η 的深度神經網絡 h_η，使得從 $\mathcal{D}_{\text{init}}$ 中任一初始值 $\theta_i(0)$ 出發時，(3.155) 總是（近似）成立。然後，我們通過求解如下最優控

制問題來訓練這個控制向量場 h_η:

$$\max_{\eta} \quad J(\eta) := -\frac{1}{M}\sum_{i=1}^{M} r_i(T), \tag{3.157a}$$

$$\text{s.t.} \begin{cases} \dot{\theta}_i(t) = h_\eta(\theta_i(t)), \quad \forall\, t \in [0,T],\ i \in [M], \\ \dot{r}_i(t) = \|\nabla_\theta u_{\theta_i(t)} \cdot h(\theta_i(t)) - F[u_{\theta_i(t)}]\|^2_{L^2(\Omega)}, \\ \theta_i(0) \in \mathcal{D}_{\text{init}}, \quad r_i(0) = 0, \quad i \in [M]. \end{cases} \tag{3.157b}$$

在 (3.157b) 中，$r_i(T)$ 表示由 $\theta_i(t)$ 在區間 $[0,T]$ 上生成的 (3.155) 兩側總平方誤差。我們可以像第 3.3 節 Neural ODE 方法中那樣構造 (3.157) 的伴隨方程並計算 $\nabla_\eta J(\eta)$，再採用任意基於梯度的優化算法來訓練代表控制向量場的神經網絡 h_η。

一旦網絡 h_η 被訓練好，解算子 S_F 的實現方式如下。對於任意新的初始條件 $g : \bar{\Omega} \to \mathbb{R}$，我們首先根據 (3.152) 計算 $\theta(0)$。這一步簡單直接，因為它只是將神經網絡擬合到定義在 Ω 上的已知函數 g。然後，從該 $\theta(0)$ 出發，求解如下 ODE 得到 $\theta(t)$:

$$\dot{\theta}(t) = h_\eta(\theta(t)), \quad \forall\, t \in [0,T],$$

這個 ODE 可以用任意數值 ODE 求解器快速而準確地求解。接著，參數取為 $\theta(t)$ 的網絡 $u_{\theta(t)}(\cdot)$ 就是演化 PDE (3.150) 在初值 g 下的解 $v^g(\cdot,t)$ 的一個近似。在實現 S_F 的這兩步過程中，求 $\theta(0)$ 的計算成本佔主導，但這仍遠低於從頭開始求解問題 (3.150) 的代價。

在實際中，有若干技巧可以提高控制向量場 h_η 的精度並加快其訓練。例如，我們可以更謹慎地選擇初值集合 $\{g_i : \bar{\Omega} \to \mathbb{R} : i \in [M]\}$ 來提升數據質量，使它們更貼近訓練後的解算子 S_F 將會應用到的那些初值。我們還可以針對訓練初值集合 $\mathcal{D}_{\text{init}}$ 中的每個初值分別求解對應的 $\theta_i(t)$，並生成多個時間點 $t_j \in (0,T]$ 上的樣本點 $\theta_i(t_j)$。這些樣本點可以構造成一個正則項，並加到 (3.157b) 中 $r_i(t)$ 的 ODE 上。更多細節可參見 [45,47]。

3.4.2 最優控制的參考文獻

最優控制是應用數學與工程中的一門核心學科，並且在過去幾十年中得到了深入研究。文獻中有大量優秀著作涵蓋了最優控制的理論與計算 [7,39,42,65,92,112]。注意，[39,92,112] 的原始版本都早在參考文獻表和現代文獻數據

庫中所顯示年份的幾十年前就已出版。

從歷史上看，最優控制源自變分法，而變分法可以追溯到數個世紀以前；不過，隨著 20 世紀中期航空航天技術和計算技術在 1960 年代和 1970 年代的發展，它逐漸成熟為一個獨立領域。早期工作將積分型代價泛函的極小化與控制函數的變分聯繫起來，形成了推廣經典 Euler–Lagrange 方程的最優性必要條件 [126]。本章沒有涉及隨機最優控制，其中 ODE 被隨機微分方程（SDE）替代，而獎勵函數則以期望形式出現。其主要思想與確定性最優控制相似，但需要認真考慮擴散效應及相關分析。感興趣的讀者可參見 [39, 42] 瞭解更多細節。

經典文獻中的一個奠基性結果是 1950 年代提出的 Pontryagin 極大值原理，它以 Hamilton 系統和伴隨（協態）演化的形式給出了最優控制的必要條件。該原理的重要性在於，它將關於控制函數的無限維優化約化為逐點條件，從而簡化了許多問題的分析與實現。與此同時，Bellman 動態規劃原理提出了一種以價值函數和遞歸最優性為核心的替代框架，它構成了連續與離散最優控制以及後來的強化學習的基礎（我們將在下一章介紹更多細節）。這兩個基礎性結果的核心思想和主要結論已在第 3.1 節中介紹，其中大部分內容遵循 [39]，並對證明細節做了一些擴展。這些結果向最優概率密度控制的推廣見第 3.2 節，其內容遵循 [46]。作為殘差網絡連續時間變體的 Neural ODE 方法由 [27] 提出，事實證明它是求解標準最優控制以及許多其他一般控制和學習問題的有力工具，正如我們在第 3.3 節所解釋的那樣。我們還在第 3.4.1 節中參考 [45, 47] 介紹了其在學習 PDE 解算子中的應用，並將在第 5.4 節中進一步介紹其在生成建模中的概率密度控制方法。

第四章　深度強化學習

強化學習（reinforcement learning）研究的是序貫決策問題，其中智能體根據觀測到的狀態採取行動並與環境進行交互。其目標是設計一個最優策略，使得在時間過程中從環境中獲得的期望累計獎勵最大化。

本章重點討論強化學習的兩個核心方面。第一部分關注其數學基礎。我們介紹強化學習系統的基本組成，並說明它們如何在 Markov 決策過程的框架下進行形式化。我們給出一系列核心理論結果，尤其是作為強化學習支柱的 Bellman 算子和 Bellman 方程，並提供嚴格的數學證明。

第二部分討論強化學習算法，包括基於模型和無模型兩類方法。基於模型的方法深度依賴於 Bellman 理論，我們介紹兩種經典算法：策略迭代和價值迭代。鑑於現實世界強化學習問題的複雜性，我們進一步討論一系列在實踐上表現優異的無模型算法。在介紹過程中，我們說明如何結合深度學習來處理大規模和高維強化學習問題。

4.1　強化學習基礎

4.1.1　強化學習中的基本概念

在強化學習中，**智能體**（agent）通常指能夠在指定時間點改變狀態、執行動作並從環境中接收反饋的主體。例如，智能體可以是機器人、汽車、無人機等。智能體的**狀態**（state）通常指其位置、速度、方向等信息的組合。智能體執行的**動作**（action）可以是加速、剎車、機械臂旋轉等。**環境**（environment）可以是圍繞智能體的物理世界（包括人類），它根據智能體的狀態和動作向其提供獎勵，同時也決定智能體的狀態轉移。**獎勵**（reward）通常表示為一個數值，智能體據此學習如何執行最優行為，以在時間過程中獲得最大的總獎勵。在強化學習中，我們通常採用離散時間設定，因為獎勵和狀態測量通常是在離散時間點進行的。

在本章中，我們用 x 表示智能體的狀態，用 u 表示動作[1]，用 r 表示獎勵。它們被表示為數或向量，以便進行數值計算。我們用 $\mathcal{X}$ 和 $\mathcal{U}$ 分別表示**狀態空間**和**動作空間**。需要注意的是，$\mathcal{X}$ 和 $\mathcal{U}$ 可以是有限集合、可數集合，或者歐氏空間的子集。我們還給出如下平穩策略的定義，它描述了智能體在任意狀態 x 下應採取的動作。

定義 4.1.1 (平穩策略，stationary policy)**.** 如果對於每個 $x \in \mathcal{X}$，$\pi(\cdot|x)$ 是定義在動作空間 $\mathcal{U}$ 上的與時間無關的概率分佈，則稱 π 為 $\mathcal{X}$ 上的一個**平穩策略**（或簡稱**策略**），即對於任意 $u \in \mathcal{U}$，有 $\pi(u|x) \geq 0$，並且

$$\int_{\mathcal{U}} \pi(u|x)\, du = 1.$$

如果 $\mathcal{U}$ 是離散集合，例如有限集合或可數集合，則積分應替換為對 $\mathcal{U}$ 的求和。我們用 Π 表示定義在 $\mathcal{X}$ 上的所有平穩策略的集合。

在實際應用中，也存在非平穩策略，即 $\pi_t(\cdot|x)$ 依賴於時間 t。然而，在大多數強化學習研究和應用中，通常採用平穩策略的設定，因此本章僅考慮平穩策略。

4.1.2 Markov 決策過程

當給定一個平穩策略 π 時，智能體可以在時刻 $t = 0$ 從狀態空間 $\mathcal{X}$ 上某個初始狀態分佈 p_0 中抽取初始狀態 X_0，並按照策略 π 執行動作 U_0。隨後，環境根據 X_0 和 U_0 提供獎勵 R_1，並將智能體轉移到新狀態 X_1。這一過程會隨時間持續進行，最終形成如下軌跡：

$$X_0,\ U_0,\ R_1,\ X_1,\ U_1,\ R_2,\ \cdots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ \cdots \tag{4.1}$$

注意，在 (4.1) 中我們使用大寫字母，是因為在強化學習問題中，這些量一般都是隨機的（除非 p_0 和 π 是確定性的，即它們只映射到特定值而不具有任何隨機性）。

在強化學習的設定中，軌跡 (4.1) 被假設或構造成一個 **Markov 過程**。更準確地說，對於任意時刻 t，隨機變量 $U_t, R_{t+1}, X_{t+1}, \cdots$ 僅依賴於 X_t 的取值，而不依賴於過去的隨機變量 $X_0, \cdots, X_{t-1}, U_{t-1}, R_t$ 的取值。在這種情況下，(4.1) 被稱為 **Markov 決策過程**（Markov decision process）。

[1]在強化學習文獻中，狀態和動作通常分別記為 s 和 a。這裡我們使用 x 和 u 以體現它們與第 三 章中最優控制的狀態和控制之間的對應關係。

环境（Environment）

- 围绕智能体的相关物理世界。
- 可能包括非生物体和生物体（例如人类）。
- 可根据智能体当前的状态 $x \in \mathcal{X}$ 及其采取的动作 $u \in \mathcal{U}$ 提供数值奖励 $r \in \mathbb{R}$。
- 可根据当前状态 $x \in \mathcal{X}$ 和动作 $u \in \mathcal{U}$ 将智能体转移到新的状态 $x' \in \mathcal{X}$。其中 x' 由某个（可能未知的）转移概率 $p(\cdot|x,u)$ 决定。

在时刻 t，智能体处于状态 $x_t \in \mathcal{X}$，并按照策略 $\pi(\cdot|x_t)$ 采取动作 $u_t \in \mathcal{U}$。环境观测到 (x_t, u_t)。

在观测 (x_t, u_t) 后，环境将智能体转移到 $x_{t+1} \in \mathcal{X}$，其分布由 $p(\cdot|x_t, u_t)$ 给出，并（可能）根据 (x_t, u_t) 提供奖励 $r_{t+1} \in \mathbb{R}$。

智能体（Agent）

- 能够与环境交互的主体。
- 通常指智能机器人、汽车、无人机等。
- 能获取当前状态 $x \in \mathcal{X}$。
- 可依据策略 $\pi(\cdot|x)$ 在状态 x 下采取动作 $u \in \mathcal{U}$。
- 若 $\pi(\cdot|x)$ 为确定性策略，则直接选择 u。
- 若 $\pi(\cdot|x)$ 为随机策略（$\mathcal{U}$ 上的概率分布），则 $u \sim \pi(\cdot|x)$。
- 强化学习的目标是在每个 $x \in \mathcal{X}$ 下找到最优策略 $\pi(\cdot|x)$，使得智能体能够在指定时间内最大化从环境获得的总奖励。

图 4.1: 典型的強化學習（RL）系統的結構。該系統由智能體和環境組成，其功能分別在對應方框中說明。智能體通過按照策略 π 採取動作與環境交互。在任意時刻 t，環境根據智能體當前狀態 x_t 和其採取的動作 u_t，提供獎勵 r_{t+1} 並將其轉移到新狀態 x_{t+1}。強化學習的目標是尋找一個最優策略 π，使智能體在時間過程中獲得最大累計獎勵。

從概率的角度來看，Markov 決策過程 (4.1) 由如下策略分佈和狀態轉移概率分佈所決定。對於任意 $x \in \mathcal{X}$，有

$$U_t \sim \pi(\cdot|X_t = x), \tag{4.2}$$

其中 $\pi \in \Pi$ 是一個平穩策略。對於任意 $x \in \mathcal{X}$ 和 $u \in \mathcal{U}$，下一狀態 X_{t+1} 服從轉移概率

$$X_{t+1} \sim p(\cdot|X_t = x, U_t = u), \tag{4.3}$$

注意，這個轉移概率 $p(\cdot|x,u)$ 完全由環境決定，不受任何策略影響。當給定一個策略 π 時，在本章的推導中我們還會頻繁使用如下狀態到狀態的轉移概率

$$p^\pi(x'|x) := \int_{\mathcal{U}} \pi(u|x)p(x'|u,x)\,du \tag{4.4}$$

其中 $x' \in \mathcal{X}$ 是任意狀態。

值得注意的是，如果給定一個策略 π，並且 (4.4) 中的狀態到狀態轉移概率 p^π 已知，那麼 Markov 決策過程 (4.1) 就退化為一個 **Markov 獎勵過程**（Markov reward process）：

$$X_0,\ R_1,\ X_1,\ R_2,\ \cdots,\ X_t,\ R_{t+1},\ X_{t+1},\ \cdots \tag{4.5}$$

因為動作將由策略 π 自動給出，而狀態轉移由 p^π 決定。Markov 獎勵過程 (4.5) 可以被觀測到，但在這一過程中策略 π 不能改變。

在某些強化學習問題中，獎勵 R_{t+1} 也是一個隨機變量，它服從某個條件概率分佈 $p_R(\cdot|X_t = x, U_t = u)$（同樣，$p_R$ 由環境決定，並且不受任何策略影響）。然而，在大多數情況下，求解強化學習問題時只需要用到 R_{t+1} 的期望，因此為簡化起見，我們直接設 $R_{t+1} = r(X_t, U_t)$，其中 $r(x,u) : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$ 是確定性的（並由環境給定），並且在給定 x 和 u 時可以被計算。

4.1.3 Bellman 算子與 Bellman 方程

有了上述所有概念之後，我們已經準備好給出強化學習問題的數學表述。為此，我們首先引入一個重要術語，稱為狀態價值函數（它可以看作最優控制中價值函數的對應物）。在強化學習中，狀態價值函數將每個狀態 $x \in \mathcal{X}$ 映射為從 $X_0 = x$ 出發的軌跡 (4.1) 過程中所獲得折扣獎勵總和的期望，如下定義所示。

定義 4.1.2 (狀態價值函數，state value function)**.** 設 $\pi \in \Pi$ 是定義在狀態空間 $\mathcal{X}$ 和動作空間 $\mathcal{U}$ 上的一個平穩策略，$\gamma \in (0,1)$ 是一個**折扣因子**（discount factor）。則**狀態價值函數**（簡稱**價值函數**），記作 v^π，是一個從 $\mathcal{X}$ 到 $\mathbb{R}$ 的映射，定義為

$$v^\pi(x) = \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \Big| X_0 = x\Big], \tag{4.6}$$

其中 $\{R_{t+1} : t \geq 0\}$ 是從 $X_0 = x$ 出發、並遵循策略分佈 (4.2) 與狀態轉移概率 (4.3) 的 Markov 決策過程軌跡 (4.1) 中所獲得的獎勵。

關於定義 4.1.2 中引入的折扣因子 γ，我們有幾點說明。這個折扣因子通常取值於 $(0,1)$，因為我們通常無法完全確定未來的情況。我們也不確定像 (4.1) 這樣的過程會持續多久，而折扣因子 $\gamma \in (0,1)$ 能夠在較弱條件下保證總獎勵始終有界（例如，對所有 t，R_t 都小於某個常數上界 $M_R > 0$）。在實際應用中，我們使用折扣因子，是因為人類和動物的行為似乎更偏好即時獎勵。在金融中，也經常考慮折扣因子，因為即時獎勵可能比延遲獎勵產生更多的利息和收益。最後但同樣重要的是，折扣因子 $\gamma \in (0,1)$ 使我們更容易為強化學習構建強有力的數學理論和數值算法。

當然，我們也可以考慮 $\gamma = 1$ 的 Markov 決策過程下的強化學習。通常這是指存在一個終止狀態 $\bar{x} \in \mathcal{X}$，使得智能體幾乎必然（在某種概率意義下）在有限時間內到達 $\bar{x}$，並且之後一直停留在 $\bar{x}$，即對所有 $u \in \mathcal{U}$，都有 $p(\bar{x}|\bar{x}, u) = 1$。儘管如此，在本章中我們主要考慮帶有折扣因子 $\gamma \in (0,1)$ 的這種常見情形。

現在我們注意到狀態價值函數 v^π 在 (4.6) 中定義後具有一個重要性質：對於每個 $x \in \mathcal{X}$，有

$$\begin{aligned}
v^\pi(x) &= \mathbb{E}[R_1|X_0 = x] + \gamma\, \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_0 = x\Big] \\
&= \int \pi(u_0|x) r(x, u_0)\, du_0 \\
&\quad + \gamma \iint \pi(u_0|x) p(x_1|x, u_0) \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_1 = x_1\Big] dx_1 du_0 \qquad (4.7) \\
&= \int \pi(u_0|x) \Big[r(x, u_0) + \gamma \int p(x_1|x, u_0) v^\pi(x_1)\, dx_1\Big] du_0 \\
&= \mathbb{E}_{U_0 \sim \pi(\cdot|x)} \Big[r(x, U_0) + \gamma \mathbb{E}_{X_1 \sim p(\cdot|x, U_0)}[v^\pi(X_1)]\Big],
\end{aligned}$$

其中我們使用了策略分佈和狀態轉移概率分佈 (4.2) 與 (4.3)，以及

$$v^\pi(x_1) = \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_1 = x_1\Big]$$

這基於定義 4.1.2（就好像 Markov 決策過程 (4.1) 是從 x_1 開始的一樣）。方程 (4.7) 被稱為 v^π 的 **Bellman 方程**。

到這裡，我們已經看到任意策略 $\pi \in \Pi$ 都可以確定一個如 (4.1) 所示的 Markov 決策過程，以及定義 4.1.2 中的狀態價值函數 $v^\pi : \mathcal{X} \to \mathbb{R}$。我們還注意到 (4.7) 給出的狀態價值函數具有如下性質：v^π 等於對其自身施加某種運算後的結果，正如 (4.7) 右端所示。這表明該運算在強化學習中的重要性。為了更清楚地描述這一運算，我們先考慮 $\mathcal{X}$ 上所有一致有界函數所構成的集合：

$$B(\mathcal{X}) := \Big\{ v : \mathcal{X} \to \mathbb{R} : \sup_{x \in \mathcal{X}} |v(x)| < \infty \Big\}. \tag{4.8}$$

容易看出，$B(\mathcal{X})$ 是一個實線性空間，即對於任意 $v_1, v_2 \in B(\mathcal{X})$ 和 $a_1, a_2 \in \mathbb{R}$，都有 $a_1 v_1 + a_2 v_2 \in B(\mathcal{X})$。此外，我們在 $B(\mathcal{X})$ 上定義 ∞-範數：對任意 $v \in B(\mathcal{X})$，$\|v\|_\infty := \sup_{x \in \mathcal{X}} |v(x)|$。於是很容易驗證，帶有該 ∞-範數的 $B(\mathcal{X})$ 是一個 Banach 空間（見附錄 A.2）。現在我們可以正式給出 (4.7) 右端運算的定義了。

定義 4.1.3 (與給定策略對應的 Bellman 算子)**.** 設 $\pi \in \Pi$ 是 $\mathcal{X}$ 上的一個平穩策略。則**與 π 相關的 Bellman 算子** T^π 是從 $B(\mathcal{X})$ 到 $B(\mathcal{X})$ 的一個映射，記為 $T^\pi : v \mapsto T^\pi v$，其中對每個 $x \in \mathcal{X}$ 有

$$T^\pi v(x) := \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x, U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)} [v(X')] \Big]. \tag{4.9}$$

注意，$T^\pi v(x)$ 應始終理解為 $(T^\pi v)(x)$。現在，藉助 (4.9) 中 Bellman 算子 T^π 的定義，我們可以將 v^π 的 Bellman 方程 (4.7) 簡潔地寫成：

$$v^\pi = T^\pi v^\pi.$$

這表明 v^π 是 T^π 的一個不動點。關於 Banach 空間中的算子及其不動點，已有大量豐富的結果。特別地，如果一個算子對某個 $\gamma \in (0,1)$ 是 γ-壓縮的，那麼由不動點理論可知它具有許多重要性質（見附錄 A.2）。接下來我們將證明，T^π 的確是 γ-壓縮的。

命題 4.1.4 (T^π 是 γ-壓縮的)**.** *對任意 $\pi \in \Pi$ 和 $\gamma \in (0,1)$，與 π 對應的*

Bellman 算子 T^π 是一個 γ-壓縮映射。也就是說，對任意 $v, \hat{v} \in B(\mathcal{X})$，有

$$\|T^\pi v - T^\pi \hat{v}\|_\infty \le \gamma \|v - \hat{v}\|_\infty. \tag{4.10}$$

证明. 對任意 $x \in \mathcal{X}$，有

$$\begin{aligned}
|T^\pi v(x) - T^\pi \hat{v}(x)| &= \Big| \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big] \\
&\qquad - \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\Big| \\
&= \Big|\gamma\, \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X') - \hat{v}(X')]\Big]\Big| \\
&= \gamma \Big| \iint \pi(u|x) p(x'|x,u)(v(x') - \hat{v}(x'))\, dx' du \Big| \\
&= \gamma \Big| \int p^\pi(x'|x)(v(x') - \hat{v}(x'))\, dx' \Big| \\
&\le \gamma \int p^\pi(x'|x) |v(x') - \hat{v}(x')|\, dx' \\
&\le \gamma \|v - \hat{v}\|_\infty \int p^\pi(x'|x)\, dx' \\
&= \gamma \|v - \hat{v}\|_\infty,
\end{aligned}$$

其中,第二個等號用到了期望的線性性;第四個等號中用了 $p^\pi(x'|x) = \int \pi(u|x)p(x'|x,u)du$;最後一個等號則使用了 $\int p^\pi(x'|x)\, dx' = 1$。由於 x 是任意的，所以命題 (4.10) 得證。 □

接下來，我們介紹定義在 $\mathcal{X}$ 上的另一個 Bellman 算子，它在推導強化學習中策略最優性條件時非常有用。注意，這個 Bellman 算子不需要事先給定任何策略。

定義 4.1.5 (Bellman 算子)**.** $\mathcal{X}$ **上的 Bellman 算子** T^* 是一個從 $B(\mathcal{X})$ 到 $B(\mathcal{X})$ 的映射，記為 $T^* : v \mapsto T^* v$，其中對每個 $x \in \mathcal{X}$ 定義:

$$T^* v(x) := \sup_{\pi\in\Pi} \Big\{ \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\Big\}. \tag{4.11}$$

我們說明一下: T^π 是 $B(\mathcal{X})$ 上的一個線性算子，而由於 (4.11) 中存在上確界，T^* 一般是非線性算子。現在我們希望證明，T^* 與 T^π 一樣，也是 γ-壓縮的。然而，由於其非線性，我們在證明前需要一個小引理。

引理 4.1.6. 設 A 是任意集合，$f, g : A \to \mathbb{R}$ 是定義在 A 上的兩個實值函數。

則

$$\left|\sup_{a\in A} f(a) - \sup_{a\in A} g(a)\right| \le \sup_{a\in A} |f(a) - g(a)|. \tag{4.12}$$

证明. 首先注意到，對任意兩個實值函數 $f_1, f_2 : A \to \mathbb{R}$，有

$$\sup_{a\in A}(f_1(a) + f_2(a)) \le \sup_{a\in A} f_1(a) + \sup_{a\in A} f_2(a). \tag{4.13}$$

這個不等式 (4.13) 很容易驗證：對任意 $a \in A$，都有

$$f_1(a) + f_2(a) \le \sup_{a\in A} f_1(a) + \sup_{a\in A} f_2(a),$$

然後對左端關於 a 在 A 上取上確界，就得到 (4.13)。

現在證明 (4.12)。我們有

$$\begin{aligned}\sup_{a\in A} f(a) &= \sup_{a\in A}\Big((f(a) - g(a)) + g(a)\Big)\\ &\le \sup_{a\in A}(f(a) - g(a)) + \sup_{a\in A} g(a)\\ &\le \sup_{a\in A}|f(a) - g(a)| + \sup_{a\in A} g(a),\end{aligned}$$

其中第一個不等式使用了 (4.13)，因此有

$$\sup_{a\in A} f(a) - \sup_{a\in A} g(a) \le \sup_{a\in A}|f(a) - g(a)|.$$

類似地，也有

$$\sup_{a\in A} g(a) - \sup_{a\in A} f(a) \le \sup_{a\in A}|f(a) - g(a)|.$$

將前面這兩個不等式結合起來，就得到 (4.12)。 □

現在我們可以證明 (4.11) 中定義的 T^* 也是一個 γ-壓縮映射。

命題 4.1.7 (T^* 是 γ-壓縮的)**.** 對任意 $\gamma \in (0,1)$，*Bellman* 算子 T^* 是一個 γ-壓縮映射。也就是說，對任意 $v, \hat{v} \in B(\mathcal{X})$，有

$$\|T^*v - T^*\hat{v}\|_\infty \le \gamma\|v - \hat{v}\|_\infty. \tag{4.14}$$

证明. 對任意 $x \in \mathcal{X}$，有

$$
\begin{aligned}
|T^* v(x) - T^* \hat{v}(x)| &= \Big| \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x,U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)}[v(X')] \Big] \\
&\qquad - \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x,U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)}[\hat{v}(X')] \Big] \Big| \\
&\le \sup_{\pi \in \Pi} \Big| \gamma \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[\mathbb{E}_{X' \sim p(\cdot|x,U)}[v(X') - \hat{v}(X')] \Big] \Big| \\
&= \gamma \sup_{\pi \in \Pi} \Big| \int p^\pi(x'|x)(v(x') - \hat{v}(x'))\, dx' \Big| \\
&\le \gamma \sup_{\pi \in \Pi} \int p^\pi(x'|x) |v(x') - \hat{v}(x')|\, dx' \\
&\le \gamma \|v - \hat{v}\|_\infty \sup_{\pi \in \Pi} \int p^\pi(x'|x)\, dx' \\
&= \gamma \|v - \hat{v}\|_\infty
\end{aligned}
$$

其中，第一個不等式使用了引理 4.1.6。由於 x 是任意的，因此命題 (4.14) 得證。 □

由於 T^π 和 T^* 都是 γ-壓縮映射，Banach 不動點定理（定理 A.2.7）意味著它們各自都有唯一不動點，並且對應的不動點迭代從任意初始值出發都收斂到該不動點。下面的定理總結了這些結論。

定理 4.1.8 (Bellman 算子具有唯一不動點)**.** *關於折扣因子 $\gamma \in (0,1)$ 下定義在 $\mathcal{X}$ 上的 Bellman 算子，如下兩條結論成立:*

- *對任意定義在 $\mathcal{X}$ 上的平穩策略 $\pi \in \Pi$，與 π 相關的 Bellman 算子 T^π 有唯一不動點 v^π。進一步地，對任意 $v \in B(\mathcal{X})$，設 $\{v_k : k = 0, 1, \dots\}$ 是由不動點迭代 $v_{k+1} = T^\pi v_k$、初值 $v_0 = v$ 所生成的序列，則當 $k \to \infty$ 時，有*

$$
\|(T^\pi)^k v - v^\pi\|_\infty = \|v_k - v^\pi\|_\infty \to 0. \tag{4.15}
$$

  且收斂速度是 γ-線性的。

- *Bellman 算子 T^* 有唯一不動點 v^*。進一步地，對任意 $v \in B(\mathcal{X})$，設 $\{v_k : k = 0, 1, \dots\}$ 是由不動點迭代 $v_{k+1} = T^\pi v_k$、初值 $v_0 = v$ 所生成的序列，則當 $k \to \infty$ 時，有*

$$
\|(T^*)^k v - v^*\|_\infty = \|v_k - v^*\|_\infty \to 0. \tag{4.16}
$$

且收斂速度是 γ-線性的。

证明. 由於 T^π 和 T^* 都是在 $B(\mathcal{X})$ 上的 γ-壓縮映射，因此唯一不動點的存在性以及不動點迭代的收斂性都可由定理 A.2.7 直接推出。 □

定義 4.1.9 ($B(\mathcal{X})$ 中的偏序)**.** 設 $v, \hat{v} \in B(\mathcal{X})$。若

$$v \leq \hat{v} \tag{4.17}$$

對每個 $x \in \mathcal{X}$ 都有 $v(x) \leq \hat{v}(x)$，則稱 $v \leq \hat{v}$。若同時有 $v \leq \hat{v}$ 且 $\hat{v} \leq v$，則稱 $v = \hat{v}$。

現在我們有關於 Bellman 算子的三個重要單調性性質。這些結果在後文中會被頻繁使用。

定理 4.1.10. 設 $v, \hat{v} \in B(\mathcal{X})$（它們不一定是 Π 中某個策略對應的價值函數），則以下結論成立:

(i) 設 $\pi \in \Pi$ 是任意一個平穩策略，T^π 是與之相關的 *Bellman* 算子，則

$$T^\pi v \leq T^* v. \tag{4.18}$$

(ii) 如果 $v \leq \hat{v}$，那麼對任意平穩策略 $\pi \in \Pi$，與 π 相關的 *Bellman* 算子 T^π 滿足

$$T^\pi v \leq T^\pi \hat{v}. \tag{4.19}$$

(iii) 如果 $v \leq \hat{v}$，那麼 *Bellman* 算子 T^* 滿足

$$T^* v \leq T^* \hat{v}. \tag{4.20}$$

证明. 任取 $x \in \mathcal{X}$。對 (i)，注意到

$$\begin{aligned}
T^\pi v(x) &= \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x, U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)} [v(X')] \Big] \\
&\leq \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x, U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)} [v(X')] \Big] \\
&= T^* v(x).
\end{aligned}$$

對 (ii)，有

$$\begin{aligned}T^\pi v(x) &= \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\\ &\le \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\\ &= T^\pi\hat{v}(x)\end{aligned}$$

其中不等式使用了對每個 $x\in\mathcal{X}$ 都有 $v(x)\le\hat{v}(x)$。

類似地，對 (iii)，有

$$\begin{aligned}T^* v(x) &= \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\\ &\le \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\\ &= T^*\hat{v}(x),\end{aligned}$$

這裡同樣使用了對每個 $x\in\mathcal{X}$ 都有 $v(x)\le\hat{v}(x)$。

由於上面所取的 $x\in\mathcal{X}$ 是任意的，因此這三個結論都成立。 □

由於強化學習的目標是尋找最優策略（我們尚未給出其數學定義；它將在後面的定義 4.1.14 中給出），因此去考慮某種意義下“貪婪”的策略是很自然的。下面的定義給出了相對於 $B(\mathcal{X})$ 中一個函數的貪婪策略的具體含義。

定義 4.1.11 (貪婪策略，greedy policy)**.** 對任意 $v\in B(\mathcal{X})$，若 π 滿足

$$T^\pi v = T^* v, \tag{4.21}$$

其中 T^π 是與 π 相關的 Bellman 算子，則稱 π **相對於** v **是貪婪的**。注意，對某個 $v\in B(\mathcal{X})$，可能存在多個相對於它的貪婪策略。

一個重要問題是：對任意 $v\in B(\mathcal{X})$，是否總存在一個相對於 v 貪婪的平穩策略 π？由於平穩策略的 Markov 性質，這個問題的答案為“是”的充要條件是：對每個 $x\in\mathcal{X}$，都存在 $u(x)\in\mathcal{U}$（不要求唯一）滿足

$$\begin{aligned}&r(x,u(x))+\gamma\mathbb{E}_{X'\sim p(\cdot|x,u(x))}[\hat{v}(X')]\\ &\qquad = \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big].\end{aligned} \tag{4.22}$$

在這種情況下，我們可以構造一個策略 π，使得它在每個 x 處都映射到 $u(x)$（如果有多個 $u(x)$ 滿足 (4.22)，我們任選其一即可）。那麼根據定義 4.1.11，該

π 就是相對於 v 的貪婪策略。

我們可以看到，對每個 $x \in \mathcal{X}$，存在滿足 (4.22) 的 $u(x)$，當且僅當上確界能夠在動作空間 $\mathcal{U}$ 中取到。很容易看出，在以下兩種情況下這會發生：(i) $\mathcal{U}$ 是有限集；或者 (ii) $\mathcal{U} \subset \mathbb{R}^m$ 是緊集，並且函數

$$r(x,\cdot) + \gamma \int p(x'|x,\cdot)v(x')\,dx' \ : \ \mathcal{U} \to \mathbb{R} \tag{4.23}$$

是連續的（因為定義在 $\mathbb{R}^m$ 中緊域上的連續函數一定能在該域上取到最大值）。條件 (i) 和 (ii) 都是充分條件，但並非必要條件。在實際設定中，對任意給定的 $v \in B(\mathcal{X})$，我們通常都能得到一個相對於它貪婪的策略。

定理 4.1.12. *設 v^* 是 T^* 的不動點。如果 π 相對於 v^* 是貪婪的，那麼 $v^\pi = v^*$。*

证明. 由於 π 相對於 v^* 是貪婪的，根據定義 4.1.11，有

$$T^\pi v^* = T^* v^*. \tag{4.24}$$

另一方面，因為 v^* 是 T^* 的不動點，所以有 $T^* v^* = v^*$。因此，我們得到 $T^\pi v^* = v^*$，這意味著 v^* 同時也是 T^π 的一個不動點。由於 v^π 是 T^π 的唯一不動點，所以必有 $v^\pi = v^*$。 □

我們已經見過這樣一個關於狀態 x 和動作 u 的函數：在給定某個狀態價值函數 v 的情況下，它評估期望總獎勵，例如 (4.22) 中所示的函數。這個函數在強化學習的算法發展中也非常有用。下面我們給出它的正式定義。

定義 4.1.13 (Q 函數)**.** 對任意 $v \in B(\mathcal{X})$，我們定義與 v 對應的**質量函數**（quality function），簡稱 **Q 函數**（Q-function）（也稱為**狀態–動作價值函數**），為

$$q(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v(X')] \tag{4.25}$$

其中 $x \in \mathcal{X}$，$u \in \mathcal{U}$。注意，$q : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$。

特別地，我們將與 v^π（T^π 的唯一不動點）對應的 Q 函數記為

$$q^\pi(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v^\pi(X')], \tag{4.26}$$

將與 v^*（T^* 的唯一不動點）對應的 Q 函數記為

$$q^*(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v^*(X')]. \tag{4.27}$$

我們還注意到，反過來，v^π 和 v^* 也都可以分別由它們對應的 Q 函數 q^π 和 q^* 表示。也就是說，

$$v^\pi(x) = \mathbb{E}_{U\sim\pi(\cdot|x)}[q^\pi(x,U)]$$

以及

$$v^*(x) = \sup_{\pi\in\Pi} \mathbb{E}_{U\sim\pi(\cdot|x)}[q^*(x,U)]. \tag{4.28}$$

如果對每個 $x \in \mathcal{X}$，(4.28) 中的上確界都能夠在 Π 中取到（例如，當 $\mathcal{U}$ 是有限集時），那麼 (4.28) 就可化為

$$v^*(x) = \max_{u\in\mathcal{U}} q^*(x,u).$$

4.1.4 策略最優性與改進理論

現在我們已經準備好正式定義強化學習中最優策略的含義。

定義 4.1.14 (最優策略，optimal policy)**.** 如果 $\pi^* \in \Pi$ 滿足

$$\mathbb{E}_{U\sim\pi^*(\cdot|x)}[q^*(x,U)] = v^*(x) \tag{4.29}$$

對每個 $x \in \mathcal{X}$ 都成立，其中 v^* 是 (4.11) 中定義的 T^* 的唯一不動點，而 q^* 定義於 (4.27)，則稱 π^* 為一個**最優策略**。

顯然，如果 π^* 相對於 v^* 是貪婪的，那麼 π^* 就是一個最優策略，因為 (4.29) 意味著 $T^{\pi^*}v^* = T^*v^* = v^*$。我們將在下面的一個定理中將這一點表述為一個正式結論。

我們還定義如下一個特殊函數 $\bar{v} \in B(\mathcal{X})$：對每個 $x \in \mathcal{X}$，

$$\bar{v}(x) := \sup_{\pi\in\Pi} v^\pi(x). \tag{4.30}$$

注意，我們並不知道 $\bar{v}$ 是否是某個策略 $\pi \in \Pi$ 的價值函數。不過，由定義 (4.30) 可知，對任意 $\pi \in \Pi$ 都有

$$v^\pi \le \bar{v}. \tag{4.31}$$

我們還可以定義與 $\bar{v}$ 對應的 Q 函數為

$$\bar{q}(x,u) := r(x,u) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,u)}[\bar{v}(X')]. \tag{4.32}$$

由 (4.30) 可見，

$$\bar{q}(x,u) = \sup_{\pi\in\Pi} q^{\pi}(x,u). \tag{4.33}$$

此外，關於 $\bar{v}$ 及其與 v^* 的關係，我們有如下重要引理。

引理 4.1.15. *如果存在一個平穩策略 $\bar{\pi}\in\Pi$ 相對於 $\bar{v}$ 是貪婪的，那麼*

$$\bar{v} = v^{\bar{\pi}} \le v^*, \tag{4.34}$$

其中 $v^{\bar{\pi}}$ 是與 $\bar{\pi}$ 相關的 $T^{\bar{\pi}}$ 的唯一不動點，而 v^* 是 T^* 的唯一不動點。

证明. 對任意 $\hat{\pi}\in\Pi$，記其對應的 Bellman 算子為 $T^{\hat{\pi}}$，其唯一不動點為 $v^{\hat{\pi}}$，滿足 $T^{\hat{\pi}}v^{\hat{\pi}} = v^{\hat{\pi}}$。

現在對任意 $x\in\mathcal{X}$，有

$$\begin{aligned}
v^{\hat{\pi}}(x) &= T^{\hat{\pi}}v^{\hat{\pi}}(x)\\
&= \mathbb{E}_{U\sim\hat{\pi}(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v^{\hat{\pi}}(X')]\Big]\\
&\le \mathbb{E}_{U\sim\hat{\pi}(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\bar{v}(X')]\Big]\\
&\le \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\bar{v}(X')]\Big]\\
&= T^*\bar{v}(x)\\
&= T^{\bar{\pi}}\bar{v}(x),
\end{aligned}$$

其中第二個等號由定義 4.1.3 對策略 $\hat{\pi}$ 得到；第一個不等號由 (4.31) 得到；倒數第二個等號由 T^* 的定義得到；最後一個等號成立是因為 $\bar{\pi}$ 相對於 $\bar{v}$ 是貪婪的，因此 $T^{\bar{\pi}}\bar{v} = T^*\bar{v}$。

由於上面所取的 $\hat{\pi}\in\Pi$ 和 $x\in\mathcal{X}$ 都是任意的，所以由 (4.31) 中 $\bar{v}(x) = \sup_{\pi\in\Pi} v^{\pi}(x)$ 的定義可知，

$$\bar{v} \le T^{\bar{\pi}}\bar{v}, \tag{4.35}$$

對 (4.35) 兩邊同時施加 $T^{\bar{\pi}}$，並利用定理 4.1.10(ii)，得到

$$T^{\bar{\pi}}\bar{v} \le (T^{\bar{\pi}})^2\bar{v}. \tag{4.36}$$

結合 (4.35) 和 (4.36)，可得 $\bar{v} \le (T^{\bar{\pi}})^2\bar{v}$。繼續對 (4.36) 兩邊施加 $T^{\bar{\pi}}$，並結合 (4.35)，就得到

$$\bar{v} \le T^{\bar{\pi}}\bar{v} \le \cdots \le (T^{\bar{\pi}})^k\bar{v}$$

對任意 k 都成立。令 $k \to \infty$，並回憶 $(T^{\bar{\pi}})^k \bar{v} \to v^{\bar{\pi}}$ 在 ∞-範數下收斂，即得

$$\bar{v} \leq v^{\bar{\pi}}. \tag{4.37}$$

由於 $\bar{\pi} \in \Pi$，由 (4.31) 知道 $v^{\bar{\pi}} \leq \bar{v}$。因此，$v^{\bar{\pi}} = \bar{v}$。

現在證明 $v^{\bar{\pi}} \leq v^*$。注意到

$$v^{\bar{\pi}} = T^{\bar{\pi}} v^{\bar{\pi}} \leq T^* v^{\bar{\pi}}, \tag{4.38}$$

其中不等號由定理 4.1.10(i) 得到。對上式兩邊施加 T^*，得到

$$T^* v^{\bar{\pi}} \leq (T^*)^2 v^{\bar{\pi}}. \tag{4.39}$$

繼續對 (4.39) 兩邊施加 T^*，並利用定理 4.1.10(iii)，得到

$$v^{\bar{\pi}} \leq T^* v^{\bar{\pi}} \leq \cdots \leq (T^*)^k v^{\bar{\pi}}$$

對任意 k 都成立，其中第一個不等號由 (4.38) 得到。令 $k \to \infty$，並回憶 $(T^*)^k \bar{v} \to v^*$ 在 ∞-範數下收斂，即得

$$v^{\bar{\pi}} \leq v^*. \tag{4.40}$$

綜上，我們得到 $\bar{v} = v^{\bar{\pi}} \leq v^*$。 □

定理 4.1.16 (策略改進定理，Policy Improvement Theorem)**.** *以下結論成立：*

(i) *對任意 $\pi_0 \in \Pi$，設 π 相對於 v^{π_0} 是貪婪的（其中 v^{π_0} 是與 π_0 相關的 Bellman 算子 T^{π_0} 的唯一不動點），即 $T^\pi v^{\pi_0} = T^* v^{\pi_0}$，則*

$$v^{\pi_0} \leq v^{\pi}. \tag{4.41}$$

*我們稱 π 是**對 π_0 的改進**。*

(ii) *如果另外對某個 $x \in \mathcal{X}$ 有 $v^{\pi_0}(x) < T^* v^{\pi_0}(x)$，那麼 $v^{\pi_0}(x) < v^{\pi}(x)$。在這種情況下，我們說 π **在 x 處嚴格改進了** π_0。*

(iii) *如果 $T^* v^{\pi^*} = v^{\pi^*}$，那麼 π^* 是一個最優策略。*

证明. 對 (i)，我們有

$$v^{\pi_0} = T^{\pi_0} v^{\pi_0} \le T^* v^{\pi_0} = T^{\pi} v^{\pi_0},$$

其中不等號由定理 4.1.10(i) 得到，最後一個等號成立是因為 π 相對於 v^{π_0} 是貪婪的。繼續對兩邊施加 T^{π}，並利用定理 4.1.10(ii)，可得

$$v^{\pi_0} \le T^{\pi} v^{\pi_0} \le \cdots \le (T^{\pi})^k v^{\pi_0}$$

對所有 k 都成立。令 $k \to \infty$，並注意到 $(T^{\pi})^k v^{\pi_0} \to v^{\pi}$ 在 ∞-範數下收斂，即得 $v^{\pi_0} \le v^{\pi}$。

對 (ii)，我們沿用 (i) 中相同的推導，得到

$$v^{\pi_0}(x) < T^* v^{\pi_0}(x) \le (T^{\pi})^k v^{\pi_0}(x)$$

對該 x 的所有 k 都成立。令 $k \to \infty$，可得 $v^{\pi_0}(x) < T^* v^{\pi_0}(x) \le v^{\pi}(x)$。

對 (iii)，由於 $T^* v^{\pi^*} = v^{\pi^*}$，可知 $v^{\pi^*} = v^*$，即 T^* 的唯一不動點。注意到 v^{π^*} 也是 T^{π^*} 的不動點，因此有 $T^{\pi^*} v^* = v^*$，即

$$\mathbb{E}_{U \sim \pi^*(\cdot|x)}[q^*(x, U)] = v^*(x)$$

對每個 $x \in \mathcal{X}$ 都成立。由定義 4.1.14，可知 π^* 是一個最優策略。 □

現在我們對最優策略的形式有了更清晰的認識。下面的定理將策略的最優性條件、它與 $\bar{v}$ 的關係，以及貪婪性質聯繫在一起。

定理 4.1.17. 以下命題彼此等價:

(i) π^* 是一個最優策略，即 $\mathbb{E}_{U \sim \pi^*(\cdot|x)}[q^*(x, U)] = v^*(x)$。

(ii) $v^* = v^{\pi^*} = \bar{v}$。

(iii) π^* 相對於 v^* 是貪婪的。

证明. 如果 (i) 成立，則有 $T^{\pi^*} v^* = v^*$，這意味著 v^* 是 T^{π^*} 的唯一不動點。因此 $v^* = v^{\pi^*}$。再由 (4.31) 知道 $v^{\pi^*} \le \bar{v}$。結合引理 4.1.15 中給出的 $\bar{v} \le v^*$，便得到 (ii)。

如果 (ii) 成立，那麼有

$$T^{\pi^*} v^* = T^{\pi^*} v^{\pi^*} = v^{\pi^*} = v^* = T^* v^*,$$

其中第一個和第三個等號由 (ii) 得到，第二個等號成立是因為 π^* 是 T^{π^*} 的不動點，最後一個等號成立是因為 v^* 是 T^* 的不動點。

如果 (iii) 成立，那麼

$$T^{\pi^*}v^* = T^*v^* = v^*,$$

其中第一個等號成立是因為 π^* 相對於 v^* 是貪婪的。根據定義 4.1.14，這說明 π^* 是一個最優策略。 □

推論 4.1.18. *如果 π^* 是一個最優策略，那麼 π^* 相對於 $\bar{v}$ 是貪婪的。*

证明. 如果 π^* 是一個最優策略，由定理 4.1.17 可知 $v^{\pi^*} = \bar{v} = v^*$。因此，

$$T^{\pi^*}\bar{v} = T^{\pi^*}v^{\pi^*} = v^{\pi^*} = v^* = T^*v^* = T^*\bar{v},$$

這表明 π^* 相對於 $\bar{v}$ 是貪婪的。 □

總結 現在我們對本節所建立的理論結果作一個簡要總結。在具有無限時間（$T = \infty$）和折扣因子 $\gamma \in (0,1)$ 的標準設定下，我們知道每一個平穩策略 $\pi \in \Pi$ 都會產生一個 Bellman 算子 $T^\pi : B(\mathcal{X}) \to B(\mathcal{X})$，它具有唯一不動點 v^π。這裡 $v^\pi : \mathcal{X} \to \mathbb{R}$ 是策略 π 的狀態價值函數，意味著 $v^\pi(x)$ 是當智能體處於狀態 $x \in \mathcal{X}$ 時未來總獎勵的期望。Q 函數（狀態–動作價值函數）$q^\pi(x,u)$ 則表示當智能體位於狀態 $x \in \mathcal{X}$ 並採取特定動作 $u \in \mathcal{U}$ 時未來總獎勵的期望。

我們還有一個概念上“逐點最優”的價值函數 $\bar{v} : \mathcal{X} \to \mathbb{R}$，它在每個 $x \in \mathcal{X}$ 處都定義為所有平穩策略 Π 上價值函數的上確界。然而，$\bar{v}$ 本身並不能說明它是否對應於 Π 中的某個策略，因此也就不知道它是否對應於某個 Bellman 算子。

與此同時，我們還有一個概念上“最優”的 Bellman 算子 $T^* : B(\mathcal{X}) \to B(\mathcal{X})$。它有唯一不動點 $v^* \in B(\mathcal{X})$，因此也對應一個 Q 函數 q^*，但並不總能保證 v^* 是 Π 中某個策略的狀態價值函數。然而，如果對每個 $x \in \mathcal{X}$，函數

$$r(x,\cdot) + \gamma \int p(x'|x,\cdot)v^*(x')\,dx' \quad : \quad \mathcal{U} \to \mathbb{R}$$

的最大值能夠在 $\mathcal{U}$ 中取得，那麼 v^* 就存在一個貪婪策略 $\pi^* \in \Pi$。（我們前面提到過，如果 $\mathcal{U}$ 是有限集，或者 $\mathcal{U}$ 是緊集且上面定義的實值函數在 $\mathcal{U}$ 上連續，那麼最大值就可以在 $\mathcal{U}$ 中取得。）更重要的是，π^* 的 Bellman 算子正是

T^*，其唯一不動點就是 v^*，而 $\bar{v}$ 也正是 v^*。這意味著根據定義，π^* 是一個最優策略，而 v^* 就是最優狀態價值函數。

4.2 強化學習算法

強化學習算法可以按照許多不同方式進行分類。這是因為強化學習算法所應用的領域和問題設定非常多樣。在本節中，我們按照該領域中最典型的一種分類方式，將強化學習算法分為兩類：基於模型（model-based）的方法和無模型（model-free）的方法。一般而言，基於模型的方法考慮這樣的問題：我們擁有能夠（近似）準確描述由環境決定的獎勵和狀態轉移概率的模型；而在無模型的方法中，我們並不具備這些信息，因此需要利用智能體與環境交互所生成的數據來求解問題。我們將在下面兩個小節中分別詳細介紹這兩類算法。

4.2.1 基於模型的強化學習算法

基於模型的強化學習算法是指應用於如下問題的算法：我們已知以下信息

- 獎勵函數 $r(x,u)$，或者定義在 $\mathbb{R}$ 上的條件獎勵概率分佈 $p_R(\cdot|x,u)$，從而我們能夠計算獎勵的期望值

$$\mathbb{E}_{R\sim p_R(\cdot|x,u)}[R] = \int r p_R(r|x,u)\,dr,$$

  對每個狀態–動作對 $(x,u)\in\mathcal{X}\times\mathcal{U}$ 都成立。為了便於討論，在下面討論基於模型算法時，我們假設函數 r 是已知的。

- 對每個狀態–動作對 $(x,u)\in\mathcal{X}\times\mathcal{U}$，定義在 $\mathcal{X}$ 上的狀態轉移概率

$$p(\,\cdot\,|\,x,u).$$

  它表示在當前狀態為 x 且執行動作 u 時，下一個狀態的概率分佈。

注意，上述信息是由問題設定中的環境決定的。具體來說，我們準確知道：在當前狀態為 x 且執行動作 u 時，可以獲得多少獎勵，以及下一個狀態是什麼（或者下一個狀態的概率分佈是什麼）。這就是我們稱這類強化學習問題為**基於模型**的原因。

當我們處理一個基於模型的強化學習問題時，通常可以直接求出該問題中的最優策略，而不需要讓智能體與環境交互並收集必要的數據來估計這些量。

在許多這類情形中，我們可以使用動態規劃或類似思想來尋找最優策略。下面我們將詳細介紹一些常用的基於模型的強化學習算法。這些算法高度依賴於我們在第 4.1 節中討論的 Markov 決策過程和 Bellman 算子的性質。

策略迭代

策略迭代（policy iteration）是一種迭代方案，可以為基於模型的強化學習生成一列持續改進的策略。在一定條件下（稍後將具體說明），這列策略可以被證明收斂到最優策略。策略迭代算法背後的理論基礎是策略改進定理（定理 4.1.16）。

策略迭代算法的典型過程如下。我們任選一個初始猜測 $v_0 \in B(\mathcal{X})$（它不一定是某個價值函數），並找到一個相對於 v_0 是貪婪的策略 π_0。現在，在第 k 次迭代中（其中任意 $k \geq 0$），第一步是求當前策略 π_k 的價值函數 v^{π_k}，這一步稱為**策略評估**（policy evaluation），因為我們試圖求得當前策略 π_k 所對應的價值函數 v^{π_k}；然後在第二步中，這一步稱為**策略改進**（policy improvement），我們尋找一個相對於 v^{π_k} 是貪婪的策略 π_{k+1}，並且可以證明 π_{k+1} 是對 π_k 的改進。隨後將迭代計數器 k 加 1，並繼續下一次迭代。我們在算法 4.2.1 中總結了策略迭代算法。

Algorithm 4.2.1 基於模型強化學習的策略迭代

Input: $v_0 \in B(\mathcal{X})$ 且 $\pi_0 \in \Pi$ 相對於 v_0 貪婪，$k = 0$。

1: **repeat**
2: （策略評估）求 v^{π_k}，使其滿足 $T^{\pi_k} v^{\pi_k} = v^{\pi_k}$。
3: （策略改進）求 $\pi_{k+1} \in \Pi$，使其相對於 v^{π_k} 貪婪。
4: $k \leftarrow k + 1$
5: **until** 收斂。

Output: π_k。

當我們執行策略迭代算法（算法 4.2.1）時，會得到一列策略

$$\pi_0,\ \pi_1,\ \ldots,\ \pi_k,\ \ldots. \tag{4.42}$$

根據貪婪策略的定義（定義 4.1.11），我們知道策略改進步驟會產生 $T^{\pi_{k+1}} v^{\pi_k} = T^* v^{\pi_k}$。此外，由策略改進定理（定理 4.1.16(i)），有

$$v^{\pi_k} \leq v^{\pi_{k+1}}$$

對每個 $k \geq 0$ 都成立。

如果狀態空間 $\mathcal{X}$ 和動作空間 $\mathcal{U}$ 都是有限集合，那麼我們知道貪婪策略的個數也是有限的，因此迭代序列 (4.42) 必將在有限步後停止，即對某個 k 有 $\pi_k = \pi_{k+1}$。此時，我們有

$$v^{\pi_k} = T^{\pi_k} v^{\pi_k} = T^{\pi_{k+1}} v^{\pi_k} = T^* v^{\pi_k}$$

其中第一個等號是因為 v^{π_k} 是 T^{π_k} 的不動點，第二個等號由 $\pi_k = \pi_{k+1}$ 得到。因此，由於 v^* 是 T^* 的唯一極小化子，我們有 $v^{\pi_k} = v^*$。再由定理 4.1.17，可知 π_k 是一個最優策略。

現在我們需要考慮策略迭代算法（算法 4.2.1）的實際操作。更具體地說，我們需要找到求解策略評估（第 2 行）和策略改進（第 3 行）兩個子問題的方法。根據問題規模的不同，我們需要採用不同的方法來解決這些子問題，下面我們將分別說明。

小規模問題 所謂小規模問題，是指狀態空間 $\mathcal{X}$ 和動作空間 $\mathcal{U}$ 都是較小的有限集合（例如元素個數在 10^3 左右）。在這種情況下，對任意策略 π，我們都可以計算狀態獎勵

$$r^\pi(x) := \mathbb{E}_{U\sim\pi(\cdot|x)}[r(x,U)] \tag{4.43}$$

以及狀態到狀態的轉移概率

$$p^\pi(x'|x) := \sum_{u\in\mathcal{U}} \pi(u|x) p(x'|x,u) \tag{4.44}$$

對所有 $x, x' \in \mathcal{X}$ 都成立。記 $n = |\mathcal{X}|$ 為 $\mathcal{X}$ 中狀態的數量。則定義在 $\mathcal{X}$ 上的 v^π 與 r^π 都可以表示為 n 維列向量，而轉移概率則可以組成一個 $n \times n$ 矩陣 $P^\pi = [p^\pi(x'|x)]_{x,x'\in\mathcal{X}}$。因此，$v^\pi$ 的 Bellman 方程

$$v^\pi(x) = T^\pi v^\pi(x) = \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\in p(\cdot|x,U)}[v^\pi(X')]\Big]$$

對所有 $x \in \mathcal{X}$ 可以寫成如下線性系統

$$v^\pi = r^\pi + \gamma P^\pi v^\pi, \tag{4.45}$$

其中 I_n 是 $n \times n$ 單位矩陣。

由 Gershgorin 圓盤定理可知，P^π 的所有特徵值都位於 $\mathbb{R}$ 中如下集合內：

$$\bigcup_{x\in\mathcal{X}} D\Big(p^\pi(x|x), \sum_{x'\neq x} p^\pi(x'|x)\Big) \tag{4.46}$$

其中 $D(c,r) := [c-r, c+r]$ 是 $\mathbb{R}$ 上的閉區間。由於對所有 $x, x' \in \mathcal{X}$ 都有 $\sum_{x'\in\mathcal{X}} p^\pi(x'|x) = 1$ 且 $p^\pi(x'|x) \geq 0$，我們知道集合 (4.46) 包含在 $[-1,1]$ 中。因此，P^π 所有特徵值的最大模不超過 1。於是只要 $\gamma < 1$，就有 $I_n - \gamma P^\pi$ 可逆，從而 (4.45) 有唯一解

$$v^\pi = (I_n - \gamma P^\pi)^{-1} r^\pi. \tag{4.47}$$

在實際中，解 (4.47) 可以使用已有的數值線性方程組求解器計算。一般情況下，其計算代價為 $O(n^3)$（如果 P^π 具有特殊結構，則代價可能更低）。這就是每次迭代中策略評估（算法 4.2.1 第 2 行）的求解過程。

對於策略改進（算法 4.2.1 第 3 行），我們可以直接把 π_{k+1} 設為如下確定性貪婪策略：對任意 $x \in \mathcal{X}$，令

$$\pi_{k+1}(u|x) = \begin{cases} 1, & \text{如果 } u = \arg\max_{u'\in\mathcal{U}} q^{\pi_k}(x,u'), \\ 0, & \text{否則}, \end{cases} \tag{4.48}$$

其中 q^{π_k} 是與 π_k 對應的 Q 函數，定義見 (4.26)。如果 $q^{\pi_k}(x,\cdot)$ 在 $\mathcal{U}$ 中有多個極大值點，則在 (4.48) 中任選其一即可。

中等規模問題 在這種情況下，我們通常有狀態空間大小 $|\mathcal{X}| \approx 10^6$ 或類似規模。此時，直接求解 (4.45) 一般代價很高。一個可行方法是：對給定策略 π，通過固定點迭代 $v_{j+1} = T^\pi v_j$（$j \geq 0$）並從某個初值 v_0 出發，來求解策略評估子問題（算法 4.2.1 第 2 行）。由於 T^π 是 γ-壓縮映射，因此根據定理 4.1.8，

$$\|v_j - v^\pi\|_\infty \leq \gamma^j \|v_0 - v^\pi\|_\infty$$

對所有 j 都成立。因此，對任意 π，我們都可以以 γ 的線性速率逼近 v^π。在實際中，我們可以在 $\|v_{j+1} - v_j\|_\infty < \varepsilon_{\text{tol}}$ 時終止固定點迭代，其中 $\varepsilon_{\text{tol}} > 0$ 是某個預先給定的容差（用戶定義的超參數），並認為策略評估（算法 4.2.1 第 2 行）已被近似求解，然後轉入策略改進（算法 4.2.1 第 3 行）。

為了求解策略改進子問題，我們可以像 (4.48) 那樣，把 π_{k+1} 設為確定性貪婪策略。當 $\mathcal{U}$ 是一個規模較小或中等的有限集合時，這在計算上是可行的。

大規模問題 所謂大規模問題，是指那些 $\mathcal{X}$ 含有 10^{10} 個狀態，甚至遠遠更多；或者 $\mathcal{X}$ 是 $\mathbb{R}^d$ 中的連續區域，且 $d \geq 5$，使得哪怕進行適度的空間離散化也會得到超過 10^{10} 個狀態（這也是維數災難問題）。在實際中，這類問題的規模往往更大，以至於在現代計算機內存中存儲價值函數或策略都是不可行的。因此，我們通常用函數逼近器來表示價值函數和策略。在這些情形下，深度神經網絡被證明是非常有效的函數逼近器。

更具體地說，在策略評估步驟（算法 4.2.1 第 2 行）中，我們有當前估計的策略 π_k。為了求出其價值函數 $v^{\pi_k}: \mathcal{X} \to \mathbb{R}$，我們把它參數化為一個深度神經網絡，並通過求解以下最小化問題來訓練它：

$$v^{\pi_k} = \arg\min_{v} \frac{1}{2} \int |v(x) - T^{\pi_k} v(x)|^2 \, dx \tag{4.49}$$

其中 T^{π_k} 是與策略 π_k 對應的 Bellman 算子，定義見 (4.9)。最小化問題 (4.49) 是關於 v 的網絡參數進行的。由於 T^{π_k} 是 γ-壓縮映射，它有唯一不動點，因此問題 (4.49) 有唯一極小化子，而這個極小化子正是所需的 v^{π_k}。

策略改進步驟（算法 4.2.1 第 3 行）要求尋找一個相對於 v^{π_k} 貪婪的策略 π_{k+1}，如 (4.48) 所示。這歸結為：對每個狀態 $x \in \mathcal{X}$，求解一個關於貪婪動作 $u \in \mathcal{U}$ 的極大化問題。這一問題的難點取決於這類極大化問題是否容易求解，而這會隨著具體應用問題而變化。因此，用來求解該策略改進子問題的方法往往需要具體問題具體分析。

儘管如此，仍有一些應用中狀態空間 $\mathcal{X}$ 極其巨大，而動作空間卻很小。例如，在圍棋中，狀態空間 $\mathcal{X}$ 包含所有可能的棋盤局面，且 $|\mathcal{X}| \approx 10^{170}$，而 $|\mathcal{U}|$ 僅有幾百個（在很多狀態 x 下，$\mathcal{U}$ 中合法動作的數量通常更少）。此外，從 (x, u) 到下一個狀態 x' 的轉移是確定性的：一旦在狀態 x（當前棋盤局面）下執行動作 u（在棋盤上落子），下一個狀態 x' 就完全確定。在這種情況下，(4.48) 中的極大化問題對於任意給定的 x 都可以通過計算機進行簡單搜索而容易求解。

然而，即便 $|\mathcal{U}|$ 很小，策略改進子問題仍然具有挑戰性：狀態空間 $\mathcal{X}$ 依舊過於龐大，不可能對每個 $x \in \mathcal{X}$ 都求出一個動作 u。在實踐中，我們可以將策略 π（即在給定任意狀態 $x \in \mathcal{X}$ 時，定義在 $\mathcal{U}$ 上的條件概率）參數化為一個深度神經網絡（稱為策略網絡），並與參數化價值函數的深度神經網絡（稱為價值網絡）一起訓練。例如，給定當前價值函數 v^{π}，我們可以從 $\mathcal{X}$ 中採樣 M 個狀態 $\{x_i : i \in [M]\}$，並對每個採樣狀態 x_i 求解

$$u_i = \arg\max_{u \in \mathcal{U}} \Big\{ r(x_i, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i, u)} [v^{\pi}(X')] \Big\}.$$

Algorithm 4.2.2 基於模型強化學習的價值迭代

Input: $v_0 \in B(\mathcal{X})$, $k = 0$。
1: **repeat**
2: （價值改進）設 $v_{k+1} = T^* v_k$。
3: $k \leftarrow k + 1$
4: **until** 收斂。
5: 找到一個相對於 v_k 貪婪的策略 π。
Output: π。

然後利用這些樣本對 $\{(x_i, u_i) : i \in [M]\}$ 來訓練策略網絡。這當然並不總是簡單且易於實際實現的。但它展示了一種處理策略改進步驟的通用思路。我們還將在第 4.2.2 節中介紹若干實際算法，以進一步應對這一問題。

價值迭代

價值迭代是另一種用於求解基於模型強化學習問題的迭代方案。價值迭代的思想是生成一列價值函數，使其直接逼近最優價值函數 v^*。當找到價值函數 v^* 後，我們就可以直接得到最優策略，即相對於 v^* 貪婪的那個策略。相比之下，策略迭代則需要在迭代過程中生成一列持續改進的策略。

由於 T^* 是 γ-壓縮映射，我們知道 v^* 是其唯一不動點，並且可以從任意初始值 v_0 出發，通過固定點迭代 $v_{k+1} = T^* v_k$ 來逼近它。價值迭代算法本質上正是在實現這一固定點迭代方案。我們在算法 4.2.2 中總結了價值迭代算法。

為了實現價值改進步驟（算法 4.2.2 第 2 行），我們對每個 x 和 k 設定

$$v_{k+1}(x) = \max_{u \in \mathcal{U}} \Big\{ r(x, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v_k(X')] \Big\}. \tag{4.50}$$

對於小規模到中等規模的問題，這通常是可行的。然而，對於大規模問題，我們需要使用函數逼近器（例如深度神經網絡）來逼近 v_k。具體而言，對每個給定的 v_k，我們可以從 $\mathcal{X}$ 中採樣 M 個狀態 $\{x_i : i \in [m]\}$，並通過求解

$$u_i = \arg\max_{u \in \mathcal{U}} \Big\{ r(x_i, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i,u)}[v_k(X')] \Big\}$$

來得到對應的 u_i，對每個 $i \in [M]$ 都如此。然後利用這些樣本對 $\{(x_i, u_i) : i \in$

$[M]\}$ 來訓練 v_{k+1}：

$$v_{k+1} = \arg\min_{v} \frac{1}{2M} \sum_{i=1}^{M} \Big| v(x_i) - \Big(r(x_i, u_i) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i,u_i)}[v_k(X')] \Big) \Big|^2,$$

其中最小化是相對於 v 的網絡參數進行的。由定理 4.1.8 可知，v_k 在 ∞-範數下以 γ 的線性速率收斂到 v^*。當 v_k 收斂後，我們可以像 (4.48) 中那樣，選擇一個相對於輸出 v_k 貪婪的策略 π。注意，只有在價值迭代算法（算法 4.2.2）的這一時刻，我們才會計算一個策略，而它是對最優策略 π^* 的一個近似。

4.2.2 無模型強化學習算法

在許多現實世界的應用中，由於環境或問題設定的複雜性，我們可能並不具備關於獎勵函數 r（或獎勵概率 $p_R(\cdot|x,u)$）以及狀態轉移概率 $p(\cdot|x,u)$ 的精確模型。在這些情況下，我們需要讓智能體執行動作並與環境進行交互。智能體可以在這些交互過程中收集數據，利用這些數據我們可以估計獎勵和狀態轉移概率，並找到能夠產生最大總回報的最優策略。這類方法稱為**無模型**強化學習算法。

在本小節中，我們將介紹一系列無模型強化學習算法。這些方法之間有許多相似之處，但由於問題設定和數據可獲得性的多樣性，它們之間也存在差異。儘管如此，我們仍應牢記第 4.1 節中建立的基本結果，因為這些結果有助於理解這些算法。

Monte Carlo 方法

原始的 Monte Carlo 方法是一種無模型方法，常用於小規模到中等規模的問題。回憶一下，對於給定策略 $\pi \in \Pi$，其價值函數定義為

$$v^\pi(x) = \mathbb{E}_{\pi,p}\Big[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \Big| X_0 = x \Big],$$

對每個 $x \in \mathcal{X}$ 都成立，其中 R_t 是在時間 t 收到的獎勵。在中等規模以內的問題中，$\mathcal{X}$ 是一個大小約為 10^6 或類似規模的有限集合，因此價值函數 $v^\pi : \mathcal{X} \to \mathbb{R}$ 可以作為一個**表格**（即大小為 $|\mathcal{X}|$ 的數組）存儲在計算機內存中，並根據觀測數據方便地進行更新，具體如下所述。

由於在無模型設定中我們並不知道獎勵函數 r 和轉移概率 p，因此需要讓智能體在環境中執行動作並收集樣本數據，以估計這些量。在這種情況下，設

我們讓智能體按照當前估計的策略 π 與環境交互，則會觀測到一個由該 π 誘導的 Markov 決策過程：

$$X_0,\ U_0,\ R_1,\ \ldots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ U_{t+1},\ldots \tag{4.51}$$

為了擴展價值函數 v^π 的概念，我們定義**總回報**為

$$G_t^\pi(x) := \sum_{\tau=t}^{\infty} \gamma^{\tau-t} R_{\tau+1}, \tag{4.52}$$

它表示從時間 t 開始且 $X_t = x$，在 t 之後所獲得的總獎勵，注意這裡相當於假設 Markov 決策過程是從時間 t 而不是從 0 開始的。我們還定義其期望總回報為

$$g_t^\pi(x) := \mathbb{E}_{\pi,p}\Big[G_t^\pi(x)\Big|X_t = x\Big]. \tag{4.53}$$

於是，對任意 t，$g_t^\pi(x)$ 都可以在乘以適當縮放因子 γ^t 後用於估計 $v^\pi(x)$。在 Monte Carlo 方法中，我們需要利用 $g_t^\pi(x)$ 的樣本，也即 $G_t^\pi(x)$ 的樣本，來逼近價值函數 v^π。

更具體地說，我們讓智能體按照當前策略 π 行動並在環境中執行動作。然後我們觀測到 Markov 獎勵過程的一個樣本軌跡，稱為一個**回合**（episode）。一個回合只是採樣得到的 Markov 決策過程 (4.51) 中去掉動作後的片段：

$$x_0,\ r_1,\ x_1,\ r_2,\ \ldots,\ x_t,\ r_{t+1},\ x_{t+1},\ \ldots, x_T,\ r_{T+1}, \tag{4.54}$$

它在實踐中會在某個有限時間 T 終止。如果我們在時間 t 觀測到 $x_t = x$，那麼就把

$$\sum_{\tau=t}^{T} \gamma^{\tau-t} r_{\tau+1} \tag{4.55}$$

視為 $G_t^\pi(x)$ 的一個樣本。注意，對於某個 $x \in \mathcal{X}$，在一個回合中，我們可能得到零個、一個或多個類似 (4.55) 的樣本，這取決於狀態 x 在該回閣中出現的次數。在實踐中，我們也可以運行多個回合，並對類似 (4.55) 的樣本取平均。簡化起見，我們仍將這個平均值記為 $G_t^\pi(x)$（這裡 t 用來表示總回報是從時間 t 到達狀態 x 後開始累積的；但由於 $G_t^\pi(x)$ 對任意 t 都是 $v^\pi(x)$ 的估計，因此 t 的具體數值並不重要）。在這種情況下，Monte Carlo 方法通過如下方式更新 $v^\pi(x)$：

$$v^\pi(x) \leftarrow v^\pi(x) + \alpha(G_t^\pi(x) - v^\pi(x)), \tag{4.56}$$

其中 $\alpha > 0$ 為步長。

Monte Carlo 方法已知在回合長度趨於無窮時可以給出價值函數 v^π 的無偏估計，但該估計的方差通常較大。得到 v^π 的估計之後，我們可以找到一個相對於 v^π 貪婪的策略 π'，然後使用更新後的策略 π' 再次重複 Monte Carlo 方法的過程，以估計新的價值函數 $v^{\pi'}$，如此往復。因此，Monte Carlo 方法會像價值迭代算法（算法 4.2.2）那樣生成一列不斷改進的策略，只不過它不需要知道獎勵函數 r 和轉移概率 p。

時序差分方法

時序差分（temporal difference，TD）是另一種與 Monte Carlo 方法密切相關的無模型方法。二者的區別在於：對於當前策略 π，TD 方法中的 v^π 表格會在遍歷形如 (4.54) 的採樣 Markov 獎勵過程中（一個回合內）持續更新；相比之下，Monte Carlo 方法只在每個回合結束時才更新該表格。

更具體地說，設 $x_t = x$，並且智能體獲得獎勵 r_{t+1} 後轉移到狀態 x_{t+1}，則我們可以通過如下方式更新 $v^\pi(x)$：

$$v^\pi(x) \leftarrow v^\pi(x) + \alpha\Big(\underbrace{\overbrace{r_{t+1} + \gamma v^\pi(x_{t+1})}^{\text{TD 目标}} - v^\pi(x)}_{\text{TD 误差}}\Big) \tag{4.57}$$

其中 $\alpha > 0$ 是步長。正如我們所見，TD 方法 (4.57) 使用所謂的 **TD 目標** $r_{t+1} + \gamma v^\pi(x_{t+1})$ 來替代 Monte Carlo 方法 (4.56) 中的 $G_t^\pi(x)$。其目標是使 (4.57) 中的 **TD 誤差**變為零，從而得到 v^π 的 Bellman 方程 (4.7)。注意，TD 方法使用當前估計的一部分，即 $v^\pi(x_{t+1})$，來估計值 $v^\pi(x)$。這一策略稱為**自舉**（bootstrapping）。與 Monte Carlo 方法相比，TD 方法得到的 v^π 估計是有偏的，但其方差通常更小。

上面討論的 TD 方法通常簡稱為 TD(0) 方法。它可以推廣為：對 $n =$

$1, 2, \ldots$，使用 n 步總回報作為 TD 目標。為此，我們記

$$
\begin{aligned}
G_{t,1}^{\pi}(x) &:= R_{t+1} + \gamma v^{\pi}(X_{t+1}), \\
G_{t,2}^{\pi}(x) &:= R_{t+1} + \gamma R_{t+2} + \gamma^2 v^{\pi}(X_{t+2}), \\
&\vdots \\
G_{t,n}^{\pi}(x) &:= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n v^{\pi}(X_{t+n}), \\
&\vdots \\
G_{t,\infty}^{\pi}(x) &:= G_t^{\pi}, \qquad \text{（假设当 } t > T+1 \text{ 时 } R_t = 0\text{）}.
\end{aligned}
$$

然後對任意 $\lambda \in [0,1]$，定義

$$
G_t^{\pi,\lambda}(x) := \sum_{n=1}^{\infty} (1-\lambda)\lambda^{n-1} G_{t,n}^{\pi}(x). \tag{4.58}
$$

注意，$(1-\lambda)\lambda^{n-1}$ 在和式 (4.58) 中起到權重的作用，並且總權重滿足 $\sum_{n=1}^{\infty}(1-\lambda)\lambda^{n-1} = 1$。於是，作為 TD(0) 方法推廣的 TD($\lambda$) 方法通過如下方式更新 $v^{\pi}(x)$：

$$
v^{\pi}(x) \leftarrow v^{\pi}(x) + \alpha(G_t^{\pi,\lambda}(x) - v^{\pi}(x)). \tag{4.59}
$$

事實上，TD(λ) 方法正是 TD(0) 方法與 Monte Carlo 方法之間的一種插值：當 $\lambda = 0$ 時，有

$$
G_t^{\pi,0}(x) = G_{t,0}^{\pi}(x) = R_{t+1} + \gamma v^{\pi}(X_{t+1})
$$

因此 TD(λ) 恰好就是 TD(0) 方法；當 $\lambda \to 1$ 時，(4.58) 中所有權重都趨近於 1，而 TD(λ) 在漸近意義下變為 Monte Carlo 方法。

對於任意策略 π，由 (4.57) 迭代更新得到的 $v^{\pi}(x)$ 在適當的步長設置和無限量回合數據的條件下，可以證明 (4.57) 收斂到 π 的真實價值函數。這裡我們省略證明。

和 Monte Carlo 方法一樣，在更新完價值函數 v^{π} 之後，我們還需要找到一個相對於 v^{π} 貪婪的策略 π'，然後使用 π' 重複 TD 更新 (4.57) 的過程，以估計新函數 $v^{\pi'}$，如此繼續。因此，TD 方法與 Monte Carlo 方法相似，但在實踐中通常被認為比後者更高效地利用數據。

SARSA 算法

SARSA算法也是一種無模型方法，它與 TD 方法有很多相似之處，並適用於小規模到中等規模的問題。不同之處在於，SARSA 算法學習的是質量函數（狀態–動作價值函數）q^π，而不是 TD 方法中的狀態價值函數 v^π。給定一個策略 π，智能體可以執行動作並生成一個 Markov 決策過程

$$X_0,\ U_0,\ R_1,\ \ldots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ U_{t+1},\ \ldots \tag{4.60}$$

為了學習 q^π，SARSA 算法同樣在計算機內存中保存一個定義在 $\mathcal{X}\times\mathcal{U}$ 上的 q^π 表格。然後通過對 (4.60) 採樣一個回合，我們觀測到如下 Markov 決策過程樣本軌跡

$$x_0,\ u_0,\ r_1,\ \ldots,\ x_t,\ u_t,\ r_{t+1},\ x_{t+1},\ u_{t+1},\ \ldots$$

並通過如下方式更新 $q^\pi(x,u)$：

$$q^\pi(x_t,u_t)\leftarrow q^\pi(x_t,u_t)+\alpha(r_{t+1}+\gamma q^\pi(x_{t+1},u_{t+1})-q^\pi(x_t,u_t)). \tag{4.61}$$

我們可以看出 (4.57) 與 (4.61) 有很強的相似性（在 (4.61) 中我們直接使用 $q^\pi(x_t,u_t)$ 而不是 $q^\pi(x,u)$，但其思想與 (4.57) 中的 $v^\pi(x_t)$ 是相同的）。(4.61) 的目標是在 x_{t+1} 是按照 $p(\cdot|x_t,u_t)$ 採樣得到的前提下，使 $r_{t+1}+\gamma q^\pi(x_{t+1},u_{t+1})$ 在期望意義下等於 $q^\pi(x_t,u_t)$，從而得到 (4.26)。由於每次這樣的更新都需要從樣本軌跡中提取一個五元組 $(x_t,u_t,r_{t+1},x_{t+1},u_{t+1})$，因此 SARSA 算法的名字正是（State, Action, Reward, State, Action）的縮寫。

對於任意策略 π，通過 SARSA 更新規則 (4.61) 迭代更新得到的函數 q^π 也可以被證明收斂到 π 的真實 Q 函數。之後，我們可以把策略更新為相對於 π 貪婪的策略，即

$$\pi'(u|x)=\begin{cases}1, & \text{如果}u=\arg\max_{u'\in\mathcal{U}} q^\pi(x,u'),\\ 0, & \text{否則}.\end{cases} \tag{4.62}$$

之所以按這種方式選擇上面的貪婪策略，是因為

$$T^{\pi'}v^\pi(x)=T^*v^\pi(x)=\max_{\hat\pi\in\Pi}\mathbb{E}_{U'\sim\hat\pi(\cdot|x)}[q^\pi(x,U')]=\max_{u'\in\mathcal{U}}q^\pi(x,u'),$$

前提是對每個 $x\in\mathcal{X}$，在 $\mathcal{U}$ 中都能取到一個極大值點。得到 π' 後，我們可以按照 π' 生成新的樣本軌跡，並在這些軌跡上繼續應用 SARSA 更新規則 (4.61)

來獲得 $q^{\pi'}$，如此反覆。這一過程會生成一列策略 $\pi_1, \pi_2, \ldots, \pi_k, \ldots$，其中對每個 k，π_{k+1} 都相對於 v^{π_k} 是貪婪的。由策略改進定理（定理 4.1.16）可知，π_k 收斂到 π^*。

在實踐中，SARSA 算法並不需要很長的回合來更新 q，因為它可以像 (4.61) 那樣在一個回合進行過程中持續更新，並可在任意時刻終止。

SARSA 算法的一個主要缺點是，它需要五元組 $(x_t, u_t, r_{t+1}, x_{t+1}, u_{t+1})$ 來實現更新 (4.61)。如果某個狀態–動作對 (x_{t+1}, u_{t+1}) 沒有被採樣到——而這種情況經常會發生，因為當前對 q 的估計可能表明在狀態 x_{t+1} 下某個動作 u_{t+1} 不太有價值——那麼我們可能永遠無法更新 $q(x_{t+1}, u_{t+1})$。

這一缺點是無模型算法中的一個常見問題，其核心在於強化學習中的**利用**（exploitation）與**探索**（exploration）之間的權衡。利用意味著算法總是根據先前得到的狀態–動作價值 q 選擇當前最優動作；而探索則保留一定概率去嘗試其他動作，從而可能發現更好的動作。現實生活中的類比是：利用意味著我們總是去當前最喜歡的餐館並嚐遍菜單上的所有菜，而探索意味著我們偶爾會隨機嘗試別的餐館，看看是否有更好的菜品。

在許多無模型強化學習算法中，探索是通過 **ϵ-貪婪策略**

$$\pi(u|x) = \begin{cases} \frac{\epsilon}{|\mathcal{U}|} + 1 - \epsilon, & \text{如果} u = \arg\max_{u' \in \mathcal{U}} q^\pi(x, u'), \\ \frac{\epsilon}{|\mathcal{U}|}, & \text{否則}. \end{cases} \tag{4.63}$$

來實現的，其中某個小參數 $\epsilon \in (0, 1)$，並且 $|\mathcal{U}| < \infty$，例如 $\mathcal{U}$ 是有限集，或者是歐氏空間中的連續緊子集。（當 $\mathcal{U}$ 無界時，我們也可以使用非均勻分佈來代替均勻分佈。）為了實現這一策略，我們可以先抽取一個 Bernoulli 樣本，其有兩個可能結果：以概率 $1 - \epsilon$ 得到正面，以概率 ϵ 得到反面；若得到正面，就選擇極大值動作 u，若得到反面，就按照均勻分佈從 $\mathcal{U}$ 中隨機選擇一個動作。

類似於 n 步時序差分方法，SARSA 算法也可以使用更長的元組

$$(x_t,\ u_t,\ r_{t+1},\ x_{t+1},\ u_{t+1},\ \ldots,\ r_{t+n},\ x_{t+n},\ u_{t+n})$$

作為 Markov 決策過程樣本軌跡中的數據。在這種情況下，我們得到 SARSA 算法的一個變體，稱為 n **步 SARSA** 算法，其更新 q 的方式為

$$\begin{aligned} q^\pi(x_t, u_t) \leftarrow\ & q^\pi(x_t, u_t) \\ & + \alpha\Big(r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n q^\pi(x_{t+n}, u_{t+n}) - q^\pi(x_t, u_t)\Big). \end{aligned}$$

Q-learning 算法

Q-learning算法是 SARSA 算法的一個變體，它把學習當前策略 π 的 Q 函數 q^π 的目標改為直接學習最優 Q 函數 q^*。這類似於價值迭代算法（算法 4.2.2）直接學習最優狀態價值函數 v^*）。回憶 (4.27)，最優狀態–動作價值函數 q^* 滿足

$$\begin{aligned} q^*(x,u) &= r(x,u) + \mathbb{E}_{X'\sim p(\cdot|x,u)}[v^*(X')] \\ &= r(x,u) + \mathbb{E}_{X'\sim p(\cdot|x,u)}[\max_{u'\in\mathcal{U}} q^*(X',u')]. \end{aligned} \tag{4.64}$$

轉移概率 $p(\cdot|x,u)$ 由環境決定，而不是由任何策略決定。因此，如果我們在一個回合中觀測到 $(x_t,u_t,r_{t+1},x_{t+1})$，那麼可以把 $v^*(x_{t+1})$ 看作 $\mathbb{E}_{X'\sim p(\cdot|x_t,u_t)}[v^*(X')]$ 的一個無偏估計。與 SARSA 算法中的更新規則類似，Q-learning 算法中對 q 的更新為

$$q(x_t,u_t) \leftarrow q(x_t,u_t) + \alpha\Big(r_{t+1} + \gamma \max_{u'\in\mathcal{U}} q(x_{t+1},u') - q(x_t,u_t)\Big). \tag{4.65}$$

在一定條件下，也可以證明以上更新得到的 q 收斂到 q^*。Q-learning 算法的其餘部分與 SARSA 算法相同。一旦通過反覆迭代 (4.65) 學到了 q^*，我們就可以像 (4.62) 那樣找到一個相對於 q^* 貪婪的策略，而它將是一個最優策略。

與 SARSA 算法相比，Q-learning 算法的主要優點在於，它只使用形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的四元組作為樣本。它們不包含 u_{t+1}，因此比 SARSA 算法對數據的需求更低。

總結 到目前為止，我們可以對目前討論過的無模型方法作一個快速總結，包括 Monte Carlo 方法、時序差分（TD）方法、SARSA 算法以及 Q-learning 算法。更具體地說，給定一個策略 π，Monte Carlo 方法使用完整回合來估計價值函數 v^π；TD(0) 方法使用當前估計的 v^π 來估計其自身（即一種自舉策略）；而 TD(λ) 方法則是 TD(0) 方法與 Monte Carlo 方法之間的一種插值。對於 Monte Carlo 和 TD 方法，我們都會得到一個相對於 v^π 貪婪的新策略，並重復這一過程。它們可以看作是策略迭代算法（算法 4.2.1）的無模型版本。

相比之下，SARSA 算法學習的是當前策略 π 的 Q 函數 q^π，然後再把 π 更新為相對於 q^π 貪婪的策略。之後，由 SARSA 算法生成的策略序列預計將收斂到最優策略 π^*。因此，SARSA 算法也可以看作是策略迭代算法的無模型版本，只不過它在每次迭代中學習的是 q^π 而不是 v^π。

Q-learning 算法則通過規則 (4.65) 直接學習最優 Q 函數 q^*。當其收斂時，

我們得到（對）q^* 的近似，並將最優策略設為相對於 q^* 貪婪的那個策略。因此，Q-learning 算法可以視為價值迭代算法（算法 4.2.2）的無模型版本，只不過它逼近的是 q^* 而不是 v^*。

所有這些方法都可用於小規模到中等規模的問題，因為它們需要將 v^π 或 q^π 作為表格存儲在計算機內存中。接下來我們考慮它們可用於大規模問題的變體。

深度 Q 網絡（DQN）算法

我們考慮 Q-learning 算法的一個變體，稱為深度 Q 網絡（deep Q-network，DQN）算法，它可以求解具有大狀態空間 $\mathcal{X}$ 和動作空間 $\mathcal{U}$ 的無模型強化學習問題。DQN 算法的關鍵在於使用函數逼近器來逼近最優 Q 函數 q^*。我們使用深度神經網絡作為這類逼近器。

回憶一下，q^* 的 Bellman 方程由 (4.64) 給出。因此，很自然地，可以考慮使用如下損失函數來訓練 q^*：

$$\frac{1}{2}\Big|q^*(x_t,u_t)-\Big(r_{t+1}+\gamma\max_{u'\in\mathcal{U}}q^*(x_{t+1},u')\Big)\Big|^2 \tag{4.66}$$

其中樣本四元組為 $(x_t,u_t,r_{t+1},x_{t+1})$。通過運行許多回合，我們可以獲得大量形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的樣本四元組，並對它們取平均，將其作為 Q 網絡的損失函數。在每次迭代中，我們可以取一個相對於當前 q^* 估計的 ϵ-貪婪策略 π，然後再次運行回合來更新 q^*。

在實踐中，DQN 算法使用一個**經驗回放**（experience replay）緩衝區來存儲先前獲得的 $(x_t,u_t,r_{t+1},x_{t+1})$。注意，一旦 (x_t,u_t) 確定，$r_{t+1}=r(x_t,u_t)$ 或 $r_{t+1}\sim p_R(\cdot|x_t,u_t)$，以及 $x_{t+1}\sim p(\cdot|x_t,u_t)$ 都與策略無關，因此由之前的策略得到的數據可以被存入經驗回放緩衝區，並在新策略對應的 q^* 訓練中繼續使用。因為同一批樣本可以被不同策略複用，這樣就提高了整體的採樣使用效率。

注意，上述 DQN 設定中的一個問題是，q^* 在損失函數 (4.66) 的兩處同時出現。這通常會導致訓練不穩定，因為一旦當前估計的 q^* 偏向某個特定的狀態–動作序列，它在後續訓練中會被進一步強化，從而造成更大的估計誤差。為緩解這一問題，我們可以設置兩個 DQN，分別記為 q 和 $\hat{q}$，稱為訓練 DQN 和目標 DQN。在更新 q 時，保持 $\hat{q}$ 不變，並求解

$$\min_q\frac{1}{2}\iiint\Big|q(x,u)-\Big(r(x,u)+\gamma\max_{u'\in\mathcal{U}}\hat{q}(x',u')\Big)\Big|^2dx'dudx \tag{4.67}$$

在進行若干次 (4.67) 更新 q 之後，我們令 $\hat{q} \leftarrow q$，然後再重複若干輪 (4.67)。從經驗上看，這常常能提升計算穩定性。然而，誤差累積問題仍然存在。

雙重深度 Q 網絡算法

注意，在上面的 DQN 算法中，我們在更新 q 時固定 $\hat{q}$。也就是說，用於更新 q 的目標由如下函數給出：

$$r(x,u) + \gamma \max_{u' \in \mathcal{U}} \hat{q}(x', u').$$

如果我們假設在上式中的極大化問題裡選擇了一個貪婪動作 $u' \in \mathcal{U}$，那麼該目標實際上等價於

$$r(x,u) + \gamma \hat{q}(x', \arg\max_{u' \in \mathcal{U}} \hat{q}(x', u')). \tag{4.68}$$

因此，如果 $\hat{q}$ 對某個 (x,u) 產生了過高估計，那麼這種誤差仍會繼續累積。

在雙重深度 Q 網絡（Double DQNs）算法中，我們僅僅把 (4.68) 改為如下形式：

$$r(x,u) + \gamma \hat{q}(x', \arg\max_{u' \in \mathcal{U}} q(x', u')),$$

並用它作為訓練 q 的目標。當 q 和 $\hat{q}$ 每隔若干次迭代交替更新時，從經驗上發現，這種做法可以顯著緩解誤差累積問題。

策略梯度算法

Monte Carlo 算法、時序差分算法和 SARSA 算法都旨在學習某個給定策略 π 的狀態價值函數 v^π 或狀態–動作價值函數 q^π。另一方面，Q-learning 算法、DQN 方法及其變體則試圖學習 q^*。只有當最優的 v^* 或 q^* 被學到之後，這些方法才將最優策略設為相對於這些價值函數貪婪的那個策略。

策略梯度算法（policy gradient）採取了不同的思路，即直接學習最優策略 π^*。在策略梯度算法中，我們同樣將策略參數化為一個深度神經網絡。該網絡要麼是從狀態空間 $\mathcal{X}$ 到動作空間 $\mathcal{U}$ 的映射，要麼是在每個 $x \in \mathcal{X}$ 上給出定義在 $\mathcal{U}$ 上的概率分佈。然而，我們需要一個標量值目標函數來訓練這個策略 π^*。具體而言，我們需要計算目標函數相對於策略的梯度（更準確地說，是相對於該策略網絡參數的梯度）。

假設當前我們已有一個策略 π，則可以用它生成一個長度為 t 的樣本回合

(t 任意):

$$z_t = (x_0,\ u_0,\ x_1,\ u_1,\ \ldots,\ x_t,\ u_t), \tag{4.69}$$

其中 $x_0 \sim p_0$，這裡 p_0 是某個初始狀態分佈（它預先給定，和任何策略都無關）；而對任意時刻 t，有 $u_t \sim \pi(\cdot|x_t)$ 由策略 π 決定，且 $x_{t+1} \sim p(\cdot|x_t, u_t)$ 由環境決定。這裡我們假設回合可能在有限時間終止，且折扣因子 $\gamma \in [0,1]$，但下面展示的推導同樣適用於無限時間範圍且折扣因子 $\gamma \in (0,1)$ 的情形。

為了下面的推導，我們對任意時間 t 定義

$$\rho_t^\pi(z_t) := p_0(x_0)\pi(u_0|x_0)p(x_1|x_0,u_0)\ldots p(x_t|x_{t-1},u_{t-1})\pi(u_t|x_t). \tag{4.70}$$

另外，我們把終止於有限時間 T 的回合 z_T 的總回報定義為

$$R(z_T) := \sum_{t=0}^{T} \gamma^t r_{t+1} = \sum_{t=0}^{T} \gamma^t r(x_t, u_t). \tag{4.71}$$

於是我們將目標函數（即由策略 π 決定的 z_T 的期望總回報）記為

$$J(\pi) := \mathbb{E}_{Z_T \sim \rho_T^\pi}[R(Z_T)] = \int R(z_T)\rho_T^\pi(z_T)\, dz_T. \tag{4.72}$$

現在我們的目標是求出 $\nabla_\pi J(\pi)$，即 J 關於 π 的梯度。注意，這裡以及下文中的 ∇_π 仍應理解為對 π 的網絡參數所取的梯度。

為了把梯度 $\nabla_\pi J(\pi)$ 寫成一個在實踐中可實現的目標函數，我們首先注意到

$$J(\pi) = \int R(z_T)\rho_T^\pi(z_T)\, dz_T = \sum_{t=0}^{T} \gamma^t \underbrace{\int r(x_t, u_t)\rho^\pi(z_T)\, dz_T}_{=:\, J_t(\pi)} \tag{4.73}$$

這是由 (4.71) 與 (4.72) 聯立得到的。我們看到，$J_t(\pi)$ 可以寫為

$$
\begin{aligned}
J_t(\pi) &= \int r(x_t, u_t)\rho_T^\pi(z_T)\, dz_T \\
&= \iint r(x_t, u_t)p_0(x_0)\pi(u_0|x_0)\cdots\pi(u_T|x_T)\,(du_T\cdots dx_{t+1})dz_t \\
&= \int \Big[r(x_t, u_t)p_0(x_0)\cdots\pi(u_t|x_t) \\
&\qquad\qquad \cdot\Big(\int p(x_{t+1}|x_t, u_t)\cdots\pi(u_T|x_T)\, du_T\cdots dx_{t+1}\Big)\Big]dz_t \\
&= \int r(x_t, u_t)\underbrace{p_0(x_0)\cdots\pi(u_t|x_t)}_{=\rho_t^\pi(z_t)}\, dz_t,
\end{aligned}
\tag{4.74}
$$

其中第二個等號來自 (4.69) 中 $\rho_T^\pi(z_T)$ 的定義以及 $dz_T = (du_T dx_T\cdots dx_{t+1})dz_t$；第三個等號來自先對 $(u_T, x_T, \ldots, x_{t+1})$ 積分；最後一個等號則是因為

$$
\int p(x_{t+1}|x_t, u_t)\cdots\pi(u_T|x_T)\, du_T\cdots dx_{t+1} = 1. \tag{4.75}
$$

現在我們先考察 $\nabla_\pi J_t(\pi)$，它由下式給出

$$
\begin{aligned}
\nabla_\pi J_t(\pi) &= \nabla_\pi\Big(\int r(x_t, u_t)\rho_t^\pi(z_t)\, dz_t\Big) \\
&= \int r(x_t, u_t)\rho_t^\pi(z_t)\Big(\nabla_\pi \log\rho_t^\pi(z_t)\Big)\, dz_t \\
&= \int r(x_t, u_t)\rho_T^\pi(z_T)\Big(\nabla_\pi \log\rho_t^\pi(z_t)\Big)\, dz_T
\end{aligned}
\tag{4.76}
$$

其中第二個等號利用了 $\nabla_\pi\rho_t^\pi(z_t) = \rho_t^\pi(z_t)\nabla_\pi\log\rho_t^\pi(z_t)$，而最後一個等號再次利用了 (4.75)。我們進一步考察 $\nabla_\pi\log\rho_t^\pi(z_t)$，並注意到由 (4.70) 有

$$
\begin{aligned}
\nabla_\pi\log\rho_t^\pi(z_t) &= \nabla_\pi\Big(\log p_0(x_0) + \sum_{s=0}^{t}\log\pi(u_s|x_s) + \sum_{s=0}^{t}\log p(x_{s+1}|x_s, u_s)\Big) \\
&= \sum_{s=0}^{t}\nabla_\pi\log\pi(u_s|x_s)
\end{aligned}
\tag{4.77}
$$

其中最後一個等號是因為上式括號中的中間一項才包含 π（第一項只包含初始分佈 p_0，最後一項只包含由環境決定的狀態轉移分佈）。因此，結合 (4.76) 與

(4.77)，我們得到 $\nabla_\pi J_t(\pi)$ 的最終形式：

$$\nabla_\pi J_t(\pi) = \int r(x_t, u_t)\Big(\sum_{s=0}^{t} \nabla_\pi \log \pi(u_s|x_s)\Big)\rho_T^\pi(z_T)\, dz_T. \tag{4.78}$$

現在有

$$\begin{aligned}
\nabla_\pi J(\pi) &= \nabla_\pi\Big(\sum_{t=0}^{T} \gamma^t J_t(\pi)\Big) \\
&= \sum_{t=0}^{T} \gamma^t \nabla_\pi J_t(\pi) \\
&= \int \sum_{t=0}^{T}\sum_{s=0}^{t} \gamma^t r(x_t, u_t)\nabla_\pi \log \pi(u_s|x_s)\rho_T^\pi(z_T)\, dz_T \\
&= \int \sum_{t=0}^{T}\Big(\sum_{s=t}^{T} \gamma^s r(x_s, u_s)\Big)\nabla_\pi \log \pi(u_t|x_t)\rho_T^\pi(z_T)\, dz_T \\
&= \mathbb{E}_{Z_T\sim\rho_T^\pi}\Big[\sum_{t=0}^{T}\Big(\sum_{s=t}^{T} \gamma^s R_{s+1}\Big)\nabla_\pi \log \pi(U_t|X_t)\Big],
\end{aligned} \tag{4.79}$$

其中第四個等號來自交換關於 t 與 s 的求和順序，然後再交換符號 s 與 t；最後一個等號則來自對所有 s 有 $R_{s+1} = r(X_s, U_s)$。

注意，(4.79) 右端在實踐中是可以近似的。如果採用 Monte Carlo 方法，那麼可以通過讓智能體按照當前估計策略 π 在環境中行為來採樣一個回合 $z_T \sim \rho_T^\pi$（這裡時間 T 取決於實際樣本回合 z_T 的長度，而非預先固定）。在這種情況下，獎勵 $r_{s+1} = r(x_t, u_t)$ 可以被存儲並用於估計 (4.79)。如果我們想利用估計的 Q 函數 $q(x_t, u_t)$（它可像上面的 DQN 和雙重 DQN 算法那樣，被近似為一個輸入來自 $\mathcal{X}\times\mathcal{U}$ 的深度神經網絡）來近似獎勵 $\sum_{s=t}^{T}\gamma^s r_{s+1}$，那麼我們注意到 $\sum_{s=t}^{T}\gamma^s r_{s+1} = \gamma^t q(x_t, u_t)$。

現在我們看到，在 (4.79) 中，唯一尚未確定的項是 $\log\pi$。如果 $\mathcal{X}$ 是一個大的有限集，或者是歐氏空間中的連續子集，我們可以把 $\pi(\cdot|x)$ 參數化為在任意 $x \in \mathcal{X}$ 下定義在 $\mathcal{U}$ 上的一個概率分佈。更具體地說，如果 $\mathcal{U} = \{u_1, \ldots, u_m\}$ 是一個包含 m 個動作的有限集合，那麼我們可以把 $\pi : \mathcal{X} \to \mathbb{R}^m$ 參數化為一個深度神經網絡，並將該網絡的最後一層操作設為 softmax 函數：

$$\mathrm{softmax}(w) := \frac{1}{\sum_{i=1}^{m} e^{w_i}}(e^{w_1}, e^{w_2}, \ldots, e^{w_m}) \in \mathbb{R}^m \tag{4.80}$$

其中 $w \in \mathbb{R}^m$。在這種情況下，我們知道對每個 $i \in [m]$ 和每個 $x \in \mathcal{X}$，都有 $\pi(u_i|x) > 0$ 且 $\sum_{i=1}^m \pi(u_i|x) = 1$，而在狀態 x 下實際執行的動作 u 應當按照定義在 $\mathcal{U}$ 上的分佈 $\pi(\cdot|x)$ 來採樣。在這種情況下，對任意 $(x,u) \in \mathcal{X} \times \mathcal{U}$，可以直接求出 $\nabla_\pi \pi(u|x)$——即 $\pi(u|x)$ 關於 π 的網絡參數的梯度。

如果 $\mathcal{U}$ 是 $\mathbb{R}^m$ 某個連續子集，其中 $m \in \mathbb{N}$，那麼我們可以設置兩個深度神經網絡，$\mu_\pi : \mathcal{X} \to \mathbb{R}^m$ 和 $\sigma_\pi : \mathcal{X} \to (0,\infty)$（我們也可以很容易將 σ_π 推廣為取值於 $(0,\infty)^m$，使得 σ_π 可以作為一個對角協方差矩陣的對角元，或者直接取為一個 $m \times m$ 的對稱正定矩陣作為協方差矩陣），並令 $\pi(\cdot\,|x) = \mathcal{N}(\,\cdot\,;\mu_\pi(x), \sigma_\pi(x)^2 I_m)$。這樣一來，我們可以獲得 $\nabla_\pi \pi(u|x)$ 的表達式以及它對 π 的參數的導數。

一旦我們知道如何計算任意 π 下的 $\nabla_\pi J(\pi)$，便可以使用梯度上升法或類似方法來更新策略 π 的網絡參數，並逼近最優策略 π^*。

Actor-Critic 算法

注意到我們上面討論的算法，要麼學習狀態（或狀態–動作）價值函數（例如 DQN 與雙重 DQN 算法），要麼學習策略（例如策略梯度算法）。Actor-Critic（AC）算法是一種新的方法，它同時學習狀態價值函數 v^π 和 actor π。在 AC 算法中，π 和 v^π 都被參數化為深度神經網絡，它們分別稱為 actor 網絡和 critic 網絡。critic 網絡的目標是評估由 actor 網絡 π 所決定的當前策略的價值函數 v^π，而 actor 網絡的目標則是利用 critic 網絡給出的反饋來改進策略 π。更具體地，回憶一下

$$\nabla_\pi J(\pi) = \mathbb{E}_{Z_T \sim \rho_T^\pi}\Big[\sum_{t=0}^T \Big(\sum_{s=t}^T \gamma^s R_{s+1}\Big) \nabla_\pi \log \pi(u_t|x_t)\Big]. \tag{4.81}$$

在 AC 算法中，我們並不通過採樣完整回合來更新 π。相反，我們使用與 TD 方法中樣本四元組 $(x_t, u_t, r_{t+1}, x_{t+1})$ 類似的思想，將 $\sum_{s=t}^T \gamma^s R_{s+1}$ 替換為

$$\gamma^t \sum_{s=t}^T \gamma^{s-t} R_{s+1} = \gamma^t (r_{t+1}) + \gamma v^\pi(x_{t+1}),$$

其中 v^π 為 critic 網絡。一旦通過梯度上升算法或類似方法更新了 actor 網絡 π，我們就轉而通過最小化

$$\frac{1}{2}|r_{t+1} + \gamma v^\pi(x_{t+1}) - v^\pi(x_t)|^2. \tag{4.82}$$

來更新 critic 網絡 v^π。在實際實現中，上式中的 $r_{t+1}+\gamma v^\pi(x_{t+1})$ 被視作目標狀態價值函數，並且 (4.82) 相對於 v^π 參數的梯度只作用於第二個 v^π。也就是說，在 AC 算法中，經驗上將 (4.82) 的近似梯度設為 $-(r_{t+1}+\gamma v^\pi(x_{t+1})-v^\pi(x_t))\cdot\nabla_{v^\pi}v^\pi(x_t)$。

信賴域策略優化（TRPO）算法

儘管 actor-critic（AC）算法建立了一個交替更新策略（actor）及其價值函數（critic）的新框架，但其經驗計算往往表現得不夠穩定。其中一個主要原因在於：相對於網絡參數的梯度通常過於激進，因此需要更好的約束來緩解穩定性問題。

回憶一下，策略梯度算法考慮瞭如下函數關於當前策略 π 的梯度

$$J(\pi)=\mathbb{E}_{Z_T\sim\rho_T^\pi}[R(Z_T)]$$

其中這裡我們還假設採樣得到的回合具有無限長度，且總回報定義為 $R(z_T)=\sum_{t=0}^{\infty}\gamma^t r(x_t,u_t)$，以便於下面的討論。我們還注意到，這個函數可以寫為

$$J(\pi)=\mathbb{E}_{X\sim p_0}[v^\pi(X)]=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)]\Big]. \tag{4.83}$$

正如我們所看到的，策略梯度步驟計算不穩定的主要原因在於：當 v^π 或 q^π 被參數化為深度神經網絡時，它們存在估計誤差。在這種情況下，使用 π 去生成新的回合可能代價高昂，因為許多高風險採樣動作可能發生，從而提高數據生成的實際成本。

為了解決這一問題，信賴域策略優化（trust-region policy optimization，TRPO）算法通過引入一個先前估計的策略 π_k（假設它在樣本回合生成過程中表現良好），將 (4.83) 中的目標函數改寫為：

$$J(\pi)=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}q^\pi(X,U)\Big]\Big]. \tag{4.84}$$

容易驗證，新的目標函數 (4.84) 與 (4.83) 中的目標函數是完全等價的。

由於 TRPO 算法要求關於策略 π 網絡參數的梯度不能過於激進，因此它提出要約束先前的 $\pi_k(\cdot|x)$ 與新的 $\pi(\cdot|x)$ 在每個 $x\in\mathcal{X}$ 處的差異，並假設未知的 Q 函數 q^π 可以由先前獲得的 q^{π_k} 很好地近似。更具體地說，TRPO 算法

提出如下方案來更新策略 π：

$$\max_{\pi} \quad \tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}q^{\pi_k}(X,U)\Big]\Big], \tag{4.85a}$$

$$\text{s.t.} \quad \mathbb{E}_{X\sim p_0}\Big[\mathrm{KL}(\pi_k(\cdot|X),\, \pi(\cdot|X))\Big] \le \delta, \tag{4.85b}$$

其中 $\delta > 0$ 是某個用戶選定的超參數。正如我們所見，這一過程會產生一列策略 $\{\pi_k : k \ge 1\}$ 和 Q 函數 $\{q^{\pi_k} : k \ge 1\}$。其優點在於，相鄰策略在 KL 散度 (4.85b) 的意義下必須足夠接近，以避免對 π_k 的網絡參數產生不穩定的更新。

基於上述思想，我們可以設計許多 TRPO 算法的變體。對於原始 TRPO 算法 (4.85)，(4.85a) 中的目標函數可以近似為 π 的仿射函數，而 (4.85b) 可以近似為關於 π 網絡參數的一個二次函數的不等式約束。藉助這些近似，求解 (4.85) 的計算代價可以大大降低，但與我們稍後介紹的某些其他變體相比，它仍然略顯昂貴。

作為與 TRPO 算法及其變體相關的最後一點說明，我們注意到，進一步穩定數值計算的一種實踐上很有效的方法是引入所謂的**基線**（baseline）。其原因在於，儘管像 (4.84) 這樣的目標函數看上去是對真實目標函數 (4.83) 的無偏估計，但其樣本方差往往較大，並可能導致不理想的收斂問題。基線函數被設計成真實目標函數的另一個無偏估計，並且當它從 (4.84) 中減去時，該估計仍保持無偏。與此同時，這個基線函數的目標是降低樣本方差，並改善 TRPO 算法及其變體在經驗上的收斂穩定性。

設計基線函數有多種選擇。一種典型做法是：對任意 $x \in \mathcal{X}$，將策略網絡 $\pi(\cdot|x)$ 設計成一個正態分佈，這樣 (4.85b) 中的 KL 散度就有具體的表達式，從而更易處理。與此同時，我們將從 (4.85a) 中減去的基線函數設計為

$$\begin{aligned}
\mathbb{E}_{X\sim p_0}[v^{\pi_k}(X)] &= \int p_0(x)v^{\pi_k}(x)\,dx \\
&= \int p_0(x)v^{\pi_k}(x)\Big(\int \pi(u|x)\,du\Big)\,dx \\
&= \iint p_0(x)\pi_k(u|x)\frac{\pi(u|x)}{\pi_k(u|x)}v^{\pi_k}(x)\,dudx \\
&= \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}v^{\pi_k}(X)\Big]\Big],
\end{aligned}$$

它同樣是 $J(\pi)$ 的一個無偏估計（假設 v^{π} 可以由 v^{π_k} 很好地近似）。因此，目標函數 (4.85a) 中的 Q 函數 $q^{\pi_k}(X,U)$ 將被替換為 $q^{\pi_k}(X,U) - v^{\pi_k}(X)$。大量數值實驗表明，這樣做能夠減輕計算不穩定的問題。

近端策略優化（PPO）算法

近端策略優化（proximal policy optimization，PPO）算法是上述 TRPO 算法的後續變體。通過對 TRPO 作出一些適度修改，PPO 算法通常展現出更強的學習性能和更快的數值計算速度。PPO 算法的思想是使用 Lagrange 方法的一個放鬆版本來處理約束 (4.85b)。這裡所謂 “放鬆版本” 是指，Lagrange 乘子的更新是以啟發式方式進行的。

更具體地說，PPO 算法的懲罰版本將 TRPO 算法中的方案 (4.85) 修改為一個無約束問題：

$$\tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}a^{\pi_k}(X,U) - \lambda \mathrm{KL}\Big(\pi_k(\cdot|X), \pi(\cdot|X)\Big)\Big]\Big],$$

其中 $a^{\pi_k}(x,u) := q^{\pi_k}(x,u) - v^{\pi_k}(x)$ 稱為**優勢函數**（注意它可以為正也可以為負，這取決於 q^{π_k} 中動作 u 所帶來的價值是否高於其期望值 v^{π_k}），它在許多無模型強化學習算法中都很常見，而 λ 起到 Lagrange （KKT）乘子的作用。在 PPO 算法中，λ 的更新方式非常簡單：若 KL 散度小於 $2\delta/3$，則將 λ 更新為 $\lambda/2$；若 KL 大於 $3\delta/2$，則更新為 2λ；否則保持不變，其中 $\delta > 0$ 同樣是用戶選取的超參數。

PPO 算法的另一個簡單而有效的版本稱為 PPO-Clip。在 PPO-Clip 中，目標函數改為如下形式：

$$\tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\min\Big(\xi_k(X,U)a^{\pi_k}(X,U),\, \bar{\xi}_k^{\pi}(X,U)a^{\pi_k}(X,U)\Big)\Big]\Big]$$

其中比值 ξ_k 定義為

$$\xi_k^{\pi}(x,u) := \frac{\pi(U|X)}{\pi_k(U|X)},$$

而其裁剪值 $\bar{\xi}_k^{\pi}(x,u)$ 被限制在區間 $[1-\epsilon, 1+\epsilon]$ 內：

$$\bar{\xi}_k^{\pi}(x,u) := \begin{cases} 1-\epsilon, & \text{如果 } \xi_k^{\pi}(x,u) \le 1-\epsilon, \\ 1+\epsilon, & \text{如果 } \xi_k^{\pi}(x,u) \ge 1+\epsilon, \\ \xi_k^{\pi}(x,u), & \text{否則}, \end{cases}$$

其中 $\epsilon \in (0,1)$ 為用戶選定的超參數。注意，之所以這樣設置裁剪值，是因為優勢函數值 a^{π_k} 可能為正，也可能為負。這一 PPO-Clip 版本比放鬆 Lagrange 方法更易實現，並且在許多經驗測試中顯得非常有效。

深度確定性策略梯度（DPG）算法

深度確定性策略梯度（deterministic policy gradient，DPG）算法是一種在學習過程中避免對動作進行採樣的方法。通過假設策略是貪婪的，DPG 算法可以在 actor-critic 算法中，直接把策略網絡（actor 網絡）併入價值網絡（critic 網絡）。

由於貪婪策略都是確定性的，因此 DPG 算法中的 actor 可以設為一個深度神經網絡 $w:\mathcal{X}\to\mathcal{U}$。換言之，確定性策略 π 可被寫為如下 one-hot 概率：

$$\pi(u|x):=\begin{cases}1,\ \text{在}w(x)\ \text{处，其中}\ w(x)=\arg\max_{u\in\mathcal{U}}q(x,u),\\ 0,\ \text{在每个}u\in\mathcal{U}\setminus\{w(x)\}\text{处}.\end{cases}$$

如果極大值點不唯一，則 $w(x)$ 可以取其中任意一個。由於 actor 網絡 w 的輸出始終是 $\mathcal{U}$ 中的一個極大值點，因此 DPG 算法中的 critic 可以設為最優 Q 函數 q^*，它同樣被參數化為定義在 $\mathcal{X}\times\mathcal{U}$ 上的深度神經網絡。假設 critic 網絡 q^* 關於參數 $u\in\mathcal{U}$ 可微，則有

$$\partial_u q^*(x,w(x))\nabla_w w(x),$$

其中 $\nabla_w w(x)$ 應理解為 actor 網絡 w 關於其網絡參數的梯度。

注意，使用貪婪策略 $w(x)$ 會導致

$$q^*(x,w(x))=\max_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}[q^*(x,U)]=v^*(x).$$

因此，我們可以定義一個關於 w 的目標函數 J 來更新 w 的參數：

$$J(w):=\mathbb{E}_{X\sim p_0}[v^*(X)]=\mathbb{E}_{X\sim p_0}[q^*(X,w(X))].$$

我們看到，$J(w)$ 關於 w 的梯度可以表示為

$$\nabla_w J(w)=\nabla_w\mathbb{E}_{X\sim p_0}[q^*(X,w(X))]=\mathbb{E}_{X\sim p_0}[\partial_u q^*(X,w(X))\nabla_w w(X)].$$

DPG 算法中最優 Q 函數 q^* 的更新與 Q-learning 算法相似。更具體地說，我們需要讓智能體按照當前估計策略 π（即 w）在環境中行為，從而採樣形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的四元組。然後，對於每個樣本四元組，我們希望最小化誤差

$$\frac{1}{2}|r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))-q^*(x_t,u_t)|^2.$$

在訓練過程中，DPG 算法將上述量關於 q^* 網絡參數的梯度近似為

$$-\Big(r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))-q^*(x_t,u_t)\Big)\cdot\nabla_{q^*}q^*(x_t,u_t),$$

其中 $\nabla_{q^*}q^*(x_t,u_t)$ 應理解為 q^* 在給定樣本對 (x_t,u_t) 處關於其網絡參數的梯度。正如我們所見，項 $r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))$ 被視為目標值，而梯度 ∇_{q^*} 並不作用於該項中的 q^*。在實踐中，上述梯度會對所有采樣四元組取平均，以更新 q^*。

DPG 算法也存在許多變體。例如，我們可以將 π 參數化為某種典型概率分佈，例如當 $\mathcal{U}$ 是某個歐氏空間中的連續子集時，將其設為正態分佈。這可以改善學習過程中的探索能力，並有可能發現比只學習貪婪動作更優的動作。

Soft Actor-Critic（SAC）算法

深度確定性策略梯度（DPG）算法有時會表現出問題性的收斂行為，因為關於 q^* 網絡參數的梯度往往並不準確。應對這一問題的一種解決思路是給當前估計策略 π 加上一個正則項。為此，回憶一下，π 的目標函數可以定義為

$$J(\pi):=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)]\Big].$$

我們希望對每個 $X\sim p_0$，向上述目標函數中加入加權熵函數

$$\beta H(\pi(\cdot|X))=-\beta\mathbb{E}_{U\sim\pi(\cdot|X)}[\log\pi(U|X)]$$

其中 $\beta>0$ 是熵項 H 的用戶選定權重。於是，對 π 進行最大化時所採用的目標函數 J 變為

$$J(\pi)=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)-\beta\log\pi(U|X)]\Big]. \tag{4.86}$$

在理論上，該目標函數 J 的極大化子具有如下閉式表達：

$$\pi(u|x)=\frac{e^{q^\pi(x,u)/\beta}}{\int e^{q^\pi(x,u)/\beta}du} \tag{4.87}$$

對每個 x 都成立。然而，這一閉式形式只在 $\mathcal{X}$ 和 $\mathcal{U}$ 的規模較小或中等、並且它們上的 $q^\pi(x,u)$ 值可以存儲為表格時才行。

對於大規模問題，我們需要將 Q 函數 $q^\pi:\mathcal{X}\times\mathcal{U}\to\mathbb{R}$ 參數化為一個深度神經網絡。對於生成回合中的一個樣本四元組 $(x_t,u_t,r_{t+1},x_{t+1})$，我們希望最

小化如下誤差:

$$\frac{1}{2}\Big|\Big(r_{t+1}+\gamma\mathbb{E}_{U_{t+1}\sim\pi(\cdot|x_{t+1})}q^{\pi}(x_{t+1},U_{t+1})\Big)-q^{\pi}(x_t,u_t)\Big|^2. \tag{4.88}$$

在實踐中，我們需要對所有形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的採樣四元組上的誤差取平均，並對該平均誤差相對於 q^{π} 的網絡參數求梯度，以更新它。

在 SAC 算法的實現中，還需要考慮高估 q^{π} 的問題。緩解這一問題的一種辦法是使用兩個深度神經網絡 q_1^{π} 和 q_2^{π}，並將如下值（而不是 (4.88) 中大括號內的那一項）作為目標:

$$r_{t+1}+\gamma\min_{j=1,2}\mathbb{E}_{U_{t+1}\sim\pi(\cdot|x_{t+1})}[q_j^{\pi}(x_{t+1},U_{t+1})]. \tag{4.89}$$

然後，將 (4.88)（其中目標設為 (4.89)）關於 $q^{\pi}(x_t,u_t)$ 的網絡參數的梯度同時用於更新 q_1^{π} 和 q_2^{π}。

儘管我們能夠更新 Q 函數網絡 q^{π} 的參數，但如何計算 (4.86) 中 $J(\pi)$ 的極大值點仍是一個問題。其原因在於，閉式表達 (4.87) 中的歸一化常數並不容易計算，因為它可能需要在一個大的動作空間 $\mathcal{U}$ 上進行積分。在這種情況下，我們可以對 π 使用重參數化技巧，例如在 $\mathcal{U}$ 是 $\mathbb{R}^m$ 的連續子集時，我們將 $\pi(\cdot|x)$ 設為正態分佈 $\mathcal{N}(\cdot;\mu_{\pi}(x),\sigma_{\pi}^2 I_m)$。當 $\mathcal{U}$ 是一個大的有限動作空間時，我們也可以將 $\pi(\cdot|x)$ 參數化為定義在 $\mathcal{U}$ 上的一個概率分佈，方法是在每個 $x\in\mathcal{X}$ 處將 π 表示為一個深度神經網絡，並令其最後一個操作為 softmax 函數，這與上面的策略梯度算法中的做法類似。無論哪種情況，只要當前估計的 Q 函數 q^{π} 已知，我們就能夠計算 $\nabla_{\pi}J(\pi)$，即 J 關於 π 的網絡參數的梯度。

4.3 備註與參考文獻

4.3.1 強化學習與最優控制

強化學習和最優控制是兩個密切相關的序貫決策框架。這兩個框架都研究智能體應如何隨時間採取行動，以優化給定的累積和/或終端性能目標，其方法也被擴展到多智能體情形。從歷史上看，最優控制起源於控制理論和應用數學，而強化學習則源自人工智能和心理學。儘管二者具有共同的基礎，這兩個領域在建模假設、求解方法以及應用領域方面仍存在差異。

在標準的最優控制設定中，控制系統通常被建模為由常微分方程（ODE）描述的受控動力系統。最優控制問題主要分為兩大類：一類是終止時間 T 固

定但終點 $x(T)$ 自由，另一類是終點 $x(T)$ 固定但終止時間 T 自由，如我們在第 3.1 節中所示。在這兩種情況下，ODE 的形式以及運行和/或終端獎勵（或等價地，代價）都是已知的，最優控制的最優性條件可以通過 Pontryagin 最大值原理和連續時間下的 Hamilton–Jacobi–Bellman 方程來刻畫 [12,23]。

相比之下，強化學習通常將決策過程建模為 Markov 決策過程，正如我們在第 4.1 節中所示。在強化學習問題中，轉移動態 $p(x'|x,u)$ 和獎勵函數可能是未知的，需要在求解最優控制（在強化學習中稱為最優策略）的過程中通過學習獲得。對於這類問題，很多不同的方法（通常是無模型算法）都被廣泛應用。我們在第 4.2 節中展示了一些具有代表性且影響較大的算法。

強化學習與最優控制的理論是緊密聯繫的。例如，當系統動力學已知且狀態空間和動作空間為連續時，強化學習可歸結為隨機最優控制。在這種情況下，動態規劃為兩個領域提供了統一框架，而 Bellman 理論支撐了兩個領域中的最優性條件和算法發展 [17]。此外，強化學習中的價值函數與最優控制中的對應概念是直接類比的，而策略梯度方法通過 Pontryagin 最大值原理和軌跡優化與最優控制具有緊密聯繫。文獻中還展示了某些強化學習目標與隨機控制表述之間的等價性，包括熵正則化控制和路徑積分控制 [74,146]。

儘管存在上述聯繫，強化學習與最優控制之間仍存在重要差異。例如，如前所述，最優控制通常假設動力學已知，而強化學習強調在最少先驗知識下從數據中學習。因此，最優控制問題的解的質量高度依賴於模型的準確性，而強化學習算法通常需要在計算效率與數據（包括提供的和生成的）質量之間進行權衡。此外，探索在強化學習中至關重要，而在標準最優控制中通常並不存在，後者通常通過魯棒或隨機建模來處理不確定性。

儘管如此，隨著近年來研究的進展，強化學習與最優控制之間的界限開始變得模糊 [115]。特別是，深度學習已成為解決兩個領域中大規模和高維問題的有力工具。使用深度神經網絡所帶來的共同問題，如穩定性、魯棒性和泛化能力，仍有待進一步的理論和算法發展來解決。為此，理解這兩個領域之間的關係有助於設計更加高效且可靠的決策系統。

4.3.2 強化學習的文獻

關於強化學習的系統性介紹可參見 [140]。文獻 [142] 對經典強化學習算法進行了系統綜述。第 4.1 節在很大程度上遵循了 [142] 中理論部分的思路，並在證明細節上進行了顯著擴展，同時通過引理 4.1.15 和定理 4.1.17 給出了新的主要結果，從而闡明瞭 $\bar{v}$ 與最優 Bellman 算子 T^* 的不動點 v^* 之間的關係。強化學習與最優控制及其聯繫在 [15] 中有深入研究。深度強化學習在過去十

年中迅速發展，近期綜述可參見 [6,44,152]。

經典強化學習算法，如策略迭代、價值迭代和時序差分方法，在 [140,142] 中有詳細討論。Q-learning 方法最早在 [155] 中提出，並在 [156] 中證明其收斂到 Bellman 算子的一個不動點。動態規劃與強化學習之間的聯繫最早在 [155] 中被提出。SARSA 算法由 [124] 提出。Monte Carlo 方法的早期工作見 [134]。最初的策略梯度方法 [141] 和 actor-critic 方法 [80] 在深度學習出現之前就已提出，但它們對現代強化學習算法的發展產生了深遠影響。開創深度強化學習的深度 Q-learning 算法在 Atari 遊戲基準上實現了人類水平的表現 [99]。幾乎在同一時期，深度強化學習的另一項突破是 AlphaGo 算法 [132]，它擊敗了圍棋世界冠軍，而圍棋曾被認為是計算機難以戰勝人類高手的經典棋類遊戲。深度強化學習還被用於發現更高效的矩陣乘法方法，從而減少乘法運算次數 [41]。

近年來，大量工作致力於改進深度強化學習算法。對偶 Q 網絡結構在 [153] 中提出，其中同一網絡的兩個分支分別預測狀態價值函數和優勢函數。標準 Q-learning 由於最大化操作會系統性地高估狀態價值。為緩解這一問題，提出了雙 Q 網絡結構 [149]。在 [43] 中，引入了帶噪聲的深度 Q 網絡，通過加入噪聲增強隨機性並促進探索，同時網絡能夠在收斂過程中逐漸降低噪聲。分佈式深度 Q 網絡結構在 [10,31] 中提出，旨在學習獎勵分佈的更完整信息，而不僅僅是其期望。一種結合多種上述 Q 網絡特性的深度網絡結構被提出，用於提升 ATARI 基準上的訓練速度和最終性能 [61]。優先經驗回放方法用於提升訓練穩定性和速度，見 [127]。優勢函數的擴展也在 [129] 中進行了研究。這些方法都是將深度學習用於價值函數逼近的變體，包括狀態價值函數、Q 函數（狀態-動作價值函數）以及優勢函數。

深度神經網絡也被用於學習強化學習問題的最優策略。信賴域策略優化（TRPO）算法在 [128] 中提出。隨後提出的近端策略優化（PPO）算法見 [130]。在 [133] 中提出了深度確定性策略梯度（DDPG）算法。一種基於 DDPG 且能夠處理連續動作空間的 actor-critic 無模型算法在 [84] 中提出。Soft actor-critic 算法見 [58,59]。該領域正處於快速發展階段。預計在不久的將來會出現更強大、更穩定、更魯棒的深度強化學習算法。

第五章　生成模型

本章介紹**生成模型**（generative model），這是過去十年中出現的一類重要的深度學習方法，用於模擬複雜的數據分佈以及生成它們的新樣本。生成模型具有廣泛的現實應用，可用於生成逼真的文本、音樂、圖像、視頻等內容，這類內容通常稱為人工智能生成內容（AI-generated content，AIGC）。

生成建模背後的原型數學問題可以表述如下。設給定數據集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含 M 個從某個未知概率分佈 p_{data}（定義在 $\mathbb{R}^d$ 上）獨立抽取的樣本。生成建模的目標是僅通過 $\mathcal{D}$ 學習如何生成 p_{data} 的新樣本。

生成模型是一個很有挑戰的問題。首先，我們只有有限數據集 $\mathcal{D}$ 可用，而 p_{data} 的解析形式是未知的。其次，樣本的維數 d 可能極其大。例如，每個 x_i 可能表示一幅圖像，此時 d 往往達到 10^4 甚至更高。

在條件生成的設定下，這個問題會變得更加複雜，此時目標是針對任意用戶指定的條件變量（通常稱為**提示詞**）y，從未知條件分佈 $p_{\text{data}}(\cdot \mid y)$ 中生成樣本。其目的是生成在性質上與用戶需求或偏好相一致的樣本。

本章主要關注無條件生成模型，因為它們的理論基礎和方法發展更加成熟。此外，許多現有的條件生成模型都是建立在無條件生成的模型和方法之上的。

近年來，生成建模取得了快速進展，但目前仍不存在一個能夠完全解決該問題的統一框架。本章將介紹文獻中幾種最具影響力的生成建模範式，並詳細考察它們的構造與性質。我們首先從經典的變分自編碼器框架以及與之密切相關、幾乎同時出現的生成對抗網絡方法開始。隨後討論更近期的發展，包括擴散模型、概率密度控制以及流匹配方法，這些方法利用（隨機）微分方程來生成樣本。最後，我們討論與這些模型相關的實踐和理論問題，並簡要介紹幾種條件生成的方法。

5.1 變分自編碼器

變分自編碼器（Variational Autoencoder，VAE）是一種網絡架構，它可以利用由 p_{data} 的有限個獨立樣本組成的數據集 $\mathcal{D}$ 來生成 p_{data} 的新樣本。VAE 模型由兩個函數組成，稱為編碼器（encoder）和解碼器（decoder）。編碼器和解碼器通常分別參數化為深度神經網絡，記為 g_η 和 f_θ，其網絡參數分別為 η 和 θ。

編碼器 g_η 學習將每個數據樣本 $x \in \mathcal{D}$ 映射到一個**潛變量**（latent variable）z，其中 z 位於**潛在空間**（latent space）$\mathbb{R}^m$ 中。編碼器的輸出 $g_\eta(x)$ 通常不是在給定輸入 x 時直接確定 z，而是被設定為某個概率分佈的參數。例如一個典型的做法是令 $g_\eta(x)$ 為向量 $\mu_\eta(x) \in \mathbb{R}^m$ 和正標量 $\sigma_\eta(x) > 0$ 所組成的一對（或者也可以是 $\mathbb{R}^m$ 中的正向量，作為對角協方差矩陣的對角元，甚至直接是一個 m-維對稱正定協方差矩陣，但考慮到這種推廣不復雜，我們這裡只討論 σ_η 為標量值的簡單情形）。然後將 $\mu_\eta(x)$ 和 $\sigma_\eta(x)$ 分別設為某個高斯分佈的均值和標準差，並從該分佈中抽取潛變量 z。更準確地說，z 是從條件概率分佈 $q_\eta(\,\cdot\,|\,x) := \mathcal{N}(\,\cdot\,;\,\mu_\eta(x), \sigma_\eta(x)^2 I_m)$ 中抽取的樣本。在實踐中，這可以很容易地實現為 $z = \mu_\eta(x) + \sigma_\eta(x)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_m)$。

解碼器 f_θ 旨在將任意 $z \in \mathbb{R}^m$ 映射到一個新樣本 $x' \in \mathbb{R}^d$。同樣，解碼器的輸出 $f_\theta(z)$ 對於每個輸入 z 也不是直接確定 x'，而是被設為向量 $\mu_\theta(z) \in \mathbb{R}^d$ 和正標量 $\sigma_\theta(z) > 0$ 所組成的一對。然後將新樣本 x' 設為從條件概率分佈 $p_\theta(\cdot|z) := \mathcal{N}(\,\cdot\,;\,\mu_\theta(z), \sigma_\theta(z)^2 I_d)$ 中抽取的樣本。在實踐中，這實現為 $x' = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_d)$。

對編碼器 g_η 的一個附加要求是，其邊緣分佈（marginal distribution）

$$q_\eta(z) := \int q_\eta(z|x) p_{\text{data}}(x)\, dx$$

應當近似於標準高斯分佈 $\mathcal{N}(0, I_m)$，從而在 g_η 和 f_θ 訓練完成後，我們可以很容易地從這個邊緣分佈 q_η 中抽樣，並將解碼器 f_θ 應用於這些樣本上，以獲得服從 p_{data} 的新樣本。

我們前面介紹了若干概率分佈：數據概率分佈 p_{data}（其解析形式未知，但我們擁有 M 個來自它的 i.i.d. 樣本）定義在 $\mathbb{R}^d$ 上，潛在空間 $\mathbb{R}^m$ 上的條件分佈 $q_\eta(\cdot|x)$ 與邊緣分佈 $q_\eta(\cdot)$，以及定義在 $\mathbb{R}^d$ 上的條件概率 $p_\theta(\cdot|z)$。由此，我們還得到定義在 $\mathbb{R}^d$ 上的邊緣分佈 $p_{\eta,\theta}(\cdot)$：

$$p_{\eta,\theta}(x) = \int p_\theta(x|z) q_\eta(z)\, dz, \quad \forall\, x \in \mathbb{R}^d, \tag{5.1}$$

原始样本所在的空间 $\mathbb{R}^d$

- 给定数据集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含 M 个样本它们是从 $\mathbb{R}^d$ 上某个未知概率分布 p_{data} 中独立抽取的。

↓ 编码器网络 g_η，网络参数 η

潜在空间 $\mathbb{R}^m$

- 对于每个 $x \in \mathcal{D}$，即 $x \sim p_{\text{data}}$，潜变量 z 是一个样本它服从 $\mathbb{R}^m$ 上的条件概率 $q_\eta(\cdot|x)$。
- 在 VAE 中，具有网络参数 η 的编码器 g_η 通常设为 $g_\eta(x) = (\mu_\eta(x), \sigma_\eta(x))$，使得 $q_\eta(\cdot|x) = \mathcal{N}(\cdot; \mu_\eta(x), \sigma_\eta(x)^2 I_m)$。因而可以实现为 $z = \mu_\eta(x) + \sigma_\eta(x)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_m)$。
- 在 VAE 中，边缘概率 $q_\eta(\cdot) = \int q_\eta(\cdot|x) p_{\text{data}}(x) dx$ 被假设为近似 $\mathcal{N}(\cdot; 0, I_m)$，从而在 f_θ 训练完成后，能够容易地从 $q_\eta(\cdot)$ 生成新样本 z。

↓ 解码器网络 f_θ，网络参数 θ

生成样本所在的空间 $\mathbb{R}^d$

- 对于每个潜变量样本 $z \sim q_\eta(\cdot)$，我们使用 $f_\theta(z)$ 生成一个 x 它服从 $\mathbb{R}^d$ 上的条件概率 $p_\theta(\cdot|z)$。
- 在 VAE 中，具有网络参数 θ 的解码器 f_θ 通常设为 $f_\theta(z) = (\mu_\theta(z), \sigma_\theta(z))$，使得 $p_\theta(\cdot|z) = \mathcal{N}(\cdot; \mu_\theta(z), \sigma_\theta(z)^2 I_d)$。因而可以实现为 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_d)$。
- 这给出联合概率 $p_{\eta,\theta}(x, z) = p_\theta(x|z) q_\eta(z)$ 以及边缘概率 $p_{\eta,\theta}(x) = \int p_{\eta,\theta}(x, z) dz$。
- 在 VAE 中，对每个 $x \in \mathbb{R}^d$，要求后验概率 $p_{\eta,\theta}(\cdot|x) = p_{\eta,\theta}(x|\cdot) q_\eta(\cdot) / p_{\eta,\theta}(x)$ 和 $q_\eta(\cdot|x)$ 在 KL 意义下接近。
- 在 VAE 中，边缘概率 $p_{\eta,\theta}$ 要求在 KL 意义下接近 p_{data}。注意 $p_{\eta,\theta}$ 是 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$ 的边缘概率，其中 $z \sim q_\eta = \mathcal{N}(0, I_m)$ 且 $\epsilon \sim \mathcal{N}(0, I_d)$，因而这一要求强制这样的生成样本 x 服从 p_{data} 分布。

图 5.1: 變分自編碼器（VAE）框架中的空間、映射（編碼器與解碼器）以及概率分佈。

其中 $p_{\eta,\theta}(x,z) := p_\theta(x|z)q_\eta(z)$ 是 (x,z) 的聯合分佈。根據 Bayes 法則，對於任意 $x \in \mathbb{R}^d$，我們還可以得到後驗分佈 $p_{\eta,\theta}(z|x)$：

$$p_{\eta,\theta}(z|x) = \frac{p_\theta(x|z)q_\eta(z)}{p_{\eta,\theta}(x)}, \quad \forall\, z \in \mathbb{R}^m. \tag{5.2}$$

將上述條件分佈設為高斯分佈不僅引入了多樣性（相比於使用確定性映射），而且使得實際實現更為方便。例如，兩個高斯分佈之間的 Kullback–Leibler（KL）散度有具體的表達式，因此可以直接寫出網絡訓練中的損失函數。我們稍後將在本節中展示這些細節。

除了上述設定外，我們還需要編碼器和解碼器所誘導的概率分佈之間滿足以下兩個重要條件：

- 生成樣本的邊緣概率分佈應當等於原始數據的概率分佈。即，

$$p_{\eta,\theta}(x) = p_{\text{data}}(x) \tag{5.3}$$

  對於所有 $x \in \mathbb{R}^d$ 都成立。

- 對於任意 $x \in \mathbb{R}^d$，假設 x 是一個新生成樣本，那麼給定 x 時潛變量 z 的後驗分佈 $p_{\eta,\theta}(z|x)$，應當與 x 是一個 p_{data} 的真樣本時對應的潛變量 z 的條件概率 $q_\eta(z|x)$ 相一致。也就是說，對於任意 $x \in \mathbb{R}^d$ 和 $z \in \mathbb{R}^m$，都有

$$p_{\eta,\theta}(z|x) = q_\eta(z|x). \tag{5.4}$$

在 VAE 框架中，(5.3) 和 (5.4) 都是通過最小化兩邊之間的 KL 散度來實現的。

數據空間 $\mathbb{R}^d$、潛在空間 $\mathbb{R}^m$、編碼器 g_η 和解碼器 f_θ 之間的關係，以及上述討論中涉及到的所有概率分佈，已在圖 5.1 中進行了總結。

現在我們開始推導構造編碼器網絡參數 η 與解碼器網絡參數 θ 的目標函數所需的數學形式，這些目標函數基於前述 KL 散度。更準確地說，為了得到 (5.3)，我們希望最小化

$$\begin{aligned}
\mathrm{KL}(p_{\text{data}}, p_{\eta,\theta}) &= \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_{\eta,\theta}(x)}\, dx \\
&= -H(p_{\text{data}}) - \int p_{\text{data}}(x) \log p_{\eta,\theta}(x)\, dx \\
&= -H(p_{\text{data}}) - \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)],
\end{aligned} \tag{5.5}$$

其中 $H(p_{\text{data}})$ 是 p_{data} 的熵，並且與 θ 和 η 無關。

因此，如果我們想對 (θ, η) 最小化 $\mathrm{KL}(p_{\text{data}}, p_{\eta,\theta})$，只需要求解

$$\max_{\theta,\eta} \ \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)]. \tag{5.6}$$

數值 $\log p_{\eta,\theta}(x)$ 被稱為點 x 處的**證據**（evidence）。(5.6) 的目標是最大化所有服從 p_{data} 的給定樣本的證據值的期望。稍後我們將進一步考察證據 $\log p_{\eta,\theta}(x)$，並說明它與 (5.4) 中兩個概率分佈之間的聯繫。

為了得到 (5.4)，我們也嘗試最小化左右兩個概率分佈之間的 KL 散度。也就是說，對每個 $x \in \mathbb{R}^d$，我們希望最小化

$$\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x)) = \int q_\eta(z|x) \log \frac{q_\eta(z|x)}{p_{\eta,\theta}(z|x)}\, dz \tag{5.7}$$

相對於 θ 和 η。如果我們希望對所有服從 p_{data} 的樣本最小化這個 KL，則需要求解

$$\min_{\theta,\eta} \ \mathbb{E}_{X\sim p_{\text{data}}}[\mathrm{KL}(q_\eta(\cdot|X),\ p_{\eta,\theta}(\cdot|X))]. \tag{5.8}$$

現在我們有兩個問題，即 (5.6) 和 (5.8)，需要求解。然而，它們之間關係密切，可以合併為一個問題。

為了看出這一點，我們需要考察證據 $\log p_{\eta,\theta}(x)$ 與 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$ 之間的關係——這兩個函數的期望分別出現在 (5.6) 和 (5.8) 中：對每個 $x \in \mathbb{R}^d$，有

$$\begin{aligned}
&\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x)) \\
&\quad = \int q_\eta(z|x) \log \frac{q_\eta(z|x)}{p_{\eta,\theta}(z|x)}\, dz \\
&\quad = \int q_\eta(z|x) \log \Big(\frac{q_\eta(z|x)}{q_\eta(z)} \frac{p_{\eta,\theta}(x)}{p_\theta(x|z)} \Big)\, dz \\
&\quad = \int q_\eta(z|x) \log \frac{q_\eta(z|x)}{q_\eta(z)}\, dz - \int q_\eta(z|x) \log p_\theta(x|z)\, dz + \log p_{\eta,\theta}(x) \\
&\quad = \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta) - \int q_\eta(z|x) \log p_\theta(x|z)\, dz + \log p_{\eta,\theta}(x),
\end{aligned} \tag{5.9}$$

其中第二個等號來自 Bayes 法則

$$p_{\eta,\theta}(z|x) = \frac{p_\theta(x|z) q_\eta(z)}{p_{\eta,\theta}(x)}.$$

將 (5.9) 重新整理可得

$$\log p_{\eta,\theta}(x) = \mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x)) + \underbrace{\int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot))}_{=:\,\mathrm{ELBO}_{\eta,\theta}(x)}. \tag{5.10}$$

因此，證據 $\log p_{\eta,\theta}(x)$ 正好等於 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$ 與所謂的**證據下界**（evidence lower bound，ELBO）之和，其中 ELBO 定義為

$$\mathrm{ELBO}_{\eta,\theta}(x) := \int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot)). \tag{5.11}$$

我們注意到，對任意 x，都有 $\mathrm{ELBO}_{\eta,\theta}(x) \le \log p_{\eta,\theta}(x)$，因為根據 (5.10)，它們的差就是 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$，而兩個概率分佈之間的 KL 散度總是非負的。

ELBO 最初之所以得名，是因為文獻中通常按如下方式推導：對任意 $x \in \mathbb{R}^d$，有

$$\begin{aligned}
\log p_{\eta,\theta}(x) &= \log \int p_{\eta,\theta}(x,z)\,dz \\
&= \log \int \frac{p_{\eta,\theta}(x,z)}{q_\eta(z|x)} q_\eta(z|x)\,dz \\
&\ge \int q_\eta(z|x)\log \frac{p_{\eta,\theta}(x,z)}{q_\eta(z|x)}\,dz \\
&= \int q_\eta(z|x)\log \Big(\frac{q_\eta(z)}{q_\eta(z|x)} p_\theta(x|z)\Big)\,dz \\
&= \int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot)) \\
&= \mathrm{ELBO}_{\eta,\theta}(x),
\end{aligned}$$

其中不等式是基於 Jensen 不等式，因為對數函數是凹函數；第三個等號來自 $p_{\eta,\theta}(x,z) = p_\theta(x|z)q_\eta(z)$。因此，如果我們想最大化證據 $\log p_{\eta,\theta}(x)$，就可以嘗試最大化 $\mathrm{ELBO}_{\eta,\theta}(x)$。

現在我們看到，(5.10) 還給出了 ELBO 的另一種解釋：最大化 $\mathrm{ELBO}_{\eta,\theta}(x)$ 不僅會增大證據 $\log p_{\eta,\theta}(x)$，還會減小每個 x 處的 $\mathrm{KL}(q_\eta(\cdot|x), p_{\eta,\theta}(\cdot|x))$。更準

確地，對 (5.10) 兩邊取期望，有

$$\begin{aligned}\mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)] = \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)] \\ - \mathbb{E}_{X\sim p_{\text{data}}}[\text{KL}(q_\eta(\cdot|X),\ p_{\eta,\theta}(\cdot|X))].\end{aligned}$$

因此，最大化問題 (5.6) 與最小化問題 (5.8) 可以通過同一種方式同時實現——求解 ELBO 最大化問題：

$$\max_{\theta,\eta}\ \mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)]. \tag{5.12}$$

在本節餘下部分，我們將進一步考察 $\text{ELBO}_{\eta,\theta}$ 的結構，並利用給定數據 $\mathcal{D}$ 將 (5.12) 寫成一個可實現的優化問題。

回顧 (5.11) 中 ELBO 的定義，我們知道 (5.12) 中的目標函數可以寫為

$$\begin{aligned}\mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)] = &\underbrace{\iint p_{\text{data}}(x)q_\eta(z|x)\log p_\theta(x|z)\,dzdx}_{=:\,A(\theta,\eta)} \\ &- \underbrace{\int p_{\text{data}}(x)\text{KL}(q_\eta(\cdot|x), q_\eta(\cdot))\,dx}_{=:\,B(\eta)}\end{aligned} \tag{5.13}$$

根據編碼器 g_η 的設定，有

$$q_\eta(z|x) = \mathcal{N}(z;\mu_\eta(x),\sigma_\eta(x)^2 I_m). \tag{5.14}$$

因此，我們可以從標準高斯 $\mathcal{N}(0, I_m)$ 中抽取 ϵ，並設

$$z_\eta(x,\epsilon) = \mu_\eta(x) + \sigma_\eta(x)\epsilon. \tag{5.15}$$

於是我們知道，$z_\eta(x,\epsilon)$ 是從概率分佈 $q_\eta(\cdot|x)$ 中抽取的一個樣本。此外，對於任意 $z\in\mathbb{R}^m$，根據解碼器 f_θ 的設定，有

$$p_\theta(x|z) = \mathcal{N}(x;\mu_\theta(z),\sigma_\theta(z)^2 I_d). \tag{5.16}$$

因此，

$$\log p_\theta(x|z) = -\frac{|x-\mu_\theta(z)|^2}{2\sigma_\theta(z)^2} - \frac{d}{2}\log\left(2\pi\sigma_\theta(z)^{2d}\right).$$

現在我們可以將 (5.13) 中定義的 $A(\theta,\eta)$ 重寫為

$$A(\theta,\eta)=\iint p_{\text{data}}(x)\mathcal{N}(\epsilon;0,I_m)\log p_\theta(x|z_\eta(x,\epsilon))\,d\epsilon dx. \tag{5.17}$$

其中 $z_\eta(x,\epsilon)$ 和 $\log p(x|z)$ 分別由 (5.15) 與 (5.16) 給出。在實際實現中，由於我們只擁有若干來自 p_{data} 的樣本，因此可以使用 Monte Carlo 積分來近似 (5.17) 中的 $A(\theta,\eta)$：對於每個樣本 $x\in\mathcal{D}$（其中 $\mathcal{D}$ 包含從 p_{data} 中獨立抽取的 M 個樣本），我們抽取一個 $\epsilon\sim\mathcal{N}(0,I_d)$，並使用

$$\frac{1}{M}\sum_{(x,\epsilon)}\mathcal{N}(\epsilon;0,I_m)\log p_\theta(x|z_\eta(x,\epsilon)),$$

來近似 (5.17) 中的 $A(\theta,\eta)$，其中求和是對這 M 組 (x,ϵ) 樣本對進行的。

接下來，我們注意到 (5.13) 中定義的 $B(\eta)$ 的被積函數 $\mathrm{KL}(q_\eta(\cdot|x),q_\eta(\cdot))$ 具有閉式表達，因為 $q_\eta(\cdot|x)$ 與 $q_\eta(\cdot)$ 都是高斯分佈。具體而言，有

$$q_\eta(z|x)=\mathcal{N}(z;\mu_\eta(x),\sigma_\eta(x)^2I_m)$$

以及

$$q_\eta(z)=\mathcal{N}(z;0,I_m).$$

因此，我們可以得到它們 KL 散度的表達式（同維高斯分佈之間的 KL 散度見 (A.12)）：

$$\mathrm{KL}(q_\eta(\cdot|x),q_\eta(\cdot))=\frac{1}{2}\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big).$$

於是，

$$B(\eta)=\frac{1}{2}\int p_{\text{data}}(x)\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big)\,dx, \tag{5.18}$$

它同樣可以通過 Monte Carlo 積分近似為

$$\frac{1}{2}\sum_{x\in\mathcal{D}}\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big).$$

現在我們已經得到了 $A(\theta,\eta)$ 與 $B(\eta)$ 的表達式 (5.17) 和 (5.18)，以及它們的可實現形式，因此可以關於 (θ,η) 最大化 (5.13) 中定義的 ELBO 期望。

在 (θ,η) 優化完成後，我們可以按如下方式從未知概率分佈 p_{data} 中生成

新樣本。首先模擬 $z \sim \mathcal{N}(0, I_m)$ 和 $\epsilon \sim \mathcal{N}(0, I_d)$，並設 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，則 x 就是一個服從概率分佈 p_{data} 的新樣本。我們可以獨立且同時地生成任意多個這樣的 (z, ϵ) 對，並計算對應的 x，這些 x 都是服從概率分佈 p_{data} 的新樣本。

5.2 生成對抗網絡

生成對抗網絡，Generative Adversarial Networks（GANs）是一種與變分自編碼器框架大約同時發展起來的深度網絡架構。GAN 的思想直接而簡單。與前面一樣，設 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是一個給定數據集，由從 $\mathbb{R}^d$ 上某個未知概率分佈 p_{data} 獨立抽取的 M 個樣本組成。GAN 的思想是構造一個**生成器**（generator）函數 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$，並將其參數化為帶參數 θ 的深度神經網絡，使得對於任意 $z \sim \mathcal{N}(0, I_d)$，$g_\theta(z)$ 都是一個服從概率分佈 p_{data} 的樣本。

現在的問題是，如何確定參數 θ，使得 $g_\theta(z)$ 確實服從未知分佈 p_{data}。為此，我們注意到我們既有來自 $\mathcal{D}$ 的真實樣本，也有由 $z \sim \mathcal{N}(0, I_d)$ 生成的樣本 $g_\theta(z)$。因此，我們可以給這些樣本賦予二元標籤，即給來自 $\mathcal{D}$ 的每個樣本賦標籤 1，給每個生成的（偽造的）樣本 $g_\theta(z)$ 賦標籤 0。更具體地，我們記

$$\mathcal{D}_{\text{real}} := \Big\{(x_i, 1) : x_i \in \mathcal{D},\ i \in [M]\Big\} \tag{5.19}$$

以及

$$\mathcal{D}_{\text{fake}} := \Big\{(g_\theta(z_j), 0) : z_j \sim \mathcal{N}(0, I_d),\ j \in [N]\Big\}, \tag{5.20}$$

其中 1 和 0 表示樣本的標籤。在 (5.20) 中，我們假設生成樣本的數量為任意的 N。

有了 (5.19) 中的兩個數據集 $\mathcal{D}_{\text{real}}$ 和 (5.20) 中的 $\mathcal{D}_{\text{fake}}$ 後，GAN 進一步引入了一個分類器（classifier），也稱為**判別器**（discriminator），它被參數化為一個帶參數 η 的**對抗網絡**（adversarial network）$a_\eta : \mathbb{R}^d \to (0, 1)$。這個對抗網絡 a_η 的訓練目標是區分 (5.20) 中來自 $\mathcal{D}_{\text{fake}}$ 的生成樣本與 (5.19) 中來自 $\mathcal{D}_{\text{real}}$ 的真實樣本。GAN 的示意圖見圖 5.2。

a_η 的功能建立在 Bernoulli 概率分佈之上。回憶一下，對於任意 $p \in (0, 1)$ 和 $X \sim \text{Bernoulli}(p)$，$X = 1$ 的概率為 p，而 $X = 0$ 的概率為 $1 - p$。因此，X 的概率質量函數可以寫為 $\text{Prob}(X = x) = p^x(1 - p)^{1-x}$，其中 $x = 0$ 或 1。基於這一表述，我們可以將 $a_\eta(x) \in (0, 1)$ 解釋為樣本 x 為真實樣本的概率。具體而言，對於任意來自 $\mathcal{D}_{\text{real}}$ 的 x_i，這一概率質量函數為 $a_\eta(x_i)$；而對於任意

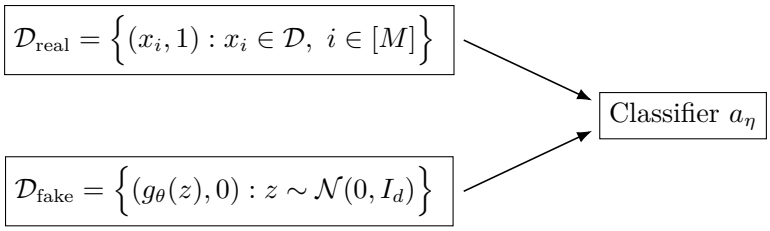


图 5.2: 生成對抗網絡（GANs）的示意圖。數據集 $\mathcal{D}_{\text{real}}$ 由 $\mathcal{D}$ 中給定的 M 個真實樣本及其標籤 1 組成。數據集 $\mathcal{D}_{\text{fake}}$ 由生成樣本 $g_\theta(z)$ 組成，其標籤為 0，其中 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$ 是參數為 θ 的生成器網絡，且 $z \sim \mathcal{N}(0, I_d)$。分類器 $a_\eta : \mathbb{R}^d \to (0, 1)$ 是參數為 η 的對抗網絡，它對每個輸入樣本賦予一個位於 $(0, 1)$ 中的概率值，表示它認為該樣本是真實樣本的概率。

來自 $\mathcal{D}_{\text{fake}}$ 的 $g_\theta(z_j)$，則為 $a_\eta(g_\theta(z_j))$。由於 $\mathcal{D}_{\text{real}}$ 和 $\mathcal{D}_{\text{fake}}$ 中的所有樣本都是獨立抽取的，並且它們的標籤已知分別為 1 和 0，因此我們可以構造它們的聯合概率分佈為

$$\Big(\prod_{i=1}^{M} a_\eta(x_i)\Big)^{1/M} \Big(\prod_{j=1}^{N} \big(1 - a_\eta(g_\theta(z_j))\big)\Big)^{1/N}, \tag{5.21}$$

其中我們去掉了所有等於 1 的項（例如 $(1 - a_\eta(x_i))^0 = a_\eta(g_\theta(z_j))^0 = 1$），並依據數據規模 M 和 N 用指數對該量做了歸一化。對 (5.21) 兩邊取對數，得到

$$E(\theta, \eta) := \frac{1}{M} \sum_{i=1}^{M} \log a_\eta(x_i) + \frac{1}{N} \sum_{j=1}^{N} \log(1 - a_\eta(g_\theta(z_j))). \tag{5.22}$$

現在我們來解釋 (5.22) 中 $E(\theta, \eta)$ 的含義。如果生成器 g_θ 固定，則分類器 a_η 應被訓練為給所有 x_i 賦值 1（或非常接近 1），並給所有 $g_\theta(z_j)$ 賦值 0（或非常接近 0）。這等價於在固定 θ 時，關於 η 最大化 $E(\theta, \eta)$。通過這種方式，分類器 a_η 可以提高其判別能力，從而正確地區分一個樣本是真實的還是生成的。

反過來，如果分類器 a_η 固定，則應訓練生成器 g_θ 去迷惑分類器，即找到一個參數 θ，使得對所有 x_i 和 z_j 都有 $a_\eta(x_i) = a_\eta(g_\theta(z_j)) = \frac{1}{2}$。這等價於在固定 η 時，關於 θ 最小化 $E(\theta, \eta)$。通過這種方式，生成器 g_θ 可以提高其生成能力，使得對分類器 a_η 來說，$g_\theta(z_j)$ 與真實樣本看起來沒有區別，因此 a_η 無論接收到什麼樣本都只能賦予概率 $\frac{1}{2}$，即 a_η 完全無法判斷該樣本是真實的還是生成的。

基於上述解釋，將訓練 θ 和 η 的問題設為如下鞍點（極小極大）問題是

很自然的:

$$\min_{\theta} \max_{\eta} \; E(\theta, \eta). \tag{5.23}$$

在實踐中，這個鞍點問題 (5.23) 可以通過交替地使用梯度下降更新 θ，然後梯度上升更新 η 來求解。其收斂可能不穩定，但在實際實現中有一些緩解這一問題的方法。

我們還可以進一步分析鞍點問題 (5.23) 的含義。設 θ 固定，且生成器 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$ 是一個可逆映射。記 p_θ 為當 $z \sim \mathcal{N}(0, I_d)$ 時生成樣本 $g_\theta(z)$ 的概率分佈。則根據變量替換公式，我們知道

$$p_\theta(x) = \mathcal{N}(g_\theta^{-1}(x); 0, I_d) \det(\nabla g_\theta^{-1}(x)). \tag{5.24}$$

若 $M, N \to \infty$，即我們擁有無限多的真實樣本和生成樣本，則 (5.22) 中的 $E(\theta, \eta)$ 變為

$$E(\theta, \eta) = \int p_{\text{data}}(x) \log a_\eta(x)\, dx + \int p_\theta(x) \log(1 - a_\eta(x))\, dx. \tag{5.25}$$

對於任意固定的 θ，我們發現 (5.25) 中 $E(\theta, \cdot)$ 的臨界點 η^* 應滿足

$$\frac{p_{\text{data}}(x)}{a_{\eta^*}(x)} - \frac{p_\theta(x)}{1 - a_{\eta^*}(x)} = 0,$$

這意味著

$$a_{\eta^*}(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_\theta(x)}$$

對每個 x 都成立。將這一最優的 η^* 代回 $E(\theta, \eta)$，可得

$$\begin{aligned} E(\theta, \eta^*) &= \int p_{\text{data}}(x) \log a_{\eta^*}(x)\, dx + \int p_\theta(x) \log(1 - a_{\eta^*}(x))\, dx \\ &= 2\,\mathrm{JS}(p_{\text{data}}, p_\theta) - 2\log 2, \end{aligned}$$

其中 $\mathrm{JS}(p, q) := (\mathrm{KL}(p, r) + \mathrm{KL}(q, r))/2$ 稱為兩個概率分佈 p 和 q 之間的 **Jensen–Shannon**（JS）散度，而 $r := (p + q)/2$ 是這兩個分佈的點態平均。（關於 JS 散度的更多細節見附錄 A.3。）因此，鞍點問題 (5.23) 等價於

$$\min_{\theta} \; \mathrm{JS}(p_{\text{data}}, p_\theta), \tag{5.26}$$

形式 (5.26) 表明，可以使用兩種概率分佈之間相當廣泛的一類差異或散

度，作為學習生成器網絡 g_θ 參數 θ 的目標函數。在原始 GAN 工作之後，一個流行的選擇是使用 Wasserstein-1 距離，即將 (5.26) 改為

$$\min_\theta \; W_1(p_{\text{data}}, p_\theta), \tag{5.27}$$

其中兩個概率分佈 p 和 q 之間的 Wasserstein-1 距離定義為

$$W_1(p,q) := \min_{\pi\in\Pi(p,q)} \iint |x-y|\pi(x,y)\,dxdy \tag{5.28}$$

而 $\Pi(x,y)$ 是定義在 $\mathbb{R}^d\times\mathbb{R}^d$ 上、邊緣分佈分別為 p 和 q 的聯合分佈集合，即

$$\Pi(p,q) := \left\{\pi:\mathbb{R}^d\times\mathbb{R}^d\to[0,\infty): \begin{array}{l} p(x)=\int\pi(x,y)\,dy, \\ q(y)=\int\pi(x,y)\,dx, \end{array} \; \forall\, x,y\in\mathbb{R}^d\right\}.$$

Wasserstein 距離具有許多重要的數學性質，包括它是在 $\mathbb{R}^d$ 上概率分佈空間中定義良好的一個度量。它是最優傳輸（optimal transport）領域中的關鍵概念之一，而該領域在過去幾十年中已被廣泛研究。關於 Wasserstein 距離的更多細節見附錄 A.3。然而，Wasserstein 距離的主要問題在於，由於其帶約束的結構和維數因素，它在數值計算中較為複雜。

為了求解最小化問題 (5.27)，我們可以將 Wasserstein-1 距離轉化為其對偶形式，而這一形式相對更容易處理。Wasserstein-1 距離 $W_1(p,q)$ 的對偶形式為

$$W_1(p,q) = \max_{f\in\mathrm{Lip}_1(\mathbb{R}^d)} \int f(x)p(x)\,dx - \int f(x)q(x)\,dx, \tag{5.29}$$

其中 $\mathrm{Lip}_1(\mathbb{R}^d)$ 是定義在 $\mathbb{R}^d$ 上的所有 1-Lipschitz 連續函數的集合，即若對任意 $x,y\in\mathbb{R}^d$ 有 $|f(x)-f(y)|\le|x-y|$，則 $f\in\mathrm{Lip}_1(\mathbb{R}^d)$。

利用 (5.29) 中 Wasserstein-1 距離的對偶形式，我們可以將最小化問題 (5.27) 重寫為一個等價形式，這就是所謂 Wasserstein GAN 模型中用於學習生成器網絡 g_θ 最優參數 θ 的最小化問題：

$$\min_\theta \max_{f\in\mathrm{Lip}_1(\mathbb{R}^d)} \int f(x)p_{\text{data}}(x)\,dx - \int f(x)p_\theta(x)\,dx. \tag{5.30}$$

(5.30) 中的第二個積分可以寫成一個更易處理的形式

$$\int f(x)p_\theta(x)\,dx = \int f(g_\theta(z))\mathcal{N}(z;0,I_d)\,dz$$

這是由變量替換公式 (5.24) 得到的。

在實際實現中，(5.30) 中的積分用 Monte Carlo 積分替代，而 $f:\mathbb{R}^d\to\mathbb{R}$ 則被參數化為一個深度神經網絡，並對其參數施加適當約束，使得 f 近似滿足 $f\in \mathrm{Lip}_1(\mathbb{R}^d)$。從這個意義上說，$f$ 在 Wasserstein GAN 中扮演著與原始 GAN 工作中的對抗網絡 a_η 相似的角色。感興趣的讀者可以參考本章末尾列出的相關文獻以獲取更多細節。

在生成器網絡 g_θ 訓練完成後，我們可以先從 $\mathcal{N}(0,I_d)$ 中模擬任意多個樣本，然後將 g_θ 應用於這些標準高斯樣本，從而得到新的樣本，而這些新樣本服從概率分佈 p_{data}。

5.3 擴散模型

擴散模型（Diffusion Models）是一類新的生成模型。其目標仍然是在僅給定數據集 $\mathcal{D}=\{x_i\in\mathbb{R}^d : i\in[M]\}$ 的情況下，學習如何生成服從 $\mathbb{R}^d$ 上某個未知概率分佈 p_{data} 的新樣本，其中該數據集由從 p_{data} 獨立抽取的 M 個樣本組成。與之前一樣，我們並不知道 p_{data} 的解析形式。

擴散模型採用了一種新穎的樣本生成方法：在 $\mathbb{R}^d$ 上、時間區間 $[0,T]$ 內構造一個**隨機微分方程**（stochastic differential equation，SDE），其中 $T>0$ 是任意選定的：

$$dZ_t=\mu_t(Z_t)dt+(\Sigma_t(Z_t))^{1/2}dW_t, \tag{5.31}$$

其中

$$\mu:\ \mathbb{R}^d\times[0,T]\to\mathbb{R}^d$$

稱為**漂移**（drift），而

$$\Sigma:\ \mathbb{R}^d\times[0,T]\to\mathbb{R}^{d\times d}$$

稱為**擴散**（diffusion）係數。μ 和 Σ 都是隨時間變化的。這個 SDE 的性質是：從 $\mathbb{R}^d$ 上某個簡單的初始化概率 p_{init} 出發（通常取為 $\mathcal{N}(0,I_d)$ 或類似形式），該 SDE (5.31) 可以將任意在時刻 $t=0$ 從 p_{init} 中抽取的初值 Z_0 沿時間正向推進到時刻 $t=T$ 的 Z_T，並使得 Z_T 服從概率分佈 p_{data}，即 Z_T 是從 p_{data} 中抽取的樣本。

注意，由於目標是生成任意多個服從 p_{data} 的新樣本 Z_T，我們需要能夠方便地從 p_{init} 中抽取初值 Z_0。我們還需要條件概率分佈具有閉式形式，以便於後續訓練損失函數的實現，這一點我們稍後會詳細說明。基於這些原因，初始化概率 p_{init} 往往選取得較為簡單，以便容易從中採樣。如上所述，p_{init} 的

一個典型選擇是定義在 $\mathbb{R}^d$ 上的標準高斯分佈 $\mathcal{N}(0, I_d)$，或者其某種平移均值或縮放方差的簡單變體。

為了實現簡單和計算高效，Σ 通常被設為單位矩陣 I_d 的一個與狀態無關的標量倍數，即 σI_d，其中 $\sigma : [0,T] \to [0,\infty)$ 僅依賴時間 t。這種簡化不影響擴散模型的核心思想，並且很容易推廣到更一般的情形。

現在擴散模型的目標可以概括如下：找到一個漂移 $\mu : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 和一個擴散係數 $\sigma : [0,T] \to [0,\infty)$，使得對於任意滿足初值 $Z_0 \sim p_{\text{init}} = \mathcal{N}(0, I_d)$ 且遵循如下 SDE 的軌道 Z_t（為簡化記號，以下記 $\mu_t(z) := \mu(z,t) \in \mathbb{R}^d$ 和 $\sigma_t := \sigma(t) \in \mathbb{R}$）：

$$dZ_t = \mu_t(Z_t)dt + \sigma_t dW_t \tag{5.32}$$

在區間 $[0,T]$ 上成立，並且有 $Z_T \sim p_{\text{data}}$。一旦找到了這樣的 μ 和 σ，我們就可以任意多次地抽取 $Z_0 \sim p_{\text{init}}$，並數值求解 SDE (5.32)（例如使用 Euler-Maruyama 方法等數值 SDE 求解器），從每個 Z_0 得到一個新的樣本 $Z_T \sim p_{\text{data}}$。

由此可見，現在的主要任務是為 (5.32) 找到這樣的 μ_t 和 σ_t。為此，擴散模型首先嚐試確定另一個 SDE

$$dX_t = f_t(X_t)dt + g_t dW_t \tag{5.33}$$

其中帶有人工設計的 $f : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 和擴散係數 $g : [0,T] \to [0,\infty)$，使得 (5.33) 恰好執行與 (5.32) 相反的過程：也就是說，對於任意選取為服從 p_{data} 的給定樣本之一的初值 X_0，我們都可以數值求解 (5.33) 並得到 $X_T \sim p_{\text{init}}$。從這個意義上說，SDE (5.33) 可以將 p_{data} 轉換為 p_{init}。設計這一新的 SDE (5.33) 的目的，是為了模擬其軌道並學習相應的時間演化概率。通過這樣做，我們就能學習到所需要的 μ_t 和 σ_t，這一點將在本節稍後說明。

在繼續之前，我們指出，一般而言，要為 (5.33) 找到合適的 f 和 g，使得從 $X_0 \sim p_{\text{data}}$ 出發時有 $X_T \sim p_{\text{init}}$，是非常困難的。我們將在本節進一步討論這一問題。儘管如此，要找到使得當 T 足夠大時 X_T 近似服從 p_{init} 的 f 和 g 卻並不困難。例如，一個常見的選擇是使用 Ornstein–Uhlenbeck（OU）過程，並使其擴散係數與漂移相匹配：

$$dX_t = -aX_t dt + \sqrt{2a}\, dW_t. \tag{5.34}$$

也就是說，

$$f_t(x) = -ax \quad \text{和} \quad g_t \equiv \sqrt{2a} \tag{5.35}$$

對所有 t 和 x 都成立，其中 $a > 0$ 是用戶選定的常數。對於該 SDE (5.34)，已知在初值 $X_0 = x_0 \sim p_{\text{data}}$ 條件下，X_t 的條件概率是高斯分佈：

$$\mathcal{N}(x_0 e^{-at}, (1 - e^{-2at}) I_d)$$

對任意 $t \geq 0$ 成立。因此，我們有

$$X_T | (X_0 = x_0) \ \sim \ \mathcal{N}(x_0 e^{-aT}, (1 - e^{-2aT}) I_d) \ \approx \ \ p_{\text{init}} := \mathcal{N}(0, I_d)$$

當 T 足夠大時，對任意 x_0 都成立，而與 p_{data} 的具體形式無關。目前，我們將接受 OU 過程 (5.34) 作為 (5.33) 的選擇，並接受 $X_0 \sim p_{\text{data}}$ 且 $X_T \sim p_{\text{init}} = \mathcal{N}(0, I_d)$ 這一設定。

現在的問題是，(5.35) 中這些簡單的 f 和 g 與 (5.32) 中所需的 μ 和 σ 有什麼關係，以及如何利用 f 和 g 來構造 μ 和 σ。

為了回答這兩個問題，我們注意到，SDE (5.33) 的 Fokker–Planck 方程（關於 Fokker–Planck 方程的細節，見附錄 A.4）為

$$\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{1}{2} g_t^2 \Delta \rho_t \tag{5.36}$$

其中 $\rho : \mathbb{R}^d \times [0, T] \to [0, \infty)$ 是每個 $t \in [0, T]$ 時刻 X_t 的概率密度，並滿足 $\rho_0 = p_{\text{data}}$ 以及 $\rho_T = p_{\text{init}}$。另一方面，我們也有 SDE (5.32) 的 Fokker–Planck 方程：

$$\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t) + \frac{1}{2} \sigma_t^2 \Delta \pi_t \tag{5.37}$$

它定義在 $\mathbb{R}^d \times [0, T]$ 上，並且求解 (5.32) 的過程 Z_t 滿足對所有 t 都有 $Z_t \sim \pi_t$。

通過比較這兩個 Fokker–Planck 方程 (5.36) 和 (5.37)，可以證明：如果 $\pi_0 = \rho_T$，並且我們根據 f_t 和 g_t 適當地選擇 μ_t 和 σ_t，則對所有 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。該結果見下述定理。

定理 5.3.1. *設 ρ 和 π 分別滿足 Fokker–Planck 方程 (5.36) 和 (5.37)，並且 ρ_t 與 π_t 處處為正，且 $\pi_0 = \rho_T$。假設對每個 $t \in [0, T]$，$f_t : \mathbb{R}^d \to \mathbb{R}^d$ 都是某個常數 $L_f > 0$ 下的 L_f-Lipschitz 連續函數。如果*

$$\mu_t(x) = -f_{T-t}(x) + \frac{1}{2} g_{T-t}^2(x) \frac{\nabla(\rho_{T-t}(x) + \pi_t(x))}{\pi_t(x)} \tag{5.38}$$

並且

$$\sigma_t = g_{T-t} \tag{5.39}$$

對所有 $x \in \mathbb{R}^d$ 和 $t \in [0,T]$ 成立，則

$$\pi_t = \rho_{T-t} \tag{5.40}$$

對所有 $t \in [0,T]$ 在 $\mathbb{R}^d$ 上幾乎處處成立。特別地，$\pi_T = \rho_0 = p_{data}$。

证明. 我們將 ρ_{T-t} 和 π_t 之間的平方 L^2 誤差的一半記為

$$\delta(t) := \frac{1}{2}\|\rho_{T-t} - \pi_t\|^2_{L^2(\mathbb{R}^d)} = \frac{1}{2}\int_{\mathbb{R}^d} |\rho_{T-t}(x) - \pi_t(x)|^2\, dx$$

其中 $t \in [0,T]$。在本證明餘下部分，為簡化記號，我們省略 $\mathbb{R}^d$ 和 (x)。現在注意到

$$\begin{aligned}
\delta'(t) &= \int (\rho_{T-t} - \pi_t)(-\partial_t \rho_{T-t} - \partial_t \pi_t)\, dx \\
&= \int (\rho_{T-t} - \pi_t)\Big[\nabla\cdot(f_{T-t}\rho_{T-t}) - \frac{1}{2}g^2_{T-t}\Delta\rho_{T-t} + \nabla\cdot(\mu_t\pi_t) - \frac{1}{2}\sigma_t^2\Delta\pi_t\Big]\, dx \\
&= \int (\rho_{T-t} - \pi_t)\Big[\nabla\cdot\Big(f_{T-t}\rho_{T-t} - \frac{1}{2}g^2_{T-t}\nabla\rho_{T-t} + \mu_t\pi_t - \frac{1}{2}\sigma_t^2\nabla\pi_t\Big)\Big]\, dx \\
&= \int (\rho_{T-t} - \pi_t)[\nabla\cdot(f_{T-t}(\rho_{T-t} - \pi_t))]\, dx \\
&= -\frac{1}{2}\int f_{T-t}\nabla|\rho_{T-t} - \pi_t|^2\, dx \\
&= \frac{1}{2}\int (\nabla\cdot f_{T-t})|\rho_{T-t} - \pi_t|^2\, dx \\
&\le \|\nabla\cdot f_{T-t}\|_{L^\infty}\delta(t) \\
&\le dL_f\delta(t),
\end{aligned}$$

其中第二個等號由 (5.36) 和 (5.37) 得到，第三個等號利用了 $\Delta\rho = \nabla\cdot(\nabla\rho)$（對 π 同理），第四個等號由 (5.38) 和 (5.39) 得到，第五和第六個等號由微積分中的散度定理得到，而最後兩個不等式則來自於

$$|\nabla\cdot f_{T-t}| \le \|\nabla\cdot f_{T-t}\|_{L^\infty(\mathbb{R}^d)} \le dL_f$$

這是由於每個時刻 t 上 f_t 的 L_f-Lipschitz 連續性所致。

根據 Grönwall 不等式，我們知道

$$\delta(t) \le \delta(0)e^{dL_f t} \le \delta(0)e^{dL_f T} = 0$$

$$\boxed{X_0 \sim \rho_0 = p_{\text{data}}} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{1}{2} g_t^2 \Delta \rho_t]{dX_t = f_t(X_t)dt + g_t dW_t} \boxed{X_T \sim \rho_T \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

$$\boxed{Z_T \sim \pi_T = p_{\text{data}}} \xleftarrow[\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t) + \frac{1}{2} \sigma_t^2 \Delta \pi_t]{dZ_t = \mu_t(Z_t)dt + \sigma_t dW_t} \boxed{Z_0 \sim \pi_0 \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

图 5.3: 擴散模型的正向過程（上）與反向過程（下）。在上圖中，時間 t 從左到右增加（從 0 到 T）；在下圖中，時間 t 從右到左增加。正向過程由 X_t 的 SDE 描述，或者等價地，由 ρ_t 的 Fokker–Planck 方程描述，其中對每個 t 都有 $X_t \sim \rho_t$。反向過程由 Z_t 的 SDE 描述，或者等價地，由 π_t 的 Fokker–Planck 方程描述，其中對每個 t 都有 $Z_t \sim \pi_t$。f_t、g_t、μ_t 和 σ_t 之間的關係由 (5.38) 和 (5.39) 給出，它們推出對每個 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。注意，(5.38) 可以寫成 (5.41)，從而給出 μ_t 關於 f_t、g_t 和 $\nabla \log \rho_t$ 的表達式。

對所有 t 都成立，因為 $\delta(0) = \frac{1}{2}\|\rho_T - \pi_0\|^2_{L^2(\mathbb{R}^d)} = 0$。這說明

$$\rho_{T-t} = \pi_t$$

對所有 $t \in [0, T]$ 在 $\mathbb{R}^d$ 上幾乎處處成立，從而完成證明。 □

定理 5.3.1 提示我們將 (5.38) 中的 μ 設為如下形式：由於對所有 t 都有 $\pi_t = \rho_{T-t}$，因此 (5.38) 可化為

$$\mu_t = -f_{T-t} + g_{T-t}^2 \frac{\nabla \rho_{T-t}}{\rho_{T-t}} = -f_{T-t} + g_{T-t}^2 \nabla \log \rho_{T-t}. \tag{5.41}$$

這表明，Fokker–Planck 方程 (5.37) 變為

$$\partial_t \pi_t = -\nabla \cdot \Big(\left(-f_{T-t} + g_{T-t}^2 \nabla \log \rho_{T-t}\right) \pi_t \Big) + \frac{1}{2} g_{T-t}^2 \Delta \pi_t. \tag{5.42}$$

因此，我們知道，與 Fokker–Planck 方程 (5.42) 相對應的 SDE (5.32) 變為

$$dZ_t = (-f_{T-t}(Z_t) + g_{T-t}^2 \nabla \log \rho_{T-t}(Z_t))dt + g_{T-t} dW_t. \tag{5.43}$$

此外，如果我們從初值 $Z_0 \sim \pi_0 = \mathcal{N}(0, I_d)$ 開始，那麼通過求解 (5.43)，就會在時間 T 得到一個新的樣本 $Z_T \sim p_{\text{data}}$。上述關係如圖 5.3 所示。

由於我們已經知道了例如 (5.35) 中給出的 f 和 g，因此在 (5.43) 中唯一尚需確定的項是對每個 t 的函數 $\nabla \log \rho_t$。為此，我們需要一種稱為**得分匹配**（score matching）的方法。得分匹配的目的是近似 $\nabla \log \rho : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，

該函數稱為 **Stein 得分函數**（Stein score function）（在本節中我們簡稱其為得分函數），其中對所有 $t \in [0,T]$，都有 $X_t \sim \rho_t$，X_t 是滿足 SDE (5.33) 的過程，而 ρ_t 是相應 Fokker–Planck 方程 (5.36) 的解。

直覺上，近似 $\nabla \log \rho$ 的方法是：將一個映射 $s : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 參數化為深度神經網絡，通過最小化對所有 x 和 t 的 $|s_t(x) - \nabla \log \rho_t(x)|^2$ 來訓練 s，並用 s_t 作為得分函數 $\nabla \log \rho_t$ 的近似。然而，這在實踐中不可行，因為我們並不知道 $\nabla \log \rho_t$ 的真實值。

我們將得分匹配作為一種實用的替代方法來近似得分函數 $\nabla \log \rho_t$。為了具體化這一過程，我們使用 OU 過程 (5.34) 作為 SDE (5.33)。正如前面提到的，在給定初值 $X_0 = x_0$ 的條件下，$X_t = x_t$ 的條件概率為

$$\rho_t(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \beta_t^2 I_d), \tag{5.44}$$

其中

$$\alpha_t = e^{-at} \quad \text{和} \quad \beta_t^2 = 1 - e^{-2at}.$$

因此，由於高斯分佈的簡單性，我們可以計算**條件**得分函數：

$$\nabla_{x_t} \log \rho_t(x_t|x_0) = -\frac{x_t - \alpha_t x_0}{\beta_t^2}. \tag{5.45}$$

在 (5.45) 中，我們特別說明梯度是相對於 x_t 的，因為 $\rho_t(x_t|x_0)$ 同時是 x_t 和 x_0 的函數。

注意, (5.45) 並不是我們希望網絡 s 去近似的邊緣得分函數 $\nabla_{x_t} \log \rho_t(x_t)$。然而，我們可以找到一種方法，利用條件得分函數 $\nabla_{x_t} \log \rho_t(x_t|x_0)$ 來表示

$\nabla_{x_t} \log \rho_t(x_t)$。為此，我們注意到

$$
\begin{aligned}
\nabla_{x_t} \log \rho_t(x_t) &= \frac{\nabla_{x_t} \rho_t(x_t)}{\rho_t(x_t)} \\
&= \frac{1}{\rho_t(x_t)} \nabla_{x_t} \Big(\int \rho_t(x_t, x_0)\, dx_0 \Big) \\
&= \frac{1}{\rho_t(x_t)} \nabla_{x_t} \Big(\int \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \Big) \\
&= \frac{1}{\rho_t(x_t)} \int \nabla_{x_t} \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \qquad (5.46) \\
&= \frac{1}{\rho_t(x_t)} \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \\
&= \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \frac{\rho_t(x_t|x_0) p_{\text{data}}(x_0)}{\rho_t(x_t)}\, dx_0 \\
&= \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \rho_t(x_0|x_t)\, dx_0 \\
&= \mathbb{E}_{X_0 \sim \rho_t(\cdot|X_t = x_t)} [\nabla_{x_t} \log \rho_t(X_t|X_0)]
\end{aligned}
$$

其中，在第二個等號中 $\rho_t(x_t, x_0)$ 是時刻 t 下 (X_t, X_0) 的聯合概率分佈；第三個等號使用了 $\rho_t(x_t, x_0) = \rho_t(x_t|x_0)\rho_0(x_0) = \rho_t(x_t|x_0)p_{\text{data}}(x_0)$。在第四個等號中，我們利用 Fubini 定理交換了關於 x_t 的梯度與關於 x_0 的積分；在第五個等號中，由於 $\rho_t(x_t)$ 對被積函數而言是常數，因此將該分母移入積分；第六個等號則使用了 Bayes 公式。

(5.46) 給出的關係展示了得分函數 $\nabla_{x_t} \log \rho_t(x_t)$ 與條件得分函數 $\nabla_{x_t} \log \rho_t(x_t|x_0)$ 之間的有用聯繫。然而，我們仍然不知道後驗概率 $\rho_t(x_0|x_t)$，也不知道如何從中採樣，以便計算 (5.46) 末尾的期望。為了將這個期望寫成某種可計算的形式，我們需要用到一個概率論中的簡單事實，如下引理所示。

引理 5.3.2 (Mean squared error)**.** 設 X 和 Y 分別是取值於 $\mathbb{R}^d$ 和 $\mathbb{R}^m$ 的兩個隨機變量。令 $w : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^k$ 為一個函數。則對任意 $y \in \mathbb{R}^m$，定義 $\phi_y : \mathbb{R}^k \to \mathbb{R}$ 為

$$\phi_y(c) := \mathbb{E}_{X \sim p_{X|Y}(\cdot|Y=y)} [|c - w(X, Y)|^2].$$

則 ϕ_y 有唯一極小值點 $c_y \in \mathbb{R}^k$，其表達式為

$$c_y := \mathbb{E}_{X \sim p_{X|Y}(\cdot|Y=y)} [w(X, Y)].$$

证明. 注意到

$$\begin{aligned}\phi_y(c) &= \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[|c-w(X,Y)|^2] \\ &= \int |c-w(x,y)|^2 p_{X|Y}(x|y)\,dx.\end{aligned}$$

由於對每個 x 都有 $p_{X|Y}(x|y)\geq 0$，可知 ϕ_y 是關於 c 的凸函數。因此，ϕ_y 的每個臨界點都是其全局極小值點。

現在我們對 ϕ_y 關於 c 求梯度，令其等於 0，並求其臨界點。為此，有

$$\nabla_c\phi_y(c) = 2\int (c-w(x,y))p_{X|Y}(x|y)\,dx = 0,$$

從而推出唯一解

$$c = \int c\,p_{X|Y}(x,y)\,dx = \int w(x,y)p_{X|Y}(x,y)\,dx = \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[w(X,Y)].$$

證畢。 □

引理 5.3.2 蘊含函數 $u: y\mapsto c_y$ 是良定義的，因為 c_y 存在且唯一。其結果總結在如下定理中。

定理 5.3.3. 設引理 *5.3.2* 中的條件成立。定義 $u:\mathbb{R}^m\to\mathbb{R}^k$，使其將任意 $y\in\mathbb{R}^m$ 映射為

$$u(y) = \arg\min_{c\in\mathbb{R}^m} \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[|c-w(X,Y)|^2].$$

則對每個 $y\in\mathbb{R}^m$，都有

$$u(y) = \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[w(X,Y)]$$

成立。

证明. 該結論由引理 5.3.2 立即得到，因此略去證明。 □

定理 5.3.3 使我們能夠將感興趣的條件期望改寫為一個最小二乘優化問

題。特別地，我們注意到 (5.46) 實際上等價於

$$\begin{aligned}\nabla_{x_t}\log\rho_t(x_t) &= \mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[\nabla_{x_t}\log\rho_t(X_t|X_0)]\\ &= \arg\min_{c\in\mathbb{R}^d}\mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[|c-\nabla_{x_t}\log\rho_t(X_t|X_0)|^2].\end{aligned}$$

這可以通過應用定理 5.3.3 看出，其中 X、Y 和 $w(X,Y)$ 分別替換為 X_0、X_t 和 $\nabla_{x_t}\log\rho_t(X_t|X_0)$（可將其視為 X_0 與 X_t 的函數）。

到這裡，我們已經構造出了一個可以用於訓練深度神經網絡 s 使得 $s_t = \nabla\log\rho_t$ 的損失函數。更具體地，由於我們希望對所有 x 和 t 都有 $s_t(x) = \nabla\log\rho_t(x)$，我們可以使用通過數值求解 SDE (5.33)（即當前情形下 (5.34) 中的 OU 過程）得到的樣本軌道中的 x_t。由此可知，對每個 $t\in[0,T]$，s_t 可以通過最小化下式學習出來：

$$\begin{aligned}&\int\mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[|s_t(x_t)-\nabla_{x_t}\log\rho_t(X_t|X_0)|^2]\,dx_t\\ =&\iint|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)|^2]\rho_t(x_0|x_t)\rho_t(x_t)\,dx_0dx_t\\ =&\iint|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)|^2]\rho_t(x_t|x_0)p_{\text{data}}(x_0)\,dx_tdx_0\end{aligned}\tag{5.47}$$

其中第二個等號使用了

$$\rho_t(x_0|x_t)\rho_t(x_t)=\rho_t(x_t,x_0)=\rho_t(x_t|x_0)\rho_0(x_0)$$

以及 $\rho_0(x_0)=p_{\text{data}}(x_0)$。

由於我們通常希望在整個時間區間 $[0,T]$ 上同時訓練 s_t，因此可以人為設定一個定義在 $[0,T]$ 上的概率密度 $\xi(t)$，滿足對所有 t 都有 $\xi(t)>0$ 且 $\int_0^T\xi(t)\,dt=1$，並通過求解

$$\min_s\int_0^T\iint\Big|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)\Big|^2\rho_t(x_t|x_0)p_{\text{data}}(x_0)\,dx_tdx_0dt,\tag{5.48}$$

來訓練網絡 $s:\mathbb{R}^d\times[0,T]\to\mathbb{R}^d$，其中最小化是相對於 s 的網絡參數進行的。

為了便於實際實現，我們還可以進一步簡化 (5.48) 中的損失函數。當我們使用 OU 過程 (5.34) 來生成樣本軌道 $\{x_t\}$ 和 ρ_t 時，條件概率 $\rho_t(x_t|x_0)$ 由 (5.44) 給出，而條件得分函數 $\nabla_{x_t}\log\rho_t(x_t|x_0)$ 則由 (5.45) 給出。事實上，由於 $\rho_t(x_t|x_0)=\mathcal{N}(x_t;\alpha_tx_0,\beta_t^2I_d)$，我們可以直接從標準高斯分佈 $\mathcal{N}(0,I_d)$ 中

獨立採樣 ϵ，並將 x_t 改寫為 x_0 和 ϵ 的函數如下：

$$x_t = \alpha_t x_0 + \beta_t \epsilon. \tag{5.49}$$

在這種情況下，條件概率 $\rho_t(x_t|x_0)$ 變得更加簡單：

$$\text{当} \quad X_t = a_t x_0 + \beta_t \epsilon \quad \text{时} \quad X_t \sim \rho_t(\cdot|x_0), \quad \text{其中} \quad \epsilon \sim \mathcal{N}(0, I_d), \tag{5.50}$$

而條件得分函數則化簡為

$$\nabla_{x_t} \log \rho_t(x_t|x_0) = -\frac{\epsilon}{\beta_t}. \tag{5.51}$$

為了抵消在 OU 過程中當 $t \to 0$ 時 $\beta_t \to 0$ 的影響，我們可以選取 $\xi(t) \propto \beta_t^2$。在實踐中，我們還可以把待訓練的網絡設為 $\varepsilon : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，而不是 (5.48) 中使用的 s，使得

$$s_t(x) = -\frac{\varepsilon_t(x)}{\beta_t} \tag{5.52}$$

對所有 x 和 t 都成立。將 (5.49)、(5.50) 和 (5.51) 代入 (5.48)，我們得到一個訓練網絡 ε 的極其簡單的最小化問題：

$$\min_{\varepsilon} \int_0^T \iint |\varepsilon_t(\alpha_t x_0 + \beta_t \epsilon) - \epsilon|^2 \mathcal{N}(\epsilon; 0, I_d) p_{\text{data}}(x_0)\, dx_0 d\epsilon dt. \tag{5.53}$$

在數值實現中，(5.53) 中的損失函數可以通過 Monte Carlo 積分（見附錄 A.1）近似為

$$\frac{1}{NM} \sum_t \sum_{(x_0, \epsilon)} |\varepsilon_t(\alpha_t x_0 + \beta_t \epsilon) - \epsilon|^2. \tag{5.54}$$

更具體地說，我們先從定義在 $(0, 1)$ 上的概率密度 ξ 中採樣 N 個時間點。然後，對每個採樣到的時間點 t，從原始數據集 $\mathcal{D}$（由服從 p_{data} 分佈的 M 個樣本組成）中選取一個樣本 x_0，並獨立地抽取一個 $\epsilon \sim \mathcal{N}(0, I_d)$。這些樣本被代入 (5.54) 中，這就是我們在實踐中用於訓練網絡 ε 的損失函數。一旦網絡 ε 訓練完成，我們就可以按 (5.52) 定義 s_t，並知道它是得分函數 $\nabla \log \rho_t$ 的一個近似。

用學習好的得分函數生成新樣本 一旦我們得到了近似得分函數 s_t，就可以在 (5.43) 中用它替代真實得分函數 $\nabla \log \rho_t$。注意，對於 OU 過程，我們已知

f 和 g 具有 (5.35) 中的簡單形式，即 $f_t = -ax$ 且 $g_t = \sqrt{2a}$。於是用於生成的 SDE (5.43) 為

$$dZ_t = (-aZ_t + 2as_t(Z_t))dt + \sqrt{2a}\, dW_t. \tag{5.55}$$

實際的樣本生成可以按照 Euler-Maruyama 格式進行：將區間 $[0, T]$ 劃分為長度相等的子區間，步長為 $h > 0$，然後計算

$$Z_{t+h} = Z_t + (-aZ_t + 2as_t(Z_t))h + \sqrt{2ah}\zeta_t \tag{5.56}$$

其中 $t = 0, h, 2h, \ldots, T$，初值 Z_0 與擾動項 ζ_t 都從 $\mathcal{N}(0, I_d)$ 中相互獨立地採樣。這樣，Z_T 就是一個新生成的樣本，並且服從概率分佈 p_{data}。

(5.56) 有一個明顯缺點：在每一個時間點 t，我們都需要從標準高斯分佈中採樣一個 ζ_t。當時間步很多且我們需要生成大量樣本時，這會非常慢。

我們可以通過將 SDE (5.55) 轉換為一個 ODE 來克服這一缺點，使得其解在每個時刻 t 都服從相同的概率分佈 π_t。為此，我們注意到在 (5.42) 中，$\Delta\pi_t = \nabla \cdot (\pi_t \nabla \log \pi_t)$。此外，由定理 5.3.1 可知，對所有 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。因此，Fokker–Planck 方程 (5.42) 可以改寫為關於 π 的連續方程：

$$\partial_t \pi_t = -\nabla \cdot \Big((-f_{T-t} + \frac{1}{2} g_{T-t}^2 \nabla \log \rho_{T-t})\, \pi_t \Big), \tag{5.57}$$

對應的 ODE 為

$$\dot{z}_t = -f_{T-t}(z_t) + \frac{1}{2} g_{T-t}^2 \nabla \log \rho_{T-t}(z_t). \tag{5.58}$$

其中 $\dot{z}_t$ 表示 z_t 關於時間的導數。在實踐中，我們可以用訓練得到的得分函數 s_{T-t} 替代 $\nabla \log \rho_{T-t}$，並從任意初值 $z_0 \sim \mathcal{N}(0, I_d)$ 出發求解該 ODE (5.58) 得到 z_T。所生成的 z_T 同樣服從所需的分佈 $\pi_T = \rho_0 = p_{\text{data}}$。

有許多數值 ODE 求解器，例如中點法和四階 Runge-Kutta 方法 [24]，它們在求解 ODE (5.58) 時比求解 SDE (5.55) 快得多。相對於訓練得到的得分函數 s_t 對真實得分函數 $\nabla \log \rho_t$ 的近似誤差而言，這些求解器帶來的數值誤差通常是非常小的。

可用於生成訓練用樣本軌跡的其它 SDE 事實上，除了 OU 過程 (5.34) 之外，還有許多其他備選過程可用於設計 (5.33)。例如，下面這類方差保持型（variance preserving，VP）SDE 是 OU 過程的一個小推廣，並且在實際實現

中經常使用:

$$dX_t = -\gamma_t dt + \sqrt{2\gamma_t} dW_t, \tag{5.59}$$

其中對每個 t，γ_t 都取值於區間 $[\gamma_{\min}, \gamma_{\max}]$ 內。這裡的上下界 $\gamma_{\min}$ 和 $\gamma_{\max}$ 是用戶指定的正數，而 $\gamma_{\min}$ 往往被設為一個較小的正數，例如 0.01，以防止 γ_t 過於接近 0 並造成數值計算不穩定。另一方面，$\gamma_{\max}$ 的選擇則較為靈活，例如可以取 20。

VP SDE (5.59) 與 OU 過程類似，其解在給定初值 $X_0 = x_0$ 條件下也服從高斯分佈。更準確地說，在給定 $X_0 = x_0$ 的條件下，X_t 的條件概率分佈為

$$\rho_t(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \beta_t^2 I_d)$$

其中

$$\alpha_t = e^{-\int_0^t \gamma_\tau d\tau} \tag{5.60}$$

並且

$$\beta_t^2 = 1 - e^{-2\int_0^t \gamma_\tau \, d\tau}. \tag{5.61}$$

對於 (5.59) 中的 γ_t，一個簡單選擇是

$$\gamma_t = \gamma_{\min} + t(\gamma_{\max} - \gamma_{\min})$$

其中 $t \in [0, 1]$。此時

$$\int_0^t \gamma_\tau \, d\tau = t\gamma_{\min} + \frac{t^2}{2}(\gamma_{\max} - \gamma_{\min}),$$

因此 (5.60) 和 (5.61) 中的 α_t 和 β_t 都有閉式表達式。我們指出，儘管名稱如此，VP SDE (5.59) 並不是真的在所有時間都保持方差不變，但它的方差確實會隨著 t 變大而趨於某個常數。

關於選擇 SDE (5.33) 的討論 由於樣本數據分佈 p_{data} 在不同應用中可能各不相同，我們希望找到某種 “通用” 的漂移 f（以及擴散係數 g），使得正向時間 SDE (5.33) 能夠把 $X_0 \sim \rho_0 = p_{\text{data}}$ 移動到 $X_T \sim \rho_T = p_{\text{init}} = \mathcal{N}(0, I_d)$。這裡所謂 “通用”，是指 f 和 g 被選為使得 SDE (5.33)，或者等價地 Fokker–Planck 方程 (5.36)，能夠在有限時間 T 內把**任意給定的** p_{data} 轉換為 $\mathcal{N}(0, I_d)$。換言之，f 和 g 的選取不應依賴於 p_{data}。然而，這並不總是容易做到，因為在實踐中 p_{data} 是未知且通常相當複雜的概率分佈。事實上，我們可以斷言，這樣

的通用 f 和 g 並不存在，如下命題所示。

命題 5.3.4. *如果存在某些 f 和 g，使得 Fokker–Planck 方程 (5.36) 能夠將 $\rho_0 = p_{data}$ 轉換到 $\rho_T = \mathcal{N}(0, I_d)$，那麼 f 和 g 必須依賴於 p_{data}。*

证明. 我們採用反證法。假設 f 和 g 不依賴於 p_{data}。根據命題 5.3.1 中所描述的結論，存在一個 π，使得對每個 t 都有 $\pi_t = \rho_{T-t}$ 在 $\mathbb{R}^d$ 上幾乎處處成立，其中 ρ_t 滿足帶有此類 f 和 g 的 (5.36)，且端點條件為 $\rho_0 = p_{\text{data}}$ 和 $\rho_T = \mathcal{N}(0, I_d)$。這意味著，$\pi$ 的 Fokker–Planck 方程 (5.42) 可寫為

$$\partial_t \pi_t = -\nabla \cdot \Big((-f_{T-t} + g_{T-t}^2 \nabla \log \pi_t) \pi_t \Big) + \frac{1}{2} g_{T-t}^2 \Delta \pi_t.$$

其初值為 $\pi_0 = \rho_T = \mathcal{N}(0, I_d)$。然而，$\pi_0$、$f$ 和 g 中都不包含任何關於 p_{data} 的信息。這與 $\pi_T = \rho_0 = p_{\text{data}}$ 相矛盾。 □

由此可見，在有限時間區間 $[0, T]$ 內將 p_{data} 轉換為 p_{init} 本身就是一個不簡單的問題，或許並不比將 p_{init} 轉換為 p_{data} 簡單多少。儘管我們使用了 OU 過程 (5.34) 使得概率分佈 ρ_T 近似於 p_{init}，但根據命題 5.3.4，我們知道不可能在有限時間 T 內精確到達 $p_{\text{init}} = \mathcal{N}(0, I_d)$。從理論上講，我們應當把通過 Fokker–Planck 方程 (5.36) 或 SDE (5.33) 求得的真實 ρ_T 作為樣本生成中的初始分佈 π_0。然而，這在實踐中並不可行，因為對 ρ_T 進行評估或從中採樣同樣是困難的。

人們自然會思考，是否存在某個合適的漂移 f（以及擴散係數 g），使得 Fokker–Planck 方程 (5.36) 的解 ρ 滿足 $\rho_0 = p_{\text{data}}$ 且 $\rho_T = \mathcal{N}(0, I_d)$（或者反過來，這正是最初的目標生成問題）。在對 p_{data} 施加某些溫和條件的情況下，這個問題的答案是肯定的。

事實上，這與薛定諤橋問題 Schrödinger bridge problem（SBP）的可行性有關：給定 $\mathbb{R}^d$ 中兩個概率分佈 p_0 和 p_1，SBP 可以寫為

$$\begin{aligned} \min_f \quad & \frac{1}{2} \int_0^T \int |f_t(x)|^2 \rho_t(x)\, dx dt, \\ \text{s.t.} \quad & \partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{\sigma^2}{2} \Delta \rho_t, \quad 0 \le t \le T, \\ & \rho_0 = p_0, \quad \rho_T = p_1, \end{aligned} \tag{5.62}$$

其中 $\sigma > 0$ 是給定的常數擴散係數。如果令 $p_0 = p_{\text{data}}$ 且 $p_1 = \mathcal{N}(0, I_d)$，那麼 SBP 不僅要求從 p_{data} 轉移到 $\mathcal{N}(0, I_d)$，從而直接回答了我們之前關於 f（或

μ）可行性的問題，而且還要求在 (5.62) 中的目標函數意義下達到最優性。然而，一般而言，求解 (5.62) 的一個可行解或最優解都是困難的。

儘管存在上面提到的近似問題，擴散模型在實踐中仍然表現良好。上述討論揭示了擴散模型的侷限性，並說明了為什麼有必要學習得分函數 $\nabla \log \rho_t$，以構造 (5.32) 中的生成漂移 μ_t。

5.4 概率密度控制

在第 5.3 節對擴散模型的推導中，我們有兩個與狀態無關的標量函數 g_t 和 σ_t。它們分別是 Fokker–Planck 方程 (5.36) 和 (5.42) 中的擴散係數，並且對所有 t 都滿足 $\sigma_t = g_{T-t}$，然而，它們在學習和生成過程中並不起關鍵作用，而只是決定了相應 SDE (5.33) 和 (5.43) 中波動的大小。

根據定理 5.3.1，我們可以直接對所有 t 取 $g_t = 0$ 和 $\sigma_t = 0$，此時 Fokker–Planck 方程 (5.36) 和 (5.42) 分別退化為如下兩個連續方程

$$\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) \tag{5.63}$$

和

$$\partial_t \pi_t = -\nabla \cdot (-f_{T-t} \pi_t) \tag{5.64}$$

它們都定義在 $\mathbb{R}^d \times [0, T]$ 上。此外，ρ 和 π 仍然滿足

$$\pi_t = \rho_{T-t}$$

對所有 $t \in [0, T]$ 成立。特別地，$\pi_0 = \rho_T$ 且 $\pi_T = \rho_0$。

我們還注意到，在這種情況下，相應的 SDE (5.33) 和 (5.43) 分別退化為如下兩個 ODE

$$\dot{x}_t = f_t(x_t) \tag{5.65}$$

和

$$\dot{z}_t = -f_{T-t}(z_t), \tag{5.66}$$

分別成立。

因此，如果我們能夠找到一個漂移 $f : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，使得 (5.63) 的解 ρ_t 滿足 $\rho_0 = p_{\text{data}}$ 且 $\rho_T = \mathcal{N}(0, I_d)$，那麼就有 $\pi_0 = \mathcal{N}(0, I_d)$ 且 $\pi_T = p_{\text{data}}$。於是，我們只需從標準高斯分佈 $\mathcal{N}(0, I_d)$ 中抽取一個樣本 z_0，將 ODE (5.66) 求解到時間 T，即可得到 z_T，它是一個服從概率分佈 $\pi_T = p_{\text{data}}$ 的新樣本。

$$\boxed{x_0 \sim \rho_0 = p_{\text{data}}} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (f_t \rho_t)]{\dot{x}_t = f_t(x_t)} \boxed{x_T \sim \rho_T \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

$$\boxed{z_T \sim \pi_T = p_{\text{data}}} \xleftarrow[\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t)]{\dot{z}_t = \mu_t(z_t)} \boxed{z_0 \sim \pi_0 \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

图 5.4: 概率控制的前向過程（上）和反向過程（下）。在上圖中，時間 t 從左到右增加（從 0 到 T）；在下圖中，時間 t 從右到左增加。前向過程由 x_t 的 ODE 描述，或者等價地，由其對應的 ρ_t 的連續方程描述，並且在每個 t 都有 $x_t \sim \rho_t$。反向過程由 z_t 的 ODE 描述，或者等價地，由其對應的 π_t 的連續方程描述，並且在每個 t 都有 $z_t \sim \pi_t$。有 $\mu_t = -f_{T-t}$，這使得對每個 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。近似關係 $\rho_T \approx \mathcal{N}(0, I_d)$ 是通過尋找使 $\mathrm{KL}(\rho_T, \mathcal{N}(0, I_d))$ 最小的 f 來實現的，如 (5.67) 所示。

這一關係如圖 5.4 所示。

現在我們可以看到，關鍵在於找到這個漂移 f。根據命題 5.3.4，我們知道若要在有限時間 T 內使 $\rho_T = \mathcal{N}(0, I_d)$，則 f 必須依賴於 p_{data}。因此，人工設計的 f 不在我們的考慮範圍之內。相反，我們必須從給定的 p_{data} 樣本中學習這個 f。

我們可以將尋找 f 的問題視為一個概率密度控制問題，即學習一個控制向量場 $f : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，使其能夠將概率密度 ρ_t 隨時間從 $\rho_0 = p_{\text{data}}$ 驅動到時刻 T 的 $\rho_T = \mathcal{N}(0, I_d)$。更準確地說，這個概率密度控制問題可以寫為

$$\min_f \quad \mathrm{KL}(\rho_T, \mathcal{N}(0, I_d)), \tag{5.67a}$$

$$\text{s.t.} \quad \partial_t \rho_t = -\nabla \cdot (f_t \rho_t), \quad 0 \le t \le T, \tag{5.67b}$$

$$\rho_0 = p_{\text{data}}. \tag{5.67c}$$

一旦找到 (5.67) 的解 f，我們就知道 $\rho_T = \mathcal{N}(0, I_d)$ 近似成立，於是便可在 ODE (5.66) 中使用該 f 來生成樣本。

為了求解 (5.67)，我們可以使用第 3.3 節中介紹的 neural ODE 方法。設 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是由從 p_{data} 中獨立抽取的 M 個樣本組成的數據集，我們可以對所有 i 設定一組初值 $x_0^{(i)} = x_i$，並對每個 $x_0^{(i)}$ 求解 ODE (5.65)。於是我們知道對所有 $t \in [0, T]$，都有 $x_t^{(i)} \sim \rho_t$。

剩下的問題是如何用點集 $\{x_T^{(i)}\}$ 來表示 (5.67a) 中的目標函數。我們注意

到該目標函數可以分解為兩部分:

$$\mathrm{KL}(\rho_T, \mathcal{N}(0, I_d)) = \underbrace{\int \rho_T(x) \log \rho_T(x)\, dx}_{=:A} - \underbrace{\int \rho_T(x) \log \mathcal{N}(x; 0, I_d)\, dx}_{=:B}. \tag{5.68}$$

很容易看出，(5.68) 中的項 B 為

$$\begin{aligned} B &= \int \rho_T(x) \log \mathcal{N}(x; 0, I_d)\, dx \\ &= -\int \rho_T(x) \Big(\frac{1}{2}|x|^2 + \frac{d}{2}\log(2\pi) \Big)\, dx \\ &= -\frac{1}{2}\int |x|^2 \rho_T(x)\, dx - \frac{d}{2}\log(2\pi). \end{aligned}$$

然後我們可以使用 Monte Carlo 積分，將 B 近似為

$$B \approx -\frac{1}{2}\sum_{i=1}^{M} |x_T^{(i)}|^2 - \frac{d}{2}\log(2\pi), \tag{5.69}$$

因為 $x_T^{(i)} \sim \rho_T$。

對於 (5.68) 中的項 A，我們注意到它是 ρ_T 的負熵。我們需要找到一種在實踐中可計算 A 的方法。為此，我們令 h_t 表示每個時刻 t 上 ρ_t 的負熵，即

$$h_t := \int \rho_t(x) \log \rho_t(x)\, dx. \tag{5.70}$$

注意，對每個 t 都有 $h_t \in \mathbb{R}$。特別地，$A = h_T$，而 h_0 是 p_{data} 的負熵，它是一個未知常數。此外，我們有

$$\begin{aligned} \dot{h}_t &= \frac{d}{dt}\Big(\int \rho_t(x) \log \rho_t(x)\, dx \Big) \\ &= \int (\log \rho_t(x) + 1) \partial_t \rho_t(x)\, dx \\ &= -\int (\log \rho_t(x) + 1) \nabla \cdot (\rho_t(x) f_t(x))\, dx \\ &= \int \nabla(\log \rho_t(x) + 1) \cdot (\rho_t(x) f_t(x))\, dx \\ &= \int \nabla \rho_t(x) \cdot f_t(x)\, dx \\ &= -\int \rho_t(x) \nabla \cdot f_t(x)\, dx, \end{aligned} \tag{5.71}$$

其中第三個等號來自連續方程 (5.63)。在實踐中，由於 $x_t^{(i)} \sim \rho_t$，我們可以使用 Monte Carlo 積分近似 (5.71) 右端的積分：

$$\dot{h}_t = -\int \rho_t(x)\nabla \cdot f_t(x)\,dx \approx -\frac{1}{M}\sum_{i=1}^{M} \nabla \cdot f_t(x_t^{(i)}). \tag{5.72}$$

因此，在實際實現中，我們可以加入一個輔助變量 h_t，並將 ODE (5.72) 與 $\{x_t^{(i)} : i \in [M]\}$ 的 ODE (5.65) 一起求解，從而得到 $A = h_T - h_0$。由於 h_0 是 p_{data} 的負熵，它是一個與 f 無關的常數，因此在計算中我們可以直接假設它為零，因為這不會改變 (5.67) 的解。

總之，我們已經將尋找 $f : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 的問題 (5.67) 形式化為如下一個在實踐中可實現的問題：

$$\min_f \quad h_T + \frac{1}{2}\sum_{i=1}^{M} |x_T^{(i)}|^2, \tag{5.73a}$$

$$\text{s.t.} \quad \dot{x}_t^{(i)} = f_t(x_t^{(i)}), \quad i \in [M], \tag{5.73b}$$

$$\dot{h}_t = -\frac{1}{M}\sum_{i=1}^{M} \nabla \cdot f_t(x_t^{(i)}), \quad 0 \le t \le T, \tag{5.73c}$$

$$x_0^{(i)} = x_i, \quad h_0 = 0, \tag{5.73d}$$

其中 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是從 p_{data} 中獨立抽取的 M 個給定樣本的集合。我們在 (5.73) 中省略了所有無關常數。與往常一樣，這裡的最小化是相對於 f 的網絡參數進行的。

我們可以使用 neural ODE 方法來求解 (5.73)，在該方法中，(5.73a) 中損失函數關於網絡 f 參數的梯度可以通過該方法中的伴隨 ODE 部分來計算。

正如之前所提到的，一旦通過求解 (5.73) 訓練好了 f，我們就可以從 $\mathcal{N}(0, I_d)$ 中抽取初值 z_0，並使用訓練好的 f 求解 ODE (5.66) 到時間 T。這樣便得到 z_T，它是一個服從概率分佈 p_{data} 的新樣本。

5.5 流匹配

流匹配（Flow matching）是一種替代性的生成模型框架，其目標是直接學習將 $\mathcal{N}(0, I_d)$ 傳輸到 p_{data} 的向量場。和之前一樣，我們只給定一個數據集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含了從 p_{data} 中獨立抽取的 M 個樣本。在 flow matching 中，我們的目標是利用這些樣本學習一個向量場 $u : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$，

使得 ODE

$$\dot{x}_t = u_t(x_t) \tag{5.74}$$

在任意初值 $x_0 \sim \mathcal{N}(0, I_d)$ 下的解 x_T 滿足 $x_T \sim p_{\text{data}}$。由於在 flow matching 方法中 T 的取值並不重要，因此在本節餘下部分我們取 $T = 1$。

我們再次將目標的時變向量場 u 參數化為一個深度神經網絡。於是，flow matching 的關鍵就在於設計一個在實踐中可實現的 u 的訓練問題。為此，我們採用與第 5.3 節擴散模型中相似的方法，先考慮一些 “條件” 向量場。

注意到，將 $\mathcal{N}(0, I_d)$ 傳輸到 p_{data} 是困難的，但將 $\mathcal{N}(0, I_d)$ 傳輸到 p_{data} 中任意一個特定樣本 z 卻是容易的。例如，對於任意固定的 $z \sim p_{\text{data}}$（即，z 是一個服從 p_{data} 的給定數據樣本），在時間區間 $[0, 1]$ 內，從任意點 $\epsilon \in \mathbb{R}^d$ 移動到 z 的最簡單方式，就是沿著從 ϵ 到 z 的直線並以恆定速度運動。也就是說，我們可以令運動軌跡 x_t 滿足

$$x_t = (1 - t)\epsilon + tz. \tag{5.75}$$

顯然有 $x_0 = \epsilon$ 且 $x_1 = z$。我們還注意到由 (5.75) 可推出兩個結論：

$$\dot{x}_t = z - \epsilon \tag{5.76}$$

以及

$$\epsilon = \frac{x_t - tz}{1 - t} \tag{5.77}$$

其中我們約定 (5.77) 右端（以及本節後面類似表達式）在 $t \to 1^-$ 時以極限意義定義，並且我們將確保這不會在後續推導中引起實際的數學問題。

結合 (5.76) 和 (5.77)，可得

$$\dot{x}_t = z - \epsilon = \frac{z - x_t}{1 - t}. \tag{5.78}$$

現在我們可以如下定義所謂的**條件向量場** $v^z : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$：固定任意 $z \in \mathbb{R}^d$，對任意 $x \in \mathbb{R}^d$ 和 $t \in [0, 1]$，定義

$$v_t^z(x) := \frac{z - x}{1 - t}. \tag{5.79}$$

它的重要性在於：從任意初始值 $\epsilon \in \mathbb{R}^d$ 出發，只要沿著這個條件向量場 v_t^z 演化，就一定會在時刻 $t = 1$ 到達 z。

我們還可以利用這個條件向量場構造 ρ^z 的一個連續方程。更準確地說，

我們對任意 z 取 $\rho_0^z = \mathcal{N}(0, I_d)$（可以將其看作在 $\mathbb{R}^d$ 中服從標準高斯分佈的大量粒子），並定義如下連續方程

$$\partial_t \rho_t^z = -\nabla \cdot (v_t^z \rho_t^z). \tag{5.80}$$

由於 v^z 的性質，我們知道當 $t \to 1^-$ 時有 $\rho_t^z \to \delta_z$（δ_z 是以點 z 為中心的 Dirac delta 函數）。儘管如此，我們仍然有對所有 x 成立的 $\rho_t^z(x) \geq 0$，並且 $\int \rho_t^z(x)\, dx = 1$，因為 ρ_t^z 描述的是這些粒子（它們自時間 0 起由 v^z 驅動）在時刻 t 的概率密度。

一個特別重要的事實是，如果初始點 ϵ 服從 $\mathcal{N}(0, I_d)$，那麼就有

$$x_t = (1-t)\epsilon + tz \sim \mathcal{N}(\,\cdot\,; tz, (1-t)^2 I_d).$$

再注意到 $x_t \sim \rho_t^z$，於是得到

$$\rho_t^z = \mathcal{N}(tz, (1-t)^2 I_d). \tag{5.81}$$

現在，藉助 ρ_t^z，我們可以定義如下關於 t、x 和 z 的函數：

$$\rho_t(x, z) := \rho_t^z(x) p_{\text{data}}(z). \tag{5.82}$$

我們注意到，ρ_t 具有若干使其成為聯合概率分佈的性質：對所有 $x, z \in \mathbb{R}^d$ 和 $t \in [0, 1]$ 都有 $\rho_t(x, z) \geq 0$，並且

$$\begin{aligned}
\iint \rho_t(x, z)\, dz dx &= \iint \rho_t^z(x) p_{\text{data}}(z)\, dz dx \\
&= \int p_{\text{data}}(z) \Big(\int \rho_t^z(x)\, dx \Big) dz \\
&= \int p_{\text{data}}(z)\, dz \\
&= 1
\end{aligned}$$

其中第三個等號用到了 $\int \rho_t^z(x)\, dx = 1$。這些都說明 $\rho_t(x, z)$ 在每個 t 上都是 (x, z) 的聯合概率分佈。

在不引起混淆的情況下，我們還將 $\rho_t(x, z)$ 關於 x 的邊緣概率記作

$$\rho_t(x) := \int \rho_t(x, z)\, dz. \tag{5.83}$$

同樣地，對所有 x 和 t 都有 $\rho_t(x) \geq 0$，且 $\int \rho_t(x)\,dx = \iint \rho_t(x,z)\,dzdx = 1$。此外，我們發現這個邊緣概率分佈滿足

$$\rho_0(x) = \int \rho_0^z(x) p_{\text{data}}(z)\,dz = \int \mathcal{N}(x;0,I_d) p_{\text{data}}(z)\,dz = \mathcal{N}(x;0,I_d),$$

其中第一個等號利用了 (5.82)，而最後兩個等號利用了 $\rho_0^z(x) = \mathcal{N}(x;0,I_d)$，它是 x 的函數而與 z 無關，並且

$$\begin{aligned}
\rho_1(x) &= \lim_{t\to 1^-} \rho_t(x) \\
&= \lim_{t\to 1^-} \int \rho_t^z(x) p_{\text{data}}(z)\,dz \\
&= \int \Big(\lim_{t\to 1^-} \rho_t^z(x)\Big) p_{\text{data}}(z)\,dz \\
&= \int \delta_z(x) p_{\text{data}}(z)\,dz \\
&= p_{\text{data}}(x).
\end{aligned} \tag{5.84}$$

（嚴格來說，(5.84) 中極限與積分的交換一般並不成立；然而，我們通常將 δ_z 看成均值為 z 且方差極小的高斯概率密度函數，並且在假設 p_{data} 為連續函數時，(5.84) 中的推導仍然成立。）這兩個邊緣分佈恰好對應於我們的初始分佈 $\mathcal{N}(0,I_d)$ 和目標分佈 p_{data}，因此我們希望找到一個向量場 $u:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$，使得連續方程

$$\partial_t \rho_t = -\nabla\cdot(u_t\rho_t) \tag{5.85}$$

確實能夠將 $\rho_0 = \mathcal{N}(0,I_d)$ 傳輸到 $\rho_1 = p_{\text{data}}$。

為此，我們注意到

$$\begin{aligned}
\partial_t \rho_t(x) &= \frac{\partial}{\partial t}\int \rho_t(x,z)\,dz \\
&= \frac{\partial}{\partial t}\int \rho_t^z(x) p_{\text{data}}(z)\,dz \\
&= \int \Big(\partial_t \rho_t^z(x)\Big) p_{\text{data}}(z)\,dz \\
&= -\int \nabla\cdot(v_t^z(x)\rho_t^z(x)) p_{\text{data}}(z)\,dz \\
&= -\nabla\cdot\Big(\int v_t^z(x)\rho_t^z(x) p_{\text{data}}(z)\,dz\Big)
\end{aligned} \tag{5.86}$$

其中第四個等號利用了 (5.80)，最後一個等號則交換了關於 x 的散度 $\nabla\cdot$ 與關

於 z 的積分次序。注意，最後一個等號中的積分是逐分量作用於被積函數的，因為該被積函數是一個向量。

比較 (5.85) 和 (5.86)，我們可以看出

$$-\nabla\cdot(u_t(x)\rho_t(x)) = -\nabla\cdot\Big(\int v_t^z(x)\rho_t^z(x)p_{\text{data}}(z)\,dz\Big)$$

換句話說，存在一個向量場 $w:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$，使得對所有 x 和 t 都有 $\nabla_x\cdot(w_t(x)\rho_t(x))=0$，並且

$$u_t(x)\rho_t(x) = \int v_t^z(x)\rho_t^z(x)p_{\text{data}}(z)\,dz + w_t(x)\rho_t(x). \tag{5.87}$$

兩邊同除以 $\rho_t(x)$，得到

$$\begin{aligned} u_t(x) &= \int v_t^z(x)\frac{\rho_t^z(x)p_{\text{data}}(z)}{\rho_t(x)}\,dz + w_t(x) \\ &= \int v_t^z(x)\frac{\rho_t(x,z)}{\rho_t(x)}\,dz + w_t(x) \\ &= \int v_t^z(x)\rho_t(z|x)\,dz + w_t(x), \end{aligned}$$

其中 $\rho_t(z|x)$ 是在時刻 t 條件於 x 給定時 z 的條件分佈（而它們在時刻 t 的聯合分佈是 $\rho_t(x,z)$）。我們可以安全地忽略 w_t（因為只要滿足 $\nabla\cdot(w_t\rho_t)=0$，將連續方程 (5.85) 中的 u_t 替換為 u_t-w_t 並不會改變其解 ρ_t），並取

$$u_t(x) = \int v_t^z(x)\rho_t(z|x)\,dz. \tag{5.88}$$

帶有任意給定 $z\in\mathbb{R}^d$ 的條件向量場 v_t^z 與目標向量場 u_t 之間的對比如圖 5.5 所示。

由定理 5.3.3 可知，對任意 x 和 t，都有

$$\begin{aligned} u_t(x) &= \int v_t^z(x)\rho_t(z|x)\,dz \\ &= \mathbb{E}_{Z\sim\rho_t(\cdot|x)}[v_t^Z(x)] \\ &= \arg\min_{c\in\mathbb{R}^d}\mathbb{E}_{Z\sim\rho_t(\cdot|x)}[|c-v_t^Z(x)|^2]. \end{aligned}$$

因此，當我們將向量場 $u:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$ 參數化為一個深度神經網絡時，可以通過對 $\rho_t(x)$（它在每個 x 上都是正的）以及 $[0,1]$ 上的均勻分佈求總體損

$$\boxed{x_0 \sim \mathcal{N}(0, I_d)} \xrightarrow[\partial_t \rho_t^z = -\nabla \cdot (v_t^z \rho_t^z)]{\dot{x}_t = v_t^z(x_t) := \frac{z - x_t}{1-t}} \boxed{\begin{array}{l} x_1 = z \\ \text{where } z \in \mathbb{R}^d \text{ is} \\ \text{arbitrary and fixed} \\ \text{beforehand} \end{array}}$$

$$\boxed{x_0 \sim \mathcal{N}(0, I_d)} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (u_t \rho_t)]{\dot{x}_t = u_t(x_t)} \boxed{x_1 \sim \rho_1 = p_{\text{data}}}$$

图 5.5: Flow matching 框架中通向任意固定點 $z \in \mathbb{R}^d$ 的條件軌跡（上）以及平均軌跡（下）。在上圖和下圖中，時間 t 都從 0 增加到 $T = 1$。條件軌跡由帶有條件向量場 v_t^z 的 ODE 描述，或者等價地由其對應的 ρ_t^z 的連續方程描述，並且在每個 t 都有 $x_t \sim \rho_t^z$，且無論 x_0 是什麼，當 $t \to 1$ 時都有 $x_t \to z$。平均軌跡由帶有目標向量場 u_t 的 ODE 描述，或者等價地由其對應的 ρ_t 的連續方程描述，並且滿足 $\rho_1 = p_{\text{data}}$。目標向量場 u_t 是利用 v_t^z 與 $z \sim p_{\text{data}}$ 的聯合信息，通過 (5.90) 來學習的。

失來訓練它:

$$\min_u \int_0^1 \mathbb{E}_{X \sim \rho_t} \mathbb{E}_{Z \sim \rho_t(\cdot|X)} [|u_t(X) - v_t^Z(X)|^2] \, dt,$$

這可以寫成更易讀的形式

$$\min_u \int_0^1 \iint \rho_t(x) \rho_t(z|x) |u_t(x) - v_t^z(x)|^2 \, dx dz dt. \tag{5.89}$$

注意到

$$\rho_t(x)\rho_t(z|x) = \rho_t(x, z) = \rho_t^z(x) p_{\text{data}}(z) = \mathcal{N}(x; tz, (1-t)^2 I_d) p_{\text{data}}(z),$$

其中第二個等號來自 (5.82)，第三個等號來自 (5.81)。將其代回 (5.89)，我們得到一個等價問題

$$\min_u \int_0^1 \iint p_{\text{data}}(z) \mathcal{N}(x; tz, (1-t)^2 I_d) |u_t(x) - v_t^z(x)|^2 \, dx dz dt. \tag{5.90}$$

(5.90) 中的目標函數還可以進一步簡化。為此，我們注意到，為了得到一個樣本 $x \sim \mathcal{N}(tz, (1-t)^2 I_d)$，我們可以令

$$x = tz + (1-t)\epsilon \tag{5.91}$$

其中 $\epsilon \sim \mathcal{N}(0, I_d)$。結合 (5.79)，我們知道 (5.90) 中的 $v_t^z(x)$ 變為

$$v_t^z(x) = \frac{z-x}{1-t} = \frac{z-(tz+(1-t)\epsilon)}{1-t} = z - \epsilon. \tag{5.92}$$

因此，我們可以將最小化問題 (5.90) 重寫為

$$\min_u \int_0^1 \iint p_{\text{data}}(z)\mathcal{N}(\epsilon; 0, I_d)|u_t(tz+(1-t)\epsilon)-(z-\epsilon)|^2 \, d\epsilon dz dt. \tag{5.93}$$

我們可以使用 Monte Carlo 積分來近似這個損失函數：

$$\frac{1}{NM}\sum_t \sum_{(z,\epsilon)} |u_t(tz+(1-t)\epsilon)-(z-\epsilon)|^2 \tag{5.94}$$

在實際實現中，我們從 $[0,1]$ 上的均勻分佈中抽取 N 個時間點。對每一個時間點 t，我們針對來自 $\mathcal{D}$ 的每個樣本 z（$\mathcal{D}$ 由從 p_{data} 中獨立抽取的 M 個樣本組成），模擬一個 $\epsilon \sim \mathcal{N}(0, I_d)$。然後將這些量全部代入經驗損失 (5.94) 中訓練 u。訓練得到的 u 就是 (5.74) 中所需向量場的一個近似。

5.6 備註與參考文獻

5.6.1 生成模型中的噪聲採樣

在第 5.1、5.3 和 5.5 節中討論的變分自編碼器框架、擴散模型以及 flow matching 方法裡，我們最終常常會得到如下形式的損失函數

$$\iint \phi(x,\epsilon)p_{\text{data}}(x)\mathcal{N}(\epsilon; 0, I_d)\, d\epsilon dx \tag{5.95}$$

其中 ϕ 是關於 (x,ϵ) 的某個標量值函數。例子包括 (5.17)、(5.53) 和 (5.93)。這裡不需要將時間變量 t 納入考慮：(5.95) 應當在每個 t 上被最小化，而在實際數值實現中我們只是人為地在 $[0,T]$ 上對 t 進行採樣。因此，t 的性質與 (x,ϵ) 不同。

我們總是使用 Monte Carlo 積分來近似形如 (5.95) 的損失函數。更具體地說，我們有一組給定樣本 $\{x_i \in \mathbb{R}^d : i \in [m]\}$，共 m 個（在本章前面的各節中我們使用的是 M，但這裡改用 m，因為我們將考慮對每個給定數據樣本抽取 n 個高斯噪聲），這些樣本被假設為從未知數據分佈 p_{data} 中獨立抽取而來。與此同時，我們還需要獨立地採樣 $\epsilon \sim \mathcal{N}(0, I_d)$，即 d 維標準高斯分佈。

對 ϵ 有兩種採樣方式。第一種方式是對每個 x_i 只採樣一個 $\epsilon_i \sim \mathcal{N}(0, I_d)$，第二種方式則是對每個 x_i 採樣多個彼此獨立的 ϵ。在理論上，第一種方式在 Monte Carlo 積分中是正確的，因為它將 (5.95) 中的 $p_{\text{data}}(x)\mathcal{N}(\epsilon; 0, I_d)$ 作為 (x, ϵ) 的聯合概率分佈。然而，第二種方式也很有吸引力，因為我們可以很容易地從 $\mathcal{N}(0, I_d)$ 中抽取任意多個樣本，而不像 x_i 這樣的樣本那樣都是給定的。那麼，如果採用第二種方式，會不會有什麼問題呢?

為了便於討論，我們考慮對每個 x_i 採樣 n 個獨立 ϵ 的情形。更準確地說，假設我們用來構造 (5.95) 的 Monte Carlo 近似的樣本為

$$(x_i, \epsilon_{ij}), \quad i \in [m],\ j \in [n], \tag{5.96}$$

其中 x_i 和 ϵ_{ij} 全部彼此獨立。於是我們用

$$\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\phi(x_i, \epsilon_{ij}). \tag{5.97}$$

來近似 (5.95)。因此，我們上面提到的兩種方式之間的區別，僅僅在於第一種方式中 $n = 1$，而第二種方式中 $n > 1$。

很容易看出，無論 n 的取值為何，(5.97) 的期望都是 (5.95)。換句話說，(5.97) 始終是 (5.95) 的一個無偏估計。

現在我們想考察 (5.97) 的方差，並看看當 $n > 1$ 時它的方差是否會更小。為此，我們先為類似情形推導一個一般性結果。

命題 5.6.1. 設 $m, n \in \mathbb{N}$，並設 $\{A_{ij} : i \in [m],\ j \in [n]\}$ 是 mn 個標量值隨機變量，滿足

1. 對所有 $i \in [m]$ 和 $j \in [n]$，都有 $\mathbb{E}[A_{ij}] = 0$。
2. 對所有 $i \in [m]$ 和 $j \in [n]$，都有 $\mathrm{Var}[A_{ij}] = v$，其中某個 $v > 0$。
3. 對每個 $i \in [m]$，A_{ij} 與 $\{A_{i'j'} : i' \in [m] \setminus \{i\},\ j \in [n]\}$ 獨立。
4. 對每個 $i \in [n]$，對所有 $j, j' \in [n]$ 且 $j \neq j'$，都有 $\mathrm{Cov}[A_{ij}, A_{ij'}] = c$，其中某個 $c > 0$。

則有

$$\mathbb{E}\Big[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\Big] = 0$$

以及

$$\mathrm{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right]=\frac{v+(n-1)c}{mn}.$$

特別地，當 $n=1$ 時，

$$\mathrm{Var}\left[\frac{1}{m}\sum_{i=1}^{m}A_{i1}\right]=\frac{v}{m}.$$

证明. 我們首先注意到

$$\mathbb{E}\Big[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\Big]=\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\mathbb{E}[A_{ij}]=0.$$

對每個 $i\in[m]$，記

$$B_i:=\sum_{j=1}^{n}A_{ij}.$$

那麼我們知道 $B_1,\ldots,B_m$ 彼此獨立。此外，有

$$\mathbb{E}[B_i]=\mathbb{E}\Big[\sum_{j=1}^{n}A_{ij}\Big]=\sum_{j=1}^{n}\mathbb{E}[A_{ij}]=0$$

以及

$$\begin{aligned}\mathrm{Var}[B_i]&=\mathrm{Var}\left[\sum_{j=1}^{n}A_{ij}\right]\\&=\sum_{j=1}^{n}\mathrm{Var}[A_{ij}]+2\sum_{1\le j<j'\le n}\mathrm{Cov}[A_{ij},A_{ij'}]\\&=nv+n(n-1)c.\end{aligned}$$

因此，

$$\begin{aligned}\operatorname{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] &= \frac{1}{(mn)^2}\operatorname{Var}\left[\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] \\ &= \frac{1}{(mn)^2}\operatorname{Var}\left[\sum_{i=1}^{m}B_i\right] \\ &= \frac{1}{(mn)^2}\sum_{i=1}^{m}\operatorname{Var}[B_i] \\ &= \frac{v+(n-1)c}{mn}.\end{aligned}$$

令 $n=1$，便得到 $\operatorname{Var}[\frac{1}{m}\sum_{i=1}^{m}A_{i1}]=\frac{v}{m}$。 □

我們可以通過計算它們的差值來比較 $\frac{1}{m}\sum_{i=1}^{m}A_{i1}$ 和 $\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}$ 的方差：

$$\begin{aligned}\operatorname{Var}\left[\frac{1}{m}\sum_{i=1}^{m}A_{i1}\right]-\operatorname{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] &= \frac{v}{m}-\frac{v+(n-1)c}{mn} \\ &= \frac{(n-1)(v-c)}{mn}. \qquad (5.98)\end{aligned}$$

因此，當 $v>c$ 時，使用 $n>1$ 可能是有益的，因為無偏估計 $\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}$ 的方差會更小。

為了將命題 5.6.1 的結果應用到我們的問題中，我們為了記號簡潔起見，假設 (5.95)，也就是 (5.97) 的期望，為零。這是因為期望的取值並不會影響方差的大小。

現在我們將命題 5.6.1 中的 A_{ij} 取為 $\phi(X_i,E_{ij})$，其中對所有 $i\in[m]$ 和 $j\in[n]$，$X_i\sim p_{\text{data}}$ 且 $E_{ij}\sim\mathcal{N}(0,I_d)$，並且它們彼此獨立，那麼命題 5.6.1 中的 v 和 c 分別為

$$v=\operatorname{Var}_{(X_i,E_{ij})}[\phi(X_i,E_{ij})]$$

以及

$$c=\operatorname{Cov}_{(X_i,E_{ij},E_{ij'})}[\phi(X_i,E_{ij}),\phi(X_i,E_{ij'})].$$

因此，我們需要檢查 v 和 c 哪一個更大。

一般來說，$\phi(x,\epsilon)$ 是一個複雜函數，例如 (5.53) 和 (5.93)。我們可以看到，在這些情形中，ϕ 包含了一些需要訓練的深度神經網絡。因此，在一般情形下，我們無法斷定 v 和 c 哪一個更大。不過，我們注意到，即使 $v-c$ 為

正（並且固定），其帶來的收益（即差值 (5.98) 的大小）也未必顯著：(i) 增大 n 會使得 $\frac{n-1}{n} \to 1$，從而差值 (5.98) 在較大 n 時趨於飽和，但 (5.97) 的計算代價卻會上升；(ii) 樣本量 m 越大（這由我們已有的數據決定，無法在 (5.97) 的計算中改變），差值 (5.98) 就越小，這說明如果我們已經有大量樣本，那麼使用較大的 n 可能並不值得。

儘管如此，我們仍然可以考慮一個簡單情形，即 $\phi(x, \epsilon) = x + \epsilon$，其中 x 和 ϵ 都是標量值。此時有 $X_i \sim p_{\text{data}}$ 定義在 $\mathbb{R}$ 上，而 $E_{ij}, E_{ij'} \sim \mathcal{N}(0, 1)$，並且它們彼此獨立。在這種情況下，

$$v = \text{Var}_{(X_i, E_{ij})}[X_i + E_{ij}] = \text{Var}[X_i] + 1$$

以及

$$c = \text{Cov}_{(X_i, E_{ij}, E_{ij'})}[X_i + E_{ij}, X_i + E_{ij'}] = \text{Var}[X_i].$$

因此，我們確實在這個簡單例子中得到了 $v > c$。此外，我們還注意到 $X_i + E_{ij}$ 與 $X_i + E_{ij'}$ 是相關的，因為它們的協方差為正。

這個簡單例子還給出了一個警示：當我們試圖通過向給定樣本中加入噪聲或類似方式來製造更多“訓練樣本”時，需要格外謹慎，因為這些人為構造出的樣本在統計上是相關的（它們的協方差非零）。因此，我們前面討論的方法中某些理論推導所依賴的假設，可能不再成立。

5.6.2 條件生成模型

本章中討論的生成模型旨在生成與觀測數據相似的新樣本，這些觀測數據服從某個未知概率分佈 p_{data}。在真實場景中，我們經常希望在給定某些輸入信息 $y \in \mathbb{R}^k$ 的條件下生成數據（這裡為了便於下文討論，我們將其表示為向量，而在不同方法中它也可以是某種指定形狀的張量），例如類別標籤、屬性、文本描述或其他模態的信息。這就引出了**條件生成**（conditional generation）問題，其目標是從條件分佈 $p_{\text{data}}(x|y)$ 中生成樣本，從而實現可控、有針對性且可解釋的生成。

現有的條件生成框架主要區別在於它們如何表示、學習以及生成服從 $p_{\text{data}}(x|y)$ 的樣本，但它們都建立在各自對應的無條件生成原型（比如本章前面介紹的那些方法）之上。它們一個共性是將條件變量 y 注入到生成過程中，或者通過概率建模顯式引入，或者通過神經網絡結構引入。下面我們簡要介紹幾種現有的條件生成方法。需要注意的是，在生成模型社區中，我們也常將這一過程稱為**引導生成**（guided generation），並將 y 稱為**引導信號**而不是

條件，這些術語有時有助於避免與推導中出現的各種條件概率分佈相混淆。

Conditional GAN 是一種基於 GAN 的條件生成模型，它在生成器 g_θ 和判別器 a_η 中同時引入引導信號 y。生成器 g_θ 是一個參數為 θ 的神經網絡，並學習從一個簡單概率分佈的樣本 z（通常像 GAN 中那樣從 $\mathcal{N}(0, I_d)$ 中抽取，定義在 $\mathbb{R}^d$ 上）和一個引導信號 $y \in \mathbb{R}^k$ 到數據空間的映射。因此，生成器 g_θ 可以用來生成服從條件概率 $p_\theta(x|y)$ 的樣本。與此同時，判別器被參數化為對抗網絡 $a_\eta(\cdot|y)$，它在給定 y 的條件下區分真實樣本和生成樣本。參數 (θ, η) 的訓練通過求解如下鞍點問題來實現：

$$\min_\theta \max_\eta \Big\{ \iint p_{\text{data}}(x, y) \log a_\eta(x|y)\, dxdy + \iint p(y) p_\theta(x|y) \log(1 - a_\eta(x|y))\, dxdy \Big\}. \tag{5.99}$$

在實際計算中，通過對給定數據集中的獨立樣本對 (x, y) 做 Monte Carlo 積分來近似 (5.99) 中第一個積分，其中 y 是與 x 對應的引導信號。(5.99) 中的概率 $p(y)$ 指的是引導信號的概率分佈，在訓練過程中通過對所有需要用到的引導信號採樣來實現。Conditional GAN 在生成清晰樣本方面尤其有效，並被廣泛應用於圖像生成和遷移。

我們也可以將從條件概率 $p_{\text{data}}(\cdot|y)$ 中採樣的思想應用到流匹配（flow matching）框架中。更具體地說，我們可以將 flow matching 方法（第 5.5 節）中的概率 $\rho_t(x)$ 替換為帶有引導信號 y 的條件概率 $\rho_t(x|y)$。為此，我們首先注意到

$$\rho_t(x|y) = \int \rho_t(x|y, z) p_{\text{data}}(z|y)\, dz = \int \rho_t^z(x) p_{\text{data}}(z|y)\, dz \tag{5.100}$$

其中 ρ_t^z 是在 (5.81) 中給出的、具有給定目標 z 的概率流，而 $p_{\text{data}}(z|y)$ 在利用 Monte Carlo 積分近似 (5.100) 時，同樣會由帶有引導變量 y 標記的給定樣本 z 來替代。注意到，在 (5.100) 中我們假設 $\rho_t(x|y, z) = \rho_t(x|z) = \rho_t^z(x)$，因為從 x 到任意指定的 $z \sim p_{\text{data}}$ 的路徑是由 x 和 z 決定的，而不是由 y 決定的，這是因為關於 y 的信息已經完全包含在 z 之中。與第 5.5 節中的推導類似，我們可以證明，起始時刻 $t = 0$ 和終止時刻 $t = 1$ 的邊緣分佈滿足

$$\rho_0(\cdot|y) = \mathcal{N}(0, I_d) \quad \text{和} \quad \rho_1(\cdot|y) = p_{\text{data}}(\cdot|y),$$

並且帶引導的向量場 $u_t(\cdot|\cdot):\mathbb{R}^d\times\mathbb{R}^k\to\mathbb{R}^d$ 為

$$u_t(x|y)=\int v_t^z(x)\rho_t(z|x,y)\,dz=\operatorname*{arg\,min}_{c\in\mathbb{R}^d}\mathbb{E}_{Z\sim\rho_t(\cdot|x,y)}\Big[|c-v_t^Z(x)|^2\Big],$$

這意味著，當 u_t 被參數化為深度神經網絡時，它可以通過求解

$$\min_u\int_0^1\mathbb{E}_{(X,Y)\sim p_{\text{data}}}\mathbb{E}_{Z\sim\rho_t(\cdot|X,Y)}\Big[|u_t(X|Y)-v_t^Z(X)|^2\Big]\,dt. \tag{5.101}$$

來訓練。注意，這裡的最小化仍然是關於 u 的網絡參數進行的。文獻表明，這種條件生成在每個引導信號 y 在給定數據集中都有大量樣本 x 與之對應的應用中最為有效，例如類別引導 [103]。然而，在一般情形下這可能是很困難的，因為許多引導信號 y 是不重複且複雜的，例如圖像的標題（captions of figures）[86]。

擴散模型也可以被修改為條件擴散模型。為此，人們發展了 classifier guidance [32, 139] 和 classifier-free guidance [64] 方法。具體而言，由 Bayes 法則可知

$$\rho_t(x|y)=\frac{\rho_t(y|x)\rho_t(x)}{p(y)}$$

其中 $p(y)$ 與上文一樣表示引導信號的分佈，而 $\rho_t(x)$ 和 $\rho_t(x|y)$ 分別是前面提到的無條件和條件概率分佈。對上式兩邊取對數，再對 x 求梯度，就得到

$$\underbrace{\nabla_x\log\rho_t(x|y)}_{\text{conditional score}}=\underbrace{\nabla_x\log\rho_t(y|x)}_{\text{classifier score}}+\underbrace{\nabla_x\log\rho_t(x)}_{\text{unconditional score}}. \tag{5.102}$$

因此，只要我們得到 classifier score，就可以獲得所需的 conditional score，並按照擴散模型中的推導訓練網絡 $s_t(x|y)$ 來近似它。

基於這一關係，classifier guidance 方法訓練一個時變函數 $\phi_t(y|x)$，並將其參數化為深度神經網絡，用以近似 classifier score $\nabla_x\rho_t(y|x)$。我們強調，這個函數 ϕ 是與擴散模型中的 unconditional score 分開訓練的；更重要的是，它是時變的，即對任意時刻 t 和 $x\sim\rho_t(\cdot)$ 都能逼近這個 $\nabla_x\rho_t(y|x)$。[86] 表明，在擴散模型中使用與 (5.75) 相同的線性插值時，帶引導的向量場為

$$u_t(x|y)=u_t(x)+\frac{1-t}{t}\phi_t(y|x) \tag{5.103}$$

同時文中還給出了一個可定製插值係數的公式表。這種方法在類別條件和文本條件兩種情形下，都比 (5.101) 更能準確逼近帶引導的向量場 [104]。

由於 classifier score 與 unconditional score 是分開學習的，因此通常需要在 (5.103) 右端第二項中的 ϕ_t 前乘上一個標量 $\gamma \in \mathbb{R}$ 來校準 classifier score 的權重。這個標量通常取為 $\gamma > 1$ [32]。

另一種不需要單獨訓練 classifier score 函數的方法，稱為 classifier-free guidance (CFG)，由 [64] 提出。由 (5.102) 可見，classifier score 就是 conditional score 與 unconditional score 的差。因此，CFG 方法只訓練一個單獨的網絡來近似 conditional score $\nabla_x \rho_t(x|y)$，並取一個固定向量 y_{null} 作為佔位符，用於輸入樣本 z 沒有引導信號時（即，$z \sim p_{\text{data}}(\cdot)$）。根據這個想法，我們可以得到如下帶引導信號的向量場 [86]：

$$u_t(x|y) = (1-\gamma)u_t(x|y_{\text{null}}) + \gamma u_t(x|y). \tag{5.104}$$

訓練 u 的損失函數中還包括另一層數據選擇：首先根據某個用戶選取的 $p \in (0,1)$，通過 Bernoulli(p) 隨機抽取 y_{null} 或某個真實引導信號 y；然後如果抽到了 y_{null}，則對 z 做無引導採樣，否則就在給定所抽引導信號 y 的條件下對 z 採樣。接下來的步驟與擴散模型中的做法相同 [64]。一些後續工作針對 CFG 採樣提出了不同的直觀解釋或理論論證，參見，例如 [22,28,57]。

5.6.3 生成模型的參考文獻

早期的生成建模工作主要由經典概率方法主導，例如高斯混合模型、隱 Markov 模型以及其他具有手工設計結構的潛變量模型。儘管這些方法在數學上基礎紮實且具有可解釋性，但它們在擴展到高維數據（如圖像、音頻信號和自然語言）時往往表現不佳。將深度神經網絡作為靈活的函數逼近器引入後，生成模型發生了重大轉變，使其能夠更有效地刻畫複雜的、多模態的數據分佈。

自編碼器的概念最早出現在 [62,123] 等早期工作中，其目標是利用潛變量建立低維表示，從而抓住給定高維數據的本質。VAE 最早由 [78] 提出。[79] 給出了關於 VAE 的系統性介紹。VAE 採用編碼器–解碼器結構，將數據映射到連續潛空間中，並通過生成式解碼器重構樣本。這一形式使得高效採樣、結構化表示學習以及穩定優化成為可能。然而，人們經常觀察到 VAE 會生成過於平滑或模糊的樣本，這反映了基於似然的目標與感知可信度之間的一種權衡。

針對 VAE，人們提出了大量改進和擴展。例如，條件 VAE 在 [136] 中被提出，並表明輸入條件在編碼後可以在潛空間中以不同方式表示。在 [137] 中，構造了 ladder VAE，通過數據相關的近似似然項遞歸地修正生成分佈。[56]

提出了一種作用於輸出變量的自迴歸模型。[81] 表明，通過使用對抗損失可以提升生成樣本的質量。過去幾年中，人們還研究了多種不同形式的後驗變分近似，包括 normalizing flows [77, 117]、圖模型 [91] 以及面向時序數據的模型 [52, 53]。

生成對抗網絡（GANs）最初由 [50] 提出。關於 GAN 及其變體的綜述可見 [54,93,154]。原始 GAN 在訓練上被認為是不穩定的 [40, 72]，自那以後人們提出了大量變體和改進 [5, 55, 94, 95, 113, 160]。其中，Wasserstein GAN 由 [5] 提出，並進一步發展出了若干變體 [1, 157]。

展示擴散模型基本思想的早期工作出現在 [135] 中，該文引入了通過 Markov 鏈將數據逐步擴散為噪聲，並學習一個逆時間過程來恢復數據分佈的概念。擴散模型在實踐上的成功則由去噪擴散概率模型（denoising diffusion probabilistic models，DDPMs）[63] 顯著推動。[103] 通過改進噪聲調度和網絡結構，對 DDPM 進行了提升，從而顯著提高了樣本保真度。此外，[75] 分析了擴散模型中的設計選擇，並提出了能夠提升訓練穩定性和效率的原則性參數化方式。這些進展使得擴散模型在許多基準任務上超越了 GAN，尤其是在模式覆蓋和訓練魯棒性方面。DDPM 與由 SDE 表示的基於 score 的生成建模之間的聯繫在 [139] 中得以建立。由 ODE 表示的確定性版本則見於 [138]。除圖像生成之外，擴散模型還已成功擴展到條件生成、超分辨率、文生圖合成以及科學計算應用等場景。典型例子包括 classifier-guided diffusion [32] 和大規模文本條件模型（如 latent diffusion models [121]），後者通過在學習得到的潛空間中操作來降低計算成本。

與擴散模型相比，flow matching 是一個更加簡潔且進化的生成建模框架 [2, 85, 89]，並且已經在語音 [82]、圖像 [36]、視頻 [111]、機器人 [18] 以及蛋白質 [71] 等大規模生成任務中得到了應用。[87, 131] 表明，Flow Matching 的線性版本可以看作 bridge matching 某種極限情形。flow matching 中使用向量場的思想與 [27, 51] 中基於似然的概率密度匹配相關。近期的改進與擴展見於 [60, 101, 147]。flow matching 框架還被推廣到了離散狀態情形 [25, 48] 以及 Riemann 流形上 [26]。[66] 進一步表明，flow matching 框架適用於一般連續時間 Markov 過程。

總體而言，包括上述所有變體在內的生成模型構成了一類靈活且具有堅實理論基礎的生成模型方法，而當前的持續研究則致力於加速採樣、提升可擴展性，並將其適用範圍拓展到更廣泛的數據模態。

附录 A　補充材料

A.1　Monte Carlo 積分

蒙特卡洛（Monte Carlo）積分是一種在高維歐幾里得空間子集上積分的近似計算方法。對於此類積分，經典的數值積分方法通常不適用。這是因為這些經典方法需要對積分區域進行空間離散化，這樣會帶來的維度災難問題，即離散化得到的格點數隨著維度以指數速度增長。相比之下，Monte Carlo 積分不需要顯式的空間離散化，而是通過遵循某種概率分佈從積分區域中抽取樣本點，並在這些樣本點上計算被積函數的值。隨後，對這些點上的函數值進行適當的平均通常可以得到對目標積分的合理近似。在本節中，我們將給出關於 Monte Carlo 積分方法及其相關數學性質的一些細節。

Monte Carlo 積分的基本思想如下。若 $\Omega \subset \mathbb{R}^d$ 為有界集合且 $h : \Omega \to \mathbb{R}$ 在 Ω 上可積，則其積分可以通過如下方法逼近：

$$\int_\Omega h(x)\,dx \approx \frac{|\Omega|}{N}\sum_{i=1}^{N} h(x^{(i)}), \tag{A.1}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 是從 Ω 上的均勻分佈獨立抽取的 N 個樣本，而 $|\Omega|$ 表示 Ω 在 $\mathbb{R}^d$ 中的體積。接下來，我們考慮該方法的一個簡單推廣（式 (A.1) 是此推廣形式的一個特例），並分析這種 Monte Carlo 積分近似的偏差和方差。

這種推廣稱為 **重要性採樣**（importance sampling）。設 Ω 是 $\mathbb{R}^d$ 的一個（可能無界的）子集，p 是定義在 Ω 上的概率密度函數，且對所有 $x \in \Omega$ 有 $p(x) > 0$。設 $h : \Omega \to \mathbb{R}$ 在 Ω 上可積，且 $X^{(1)}, \ldots, X^{(N)}$ 是從分佈 p 中抽取的 N 個獨立同分布隨機變量。記

$$I_h := \int_\Omega h(x)\,dx \quad \text{和} \quad J_h^N := \frac{1}{N}\sum_{i=1}^{N} \frac{h(X^{(i)})}{p(X^{(i)})}. \tag{A.2}$$

則 J_h^N 是對目標積分 J_h 的一個統計估計。注意，J_h^N 是一個樣本複雜度為 N 的統計量，而 I_h 是一個待由 J_h^N 近似的常數。

我們可以看到，(A.1) 是 (A.2) 在均勻分佈 $p(x) = 1/|\Omega|$（對所有 $x \in \Omega$）下的特例。然而，(A.1) 要求 Ω 是 $\mathbb{R}^d$ 的有界子集，而 (A.2) 則允許處理無界 Ω 的情況。

現在我們來分析 J_h^N 對 I_h 的近似誤差。首先注意到

$$\begin{aligned}\mathbb{E}[J_h^N] &= \mathbb{E}\Big[\frac{1}{N}\sum_{i=1}^{N}\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{X^{(i)}\sim p}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big]\\ &= \frac{1}{N}\sum_{i=1}^{N}\int_{\Omega}\frac{h(x)}{p(x)}p(x)\,dx = J_h,\end{aligned}$$

這說明 J_h^N 是 I_h 的無偏估計。

此外，對於每個 $X^{(i)}$，有

$$\mathrm{Var}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \mathbb{E}\Big[\Big|\frac{h(X^{(i)})}{p(X^{(i)})}\Big|^2\Big] - \Big|\mathbb{E}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big]\Big|^2 = \int_{\Omega}\frac{h(x)^2}{p(x)}\,dx - I_h^2.$$

因此，

$$\begin{aligned}\mathrm{Var}[J_h^N] &= \mathrm{Var}\Big[\frac{1}{N}\sum_{i=1}^{N}\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \frac{1}{N}\,\mathrm{Var}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big]\\ &= \frac{1}{N}\Big(\int_{\Omega}\frac{h(x)^2}{p(x)}\,dx - I_h^2\Big) = O\Big(\frac{1}{N}\Big),\end{aligned} \tag{A.3}$$

這表明 J_h^N 的方差以 $1/N$ 的速度衰減，其中 N 是採樣點數量。同時，有

$$I_h^2 = \Big(\int_{\Omega}h(x)dx\Big)^2 = \Big(\int_{\Omega}\frac{h(x)}{p(x)^{1/2}}p(x)^{1/2}\,dx\Big)^2 \le \int_{\Omega}\frac{h(x)^2}{p(x)}\,dx.$$

上式當且僅當 $p \propto h$ 時取等號，而且此時 (A.3) 中方差的常數項達到最小。

然而，一般情況下，我們很難找到 p 使得 $p \propto h$，因此重要性採樣的思想是選擇一個相對簡單的概率分佈（如高斯混合分佈）作為 p，並使其儘可能接近 h。選擇高斯混合分佈的原因在於可以方便地從 p 中採樣 $X^{(i)}$ 並計算 $h(X^{(i)})/p(X^{(i)})$。具體而言，可以將各高斯分佈的均值設在 h 取得峰值的位置，標準差根據 h 的形狀設定，並調整混合權重使 p 儘可能與 h 成比例。

總體而言，通過增加樣本數量 N 併合理選擇 p，可以減小 J_h^N 對 I_h 的近似誤差。同時，收斂速率 $O(1/N)$ 不依賴於維度 d，因此在使用空間離散化的

情形下，Monte Carlo 方法可以有效地緩解維度災難問題。

然而，在實際應用中，Monte Carlo 積分往往更加複雜。例如，我們常需用 Monte Carlo 積分來估計下式對 θ 的梯度：

$$I_h(\theta) := \frac{1}{2}\int_\Omega |h(x,\theta)|^2 dx,$$

其中 $h : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^n$，$\theta \in \mathbb{R}^m$ 表示待優化的神經網絡參數。由於大多數訓練算法是基於一階優化方法，我們需要近似

$$\nabla_\theta I_h(\theta) = \int_\Omega \nabla_\theta h(x,\theta) h(x,\theta)\, dx,$$

該量是一個 m 維向量。在使用 Monte Carlo 積分時，$\nabla_\theta I_h(\theta)$ 的不同分量可能具有不同方差，因此需要謹慎選擇用於重要性採樣的分佈 p，儘量減小近似方差。正如第 二 章所示，這個隨機梯度的方差對隨機梯度方法（及類似算法）的收斂性具有重要影響。

A.2 Banach 空間與不動點理論

在本節中，我們對 Banach 空間與不動點理論進行簡要回顧。本節中的結果是第 4.1 節中 Bellman 算子與 Markov 決策過程理論的基礎。

我們首先給出 Banach 空間的定義，即完備賦範線性空間。我們先介紹相關概念，然後給出正式定義。隨後介紹 Banach 不動點定理。最後，我們證明在第 4.1 節中考慮的 $B(\mathcal{X})$ 是一個 Banach 空間，從而可以應用該定理。

定義 A.2.1 (線性空間)**.** 設 X 為一個集合。如果在 X 上定義了加法和數乘運算（即對任意 $x, y \in X$ 和 $a \in \mathbb{R}$，有 $x + y \in X$ 且 $ax \in X$），並且對任意 $x, y, z \in X$ 和 $a, b \in \mathbb{R}$ 滿足以下性質，則稱 X 為 **線性空間**（亦稱 **向量空間**）：

1. $x + y = y + x$；
2. $(x + y) + z = x + (y + z)$；
3. 存在 $0 \in X$ 使得對所有 $x \in X$，有 $x + 0 = 0 + x$；
4. 對任意 $x \in X$，存在 $x' \in X$（記為 $-x$）使得 $x + x' = 0$；
5. $a(bx) = (ab)x$；
6. $1 \cdot x = x$，其中 $1 \in \mathbb{R}$；

7. $(a+b)x = ax + bx$;

8. $a(x+y) = ax + ay$。

定義 A.2.2 (範數)**.** 函數 $\|\cdot\| : X \to \mathbb{R}$ 稱為一個 **範數**（norm），如果對任意 $x, y, z \in X$ 和 $a \in \mathbb{R}$，以下性質成立：

1.（正定性）$\|x\| \geq 0$；且當且僅當 $x = 0$ 時 $\|x\| = 0$。

2.（齊次性）$\|ax\| = |a|\|x\|$。

3.（三角不等式）$\|x+y\| \leq \|x\| + \|y\|$。

如果線性空間 X 上定義了一個範數，則稱 $(X, \|\cdot\|)$ 為一個 **賦範線性空間**。

定義 A.2.3 (距離)**.** 設 X 為一個集合。若函數 $d : X \times X \to \mathbb{R}$ 滿足對任意 $x, y, z \in X$：

1. $d(x,y) \geq 0$；且當且僅當 $x = y$ 時 $d(x,y) = 0$;

2. $d(x,y) = d(y,x)$；以及

3. $d(x,z) \leq d(x,y) + d(y,z)$,

則稱 d 為一個 **距離**（或 **度量**）。

如果 X 是一個帶範數 $\|\cdot\|$ 的線性空間，則函數 $d : X \times X \to \mathbb{R}$ 定義為 $d(x,y) := \|x - y\|$ 是一個距離（這可以通過驗證定義 A.2.3 中的條件容易得到）。在下文中，我們始終假設賦範線性空間 $(X, \|\cdot\|)$ 上的距離 d 由此方式定義。

定義 A.2.4 (Cauchy 序列)**.** 在賦範線性空間 $(X, \|\cdot\|)$ 中，一個序列 $\{x_k\}$ 稱為 **Cauchy 序列**，如果對任意 $\epsilon > 0$，存在 $K \in \mathbb{N}$，使得對所有 $k, j \geq K$，都有 $\|x_k - x_j\| < \epsilon$。

定義 A.2.5 (Banach 空間)**.** 如果賦範線性空間 $(X, \|\cdot\|)$ 中的每個 Cauchy 序列都收斂到 X 中的某一點，則稱其為 **完備的**。一個完備的賦範線性空間 $(X, \|\cdot\|)$ 稱為 **Banach 空間**。

若上下文中範數 $\|\cdot\|$ 已明確，我們將 $(X, \|\cdot\|)$ 簡記為 X。現在我們考慮 Banach 空間上的運算。

定義 A.2.6 (Lipschitz、壓縮映射與不動點)**.** 設 X 為 Banach 空間。若映射（算子）$T : X \to X$ 滿足存在常數 $\gamma > 0$ 使得

$$\|Tx - T\hat{x}\| \leq \gamma\|x - \hat{x}\|$$

對任意 $x, \hat{x} \in X$ 成立，則稱 T 在 X 上是 **Lipschitz** 的，常數為 γ。當 $\gamma = 1$ 時，稱 T 為 **非擴張**映射；當 $\gamma \in (0, 1)$ 時，稱 T 為 **γ-壓縮映射**（或稱 T 是 **γ-壓縮的**）。若存在 $x \in X$ 滿足 $Tx = x$，則稱 x 為 T 的一個 **不動點**。

定理 A.2.7 (Banach 不動點定理)**.** *設 X 為 Banach 空間，$T : X \to X$ 為一個 γ-壓縮映射，其中 $\gamma \in (0, 1)$。則有:*

(i) T 在 X 中存在唯一的不動點 x^*。

(ii) 對任意 $x \in X$，定義序列 $x_{k+1} = Tx_k$ 且 $x_0 = x$（該構造稱為 **不動點迭代***）。則對任意 $k \in \mathbb{N}$，有*

$$\|x_{k+1} - x_k\| \leq \gamma\|x_k - x_{k-1}\|, \tag{A.4}$$

以及

$$\|x_k - x^*\| \leq \gamma^k\|x - x^*\|. \tag{A.5}$$

稱 x_k 以速率 γ 線性收斂到 x^*。

证明. 我們同時證明上述兩個結論。取任意 $x \in X$ 並令 $x_0 = x$。設 $\{x_k\}$ 為由 γ-壓縮映射 T 生成的不動點迭代序列。則 (A.4) 由 T 的定義直接得到。進一步，對任意 $k \in \mathbb{N}$，有

$$\begin{aligned}
\|x_{k+1} - x_k\| &= \|Tx_k - Tx_{k-1}\| \\
&\leq \gamma\|x_k - x_{k-1}\| \\
&\leq \cdots \\
&\leq \gamma^k\|x_1 - x_0\|.
\end{aligned}$$

因此，對任意 $j \in \mathbb{N}$，

$$\begin{aligned}\|x_{k+j} - x_k\| &\le \|x_{k+j} - x_{k+j-1}\| + \cdots + \|x_{k+1} - x_k\| \\ &\le (\gamma^{k+j-1} + \cdots + \gamma^k)\|x_1 - x_0\| \\ &\le \frac{\gamma^k}{1-\gamma}\|x_1 - x_0\|.\end{aligned}$$

因此，當 k 足夠大時，$\|x_{k+j} - x_k\|$ 可任意小（與 j 無關），從而 $\{x_k\}$ 為 X 中的 Cauchy 序列。由於 X 完備，x_k 收斂到某個 $x^* \in X$。此外，由 $\lim_{k\to\infty} \|Tx_k - Tx^*\| = \lim_{k\to\infty} \|x_{k+1} - x^*\| = 0$，可得 $\|Tx^* - x^*\| = 0$，即 $x^* = Tx^*$，因此 x^* 為不動點。

接下來，

$$\begin{aligned}\|x_k - x^*\| &= \|Tx_{k-1} - Tx^*\| \\ &\le \gamma\|x_{k-1} - x^*\| \\ &\le \cdots \\ &\le \gamma^k\|x_0 - x^*\| \\ &= \gamma^k\|x - x^*\|,\end{aligned}$$

這證明了 (A.5)。

若存在兩個不同的不動點 x^* 和 $\hat{x}$，則 $\|x^* - \hat{x}\| > 0$，但

$$\|x^* - \hat{x}\| = \|Tx^* - T\hat{x}\| \le \gamma\|x^* - \hat{x}\| < \|x^* - \hat{x}\|,$$

產生矛盾。因此不動點唯一。 □

現在我們考慮第 4.1 節中的空間 $B(\mathcal{X})$。回顧 $\mathcal{X}$ 是一個集合，它可以是包含有限個或可數多個元素的集合，或者是某個歐幾里得空間的子集。接下來我們定義

$$B(\mathcal{X}) := \Big\{v : \mathcal{X} \to \mathbb{R} : \sup_{x\in\mathcal{X}} |v(x)| < \infty\Big\}. \tag{A.6}$$

即，$B(\mathcal{X})$ 是定義在 $\mathcal{X}$ 上的所有一致有界實值函數所組成的集合。我們還在 $B(\mathcal{X})$ 上定義 ∞-範數：

$$\|v\|_\infty := \sup_{x\in\mathcal{X}} |v(x)|. \tag{A.7}$$

顯然，$(B(\mathcal{X}), \|\cdot\|_\infty)$ 是一個賦範線性空間。接下來，我們想說明 $(B(\mathcal{X}), \|\cdot\|_\infty)$

是完備的。

為此，設 $\{v_k\}$ 是 $(B(\mathcal{X}), \|\cdot\|_\infty)$ 中的一個 Cauchy 序列。也就是說，對任意 $\epsilon > 0$，存在 $K \in \mathbb{N}$，使得當 $k, j \geq K$ 時，

$$\|v_k - v_j\|_\infty < \epsilon.$$

因此，對任意 $x \in \mathcal{X}$，有

$$|v_k(x) - v_j(x)| \leq \|v_k - v_j\|_\infty < \epsilon \tag{A.8}$$

對所有 $k, j \geq K$ 成立。這說明 $\{v_k(x)\}$ 是 $\mathbb{R}$ 中的一個 Cauchy 數列。因此，$\{v_k(x)\}$ 在 $\mathbb{R}$ 中收斂，並有極限，記為 $v(x)$。於是對每個 $x \in \mathcal{X}$，定義 $v : \mathcal{X} \to \mathbb{R}$ 為該極限。接著，對任意 $x \in \mathcal{X}$，令 (A.8) 中 $j \to \infty$，得到

$$|v_k(x) - v(x)| \leq \epsilon$$

只要 $k \geq K$ 就成立。由於 x 是任意的，因此我們得到

$$\|v_k - v\|_\infty \leq \epsilon \tag{A.9}$$

對所有 $k \geq K$ 成立。由於 ϵ 是任意的，可知當 $k \to \infty$ 時，$\|v_k - v\|_\infty \to 0$。此外，由 (A.9) 可見，對任意 $k \geq K$，有 $\|v\|_\infty \leq \epsilon + \|v_k\|_\infty < \infty$，這說明 $v \in B(\mathcal{X})$。綜上，$\{v_k\}$ 在 ∞-範數下收斂，且其極限 v 仍屬於 $B(\mathcal{X})$。因此，$B(\mathcal{X})$ 是完備的。由定義 A.2.5，$B(\mathcal{X})$ 是一個 Banach 空間。

A.3 信息論基礎

在本節中，我們對信息論的基礎知識及其與概率的關係作一個簡要回顧。

定義 A.3.1 (熵)**.** 設 X 是一個服從 $\mathbb{R}^n$ 上概率分佈 p 的隨機變量。則 X 的**熵**（entropy）定義為

$$H(X) := \mathbb{E}_{X\sim p}[-\log p(X)] = -\int p(x)\log p(x)\,dx. \tag{A.10}$$

如果 X 是離散隨機變量，即 X 的所有可能取值組成的集合 $\mathcal{X}$ 是有限集或可數無限集，則 (A.10) 中的積分應替換為對 $\mathcal{X}$ 的求和。除非某個公式僅適用於求和情形，否則本節統一使用積分記號。注意，我們總採用約定 $0\log 0 = 0$，

因為 $\lim_{z\to 0^+} z\log z = 0$。

隨機變量的熵（等價地，其概率分佈的熵）刻畫了平均不確定性水平。熵對隨機變量的均值（期望）是平移不變的，即隨機變量的均值發生平移時，熵不變。方差和維數是決定熵值的主要因素，但對於一般隨機變量而言，還有其他因素也會影響熵的大小。

如果 X 是離散隨機變量，則 $H(X) \geq 0$。要說明這一點，只需注意對每個 $x \in \mathcal{X}$，由於 $p(x) \in [0,1]$，有 $p(x)\log p(x) \in [-e^{-1}, 0]$。

如果 X 是連續隨機變量，則 $H(X)$ 可以為負。一個特殊且重要的例子是 d 維高斯隨機變量 $X \sim \mathcal{N}(\mu, \Sigma)$ 的熵。可以驗證，

$$H(X) = \frac{d}{2}(1 + \log 2\pi) + \frac{1}{2}\log\det(\Sigma).$$

由此可見，當 $\Sigma = \sigma^2 I_n$ 且 $\sigma \to 0$ 時，$H(X) \to -\infty$。

定義 A.3.2 (條件熵)**.** 設 X 和 Y 是兩個隨機變量，它們在 $\mathbb{R}^n \times \mathbb{R}^m$ 上具有聯合概率分佈 p。則在給定 X 的條件下，Y 的 **條件熵** 定義為

$$H(Y|X) := -\iint p(x,y)\log\frac{p(x,y)}{p_X(x)}\,dxdy, \tag{A.11}$$

其中 p_X 是 X 的邊緣分佈。

設 $H(X,Y)$ 表示 (X,Y) 的聯合熵，即

$$H(X,Y) = -\iint p(x,y)\log p(x,y)dxdy.$$

則有

$$H(Y|X) = H(X,Y) - H(X).$$

容易驗證，當且僅當 Y 完全由 X 決定（即 Y 是 X 的函數）時，$H(Y|X) = 0$。並且當且僅當 X 與 Y 相互獨立時，$H(Y|X) = H(Y)$。

定義 A.3.3 (Bregman 散度)**.** 設 F 是一個凸泛函，其一階變分為 δF。則與 F 相關的 **Bregman 散度** 定義為

$$D_F(g,f) := F(g) - F(f) - \int \delta F(f)\cdot(g-f),$$

其中 f 與 g 是任意兩個函數。

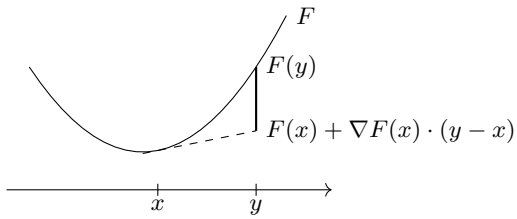


图 A.1: 與 F（凸曲線）相關的 Bregman 散度示意圖。經過點 $(x, F(x))$ 且斜率為 $\nabla F(x)$ 的切線用虛線表示。在點 y 處，函數 F 與該切線的取值分別為 $F(y)$ 和 $F(x) + \nabla F(x) \cdot (y - x)$。它們的差（點 y 上方豎直實線段的長度）就是 Bregman 散度 $D_F(y, x) = F(y) - F(x) - \nabla F(x) \cdot (y - x)$。由於凸性，$F$ 在任意處都位於切線之上或與其重合，因此對所有 x, y 都有 $D_F(y, x) \geq 0$。

在實際中，Bregman 散度大多用於有限維情形。此時，$F : \Omega \to \mathbb{R}$ 是一個凸可微函數，且 $\Omega \subset \mathbb{R}^n$ 是一個凸集。於是，與 F 相關的 **Bregman 散度**定義為

$$D_F(y, x) := F(y) - F(x) - \nabla F(x) \cdot (y - x),$$

對任意 $x, y \in \Omega$ 成立。有限維情形下 Bregman 散度 $D_F(y, x)$ 的示意圖見圖 A.1。

當 F 不可微時，Bregman 散度仍可定義，此時 $\nabla F(x)$ 可以由 F 在 x 處某個指定的次梯度代替。不過該次梯度需要在定義中明確說明。這裡我們不考慮這一推廣。

Bregman 散度的一個簡單例子是 $F(x) = \frac{1}{2}|x|^2$，其中 $x \in \mathbb{R}^n$。此時可驗證

$$D_F(y, x) = \frac{1}{2}|y - x|^2.$$

容易驗證，與 F 相關的 Bregman 散度具有如下性質，其中 F 可以是泛函或函數。以下統一使用 F 為函數時的記號。

1. 非負性：對所有 x, y，都有 $D_F(y, x) \geq 0$，因為 F 是凸的。若 F 嚴格凸，則當且僅當 $y = x$ 時，$D_F(y, x) = 0$。

2. 關於第一個變量的凸性：若 F 是（嚴格）凸的，則對任意 x，$D_F(\cdot, x)$ 也是（嚴格）凸的。

3. 關於第一個變量的凸性：對任意 x, y, z 和 $\lambda \in [0, 1]$，有

$$D_F((1 - \lambda)y + \lambda z, x) \leq (1 - \lambda)D_F(y, x) + \lambda D_F(z, x).$$

然而，Bregman 散度不是標準意義下的距離：它不具有對稱性，也不滿足三角不等式。

定義 A.3.4 (Kullback–Leibler 散度)**.** 兩個概率分佈 p 和 q 之間的 **Kullback–Leibler (KL) 散度** 定義為與熵泛函相關的 Bregman 散度，其中

$$F(\rho) := -\int \rho(x)\log\rho(x)dx$$

對任意 $\rho \in P := \{\rho : \mathbb{R}^n \to \mathbb{R} : \rho \geq 0, \int \rho = 1\}$ 成立。更具體地，

$$\mathrm{KL}(p,q) := D_F(p,q) = \int p(x)\log\frac{p(x)}{q(x)}\,dx$$

若 F 是定義在 $\mathcal{X}$ 上離散隨機變量的熵，則

$$\mathrm{KL}(p,q) := D_F(p,q) = \sum_{x\in\mathcal{X}} p(x)\log\frac{p(x)}{q(x)}$$

對 $\mathcal{X}$ 上任意概率分佈 p 和 q 成立。

由於 KL 散度是 Bregman 散度的一個特例，因此它具有後者的全部性質。特別地，我們應始終注意到 KL 散度是非負的，並且當且僅當處處有 $p = q$ 時，$\mathrm{KL}(p,q) = 0$（因為當 F 取為 p 的熵時，F 是嚴格凸的）。此外，KL 散度**不是**距離，因為它不對稱，即便 $\mathrm{KL}(p,q)$ 和 $\mathrm{KL}(q,p)$ 都存在，它們一般也不相等；並且 KL 散度不滿足三角不等式。接下來，我們進一步考察 KL 散度及其更多性質，因為在深度學習研究與應用中，它由於便於衡量兩個概率分佈之間的差異而被廣泛使用。

假設 p 和 q 是定義在 $\mathbb{R}^n$ 上的兩個概率分佈。由於對數項的存在，它們之間的 KL 散度並不總是存在，尤其是在 p 和 q 的支撐集不相同時。

如果對某個 $x \in \mathbb{R}^n$ 有 $p(x) > 0$ 且 $q(x) = 0$，則

$$p(x)\log\frac{p(x)}{q(x)} = +\infty.$$

如果所有這類點組成的集合在 $\mathbb{R}^n$ 中具有正測度，則 $\mathrm{KL}(p,q)$ 無定義，因為按其定義相應積分不存在。在這種情況下，我們不能用 KL 散度來衡量 p 與 q 的差異，也無法對 $\mathrm{KL}(p,q)$ 進行諸如求梯度之類的操作。這發生在 $p(x) > 0$ 出現在 q 的支撐之外的情形。

如果對任意 $x \in \mathbb{R}^n$ 有 $p(x) = 0$ 且 $q(x) > 0$，則不會出現上述求值問題，

因為

$$p(x)\log\frac{p(x)}{q(x)}=0.$$

在這種情況下，KL 散度是有定義的，因為 KL 中的積分是有限的。

總而言之，$\mathrm{KL}(p,q)$ 的使用僅限於 p 的支撐包含於 q 的支撐這一情形。否則，我們可能需要使用 $\mathrm{KL}(q,p)$ 或其他距離、散度來衡量 p 與 q 的差異。

當我們在深度學習模型中使用 KL 散度時，尤其要對上述問題保持謹慎。更具體地，回憶第 五 章中的若干生成模型，我們使用 KL 散度來衡量給定數據分佈 p_{data}（通常表現為數據集 $\mathcal{D}=\{x_i\in\mathbb{R}^d : i\in[M]\}$，其中 x_i 是獨立地從 p_{data} 中抽取的樣本）與生成分佈或推前分佈 p_θ 之間的差異，其中 θ 表示某個待訓練深度神經網絡的參數，p_θ 是由該網絡誘導出的概率分佈。在這種情況下，我們常常希望通過求解

$$\min_\theta\ \Big\{\mathrm{KL}(p_{\mathrm{data}},p_\theta)=-H(p_{\mathrm{data}})-\int p_{\mathrm{data}}(x)\log p_\theta(x)\,dx\Big\}.$$

來找到最優參數 θ。由於 p_{data} 是未知且固定的，$H(p_{\mathrm{data}})$ 與 θ 無關，因此上述問題可化為

$$\max_\theta\ \int p_{\mathrm{data}}(x)\log p_\theta(x)\,dx \quad\approx\quad \max_\theta\ \frac{1}{M}\sum_{i=1}^{M}\log p_\theta(x_i),$$

其中近似來自 Monte Carlo 積分，並利用了 $x_i\sim p_{\mathrm{data}}$。我們可以看出，這要求對所有 $x_i\in\mathcal{D}$，都有 $p_\theta(x_i)>0$。另一方面，由於使用神經網絡，p_θ 往往具有某種連續性或光滑性。因此，在 p_{data} 的支撐之外的點 x 處，$p_\theta(x)>0$ 是很可能發生的。這雖然不會導致計算問題，但卻可能推斷或生成不屬於 p_{data} 的樣本。這可能導致對 θ 的估計不準確，甚至得到目標問題的錯誤解。在這種情況下，可能需要對方法作適當修正，或採用其他替代方法。

當所涉及的概率分佈具有某些特殊形式時，KL 散度可以安全使用，並且常常有具體的表達式。下面給出兩個例子。

例 A.3.5 (兩個高斯分佈之間的 KL 散度)**.** 設 p_1 和 p_2 是定義在 $\mathbb{R}^n$ 上的兩個高斯概率分佈。具體而言，

$$p_1=\mathcal{N}(\mu_1,\Sigma_1),\qquad p_2=\mathcal{N}(\mu_2,\Sigma_2),$$

其中 $\mu_1,\mu_2\in\mathbb{R}^n$，且 $\Sigma_1,\Sigma_2\in\mathbb{R}^{n\times n}$ 是對稱正定矩陣。從 p_1 到 p_2 的 KL 散

度定義為

$$\mathrm{KL}(p_1, p_2) = \int p_1(x) \log \frac{p_1(x)}{p_2(x)}\, dx = \mathbb{E}_{X\sim p_1}\Big[\log \frac{p_1(X)}{p_2(X)}\Big]. \tag{A.12}$$

注意，高斯概率密度 $\mathcal{N}(x;\mu,\Sigma)$ 的對數為

$$\log \mathcal{N}(x;\mu,\Sigma) = -\frac{n}{2}\log(2\pi) - \frac{1}{2}\log\det(\Sigma) - \frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu).$$

將 p_1 與 p_2 的表達式代入 (A.12)，可得

$$\begin{aligned}\log \frac{p_1(X)}{p_2(X)} = \frac{1}{2}\log\frac{\det(\Sigma_2)}{\det(\Sigma_1)} \\ - \frac{1}{2}\Big[(X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1) - (X-\mu_2)^\top \Sigma_2^{-1}(X-\mu_2)\Big].\end{aligned} \tag{A.13}$$

對 (A.13) 兩邊關於 $X \sim p_1 = \mathcal{N}(\mu_1, \Sigma_1)$ 取期望，我們分別計算上式中的二次項。具體而言，

$$\begin{aligned}\mathbb{E}_{X\sim p_1}\big[(X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1)\big] &= \mathbb{E}_{X\sim p_1}\Big[\mathrm{tr}\Big((X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1)\Big)\Big] \\ &= \mathbb{E}_{X\sim p_1}\Big[\mathrm{tr}\Big(\Sigma_1^{-1}(X-\mu_1)(X-\mu_1)^\top\Big)\Big] \\ &= \mathrm{tr}\Big(\Sigma_1^{-1}\mathbb{E}_{X\sim p_1}\Big[(X-\mu_1)(X-\mu_1)^\top\Big]\Big) \\ &= \mathrm{tr}(\Sigma_1^{-1}\Sigma_1) \\ &= n.\end{aligned} \tag{A.14}$$

此外，有

$$X - \mu_2 = (X-\mu_1) + (\mu_1 - \mu_2),$$

因此

$$\mathbb{E}_{X\sim p_1}\big[(X-\mu_2)^\top \Sigma_2^{-1}(X-\mu_2)\big] = \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_1-\mu_2)^\top \Sigma_2^{-1}(\mu_1-\mu_2), \tag{A.15}$$

其中交叉項由於 $\mathbb{E}_{X\sim p_1}[X-\mu_1] = 0$ 而消失。

結合 (A.12)、(A.13)、(A.14) 和 (A.15)，得到 KL 散度的閉式表達

$$\mathrm{KL}(p_1,p_2) = \frac{1}{2}\Big[\log\frac{\det(\Sigma_2)}{\det(\Sigma_1)} - n + \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_2-\mu_1)^\top \Sigma_2^{-1}(\mu_2-\mu_1)\Big]. \tag{A.16}$$

如果 $\Sigma_1 = \Sigma_2 = \Sigma$，則 (A.16) 退化為

$$\mathrm{KL}(p_1, p_2) = \frac{1}{2}(\mu_2 - \mu_1)^\top \Sigma^{-1}(\mu_2 - \mu_1).$$

特別地，當且僅當 $\mu_1 = \mu_2$ 且 $\Sigma_1 = \Sigma_2$ 時，$\mathrm{KL}(p_1, p_2) = 0$。

例 A.3.6 (兩個 Poisson 分佈之間的 KL 散度)**.** 設 $p_1 = \mathrm{Poisson}(\lambda_1)$ 且 $p_2 = \mathrm{Poisson}(\lambda_2)$，其中 $\lambda_1, \lambda_2 > 0$。從 p_1 到 p_2 的 KL 散度定義為

$$\mathrm{KL}(p_1, p_2) = \sum_{x=0}^{\infty} p_1(x) \log \frac{p_1(x)}{p_2(x)}. \tag{A.17}$$

回憶參數為 λ 的 Poisson 分佈的概率質量函數為

$$\Pr(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}, \quad \text{其中} x = 0, 1, 2, \ldots.$$

將其代入 (A.17)，得到

$$\mathrm{KL}(p_1, p_2) = \sum_{x=0}^{\infty} p_1(x) \log \frac{p_1(x)}{p_2(x)} = \sum_{x=0}^{\infty} \frac{\lambda^x e^{-\lambda}}{x!} \Big(x \log \frac{\lambda_1}{\lambda_2} + \lambda_2 - \lambda_1 \Big).$$

注意到對於 $X \sim p_1$，有 $\mathbb{E}[X] = \sum_{x=0}^{\infty} x \cdot \frac{\lambda^x e^{-\lambda}}{x!} = \lambda_1$，從而得到閉式表達

$$\mathrm{KL}(p_1, p_2) = \lambda_1 \log \frac{\lambda_1}{\lambda_2} + \lambda_2 - \lambda_1.$$

容易驗證，當且僅當 $\lambda_1 = \lambda_2$ 時，$\mathrm{KL}(p_1, p_2) = 0$。

KL 散度有若干變體，它們具有對稱性，並且在 p 和 q 不共享相同支撐時仍然有定義。其中最著名的是 Jensen–Shannon 散度，其定義如下。

定義 A.3.7 (Jensen–Shannon 散度)**.** 兩個概率分佈 p 和 q 之間的 **Jensen–Shannon (JS) 散度** 定義為

$$\mathrm{JS}(p, q) := \frac{1}{2}\Big(\mathrm{KL}(p, r) + \mathrm{KL}(q, r) \Big) \tag{A.18}$$

其中 $r = (p + q)/2$ 是 p 與 q 的逐點平均。

注意到 r 本身也是一個概率分佈，且其支撐包含了 p 和 q 的支撐。因此，$\mathrm{JS}(p, q)$ 總是良定義且取有限值。事實上，可以驗證對任意 p 和 q，都有 $\mathrm{JS}(p, q) \in [0, \log 2]$。當 p 和 q 的支撐不相交時，取得最大值 $2 \log 2$。不過，需

要注意的是，這並不一定是優點，因為此時 $\mathrm{JS}(p,q)$ 關於 p 或 q 的廣義梯度變為零，從而在實際中無法使用基於梯度的優化算法來更新它們（更準確地說，是無法更新生成它們或將其推前的神經網絡參數；例如 p 可能是由某生成網絡得到的概率分佈，或者由某神經網絡逼近的推前 push-forward 映射得到的）。

儘管如此，容易驗證 $\mathrm{JS}(p,q) \geq 0$，且當且僅當 $p = q$ 時，$\mathrm{JS}(p,q) = 0$。此外，不同於 KL 散度，JS 散度是對稱的，即對定義在 $\mathbb{R}^n$ 上的任意概率分佈 p 和 q，都有 $\mathrm{JS}(p,q) = \mathrm{JS}(q,p)$。

儘管 JS 散度僅是 KL 散度的一個簡單變形，並帶來少量新性質，但它在使用上更困難，因為對許多典型概率分佈而言，它沒有具體的表達式。例如，兩個高斯分佈之間的 JS 散度不像 (A.16) 中的 KL 散度那樣有具體的表達式，因為 (A.18) 中的 r 變成了兩個高斯分佈的混合分佈，而從 p（或 q）到 r 的 KL 散度沒有具體的表達式。

另外存在一種距離，它可以以更優美的數學方式度量兩個概率分佈之間的差異，並克服上述 KL 與 JS 散度的問題。這就是下面要定義的 Wasserstein 距離。

定義 A.3.8 (Wasserstein 距離)**.** 設

$$P := \{p : \mathbb{R}^n \to \mathbb{R} : p(x) \geq 0, \forall\, x \in \mathbb{R}^n, \int p(x)dx = 1\}$$

為定義在 $\mathbb{R}^n$ 上的概率分佈空間。對 $s \in [1,\infty), p, q \in P$ 之間的 **Wasserstein-s 距離** 定義為

$$W_s(p,q) := \inf_{\pi \in \Pi(p,q)} \Big(\int |x-y|^s \pi(x,y)\, dx \Big)^{1/s}, \tag{A.19}$$

其中 $\Pi(p,q)$ 表示定義在 $\mathbb{R}^n \times \mathbb{R}^n$ 上、以 p 和 q 為邊緣分佈的聯合概率分佈全體，即

$$\Pi(p,q) := \Big\{ \pi : \mathbb{R}^d \times \mathbb{R}^d \to [0,\infty) : \begin{array}{l} p(x) = \int \pi(x,y)\, dy, \\ q(y) = \int \pi(x,y)\, dx, \end{array} \ \forall\, x, y \in \mathbb{R}^d \Big\}.$$

當 $s = \infty$ 時，$p, q \in P$ 之間的 Wasserstein-∞ 距離定義為

$$W_\infty(p,q) := \lim_{s \to \infty} W_s(p,q).$$

(A.19) 的最優解 π^* 稱為從 p 到 q 的一個 **最優傳輸**。

作為一個例子，定義在 $\mathbb{R}^d$ 上的兩個高斯分佈 $p_1 = \mathcal{N}(\mu_1, \Sigma_1)$ 和 $p_2 = \mathcal{N}(\mu_2, \Sigma_2)$ 之間的 Wasserstein-2 距離可表示為

$$W_2(p_1, p_2) = \Big(|\mu_1 - \mu_2|^2 + \mathrm{tr}\Big(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{1/2}\Sigma_2\Sigma_1^{1/2})^{1/2}\Big)\Big)^{1/2}.$$

Wasserstein 距離具有許多重要的數學性質。其中一個基本性質是，W_s 在嚴格數學意義下確實是一個距離（見定義 A.2.3）。即，W_s 滿足如下性質：

- 正定性：對所有 $p, q \in P$，有 $W_s(p, q) \geq 0$；並且當且僅當 $p = q$ 在 $\mathbb{R}^n$ 上幾乎處處成立時，$W_s(p, q) = 0$。
- 對稱性：對任意 $p, q \in P$，有 $W_s(p, q) = W_s(q, p)$。
- 三角不等式：對任意 $p, q, r \in P$，有 $W_s(p, q) \leq W_s(p, r) + W_s(r, q)$。

此外，即使 p 和 q 的支撐不同，$W_s(p, q)$ 仍然是良定義的。這相比 KL 散度是一個顯著優勢。進一步地，Wasserstein 距離具有許多有用的物理解釋。例如，Wasserstein-1 距離也稱為 **Earth mover distance, EMD**，它衡量將概率質量 p 運輸為 q 的最優代價，其中將單位質量從位置 x 移動到位置 y 的代價與距離 $|x - y|$ 成正比。

Wasserstein 距離是最優傳輸領域中的一個基本概念，而最優傳輸理論在過去幾十年中得到了廣泛研究。最優傳輸理論與數學的許多分支相關聯，包括幾何分析、偏微分方程、概率論和最優控制。許多重要結果可見 [3,125,151]。

然而，Wasserstein 距離的主要問題在於它們在數值上難以計算。這是由於其定義中包含約束條件，以及其在實際中常用於高維空間。在簡單情形下，當 p 和 q 是定義在一個較小有限集 $\mathcal{X}$ 上的概率分佈時，它們實際上是歸一化直方圖，而 (A.19) 退化為一個線性規劃問題，可以由現有優化算法求解。通過在 (A.19) 的目標函數中加入聯合概率分佈的熵正則項，可以顯著加速計算，並且可通過求解該問題的對偶形式、使用稱為 Sinkhorn 算法得到解。然而，這一技術並不適用於高維問題或定義在諸如 $\mathbb{R}^n$ 這類連續空間上的問題，原因是計算量過大。近年來，也發展出了一些基於深度學習的方法來計算 Wasserstein 距離。例如，可以將問題 (A.19) 轉化為其對偶形式，然後將對偶變量參數化為深度神經網絡，並通過訓練來求解對偶問題，從而得到 (A.19) 的解。相關結果可見 [108]。但是，到目前為止，針對一般深度學習模型，仍缺乏一種在計算上高效且能準確逼近 Wasserstein 距離的方法。這阻礙了 Wasserstein 距離在深度學習中的廣泛應用。

本節最後要介紹的概念是一個經典概念，稱為互信息。與上面的各種散度

和距離不同，互信息利用隨機變量之間的聯合概率分佈來衡量它們的依賴關係。互信息的正式定義如下。

定義 A.3.9 (互信息)**.** 設 X 和 Y 是兩個具有聯合概率分佈 $p(x,y)$ 的隨機變量。X 與 Y 之間的 **互信息**（mutual information）定義為

$$\mathrm{MI}(X,Y) = \mathrm{KL}(p, p_X p_Y) = \iint p(x,y)\frac{p(x,y)}{p_X(x)p_Y(y)}\,dxdy, \tag{A.20}$$

其中 p_X 和 p_Y 分別是 X 與 Y 的邊緣概率分佈。

注意，互信息是非負的，因為它被定義為從聯合概率分佈 p 到邊緣概率分佈乘積 $p_X p_Y$ 的 KL 散度。此外，我們可以看出，當且僅當 $p = p_X p_Y$ 時，$\mathrm{MI}(X,Y) = 0$，即 X 與 Y 相互獨立。互信息也是對稱的。不過，我們強調互信息在任何意義下都不是距離。儘管如此，較大的 $\mathrm{MI}(X,Y)$ 表明 X 與 Y 存在較強的依賴關係。事實上，兩個隨機變量的互信息與其（條件）熵 $H(X|Y)$ 之間有如下關係：

$$\begin{aligned}
\mathrm{MI}(X,Y) &= \int p(x,y)\log\frac{p(x,y)}{p_X(x)p_Y(y)}\,dxdy \\
&= \int p(x,y)\log\frac{p_{X|Y}(x|y)}{p_X(x)}\,dxdy \\
&= \int p(x,y)\log p_{X|Y}(x|y)\,dxdy - \int p_X(x)\log p_X(x)\,dx \\
&= H(X) - H(X|Y).
\end{aligned}$$

這再次說明，當 X 與 Y 獨立時，$\mathrm{MI}(X,Y) = 0$，此時 $H(X|Y) = H(X)$。另一個極端情形是 X 完全由 Y 決定（即 X 是 Y 的函數），此時 $H(X|Y) = 0$ 且 $\mathrm{MI}(X,Y) = H(X)$。

A.4 隨機微分方程基礎

隨機微分方程（stochastic differential equation，SDE）在現代深度學習研究與應用中的問題建模、模型構造和推導中變得愈發常見。例如，在第 5.3 節的擴散模型中，我們已經看到了 SDE 的大量應用。

在本節中，我們將簡要回顧 SDE 的一些基礎知識。具體而言，我們將介紹 Wiener 過程、隨機積分與隨機微分、Itô 公式以及 Fokker–Planck 方程的定義。

考慮一個粒子（作為隨機遊走者）在實直線 $\mathbb{R}$ 上運動。在時刻 $t=0$，它位於原點。此後，它在每個單位時間 Δt 內以步長 Δx 運動。也就是說，在每個 Δt 內，它以概率 $\frac{1}{2}$ 向左移動（位置變化為 $-\Delta x$），以概率 $\frac{1}{2}$ 向右移動（位置變化為 Δx）。

設 $X(t)$ 表示粒子在經過 n 步、時刻 $t=n\Delta t$ 時的位置。為了研究 $X(t)$ 的概率分佈，我們考慮獨立隨機變量 $X_i \sim \mathrm{Bernoulli}(\frac{1}{2})$，其中 $i=1,2,\ldots$，因此 $X_i=1$ 或 0 的概率均為 $\frac{1}{2}$。顯然，$\mathbb{E}[X_i]=\frac{1}{2}$ 且 $\mathrm{Var}[X_i]=\frac{1}{4}$。

令 $S_n := \sum_{i=1}^n X_i$，則 S_n 表示粒子在前 n 步中向右移動的次數，而 $n-S_n$ 表示向左移動的次數。於是有 $\mathbb{E}[S_n]=\frac{n}{2}$ 且 $\mathrm{Var}[S_n]=\frac{n}{4}$。進一步，

$$X(t) = S_n\Delta x - (n-S_n)\Delta x = (2S_n - n)\Delta x.$$

因此，$\mathbb{E}[X(t)]=0$，並且

$$\mathrm{Var}[X(t)] = (\Delta x)^2\,\mathrm{Var}[2S_n-n] = (\Delta x)^2 4\,\mathrm{Var}[S_n] = (\Delta x)^2 n = \frac{(\Delta x)^2}{\Delta t}t.$$

不失一般性，設 $\frac{(\Delta x)^2}{\Delta t}=1$，於是容易驗證

$$X(t) = (2S_n-n)\Delta x = \frac{S_n - \frac{n}{2}}{\sqrt{\frac{n}{4}}}\sqrt{t}.$$

現在，對任意固定的 t，令 $n\to\infty$ 且保持 $\frac{(\Delta x)^2}{\Delta t}=1$，由 Laplace–De Moivre 定理可得

$$\begin{aligned}
\lim_{n\to\infty}\Pr(a\le X(t)\le b) &= \lim_{n\to\infty}\Pr\Big(\frac{a}{\sqrt{t}} \le \frac{S_n-\frac{n}{2}}{\sqrt{\frac{n}{4}}} \le \frac{b}{\sqrt{t}}\Big)\\
&= \frac{1}{\sqrt{2\pi}}\int_{a/\sqrt{t}}^{b/\sqrt{t}} e^{-\frac{x^2}{2}}\,dx\\
&= \frac{1}{\sqrt{2\pi t}}\int_a^b e^{-\frac{x^2}{2t}}\,dx,
\end{aligned}$$

這說明 $X(t)\sim\mathcal{N}(0,t)$。換言之，如果粒子按連續時間的意義運動，則其位置滿足 $X(t)\sim\mathcal{N}(0,t)$。這導出了 **Wiener 過程**（也常稱為 **Brownian motion**）的定義如下。

定義 A.4.1 (Wiener 過程)**.** 若實值隨機過程 $W:[0,\infty)\to\mathbb{R}$ 滿足：

1. 從原點出發：$W(0)=0$ 幾乎處處成立；

2. 增量獨立：對任意 $0 < t_1 < t_2 < \cdots < t_k$，隨機變量

$$W(t_1),\ W(t_2) - W(t_1),\ \ldots,\ W(t_k) - W(t_{k-1})$$

相互獨立；

3. 增量服從正態分佈：對任意 $t > s \geq 0$，有 $W(t) - W(s) \sim \mathcal{N}(0, t-s)$；以及

4. 路徑連續：映射 $t \mapsto W(t)$ 關於 t 連續。

則稱 W 為一個 **Wiener 過程**。

注意，由於 $W(t) \sim \mathcal{N}(0,t)$，對任意 t 都有 $\mathbb{E}[W(t)] = 0$ 且 $\mathrm{Var}[W(t)] = \mathbb{E}[W(t)^2] = t$。

為了嚴格定義 SDE，我們需要一系列來自測度論的數學概念。下面我們只作簡要介紹，更詳細內容可參見 [38,93,106]。

設 (Ω, Γ, μ) 是一個概率空間，其中此處取 $\Omega = \mathbb{R}^d$，Γ 是 Ω 上的一個 σ-代數，而 μ 是一個概率測度，使得 Γ 中所有元素在 μ 下都是可測的。這裡，Ω 的一個 σ-代數被定義為 Ω 的若干子集所構成的集合，它包含空集 $\emptyset$，並且對補集和可數並運算封閉。若 A 是 Ω 的某些子集組成的集合，而 $\Gamma(A)$ 是所有包含 A 的 σ-代數的交，則稱 $\Gamma(A)$ 為由 A 生成的 σ-代數。可以證明，$\Gamma(A)$ 確實也是一個 σ-代數。由於隨機變量可以視作 μ-可測函數，因此我們也可以將 $\Gamma(\{X_\alpha : \alpha \in \mathcal{A}\})$ 定義為由隨機變量族 $\{X_\alpha : \alpha \in \mathcal{A}\}$ 在 Ω 上生成的 σ-代數，其中 $\mathcal{A}$ 是一個可以是無限的指標集，而這裡的 X_α 指的是 $\mathbb{R}$ 中 Lebesgue 可測集合在映射 X_α 下所有原像組成的集合。

我們用 $\mathcal{W}(t) := \Gamma(\{W(s) : 0 \leq s \leq t\})$ 表示由 Wiener 過程截至時刻 t 生成的 σ-代數。我們也稱 $\mathcal{W}(t)$ 為 Wiener 過程 W 在時刻 t 之前的歷史。我們再用生成的 σ-代數 $\mathcal{W}^+(t) := \Gamma(\{W(s) - W(t) : s \geq t\})$ 表示 Wiener 過程 W 在時刻 t 之後的未來。若一族 σ-代數 $\{\mathcal{F}(t) \subset \Gamma : t \geq 0\}$ 相對於 W 滿足：(i) 對所有 $0 \leq s < t$，有 $\mathcal{F}(s) \subset \mathcal{F}(t)$；(ii) 對所有 $t \geq 0$，有 $\mathcal{W}(t) \subset \mathcal{F}(t)$；且 (iii) 對所有 $t \geq 0$，$\mathcal{F}(t)$ 與 $\mathcal{W}^+(t)$ 相互獨立，則稱其相對於 W 是非預見的（也稱適應的）。我們也稱 $\mathcal{F}(t)$ 是截至時刻 t 的一個濾子（filter）。我們可以非正式地將 $\mathcal{F}(t)$ 理解為 “包含截至時刻 t 的所有信息” 的 σ-代數，也就是說，對任意 t，所有由 Wiener 過程截至時刻 t 所生成的隨機變量，以及一些與 Wiener 過程獨立的初始隨機變量，都在 $\mathcal{F}(t)$ 下可測。

若實值隨機過程 G 對任意 $t \geq 0$ 都滿足 $G(t)$ 關於 $\mathcal{F}(t)$ 可測，則稱 G 相對於 $\mathcal{F}$ 是非預見的。此外，用 $L^2(0,T)$ 表示所有滿足 $\mathbb{E}[\int_0^T |G(t)|^2 dt] < \infty$

的實值非預見隨機過程 G 所組成的空間。類似地，用 $L^1(0,T)$ 表示所有滿足 $\mathbb{E}[\int_0^T |F(t)|dt] < \infty$ 的實值非預見隨機過程 F 所組成的空間。$L^2(0,T)$ 中一類特殊的隨機過程稱為階梯過程：設 $0 = t_0 < t_1 < \cdots < t_K = T$ 是區間 $[0,T]$ 的一個劃分，且對 $t = 0,\ldots,K-1$，在每個區間 $[t_k, t_{k+1})$ 上有 $G(t) = G_k$，其中 G_k 是一個 $\mathcal{F}(t_k)$-可測隨機變量，則稱 G 為一個階梯過程。對於這樣一個屬於 $L^2(0,T)$ 的階梯過程 G，我們將其在 $(0,T)$ 上的 Itô 隨機積分定義為

$$\int_0^T G(t)\,dW(t) := \sum_{k=0}^{K-1} G_k(W(t_{k+1}) - W(t_k)),$$

它是一個隨機變量。可以證明，對任意 $G \in L^2(0,T)$，存在一列階梯過程 $G_n \in L^2(0,T)$，使得

$$\mathbb{E}\Big[\int_0^T |G - G_n|^2\,dt\Big] \to 0$$

當 $n \to \infty$ 時成立。這裡各 G_n 可能對應於 $[0,T]$ 上不同的劃分，並且在不同子區間上取不同的隨機變量。然後，Itô 隨機積分的下列性質可以先對階梯過程證明，再推廣到 $L^2(0,T)$ 中所有隨機過程：對任意 $a,b \in \mathbb{R}$ 和 $G,H \in L^2(0,T)$，

$$\int_0^T (aG(t) + bH(t))\,dW(t) = a\int_0^T G(t)\,dW(t) + b\int_0^T H(t)\,dW(t),$$

$$\mathbb{E}\Big[\int_0^T G(t)\,dW(t)\Big] = 0,$$

$$\mathbb{E}\Big[\int_0^T G(t)\,dW(t)\cdot\int_0^T H(t)\,dW(t)\Big] = \int_0^T G(t)\cdot H(t)\,dt.$$

基於上述 Itô 隨機積分的定義，我們可以如下定義實值隨機過程 X：

$$X(t) = X(0) + \int_0^t f(s,X(s))\,ds + \int_0^t g(s,X(s))\,dW(s)$$

對任意 $t \in [0,T]$，其中 $f,g : [0,T]\times\mathbb{R} \to \mathbb{R}$ 滿足 $f(\cdot,X(\cdot)) \in L^1(0,T)$ 且 $g(\cdot,X(\cdot)) \in L^2(0,T)$。在這種情況下，我們也稱 X 具有隨機微分

$$dX(t) = f(t,X(t))\,dt + g(t,X(t))\,dW(t) \tag{A.21}$$

對所有 $t \geq 0$ 成立，初值為 $X(0)$。我們也稱 X 服從 SDE (A.21)。SDE 可以推廣到多維情形，其中 $X(t) \in \mathbb{R}^d$，$W(t) = (W_1(t),\ldots,W_m(t))^\top \in \mathbb{R}^m$，這裡每個 $W_i(t)$ 都是一維 Wiener 過程，且 W 中這 m 個 Wiener 過程彼此

獨立（我們稱 W 為標準 m 維 Wiener 過程），同時 $f:[0,T]\times\mathbb{R}^d\to\mathbb{R}^d$，$g:[0,T]\times\mathbb{R}^d\to\mathbb{R}^{d\times m}$，並且仍滿足相應的可積性條件。我們通常稱 f 為漂移項，g 為擴散係數。

隨機分析中最重要的性質之一稱為 Itô 公式（也稱 Itô 鏈式法則），其表述如下。證明可見例如 [38,106]。

引理 A.4.2 (Itô 公式)**.** 設 X 滿足 *SDE* (A.21)，且 $u:[0,T]\times\mathbb{R}\to\mathbb{R}$ 連續（並且 $\partial_t u$、$\partial_x u$ 和 $\partial^2_{xx}u$ 都連續）。令 $Y(t):=u(t,X(t))$，則 Y 滿足

$$dY(t)=\Big(\partial_t u+\partial_x u\cdot f+\frac{1}{2}g^2\partial^2_{xx}u\Big)dt+(\partial_x u\cdot g)dW(t) \tag{A.22}$$

其中，為簡化記號，下文統一記 $u=u(t,X(t))$、$f=f(t,X(t))$ 和 $g=g(t,X(t))$。

多維版本的公式 (A.22) 為

$$dY(t)=\Big(\partial_t u+\partial_x u\cdot f+\frac{1}{2}\langle\nabla^2 u,gg^\top\rangle\Big)dt+\partial_x u\cdot g\cdot dW(t), \tag{A.23}$$

其中 $X(t)\in\mathbb{R}^d$, $W(t)\in\mathbb{R}^m$ 是標準 m 維 *Wiener* 過程，$f:[0,T]\times\mathbb{R}^d\to\mathbb{R}^d$，$g:[0,T]\times\mathbb{R}^d\to\mathbb{R}^{d\times m}$，$u:[0,T]\times\mathbb{R}^d\to\mathbb{R}$，$\nabla^2 u$ 表示 u 的 *Hessian* 矩陣，且對矩陣 $A=[A_{ij}]_{i,j=1}^d$ 和 $B=[B_{ij}]_{i,j=1}^d$，定義 $\langle A,B\rangle:=\mathrm{tr}(A^\top B)=\sum_{i,j=1}^d A_{ij}B_{ij}$。

下面通過一個例子說明如何利用 Itô 公式求隨機積分。不過需要注意的是，一般情況下隨機積分通常難以解析求值。

例 A.4.3. 求

$$\int_0^T W(t)\,dW(t).$$

設 $X(t)=W(t)\in\mathbb{R}$，並令 $u(x):=x^2$。則 $\nabla u(x)=2x$，$\nabla^2 u(x)=2$。由 Itô 公式，有

$$\begin{aligned}d(W(t)^2)&=d\,u(X(t))\\&=\Big(\nabla u(X(t))\cdot 0+\frac{1}{2}\nabla^2 u(X(t))\cdot 1\Big)\,dt+\nabla u(X(t))\cdot 1\,dW(t)\\&=dt+2W(t)\,dW(t).\end{aligned}$$

對兩邊在 $(0,T)$ 上積分，得到

$$\int_0^T d(W(t)^2)=\int_0^T dt+2\int_0^T W(t)\,dW(t). \tag{A.24}$$

注意到 (A.24) 左邊為

$$\int_0^T d(W(t)^2) = W(T)^2 - W(0)^2 = W(T)^2,$$

而右邊為

$$T + 2\int_0^T W(t)\,dW(t).$$

因此，對 (A.24) 進行整理可得

$$\int_0^T W(t)\,dW(t) = \frac{W(T)^2}{2} - \frac{T}{2}.$$

現在我們從 $X(t)$ 的概率分佈角度來考察該過程。設 SDE (A.21) 中的 $X(t)$ 在任意時刻 t 都服從概率分佈 $\rho(\cdot,t)$。我們希望找出 $\rho(\cdot,t)$ 的性質。

為此，我們給出一個啟發式但不夠嚴格的推導。不過，所得出的 Fokker–Planck 方程結論是正確的。更嚴格的推導可參見 [118]。

設 $\phi:\mathbb{R}^d\to\mathbb{R}$ 是任意光滑測試函數。假設有 N 個相互獨立的粒子 $X_i(t)$，它們在任意時刻 t 都服從分佈 $\rho(\cdot,t)$。則通過 Monte Carlo 積分，我們有如下近似

$$\int \rho(x,t)\phi(x)\,dx \approx \frac{1}{N}\sum_{i=1}^N \phi(X_i(t)). \tag{A.25}$$

對兩邊關於時間求導，則 (A.25) 左邊變為

$$\frac{d}{dt}\int \rho(x,t)\phi(x)\,dx = \int \partial_t\rho(x,t)\phi(x)\,dx. \tag{A.26}$$

由 Itô 公式，有

$$\begin{aligned} d\,\phi(X_i(t)) = &\Big(\nabla\phi(X_i(t))f_t(X_i(t)) + \frac{1}{2}\langle\nabla^2\phi(X_i(t)), g_t(X_i(t))g_t(X_i(t))^\top\rangle\Big)dt \\ &+ \nabla\phi(X_i(t))f_t(X_i(t))dW(t), \end{aligned} \tag{A.27}$$

其中為簡便起見，我們將 $f(t,\cdot)$ 和 $g(t,\cdot)$ 分別記作 $f_t(\cdot)$ 和 $g_t(\cdot)$。下文也將 $\rho(\cdot,t)$ 記作 $\rho_t(\cdot)$。將 (A.27) 代入 (A.25) 右邊的時間導數，並令 $N\to\infty$ 取連續極限，可得到

$$\int \rho_t(x)\Big(\nabla\phi(x)\cdot f_t(x) + \frac{1}{2}\langle\nabla^2\phi(x), g_t(x)g_t(x)^\top\rangle\Big)\,dx.$$

再對其做分部積分，得到

$$\int \Big(-\nabla \cdot (\rho_t(x) f_t(x)) + \frac{1}{2} \langle \nabla^2, g_t(x) g_t(x)^\top \rho(x,t) \rangle \Big) \phi(x)\, dx, \tag{A.28}$$

其中

$$\langle \nabla^2, A(x) \rangle := \sum_{i,j=1}^{d} \partial^2_{x_i x_j} A_{ij}(x)$$

對任意矩陣值函數 $A(x) = [A_{ij}(x)]_{i,j=1}^d$（$x \in \mathbb{R}^d$）成立。現在，由於 (A.26) 與 (A.28) 對任意 ϕ 都相等，從而推出

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)) + \frac{1}{2} \langle \nabla^2, g_t(x) g_t(x)^\top \rho_t(x) \rangle. \tag{A.29}$$

我們稱 (A.29) 為 ρ 的 **Fokker–Planck 方程**。

注意，在許多人為設計的 SDE 中，例如第 5.3 節中的擴散模型，擴散係數常被設為與狀態無關的函數 $g_t = \sigma_t I_d$，其中 $\sigma_t \in \mathbb{R}$ 可以依賴於時間 t，但不依賴於狀態 x。此時，Fokker–Planck 方程可簡化為

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)) + \frac{\sigma_t^2}{2} \Delta \rho_t(x).$$

在沒有擴散項的情形，即 $g = 0$ 時，我們得到所謂的 **連續方程**：

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)), \tag{A.30}$$

相應的 SDE (A.21) 退化為 ODE

$$\dot{x}(t) = f(t, x(t)).$$

方程 (A.30) 在本書中多次出現，例如在第 3.3 節的 Neural ODE 方法和第 5.5 節的 flow matching 方法中。順便提一句，連續方程通常是指遵循某種守恆律的一類方程，而 (A.30) 是一個保持概率守恆的特殊例子。

参考文献

[1] J. Adler and S. Lunz. Banach wasserstein gan. *Advances in neural information processing systems*, 31, 2018.

[2] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*, 2023.

[3] L. Ambrosio, E. Brué, D. Semola, et al. *Lectures on optimal transport*, volume 130. Springer, 2021.

[4] W. F. Ames. *Numerical methods for partial differential equations*. Academic press, 2014.

[5] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning*, pages 214–223, 2017.

[6] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath. Deep reinforcement learning: A brief survey. *IEEE signal processing magazine*, 34(6):26–38, 2017.

[7] M. Bardi, I. C. Dolcetta, et al. *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*, volume 12. Springer, 1997.

[8] A. R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information theory*, 39(3):930–945, 2002.

[9] S. Bartels et al. *Numerical methods for nonlinear partial differential equations*, volume 47. Springer, 2015.

[10] M. G. Bellemare, W. Dabney, and R. Munos. A distributional perspective on reinforcement learning. In *International conference on machine learning*, pages 449–458. PMLR, 2017.

[11] A. Ben-Tal and A. Nemirovski. *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*, volume 2. Society for Industrial and Applied Mathematics, 1987.

[12] D. Bertsekas. *Dynamic programming and optimal control: Volume I*, volume 4. Athena scientific, 2012.

[13] D. Bertsekas. *Convex optimization algorithms*. Athena Scientific, 2015.

[14] D. Bertsekas. *Nonlinear programming*. Athena Scientific, 2016.

[15] D. Bertsekas. *Reinforcement learning and optimal control*, volume 1. Athena Scientific, 2019.

[16] D. P. Bertsekas, A. Nedić, and A. E. Ozdaglar. *Convex analysis and optimization*. Athena Scientific Belmont, 2003.

[17] D. P. Bertsekas and J. N. Tsitsiklis. Neuro-dynamic programming: an overview. In *Proceedings of 1995 34th IEEE conference on decision and control*, volume 1, pages 560–564. IEEE, 1995.

[18] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π: A vision-language-action flow model for general robot control. *CoRR*, 2024.

[19] J. M. Borwein and A. S. Lewis. *Convex analysis and nonlinear optimization: theory and examples*, volume 3. Springer, 2006.

[20] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.

[21] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[22] A. Bradley and P. Nakkiran. Classifier-free guidance is a predictor-corrector. In *NeurIPS 2024 Workshop on Mathematics of Modern Machine Learning*, 2024.

[23] A. E. Bryson. *Applied optimal control: optimization, estimation and control*. Routledge, 2018.

[24] R. L. Burden and J. D. Faires. *Numerical Analysis 9th edition*. Boston, MA: Brooks/Cole, 2011.

[25] A. Campbell, J. Yim, R. Barzilay, T. Rainforth, and T. Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning*, pages 5453–5512. PMLR, 2024.

[26] R. T. Chen and Y. Lipman. Flow matching on general geometries. In *International Conference on Learning Representations*, 2024.

[27] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.

[28] M. Chidambaram, K. Gatmiry, S. Chen, H. Lee, and J. Lu. What does guidance do? a fine-grained analysis in a simple setting. *Advances in Neural Information Processing Systems*, 37:84968–85005, 2024.

[29] E. K. Chong and S. H. Zak. *An introduction to optimization*, volume 76. John Wiley & Sons, 2013.

[30] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[31] W. Dabney, M. Rowland, M. Bellemare, and R. Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[32] P. Dhariwal and A. Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.

[33] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

[34] W. E and B. Yu. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.

[35] S. Ellacott. Aspects of the numerical analysis of neural networks. *Acta numerica*, 3:145–202, 1994.

[36] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.

[37] L. C. Evans. *Partial differential equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 1998.

[38] L. C. Evans. *An introduction to stochastic differential equations*, volume 82. American Mathematical Soc., 2012.

[39] L. C. Evans. *An introduction to mathematical optimal control theory spring, 2024 version*. University of California, Berkeley, 2024.

[40] F. Farnia and A. Ozdaglar. Do gans always have nash equilibria? In *International Conference on Machine Learning*, pages 3029–3039. PMLR, 2020.

[41] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[42] W. Fleming and R. Rishel. *Deterministic and Stochastic Optimal Control*. Springer, 1975.

[43] M. Fortunato, M. G. Azar, B. Piot, J. Menick, M. Hessel, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, C. Blundell, and S. Legg. Noisy networks for exploration. In *International Conference on Learning Representations*, 2018.

[44] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, J. Pineau, et al. An introduction to deep reinforcement learning. *Foundations and Trends® in Machine Learning*, 11(3-4):219–354, 2018.

[45] N. Gaby and X. Ye. Approximation of solution operators for high-dimensional pdes. *Journal of Computational Physics*, 524:113709, 2025.

[46] N. Gaby and X. Ye. Hamiltonian theory and computation of optimal probability density control in high dimensions. *arXiv preprint arXiv:2505.18362*, 2025.

[47] N. Gaby, X. Ye, and H. Zhou. Neural control of parametric solutions for high-dimensional evolution pdes. *SIAM Journal on Scientific Computing*, 46(2):C155–C185, 2024.

[48] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Chen, G. Synnaeve, Y. Adi, and Y. Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024.

[49] G. Golub and C. van Van Loan. *Matrix Computations*. The Johns Hopkins University Press, 3rd edition, 1996.

[50] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[51] W. Grathwohl, R. T. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.

[52] K. Gregor, F. Besse, D. Jimenez Rezende, I. Danihelka, and D. Wierstra. Towards conceptual compression. In *Advances in neural information processing systems*, volume 29, 2016.

[53] K. Gregor, G. Papamakarios, F. Besse, L. Buesing, and T. Weber. Temporal difference variational auto-encoder. In *International Conference on Learning Representations*, 2019.

[54] J. Gui, Z. Sun, Y. Wen, D. Tao, and J. Ye. A review on generative adversarial networks: Algorithms, theory, and applications. *IEEE transactions on knowledge and data engineering*, 35(4):3313–3332, 2021.

[55] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, pages 5767–5777, 2017.

[56] I. Gulrajani, K. Kumar, F. Ahmed, A. A. Taiga, F. Visin, D. Vazquez, and A. Courville. Pixelvae: A latent variable model for natural images. In *International Conference on Learning Representations*, 2017.

[57] Y. Guo, H. Yuan, Y. Yang, M. Chen, and M. Wang. Gradient guidance for diffusion models: An optimization perspective. *Advances in Neural Information Processing Systems*, 37:90736–90770, 2024.

[58] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[59] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[60] E. Heitz, L. Belcour, and T. Chambon. Iterative α-(de) blending: A minimalist deterministic diffusion model. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–8, 2023.

[61] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[62] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

[63] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[64] J. Ho and T. Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

[65] L. M. Hocking. *Optimal control: an introduction to the theory with applications*. Oxford University Press, 1991.

[66] P. Holderrieth, M. Havasi, J. Yim, N. Shaul, I. Gat, T. Jaakkola, B. Karrer, R. T. Chen, and Y. Lipman. Generator matching: Generative modeling with arbitrary markov processes. In *International Conference on Learning Representations*, 2025.

[67] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257, 1991.

[68] K. Hornik. Some new results on neural network approximation. *Neural networks*, 6(8):1069–1072, 1993.

[69] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.

[70] K. Hornik, M. Stinchcombe, H. White, and P. Auer. Degree of approximation results for feedforward networks approximating unknown mappings and their derivatives. *Neural computation*, 6(6):1262–1275, 1994.

[71] G. Huguet, J. Vuckovic, K. Fatras, E. Thibodeau-Laufer, P. Lemos, R. Islam, C. Liu, J. Rector-Brooks, T. Akhound-Sadegh, M. Bronstein, et al. Sequence-augmented se (3)-flow matching for conditional protein generation. *Advances in neural information processing systems*, 37:33007–33036, 2024.

[72] C. Jin, P. Netrapalli, and M. Jordan. What is local optimality in nonconvex-nonconcave minimax optimization? In *International conference on machine learning*, pages 4880–4889. PMLR, 2020.

[73] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[74] H. J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of statistical mechanics: theory and experiment*, 2005(11):P11011, 2005.

[75] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.

[76] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In Y. Bengio and Y. LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[77] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems*, pages 4743–4751, 2016.

[78] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *International Conference on Learning Representations*, 2014.

[79] D. P. Kingma, M. Welling, et al. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019.

[80] V. Konda and J. Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.

[81] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther. Autoencoding beyond pixels using a learned similarity metric. In *International conference on machine learning*, pages 1558–1566. PMLR, 2016.

[82] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz, M. Williamson, V. Manohar, Y. Adi, J. Mahadeokar, et al. Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in neural information processing systems*, 36:14005–14034, 2023.

[83] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural networks*, 6(6):861–867, 1993.

[84] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[85] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[86] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.

[87] G.-H. Liu, A. Vahdat, D.-A. Huang, E. Theodorou, W. Nie, and A. Anandkumar. I2sb: Image-to-image schrödinger bridge. In *International Conference on Machine Learning*, pages 22042–22062. PMLR, 2023.

[88] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

[89] X. Liu, C. Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[90] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[91] L. Maaløe, C. K. Sønderby, S. K. Sønderby, and O. Winther. Auxiliary deep generative models. In *International conference on machine learning*, pages 1445–1453. PMLR, 2016.

[92] J. Macki and A. Strauss. *Introduction to optimal control theory*. Springer Science & Business Media, 2012.

[93] X. Mao. *Stochastic differential equations and applications*. Elsevier, 2007.

[94] X. Mao, Q. Li, H. Xie, R. Y. Lau, Z. Wang, and S. Paul Smolley. Least squares generative adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2794–2802, 2017.

[95] L. Metz, B. Poole, D. Pfau, and J. Sohl-Dickstein. Unrolled generative adversarial networks. In *International Conference on Learning Representations*, 2017.

[96] H. N. Mhaskar. Neural networks for optimal approximation of smooth and analytic functions. *Neural computation*, 8(1):164–177, 1996.

[97] H. N. Mhaskar and C. A. Micchelli. Dimension-independent bounds on the degree of approximation by neural networks. *IBM Journal of Research and Development*, 38(3):277–284, 1994.

[98] H. N. Mhaskar and C. A. Micchelli. Degree of approximation by neural and translation networks with a single hidden layer. *Advances in applied mathematics*, 16(2):151–183, 1995.

[99] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[100] H. Montanelli and Q. Du. New error bounds for deep relu networks using sparse grids. *SIAM Journal on Mathematics of Data Science*, 1(1):78–92, 2019.

[101] K. Neklyudov, R. Brekelmans, D. Severo, and A. Makhzani. Action matching: Learning stochastic dynamics from samples. In *International conference on machine learning*, pages 25858–25889. PMLR, 2023.

[102] Y. Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Springer Publishing Company, Incorporated, 1 edition, 2014.

[103] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pages 8162–8171. PMLR, 2021.

[104] A. Q. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. Mcgrew, I. Sutskever, and M. Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. In *International Conference on Machine Learning*, pages 16784–16804. PMLR, 2022.

[105] J. Nocedal and S. Wright. *Numerical optimization*. Springer Science & Business Media, 2006.

[106] B. Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer Science & Business Media, 2010.

[107] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[108] G. Peyré and M. Cuturi. Computational optimal transport. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.

[109] A. Pinkus. Approximation theory of the mlp model in neural networks. *Acta numerica*, 8:143–195, 1999.

[110] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. Why and when can deep-but not shallow-networks avoid the curse of dimensionality: a review. *International Journal of Automation and Computing*, 14(5):503–519, 2017.

[111] A. Polyak, A. Zohar, A. Brown, A. Tjandra, A. Sinha, A. Lee, A. Vyas, B. Shi, C.-Y. Ma, C.-Y. Chuang, et al. Movie gen: A cast of media foundation models. *CoRR*, 2024.

[112] L. S. Pontryagin. *Mathematical theory of optimal processes*. Routledge, 2018.

[113] G.-J. Qi. Loss-sensitive generative adversarial networks on lipschitz densities. *International Journal of Computer Vision*, 128(5):1118–1140, 2020.

[114] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[115] B. Recht. A tour of reinforcement learning: The view from continuous control. *Annual Review of Control, Robotics, and Autonomous Systems*, 2(1):253–279, 2019.

[116] S. J. Reddi, S. Kale, and S. Kumar. On the convergence of adam and beyond. In *International Conference on Learning Representations*, 2018.

[117] D. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.

[118] H. Risken. *The Fokker-Planck equation: methods of solution and applications*. Springer Verlag, 1985.

[119] H. Robbins and S. Monro. A stochastic approximation method. *The annals of mathematical statistics*, pages 400–407, 1951.

[120] R. Rockafellar. *Convex Analysis*. Princeton University Press, 1996.

[121] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[122] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. *CoRR*, abs/1505.04597, 2015.

[123] D. Rumelhart, G. Hinton, and R. Williams. Learning internal representations by error propagation. In *Neurocomputing: foundations of research*, pages 673–695. 1988.

[124] G. A. Rummery and M. Niranjan. *On-line Q-learning using connectionist systems*, volume 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

[125] F. Santambrogio. Optimal transport for applied mathematicians. *Birkäuser, NY*, 55:58–63, 2015.

[126] R. Sargent. Optimal control. *Journal of Computational and Applied Mathematics*, 124(1-2):361–371, 2000.

[127] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.

[128] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.

[129] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[130] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[131] Y. Shi, V. De Bortoli, A. Campbell, and A. Doucet. Diffusion schrödinger bridge matching. *Advances in Neural Information Processing Systems*, 36:62183–62223, 2023.

[132] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[133] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. Pmlr, 2014.

[134] S. P. Singh and R. S. Sutton. Reinforcement learning with replacing eligibility traces. *Machine learning*, 22(1):123–158, 1996.

[135] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.

[136] K. Sohn, H. Lee, and X. Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

[137] C. K. Sønderby, T. Raiko, L. Maaløe, S. K. Sønderby, and O. Winther. Ladder variational autoencoders. *Advances in neural information processing systems*, 29, 2016.

[138] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[139] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[140] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. Bradford Books, 2018.

[141] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.

[142] C. Szepesvári. *Algorithms for reinforcement learning*. Springer nature, 2022.

[143] J. W. Thomas. *Numerical partial differential equations: conservation laws and elliptic equations*, volume 33. Springer Science & Business Media, 2013.

[144] J. W. Thomas. *Numerical partial differential equations: finite difference methods*, volume 22. Springer Science & Business Media, 2013.

[145] T. Tieleman and G. Hinton. Rmsprop: Divide the gradient by a running average of its recent magnitude. coursera: Neural networks for machine learning. *COURSERA Neural Networks Mach. Learn*, 17:6, 2012.

[146] E. Todorov. Linearly-solvable markov decision problems. *Advances in neural information processing systems*, 19, 2006.

[147] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

[148] L. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM: Society for Industrial and Applied Mathematics, 1997.

[149] H. Van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

[150] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[151] C. Villani. *Optimal transport: old and new*, volume 338. Springer Science & Business Media, 2008.

[152] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4):5064–5078, 2022.

[153] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.

[154] Z. Wang, Q. She, and T. E. Ward. Generative adversarial networks in computer vision: A survey and taxonomy. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

[155] C. J. Watkins. *Learning from delayed rewards*. PhD thesis, University of Cambridge, 1989.

[156] C. J. Watkins and P. Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[157] J. Wu, Z. Huang, J. Thoma, D. Acharya, and L. Van Gool. Wasserstein divergence for gans. In *Proceedings of the European conference on computer vision (ECCV)*, pages 653–668, 2018.

[158] D. Yarotsky. Error bounds for approximations with deep relu networks. *Neural Networks*, 94:103–114, 2017.

[159] Y. Zang, G. Bao, X. Ye, and H. Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, page 109409, 2020.

[160] J. Zhao, M. Mathieu, and Y. LeCun. Energy-based generative adversarial networks. In *International Conference on Learning Representations*, 2017.

索引

Actor-Critic 方法, 216
AdaGrad, 111
Adam, 112
AdamW, 113
Armijo 條件, 74
Automatic differentiation
    reverse mode, 66

Banach 不動點定理, 273
Banach 空間, 272
Bellman 方程, 186
Bellman 算子, 186, 187
BFGS 方法, 83, 100
Bregman 散度, 276

Cauchy 序列, 272
Control Hamiltonian, 128

Double DQNs 算法, 212
DPG 方法, 220
DQN 算法, 211

Epoch, 103
Euler–Lagrange 方程, 121

Fokker–Planck 方程, 239, 250, 290

Hamilton–Jacobi–Bellman 方程, 146
Hamiltonian, 124
    動力學, 123
Hamiltonian 動力學, 124
Hessian, 50

Itô 公式, 288
Itô 鏈式法則, 288
Itô 隨機積分, 287

Jacobian, 50
Jensen–Shannon 散度, 281

Karush–Kuhn–Tucker
    KKT 乘子, 58
    KKT 條件, 58
    KKT 點, 58
    定理, 56
Kullback–Leibler 散度, 278

Lagrange
    乘子, 55
    條件, 87, 94
    法, 90
Lagrangian
    函數, 89, 120

方法, 89, 90
點, 86
Lipschitz 連續, 105

Markov 決策過程, 182
Markov 獎勵過程, 184
Markov 過程, 182
Monte Carlo 方法, 204
Monte Carlo 積分, 40, 269
Muon, 114

Neural ODE 方法, 164
Newton 方向, 78
Newton 法, 78
Newton–CG 法, 82

Ornstein–Uhlenbeck 過程, 238

Pontryagin 極大值原理, 128, 134, 144
PPO 方法, 219

Q 函數, 192
Q 學習算法, 210

RMSProp, 112

SAC 方法, 222
SARSA 方法, 208
Sherman–Morrison–Woodbury 公式, 83

Variational method, 120

Wasserstein 距離, 282
Weiner 過程, 285
Wolfe 條件, 75

下降方向, 73
不動點迭代, 90, 189
二次罰函數, 84
二次罰函數法, 84
仿射函數, 1, 114
伴隨方程, 133
似然函數, 2
對數, 3

值函數, 146
狀態, 184
狀態–動作, 192
偏微分方程, 38, 172
價值
迭代, 203
價值改進, 203
優勢函數, 219
充分下降條件, 74
充分條件
二階, 52, 60
內點, 46
全局極小值點, 48
共軛梯度法, 81
凸優化, 47, 49
凸函數, 47
強, 78
凸集, 47
凹函數, 47
切空間, 53
判別器, 233
利用, 209
割線方程, 83
動作, 181
動量, 111
協態, 133
反向傳播, 62

可行方向, 51
可行集, 46
向量場
　條件生成, 254
　生成, 253
回溯線搜索, 74
基本隨機梯度方法, 100
增廣 Lagrange 函數, 93, 96
增廣 Lagrange 方法, 92, 94
多層感知機, 8
多重變分, 138
奇異值, 115
奇異值分解, 114
對角縮放, 111
小批量隨機方法
　二階, 104
小批量隨機梯度方法, 103
局部極小值點, 48, 51
層
　輸入, 7
　輸出, 7
　隱藏, 7
布朗運動, 285
常微分方程, 123
廣義動量, 123
強化學習
　基於模型, 198
得分函數, 242
　條件, 242, 243
得分匹配, 241
必要條件
　一階, 51
　二階, 52, 55, 58

批量梯度方法, 100
探索, 209
控制, 120
　容許集, 120
損失 hansh, 46
損失函數, 38, 45, 98, 104, 259
　期望, 99
　經驗, 99
擬 Newton 法, 82, 83
擴散, 237
　係數, 237
擴散模型, 237
擾動函數, 132, 139
收斂
　局部, 79
收斂速率
　二次, 80
　次線性, 77, 103, 117
　線性, 92, 102, 117, 189, 273
　超線性, 84
時序差分方法, 206
普適逼近定理, 21
智能體, 181
曲率條件, 74
最優傳輸, 282, 283
最優控制, 127
　固定終止時刻且終點自由, 128
最速下降法, 74
有約束優化, 46, 84
本性上確界, 12
條件數, 81
梯度, 49
梯度下降法, 73
極大似然估計, 3
極小值點

全局, 85, 94, 244
局部, 54–56, 58, 60, 61, 79
正則化, 113
正規點
等式和不等式約束, 56
等式約束, 53
步長, 73
活躍集, 56
流匹配, 253
深度學習, 9
激活函數, 8
CELU, 27
ELU, 26
GELU, 27
ReLU, 10, 26
sigmoid, 24
SwiGLU, 27
swish, 27
tanh, 26

無約束優化, 46, 73
熵, 275
條件, 276
狀態, 181
獎勵, 181
終端, 128
運行, 128
環境, 181
生成
引導, 263
生成器, 233
異常問題, 144
病態, 78
目標函數, 38, 45, 46, 71, 99
確定性優化, 71
神經網絡
LSTM, 33
Wasserstein 生成, 236
前饋, 11
卷積, 29
圖, 31
弱對抗, 41
循環, 32
殘差, 28
深度, 8
淺度, 8
物理信息, 39, 40
生成對抗, 233
空間
Sobolev, 12
策略
ϵ-貪婪, 209
平穩, 182
改進, 199
改進定理, 195
最優, 193, 197
評估, 199
貪婪, 191, 197
迭代, 199
非平穩, 182
策略梯度方法, 212
範數, 10, 12, 272
Frobenius, 115
簡單變分, 131
約束集, 46
終止判據, 75
網絡架構, 10
線性空間, 271
賦範, 272

聚點, 85
胡信息, 284
臨界點, 6
自動微分, 62
    Hessian, 71
    Hessian-vector, 70
    Jacobian, 67
    前向掃描, 64
    前向模式, 64, 68
    反向掃描, 66
    反向模式, 71
    梯度, 66
    計算圖, 62
自治, 166, 173
自適應方法, 111
解算子, 174
計算單元, 11
證據, 229
證據下界 (ELBO), 230
變分, 120
變分自編碼器, 226
    編碼器, 226
    解碼器, 226
變分錐, 139
超參數, 77
距離, 272

連續方程, 247, 250, 290
邏輯函數, 4
重要性採樣, 269
鏈式法則, 62
隨機優化, 98
隨機微分方程, 237, 284
非活躍集, 56
鞍點問題, 234, 264